

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO ĐỒ ÁN CHUYÊN NGÀNH

XÂY DỰNG HỆ THỐNG QUẢN LÝ THƯ VIỆN TRỰC TUYẾN
TÍCH HỢP AI HỖ TRỢ TÌM KIẾM VÀ GỢI Ý SÁCH

ProjectID: HK251-DAGD1-512
NHÓM: HAI ANH TRAI QUẢNG TRỊ

Giảng viên hướng dẫn: TS. Trương Tuấn Anh
CN. Dương Huỳnh Anh Đức
Thành viên: Võ Quang Thắng - 2213214
Hồ Sỹ Thắng - 2213188

TPHCM, tháng 5 năm 2026

LỜI CẢM ƠN

Trong suốt thời gian học tập và nghiên cứu tại **Trường Đại học Bách Khoa - Đại học Quốc gia Thành phố Hồ Chí Minh**, chúng em đã nhận được sự quan tâm, giảng dạy và hướng dẫn tận tình từ quý thầy cô trong khoa. Đồ án chuyên ngành này là minh chứng cho sự nỗ lực không ngừng của chúng em trong việc vận dụng những kiến thức đã được trang bị để giải quyết các vấn đề thực tiễn trong lĩnh vực công nghệ thông tin.

Trước tiên, chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến **TS. Trương Tuấn Anh**, người thầy đã tận tâm hướng dẫn, định hướng và đồng hành cùng chúng em trong suốt quá trình thực hiện đồ án. Thầy không chỉ truyền đạt kiến thức chuyên môn mà còn định hướng phương pháp nghiên cứu khoa học, giúp chúng em có cái nhìn sâu sắc hơn về vấn đề đang giải quyết. Những lời khuyên, góp ý quý báu của thầy là động lực to lớn giúp chúng em vượt qua mọi khó khăn, thử thách để hoàn thành tốt đồ án này.

Chúng em cũng xin gửi lời cảm ơn chân thành đến **CN. Dương Huỳnh Anh Đức** đã đồng hành, hỗ trợ và đóng góp nhiều ý kiến quý báu trong quá trình thực hiện đồ án, giúp chúng em có thêm góc nhìn thực tiễn và áp dụng hiệu quả các giải pháp kỹ thuật.

Chúng em xin bày tỏ lòng biết ơn sâu sắc đến toàn thể quý thầy cô **Khoa Khoa học và Kỹ thuật Máy tính** đã tận tình giảng dạy, truyền đạt kiến thức và kinh nghiệm quý báu trong suốt những năm học vừa qua. Nền tảng kiến thức vững chắc mà các thầy cô đã trang bị là hành trang vô cùng quý giá để chúng em tự tin bước vào môi trường làm việc thực tế.

Chúng em cũng xin cảm ơn gia đình, bạn bè đã luôn bên cạnh, động viên và tạo điều kiện tốt nhất để chúng em có thể hoàn thành tốt chương trình học tập cũng như đồ án chuyên ngành này.

Do thời gian và kinh nghiệm có hạn, đồ án không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự góp ý, chỉ bảo của quý thầy cô để đồ án được hoàn thiện hơn và chúng em có thêm kinh nghiệm quý báu cho công việc sau này.

Xin chân thành cảm ơn!

Tp. Hồ Chí Minh, tháng 05 năm 2026

Nhóm sinh viên thực hiện

TÓM TẮT

Đồ án chuyên ngành này tập trung vào việc nghiên cứu, thiết kế và xây dựng **Hệ thống Quản lý Thư viện Trực tuyến tích hợp AI**, nhằm giải quyết các hạn chế của các hệ thống thư viện truyền thống như khả năng tìm kiếm kém hiệu quả và thiếu sự cá nhân hóa. Hệ thống được phát triển với mục tiêu số hóa toàn diện quy trình vận hành thư viện, đồng thời ứng dụng trí tuệ nhân tạo để nâng cao trải nghiệm đọc và khám phá tri thức của người dùng.

Về mặt kỹ thuật, hệ thống được xây dựng dựa trên kiến trúc **Modular Monolith** với Clean Architecture + Domain-Driven Design cho từng module, đảm bảo tính bảo mật và khả năng mở rộng. Backend sử dụng **Spring Boot (Java)** để xử lý các nghiệp vụ phức tạp, kết hợp với Frontend **React + Vite** mang lại giao diện hiện đại, tốc độ build nhanh và triển khai đơn giản qua NGINX. Các chức năng chính bao gồm: quản lý tài nguyên sách, quản lý độc giả, quy trình mượn trả tự động, kiểm soát phí phạt và hệ thống báo cáo thống kê trực quan cho thủ thư.

Điểm đột phá của đồ án là việc tích hợp sâu các công nghệ AI/NLP tiên tiến. Cụ thể, hệ thống ứng dụng mô hình **Sentence-BERT (SBERT)** để thực hiện tìm kiếm ngữ nghĩa (Semantic Search), giúp xử lý các truy vấn tiếng Việt phức tạp mà không phụ thuộc vào từ khóa chính xác. Song song đó, hệ thống gợi ý sách sử dụng phương pháp lai (**Hybrid Approach**), kết hợp giữa Lọc cộng tác (Collaborative Filtering) và Lọc dựa trên nội dung (Content-Based), giúp đưa ra các đề xuất sách phù hợp nhất dựa trên lịch sử và hành vi của từng độc giả.

Kết quả của đồ án là một hệ thống phần mềm hoàn chỉnh, có giao diện thân thiện, đáp ứng đầy đủ các yêu cầu nghiệp vụ của một thư viện đại học. Hệ thống đã chứng minh được tính khả thi trong việc kết hợp giữa quản lý truyền thống và công nghệ thông minh, mang lại giá trị thực tiễn cao trong việc thúc đẩy văn hóa đọc và chuyển đổi số trong môi trường giáo dục.

PHÂN CÔNG CÔNG VIỆC

STT	Nội dung công việc / Chương mục	Sinh viên thực hiện	Ghi chú
1	Nghiên cứu tổng quan và Cơ sở lý thuyết - Chương 1: Giới thiệu dự án - Chương 2: Kiến thức và công nghệ nền tảng - Chương 3: Công trình liên quan	Võ Quang Thắng Hồ Sỹ Thắng	Làm chung
2	Phân tích yêu cầu và Hệ thống đề xuất - Chương 4: Hệ thống đề xuất - Phân tích Use Case, Activity Diagram - Thiết kế cơ sở dữ liệu (ERD, Schema)	Võ Quang Thắng Hồ Sỹ Thắng	Làm chung
3	Thiết kế kiến trúc hệ thống (Chương 5) - Kiến trúc Modular Monolith + Clean Architecture - Thiết kế AI Service (FastAPI) - Luồng giao tiếp giữa các thành phần	Võ Quang Thắng Hồ Sỹ Thắng	Làm chung
4	Hiện thực Back-end (Spring Boot) - Xây dựng REST API và xử lý nghiệp vụ - Quản lý giao dịch, Domain Events - Xác thực và phân quyền (JWT)	Võ Quang Thắng	Phụ trách chính
5	Hiện thực module AI (Python FastAPI) - Tìm kiếm ngữ nghĩa (SBERT + pgvector) - Hệ thống gợi ý lai ghép (content-based, ALS, trending fallback) - Pipeline xử lý PDF, embedding và reduce metadata bằng Gemini	Hồ Sỹ Thắng	Phụ trách chính
6	Hiện thực Front-end (React + Vite) - Thiết kế giao diện người dùng (UI/UX) - Xây dựng các trang đọc giả, thủ thư và admin - Tích hợp API Back-end, thông báo realtime và module AI	Võ Quang Thắng Hồ Sỹ Thắng	Làm chung
7	Kiểm thử hệ thống (Chương 7) - Kiểm thử Backend: JUnit 5, Mockito, MockMvc, Testcontainers - Kiểm thử Frontend: Jest, Testing Library, service contract - Kiểm thử API và các luồng nghiệp vụ chính	Võ Quang Thắng	Phụ trách chính

Tiếp tục ở trang sau

Bảng – Tiếp theo từ trang trước

STT	Nội dung công việc / Chương mục	Sinh viên thực hiện	Ghi chú
8	Kiểm thử và Đánh giá module AI - Kiểm thử PDF processing, chunking và metadata quality - Kiểm thử SQL contract cho schema AI/public - Kiểm thử live Gemini report khi có quota API	Hồ Sỹ Thắng	Phụ trách chính
9	Triển khai hệ thống (Chương 8) - Đóng gói Docker (Multi-stage Build) - Cấu hình Docker Compose, NGINX Reverse Proxy - Triển khai lên VPS, cấu hình SSL/TLS	Võ Quang Thắng	Phụ trách chính
10	Tổng kết và Hoàn thiện báo cáo - Chương 8: Tổng kết - Tổng hợp báo cáo, chỉnh sửa format - Chuẩn bị slide thuyết trình	Võ Quang Thắng Hồ Sỹ Thắng	Làm chung

Mục lục

1	Giới thiệu	1
1.1	Động cơ của đề tài	1
1.2	Mục tiêu	1
1.3	Phạm vi	2
1.4	Ý nghĩa	2
1.4.1	Ý nghĩa thực tiễn	2
1.4.2	Ý nghĩa khoa học	3
1.5	Cấu trúc báo cáo	3
2	Kiến thức và công nghệ nền tảng	4
2.1	Các Lĩnh vực Công nghệ Chính	4
2.2	Nền tảng Kiến thức Khoa học	4
2.3	Kết chương	5
3	Công trình liên quan	6
3.1	Các công trình nghiên cứu khoa học	6
3.2	Các hệ thống và phần mềm tương tự	6
3.3	Kết chương và xác định khoảng trống nghiên cứu	7
4	Hệ thống đề xuất	8
4.1	Mô tả hệ thống	8
4.2	Hướng tiếp cận thiết kế	8
4.3	Yêu cầu	9
4.3.1	Yêu cầu chức năng	9
4.3.2	Yêu cầu phi chức năng	10
4.3.3	Yêu cầu dữ liệu	10
4.4	Kết chương	11
5	Phân tích và Thiết kế	12
5.1	Phân tích	12
5.1.1	Mô hình hóa tương tác (Biểu đồ Usecase)	12
5.1.2	Mô hình hoá quy trình nghiệp vụ (Activity diagram)	17
5.2	Giải pháp công nghệ	27
5.2.1	Front-end	27
5.2.2	Back-end	28
5.3	Ứng dụng AI vào hệ thống	30
5.3.1	Module 1: Xử lý PDF file	31
5.3.2	Module 2: Hỗ trợ tìm kiếm ngữ nghĩa (Semantic Search)	32
5.3.3	Module 3: Gợi ý cá nhân hoá (Recommendation)	33
5.4	Thiết kế	35

5.4.1	Kiến trúc hệ thống	35
5.4.2	Thiết kế cơ sở dữ liệu	38
5.4.3	Thiết kế chi tiết bên trong	40
5.4.4	Thiết kế giao diện	54
6	Hiện thực hệ thống	57
6.1	Hiện thực ứng dụng	57
6.1.1	Hiện thực Frontend	57
6.1.2	Hiện thực Backend	78
6.1.3	Hiện thực AI Service	93
7	Kiểm thử phần mềm	104
7.1	Kiểm thử Frontend	104
7.1.1	Môi trường và cấu hình kiểm thử	104
7.1.2	Phạm vi test đã hiện thực	105
7.1.3	Nhóm test ánh xạ lỗi nghiệp vụ	106
7.1.4	Nhóm test đa ngôn ngữ và notification	107
7.1.5	Nhóm test theme light/dark/system	108
7.1.6	Nhóm test helper đặt trước	109
7.1.7	Nhóm test xác thực và phiên đăng nhập	109
7.1.8	Nhóm test upload và component dùng chung	110
7.1.9	Nhóm test trang liên hệ	111
7.1.10	Nhóm test service contract	111
7.1.11	Kết quả thực thi	113
7.1.12	Đánh giá và giới hạn	113
7.2	Kiểm thử Backend	113
7.2.1	Môi trường và cấu hình kiểm thử Backend	113
7.2.2	Phạm vi test Backend đã hiện thực	114
7.2.3	Nhóm test Catalog	115
7.2.4	Nhóm test Circulation và Reservation	116
7.2.5	Nhóm test Auth, User, Recommendation và AI	116
7.2.6	Đối soát test với các tính năng admin final	117
7.2.7	Kết quả thực thi Backend	117
7.2.8	Đánh giá và giới hạn	117
7.3	Kiểm thử AI Service	118
7.3.1	Môi trường kiểm thử AI	118
7.3.2	Phạm vi test AI đã hiện thực	119
7.3.3	Nhóm test PDF processing và chunking	119
7.3.4	Nhóm test LLM metadata quality	120
7.3.5	Nhóm test ETL pipeline và database contract	120
7.3.6	Nhóm test API gateway, recommender và worker	121
7.3.7	Kết quả thực thi AI	121
7.3.8	Kiểm thử tích hợp Gemini thật	122
7.3.9	Đánh giá và giới hạn	122
8	Triển khai hệ thống	124
8.1	Container hóa với Docker	124
8.1.1	Giới thiệu Docker	124
8.1.2	Cấu hình môi trường phát triển cục bộ	124

8.2	Dockerfile đa giai đoạn	125
8.2.1	Backend Dockerfile	125
8.2.2	Frontend Dockerfile	125
8.3	Nginx — Reverse Proxy	125
8.3.1	Vai trò của Nginx	125
8.3.2	Cấu hình định tuyến	125
8.4	Môi trường sản xuất	126
8.4.1	Docker Compose sản xuất	126
8.4.2	Quản lý biến môi trường và bảo mật	126
8.4.3	Flyway Migration tự động	127
8.4.4	Sơ đồ kiến trúc triển khai	127
8.5	Quy trình triển khai	127
8.6	Hạ tầng mạng: VPS, Domain và HTTPS	128
8.6.1	Máy chủ ảo (VPS)	128
8.6.2	Tên miền (Domain)	129
8.6.3	HTTPS và SSL/TLS Certificate	129
8.6.4	Tổng chi phí vận hành	130
9	Tổng kết	131
9.1	Kết quả đạt được	131
9.1.1	Kiến trúc phần mềm	131
9.1.2	Tích hợp Trí tuệ nhân tạo	131
9.1.3	Kiểm thử	131
9.1.4	Triển khai hệ thống	132
9.2	Nhận xét và đánh giá	132
10	Tài nguyên và Mã nguồn dự án	133
	Tài liệu tham khảo	134

Danh sách hình vẽ

5.1	Biểu đồ Usecase tổng quát theo phạm vi hệ thống đã hiện thực	12
5.2	Đăng ký tài khoản — Activity Diagram	18
5.3	Đăng nhập tài khoản — Activity Diagram	19
5.4	Tìm kiếm sách — Activity Diagram	20
5.5	Gợi ý sách — Activity Diagram	21
5.6	Mượn sách — Activity Diagram	22
5.7	Xử lý hàng đợi đặt trước — Activity Diagram	23
5.8	Trả sách — Activity Diagram	24
5.9	Đánh giá và bình luận sách — Activity Diagram	25
5.10	Tạo sách với AI — Activity Diagram	26
5.11	Ranh giới trách nhiệm giữa frontend, backend và AI service	31
5.12	Module 1 – Luồng xử lý PDF file sau khi thủ thư upload	31
5.13	Module 2 – Luồng thực thi semantic search	32
5.14	Module 3 – Luồng xử lý hybrid recommendation	34
5.15	So sánh các đặc tính của các kiểu kiến trúc (Nguồn: Mark Richards, Neal Ford, <i>Fundamentals of Software Architecture</i> , O'Reilly, 2nd ed., 2020) [1].	36
5.16	Sơ đồ Quan hệ Thực thể (ERD) tổng thể của hệ thống	39
5.17	Ánh xạ cơ sở dữ liệu quan hệ ở dạng tổng quát	40
5.18	Sơ đồ kiến trúc tổng thể hệ thống	41
5.19	Các Bounded Contexts trong hệ thống	42
5.20	Clean architecture (R. C. Martin, <i>Clean Architecture: A Craftsman's Guide to Software Structure and Design</i> , Pearson Education, 2018 [2])	43
5.21	Cấu trúc Clean Architecture kết hợp Domain-Driven Design trong một Module	44
5.22	Sơ đồ luồng dữ liệu và Kiến trúc chi tiết AI Service	48
5.23	Kiến trúc chi tiết AI Service và Luồng dữ liệu	49
5.24	Giao diện Public: Trang chủ và Tìm kiếm	54
5.25	Giao diện Chi tiết sách	55
5.26	Giao diện Độc giả: Dashboard và Quản lý mượn sách	55
5.27	Giao diện Thủ thư: Quản lý danh mục và Lưu thông	56
6.1	Cấu trúc thư mục mã nguồn Frontend	57
6.2	Quan hệ giữa các nhóm module Frontend	58
6.3	Luồng tải màn hình bằng lazy loading	59
6.4	Luồng xử lý route và phân quyền trên Frontend	59
6.5	Thiết lập theme và ngôn ngữ trên giao diện	63
6.6	Pipeline xử lý request và response bằng Axios interceptor	64
6.7	Giao diện tìm kiếm sách ở nhóm trang công khai	67
6.8	Giao diện chi tiết sách	69
6.9	Trang liên hệ tích hợp Google Maps	70

6.10	Luồng mượn hoặc đặt trước từ trang chi tiết sách	70
6.11	Giao diện đánh giá và phản hồi sách	70
6.12	Giao diện dashboard độc giả	72
6.13	QR nhận sách trong giao diện độc giả	72
6.14	Dữ liệu điều hướng trong khu vực độc giả	73
6.15	Giao diện tra cứu và xác nhận mượn sách của thủ thư	73
6.16	Các luồng nghiệp vụ tại quầy thủ thư	74
6.17	Modal QR thanh toán phí phạt qua PayOS	74
6.18	Giao diện quản lý ấn phẩm của thủ thư	75
6.19	Luồng upload tài liệu PDF của thủ thư	75
6.20	UploadPanel hiển thị tiến độ upload tài liệu	76
6.21	Luồng nhận và hiển thị thông báo thời gian thực	76
6.22	Các điểm tích hợp AI trong Frontend	77
6.23	Cấu trúc thư mục mã nguồn Backend	79
6.24	Tài liệu API Backend trên Swagger/OpenAPI	80
6.25	Tổ chức Maven module của Backend	80
6.26	Phân lớp Clean Architecture trong một Backend module	81
6.27	Luồng chuẩn hóa response và exception	83
6.28	Luồng xác thực và phân quyền Backend	84
6.29	Luồng xác thực Backend	86
6.30	Luồng hiện thực quản lý ấn phẩm trong Catalog module	87
6.31	Nhóm API Catalog trên Swagger	88
6.32	Luồng xử lý yêu cầu mượn sách	89
6.33	Luồng trả sách và xử lý hàng chờ	89
6.34	Nhóm API Circulation trên Swagger	90
6.35	Luồng phát thông báo bất đồng bộ	91
6.36	Dữ liệu thông báo sau khi Kafka consumer xử lý	92
6.37	Các tích hợp hạ tầng và dịch vụ ngoài của Backend	92
6.38	Callback cập nhật trạng thái xử lý tài liệu từ AI service	93
6.39	Cấu trúc thư mục mã nguồn AI service	94
6.40	Dữ liệu vector của tài liệu trong PostgreSQL/pgvector	96
6.41	Trạng thái ETL của từng ấn phẩm trong database	96
6.42	Luồng ETL tài liệu PDF trong AI service	97
6.43	Summary, audience và tag AI sau khi ETL hoàn tất	100
6.44	Luồng xử lý semantic search	101
6.45	Kết quả API semantic search	101
6.46	Kết quả gợi ý sách trong bảng public.ai_recommendations	103
7.1	Kết quả chạy unit/context test Frontend	106
7.2	Kết quả chạy bộ test Backend	114
7.3	Kết quả chạy bộ test AI Service	119
7.4	Artifact chạy live Gemini trên PDF mẫu	122
8.1	Docker	124
8.2	Kiến trúc triển khai hệ thống	126
8.3	Luồng traffic qua các container trong môi trường sản xuất	128
8.4	Luồng kết nối từ tên miền đến hệ thống	129

Danh sách bảng

4.1	Bảng đặc tả yêu cầu hệ thống	9
5.1	Đối chiếu nhóm use case với hiện thực hệ thống	13
5.3	Bảng đặc tả Use Case UC-01: Tìm kiếm sách	14
5.4	Bảng đặc tả Use Case UC-02: Đăng ký tài khoản	15
5.5	Bảng đặc tả Use Case UC-08: Mượn sách	16
5.6	Bảng đặc tả Use Case UC-09: Trả sách	17
5.8	So sánh công nghệ Front-end theo tiêu chí của hệ thống	27
5.10	So sánh công nghệ Back-end theo tiêu chí của hệ thống	29
5.12	Ranh giới trách nhiệm khi tích hợp AI	31
5.14	Thiết kế Module 1 – Xử lý PDF file	32
5.16	Thiết kế Module 2 – Hỗ trợ tìm kiếm ngữ nghĩa	33
5.18	Ba nguồn tín hiệu trong Module 3 – Recommendation	33
5.20	Các điểm kiểm soát rủi ro khi dùng AI	35
5.21	Đánh giá kiến trúc lại so với các chỉ số	38
6.1	Cấu hình triển khai Frontend	58
6.3	Phân vùng giao diện theo vai trò	60
6.5	Các màn hình chính trong Admin Console	61
6.7	Context được hiện thực trong Frontend	62
6.9	Cơ chế cá nhân hóa giao diện	63
6.11	Service layer theo domain nghiệp vụ	64
6.13	Chức năng của nhóm trang công khai	66
6.15	Bộ lọc và sắp xếp trên trang tìm kiếm	68
6.17	State và API của nhóm trang công khai	69
6.19	Các thao tác trên hệ thống đánh giá sách	71
6.21	Các màn hình nghiệp vụ của độc giả	72
6.23	Các màn hình nghiệp vụ của thủ thư	74
6.25	Tóm tắt điểm tích hợp AI trên giao diện	77
6.27	Trách nhiệm của các Backend module	81
6.29	Lý do lựa chọn mô hình Backend	81
6.31	Quy ước hiện thực theo lớp	82
6.33	Nhóm mã lỗi nghiệp vụ	83
6.35	Nhóm API Backend cho Admin và quản trị vận hành	85
6.37	Các endpoint xác thực chính	86
6.39	Nhóm API của Catalog module	87
6.41	Các nhóm use case chính trong Circulation module	89
6.43	Scheduler được hiện thực trong Circulation module	91
6.45	Cấu hình hạ tầng Backend	92

6.47	Các module chính của AI service	94
6.49	Các quyết định kỹ thuật trong AI service	95
6.51	Các bảng dữ liệu AI quan trọng	95
6.53	Các tham số ETL chính	97
6.55	Kết quả AI ghi ngược vào dữ liệu ẩn phẩm	99
6.57	Chiến lược gợi ý sách	102
6.59	API chính của AI service	103
7.1	Cấu hình kiểm thử Frontend	104
7.3	Các file test Frontend đã hiện thực	105
7.5	Test case cho ánh xạ lỗi nghiệp vụ	106
7.7	Test case cho đa ngôn ngữ	107
7.9	Test case cho ThemeContext	108
7.11	Các nhánh kiểm thử reservation utility	109
7.13	Test case cho AuthContext	110
7.15	Test case cho upload và component dùng chung	110
7.17	Test case cho service layer	112
7.19	Kết quả kiểm thử Frontend	113
7.21	Cấu hình kiểm thử Backend	114
7.23	Tổng hợp test Backend theo module	115
7.25	Test case Backend cho Catalog	115
7.27	Test case Backend cho mượn-trả và đặt trước	116
7.29	Test case Backend cho Auth/User/Recommendation/AI	117
7.31	Kết quả kiểm thử Backend	117
7.33	Cấu hình kiểm thử AI Service	118
7.35	Các file test AI Service đã hiện thực	119
7.37	Test case chất lượng metadata AI	120
7.39	Test case ETL pipeline và SQL contract	121
7.41	Kết quả kiểm thử AI Service	122
8.1	Các nhóm biến môi trường của hệ thống	127
8.2	So sánh các nhà cung cấp VPS phổ biến	129
8.3	Chi phí vận hành hệ thống ước tính	130
10.1	Danh sách các tài nguyên trực tuyến của hệ thống	133

Chương 1: Giới thiệu

Chương này trình bày tổng quan về đề tài, bao gồm lý do lựa chọn, mục tiêu cần đạt được, phạm vi nghiên cứu và phát triển, ý nghĩa khoa học–thực tiễn của dự án, đồng thời giới thiệu cấu trúc tổng thể của báo cáo.

1.1 Động cơ của đề tài

Trong bối cảnh chuyển đổi số giáo dục đang diễn ra mạnh mẽ, việc hiện đại hóa các hệ thống thư viện tại trường đại học là một yêu cầu cấp thiết. Các hệ thống thư viện hiện tại, như Koha hay OPAC-HCMUT, dù đáp ứng được nhu cầu quản lý cơ bản nhưng vẫn còn nhiều hạn chế. Cụ thể, các hệ thống này chủ yếu dựa trên tìm kiếm từ khóa đơn giản, chưa khai thác tốt nội dung số của tài liệu PDF, chưa có cơ chế gợi ý dựa trên hành vi người dùng, và giao diện chưa thật sự tối ưu cho trải nghiệm sử dụng trên nhiều thiết bị. Thêm vào đó, các nghiệp vụ như đặt trước, nhắc hạn trả, thông báo thời gian thực, theo dõi phí phạt và thống kê xu hướng đọc thường còn rời rạc, gây tốn nhiều thời gian cho thủ thư.

Từ thực tiễn đó, có một nhu cầu rõ rệt từ các bên liên quan:

- **Sinh viên và giảng viên:** Cần một công cụ tìm kiếm tài liệu nhanh chóng, chính xác, hỗ trợ cả truy vấn tiếng Việt/tiếng Anh và gợi ý tài liệu liên quan để phục vụ học tập, nghiên cứu.
- **Thủ thư và nhà quản lý:** Mong muốn có một hệ thống tự động hóa các công việc lặp lại như quản lý ấn phẩm, bản sao, mượn–trả, đặt trước, phí phạt, thông báo và báo cáo thống kê.
- **Nhà trường:** Hướng tới việc thúc đẩy văn hóa đọc và năng lực số của sinh viên thông qua một nền tảng thư viện hiện đại, có khả năng mở rộng và tích hợp với các dịch vụ số khác.

Vì vậy, việc phát triển một **Hệ thống quản lý thư viện trực tuyến tích hợp AI hỗ trợ tìm kiếm và gợi ý sách** là cần thiết, không chỉ để giải quyết các hạn chế của hệ thống cũ mà còn để thử nghiệm cách kết hợp phần mềm quản lý thư viện truyền thống với AI service xử lý nội dung tài liệu.

1.2 Mục tiêu

Đề tài đặt ra mục tiêu tổng quát là xây dựng một hệ thống quản lý thư viện trực tuyến thông minh, vừa đảm bảo các chức năng quản lý cốt lõi, vừa tích hợp AI/NLP để nâng cao trải nghiệm tìm kiếm, khám phá và đọc tài liệu của người dùng.

Các mục tiêu cụ thể bao gồm:

- **Xây dựng hệ thống quản lý thư viện trực tuyến:** Cung cấp các chức năng chính như quản lý ấn phẩm, bản sao, tác giả, nhà xuất bản, danh mục, tag, người dùng, mượn–trả, đặt trước, phí phạt và thông báo.
- **Ứng dụng AI/NLP để nâng cao trải nghiệm:**
 - Phát triển công cụ tìm kiếm kết hợp lexical search và semantic search, hỗ trợ truy vấn tiếng Việt/tiếng Anh thông qua tag canonical và alias dịch.
 - Xây dựng hệ thống gợi ý sách dựa trên hành vi người dùng, kết hợp content-based recommendation, collaborative filtering và fallback theo xu hướng.

- Tích hợp pipeline xử lý PDF để sinh tóm tắt nội dung, audience và đúng 5 tag học thuật phục vụ tìm kiếm, gợi ý và hiển thị chi tiết sách.
- **Hỗ trợ công tác quản lý:** Tự động hóa thống kê, dashboard, nhắc hạn trả, xử lý quá hạn, hàng chờ đặt trước và thanh toán phí phạt, giúp thủ thư dễ dàng theo dõi vận hành thư viện.
- **Đảm bảo khả năng triển khai và mở rộng:** Xây dựng backend theo mô hình modular monolith, frontend SPA tách biệt và AI service độc lập để thuận tiện triển khai, kiểm thử và tách module trong tương lai.

1.3 Phạm vi

Đề tài sẽ tập trung vào các phạm vi nghiên cứu và phát triển như sau:

- **Về chức năng quản lý:** Hệ thống thực hiện các chức năng cốt lõi bao gồm quản lý ấn phẩm, bản sao, tác giả, nhà xuất bản, danh mục, tag, người dùng, vai trò, mượn-trả, đặt trước, hàng chờ, hạn trả, phí phạt, thanh toán, thông báo và dashboard thống kê.
- **Về ứng dụng AI:**
 - Xây dựng hệ thống tìm kiếm kết hợp từ khóa, tag/category alias và vector semantic search; ưu tiên lexical search với truy vấn ngắn để giữ tính chính xác và dùng semantic search để mở rộng khả năng khám phá.
 - Tích hợp hệ thống gợi ý cá nhân hóa dựa trên lịch sử xem, wishlist, mượn sách, đánh giá và dữ liệu vector nội dung.
 - Phát triển mô-đun AI ETL xử lý PDF, làm sạch văn bản, chia chunk, sinh embedding, tóm tắt nội dung, audience và tag; phần dịch metadata tiếng Anh đầy đủ được tắt mặc định để giảm tải LLM, chỉ giữ alias tag tiếng Anh phục vụ tìm kiếm song ngữ.
- **Về nền tảng công nghệ:** Hệ thống gồm frontend React/Vite, backend Spring Boot modular monolith, PostgreSQL/Flyway, Kafka/WebSocket, S3-compatible storage, PayOS và AI service Python/FastAPI/Celery dùng embedding model kết hợp Gemini ở bước reduce metadata.
- **Về phạm vi triển khai:** Sản phẩm của đề tài là hệ thống web hoàn chỉnh ở mức đồ án tốt nghiệp, được kiểm chứng bằng dữ liệu và kịch bản nghiệp vụ phù hợp với quy mô thư viện cấp khoa hoặc trường; một số tích hợp ngoài như PayOS, S3 và Gemini phụ thuộc cấu hình môi trường triển khai.

1.4 Ý nghĩa

1.4.1 Ý nghĩa thực tiễn

- **Nâng cao trải nghiệm người dùng:** Giúp sinh viên và giảng viên tìm kiếm tài liệu nhanh hơn, xem được tag/tóm tắt nội dung, nhận gợi ý tài liệu liên quan và theo dõi trạng thái mượn-trả/đặt trước trực tuyến.
- **Tối ưu hóa công tác quản lý:** Giảm tải công việc thủ công cho thủ thư thông qua dashboard, scheduler, notification, quản lý phí phạt, thanh toán và các công cụ tra cứu nghiệp vụ.
- **Thúc đẩy chuyển đổi số:** Góp phần xây dựng mô hình thư viện số hiện đại, phù hợp với chủ trương chuyển đổi số trong giáo dục và phát triển năng lực số cho sinh viên.

1.4.2 Ý nghĩa khoa học

- **Ứng dụng AI/NLP vào bài toán thực tế:** Đề tài áp dụng xử lý văn bản PDF, embedding, semantic search và hệ thống gợi ý vào bài toán tìm kiếm tài liệu trong thư viện đại học.
- **Kết hợp mô hình và hệ thống:** Dự án là cầu nối giữa kỹ thuật AI và hệ thống phần mềm hoàn chỉnh: AI service không đứng riêng lẻ mà được nối với catalog, search, recommendation, notification và giao diện người dùng.
- **Đề xuất giải pháp khả thi:** Hệ thống áp dụng hướng hybrid search và hybrid recommendation ở mức phù hợp với dữ liệu và hạ tầng của đề án, đồng thời có cơ chế fallback để tránh phụ thuộc tuyệt đối vào LLM.

1.5 Cấu trúc báo cáo

Ngoài các phần đầu mục như Tóm tắt, Mục lục, Danh mục hình ảnh và bảng biểu, nội dung chính của báo cáo được cấu trúc thành 10 chương như sau:

- **Chương 1 - Giới thiệu:** Trình bày tổng quan về động cơ, mục tiêu, phạm vi và ý nghĩa của đề tài, đồng thời giới thiệu cấu trúc tổng thể của báo cáo.
- **Chương 2 - Kiến thức và công nghệ nền tảng:** Trình bày các cơ sở lý thuyết, kiến thức chuyên ngành và các công nghệ cốt lõi được sử dụng để phát triển dự án.
- **Chương 3 - Công trình liên quan:** Phân tích, so sánh các nghiên cứu và hệ thống tương tự để xác định khoảng trống nghiên cứu mà đề tài giải quyết.
- **Chương 4 - Hệ thống đề xuất:** Mô tả tổng quan hệ thống được đề xuất, bao gồm đối tượng người dùng, yêu cầu chức năng, phi chức năng, yêu cầu dữ liệu và các luồng nghiệp vụ chính.
- **Chương 5 - Phân tích và Thiết kế:** Trình bày chi tiết quá trình phân tích và thiết kế hệ thống, bao gồm kiến trúc phần mềm, thiết kế cơ sở dữ liệu, thiết kế AI Service và thiết kế giao diện.
- **Chương 6 - Hiện thực hệ thống:** Mô tả quá trình hiện thực frontend, backend và AI service, bao gồm giao diện cho Độc giả/Thủ thư, API nghiệp vụ, pipeline xử lý dữ liệu AI và các tích hợp hạ tầng.
- **Chương 7 - Kiểm thử:** Trình bày chiến lược kiểm thử phân tầng cho frontend, backend và AI service, bao gồm unit/context test, controller/integration test, SQL contract test, AI metadata quality test và live Gemini artifact.
- **Chương 8 - Triển khai hệ thống:** Mô tả quá trình đóng gói Docker, cấu hình NGINX, CI/CD và triển khai lên môi trường VPS thực tế.
- **Chương 9 - Tổng kết:** Tóm tắt kết quả đạt được, nhận xét ưu điểm và hạn chế còn tồn tại của hệ thống.
- **Chương 10 - Tài nguyên và Mã nguồn:** Cung cấp các đường dẫn đến mã nguồn (GitHub) và bản thiết kế giao diện (Figma) của dự án.

Chương 2: Kiến thức và công nghệ nền tảng

Chương này trình bày tổng quan về các khái niệm, kiến thức và các lĩnh vực công nghệ nền tảng được vận dụng để xây dựng "Hệ thống quản lý thư viện trực tuyến tích hợp AI hỗ trợ tìm kiếm và gợi ý sách".

2.1 Các Lĩnh vực Công nghệ Chính

Dự án được định hướng phát triển dựa trên kiến trúc phần mềm hiện đại, trong đó các thành phần chính được phân tách rõ ràng để thuận lợi cho việc bảo trì và mở rộng sau này.

- **Công nghệ phía Server (Back-End):** Đây là thành phần trung tâm, chịu trách nhiệm xử lý toàn bộ logic nghiệp vụ của hệ thống, cung cấp các giao diện lập trình ứng dụng (API), quản lý dữ liệu mượn-trả, và thực hiện các tác vụ thống kê.
- **Công nghệ phía Client (Front-End):** Thành phần này đảm nhiệm việc xây dựng giao diện người dùng, giúp người dùng cuối (thủ thư, bạn đọc) tương tác với hệ thống một cách trực quan và thân thiện trên nền tảng web.
- **Hệ quản trị Cơ sở dữ liệu:** Hệ thống sử dụng một hệ quản trị cơ sở dữ liệu quan hệ để tổ chức, lưu trữ và truy xuất toàn bộ dữ liệu của thư viện, bao gồm thông tin sách, người dùng, và lịch sử giao dịch.
- **Trí tuệ Nhân tạo (AI) và Học máy (Machine Learning):**
 - **Hệ thống gợi ý (Recommendation System):** Trọng tâm của dự án là áp dụng mô hình **Hybrid**, một phương pháp kết hợp ưu điểm của hai kỹ thuật chính:
 1. **Lọc dựa trên nội dung (Content-Based Filtering):** Gợi ý tài liệu dựa trên sự tương đồng về thuộc tính và nội dung của chúng.
 2. **Lọc cộng tác (Collaborative Filtering):** Đề xuất tài liệu dựa trên hành vi và sở thích của các nhóm người dùng tương tự nhau.
 - **Xử lý Ngôn ngữ Tự nhiên (NLP):** Các kỹ thuật NLP được ứng dụng để nâng cao trải nghiệm người dùng, bao gồm: phát triển công cụ **tìm kiếm ngữ nghĩa** cho tiếng Việt và tích hợp khả năng **tự động tóm tắt nội dung sách**.

2.2 Nền tảng Kiến thức Khoa học

Để triển khai các lĩnh vực công nghệ trên, nhóm nghiên cứu cần vận dụng các nền tảng kiến thức sau:

- **Kiến trúc và Phát triển Hệ thống:**
 - Hiểu biết về kiến trúc **Client-Server** và nguyên lý thiết kế **RESTful API** để đảm bảo sự giao tiếp hiệu quả giữa Front-End và Back-End.
 - Kiến thức về thiết kế và mô hình hóa cơ sở dữ liệu quan hệ (ví dụ: ERD) để xây dựng một cấu trúc lưu trữ tối ưu.
- **Khoa học Dữ liệu và Trí tuệ Nhân tạo:**

- Nguyên lý các thuật toán gợi ý: Nắm vững bản chất, ưu và nhược điểm của các phương pháp Content-Based, Collaborative Filtering, và cách chúng bổ trợ cho nhau trong một mô hình Hybrid.
- Các kỹ thuật xử lý ngôn ngữ tự nhiên: Có kiến thức về các phương pháp tiền xử lý văn bản, mô hình hóa ngôn ngữ (word embedding), và các thuật toán đo lường độ tương đồng ngữ nghĩa (ví dụ: cosine similarity).

2.3 Kết chương

Nội dung chương này đã hệ thống hóa các lĩnh vực công nghệ và nền tảng kiến thức khoa học cần thiết cho dự án. Các thành phần công nghệ chính bao gồm **Back-End**, **Front-End**, **Cơ sở dữ liệu quan hệ** và các module **Trí tuệ nhân tạo**. Trọng tâm của dự án nằm ở việc áp dụng các kiến thức về **Hệ thống gợi ý Hybrid** và **Xử lý Ngôn ngữ Tự nhiên** để tạo ra một sản phẩm thông minh và khác biệt.

Với các nền tảng kiến thức đã trình bày, nhóm xác nhận đã bao quát đầy đủ các yêu cầu lý thuyết cần thiết để triển khai dự án, từ việc xây dựng một ứng dụng web hoàn chỉnh đến tích hợp các tính năng AI nâng cao theo đúng mục tiêu đề ra.

Chương 3: Công trình liên quan

Chương này tập trung phân tích các công trình nghiên cứu khoa học và các hệ thống phần mềm đã có liên quan đến lĩnh vực thư viện số và hệ thống gợi ý. Từ đó, chương sẽ chỉ ra những khoảng trống nghiên cứu và định vị hướng đi của đề tài.

3.1 Các công trình nghiên cứu khoa học

Qua việc tổng hợp các công trình nghiên cứu, có thể thấy các hướng tiếp cận phổ biến trong việc xây dựng hệ thống gợi ý sách cho thư viện có những đặc điểm sau:

- **Ưu điểm và xu hướng chung:**

- Hầu hết các mô hình gợi ý đều theo hướng **hybrid**, kết hợp giữa Lọc dựa trên nội dung (Content-Based) và Lọc cộng tác (Collaborative Filtering) để cân bằng giữa độ cá nhân hóa và khả năng khai thác hành vi người dùng.
- Một số nghiên cứu cải tiến kết quả bằng cách thêm các cơ chế xếp hạng (ranking) hoặc gợi ý theo cụm người dùng (clustering), đặc biệt hiệu quả trong môi trường dữ liệu vừa và nhỏ.
- Việc đánh giá mô hình thường dựa trên dữ liệu thực tế từ các thư viện, giúp kiểm chứng rõ ràng hiệu quả của thuật toán.

- **Hạn chế phổ biến:**

- Nhiều công trình tập trung sâu vào phần mô hình thuật toán nhưng ít chú trọng đến việc triển khai thành một hệ thống hoàn chỉnh với đầy đủ backend và frontend.
- Các hướng đi hiện đại như tìm kiếm ngữ nghĩa (semantic search) hay các mô hình Transformer tuy tiềm năng nhưng đòi hỏi chi phí tính toán và nguồn lực lớn, vượt quá khả năng của nhiều dự án quy mô nhỏ.

3.2 Các hệ thống và phần mềm tương tự

Hiện nay, có nhiều hệ thống quản lý thư viện điện tử đã được triển khai rộng rãi, tiêu biểu có thể kể đến:

- **Opac-HCMUT-VNU:** Là hệ thống của Đại học Quốc Gia TP.HCM, đáp ứng tốt các nghiệp vụ tra cứu và mượn-trả cơ bản. Tuy nhiên, chức năng tìm kiếm vẫn chủ yếu dựa vào từ khóa và lọc và chưa tích hợp tính năng gợi ý thông minh, tìm kiếm theo ngữ nghĩa.
- **Koha:** Là một phần mềm quản lý thư viện mã nguồn mở rất phổ biến, cung cấp đầy đủ chức năng quản lý và báo cáo. Mặc dù vậy, giao diện của Koha đã cũ, chưa hỗ trợ gợi ý tự động và chưa có cơ chế phân tích hành vi người dùng.
- **Aleph (Ex Libris):** Đây là một hệ thống thương mại quy mô lớn, tích hợp nhiều cơ sở dữ liệu học thuật. Nhược điểm chính của hệ thống này là chi phí triển khai cao và không phù hợp cho các thư viện quy mô nhỏ như cấp khoa hoặc trường đại học nhỏ.

3.3 Kết chương và xác định khoảng trống nghiên cứu

Từ việc phân tích các công trình khoa học và hệ thống thực tế, nhóm nhận thấy một số **khoảng trống (gaps)** quan trọng mà đề tài này hướng đến giải quyết:

1. **Khoảng trống giữa Mô hình và Thực thi:** Nhiều nghiên cứu tập trung vào việc xây dựng các mô hình gợi ý phức tạp nhưng lại bỏ qua việc triển khai chúng thành một hệ thống phần mềm hoàn chỉnh, có tính ứng dụng thực tế. Đa số các mô hình hiện nay tập trung nhiều vào thuật toán gợi ý, trong khi ít chú trọng đến việc triển khai thành hệ thống hoàn chỉnh.
2. **Khoảng trống về Trải nghiệm Người dùng:** Các hệ thống hiện tại, kể cả mã nguồn mở như Koha, thường có giao diện cũ, chưa tối ưu cho các thiết bị hiện đại và thiếu các tính năng thông minh giúp nâng cao trải nghiệm tìm kiếm, khám phá tài liệu của người dùng.
3. **Khoảng trống về Tính khả thi:** Các giải pháp gợi ý tiên tiến thường yêu cầu nguồn dữ liệu lớn hoặc xử lý phức tạp, vượt quá phạm vi và nguồn lực của một nhóm sinh viên.

Hướng đi của dự án: Đề tài này không đặt nặng việc phát triển một mô hình AI quá chuyên sâu. Thay vào đó, dự án tập trung vào việc **bắc cầu cho các khoảng trống trên** bằng cách:

- Xây dựng một **hệ thống hoàn chỉnh** với kiến trúc frontend-backend tách biệt, dễ bảo trì và mở rộng.
- Ưu tiên **trải nghiệm người dùng** với giao diện hiện đại và tích hợp các tính năng gợi ý, tìm kiếm thông minh một cách liền mạch.
- Áp dụng một **giải pháp AI khả thi** (mô hình hybrid cơ bản) để tạo ra một sản phẩm hoạt động ổn định, có giá trị thực tiễn và phù hợp với bối cảnh của một đồ án tốt nghiệp.

Chương 4: Hệ thống đề xuất

4.1 Mô tả hệ thống

Hệ thống được đề xuất là một **hệ thống quản lý thư viện trực tuyến tích hợp AI hỗ trợ tìm kiếm, gợi ý và khai thác nội dung tài liệu PDF**. Mục tiêu của dự án không chỉ là tạo một trang tra cứu sách, mà là xây dựng một nền tảng có đủ nghiệp vụ thư viện: quản lý ấn phẩm, bản sao, người dùng, mượn-trả, đặt trước, phí phạt, thông báo và dashboard vận hành. Trên nền nghiệp vụ đó, AI được dùng ở những điểm thật sự có giá trị như làm giàu metadata từ PDF, tìm kiếm ngữ nghĩa và gợi ý tài liệu theo hành vi người dùng.

Hệ thống được thiết kế để phục vụ hai nhóm người dùng chính, mỗi nhóm có vai trò và nhu cầu riêng biệt:

- **Bạn đọc (Sinh viên, Giảng viên, Nghiên cứu viên):** Là nhóm người dùng trực tiếp khai thác tài nguyên thư viện. Họ cần tìm kiếm tài liệu nhanh, xem tình trạng bản sao, đọc tóm tắt AI, thêm sách vào wishlist, đánh giá sách, gửi yêu cầu mượn, theo dõi sách đang mượn, đặt trước sách, xem phí phạt và nhận thông báo kịp thời.
- **Thủ thư (Người quản lý thư viện):** Là nhóm người dùng vận hành hệ thống. Thủ thư quản lý ấn phẩm, tác giả, nhà xuất bản, danh mục, tag, bản sao vật lý, upload bìa và PDF, xử lý mượn-trả tại quầy, xác nhận nhận sách, quản lý đặt trước, ghi nhận sách hư/mất, xử lý phí phạt bằng tiền mặt hoặc PayOS và theo dõi dashboard thống kê.

4.2 Hướng tiếp cận thiết kế

Hệ thống Quản lý Thư viện được thiết kế theo phương pháp “từ dưới lên” (bottom-up design). Hướng tiếp cận này phù hợp với hệ thống hiện tại vì mỗi phần nghiệp vụ có thể được xây dựng và kiểm thử tương đối độc lập trước khi ghép thành luồng hoàn chỉnh trên giao diện. Backend được tổ chức theo modular monolith, frontend là React SPA, còn AI service là một service riêng xử lý PDF, semantic search và recommendation.

Quá trình thiết kế được thực hiện theo các bước sau:

1. Xác định các module cốt lõi của hệ thống như auth, user, catalog, circulation, recommendation và AI service.
2. Xây dựng và kiểm thử từng module theo trách nhiệm riêng: xác thực, quản lý ấn phẩm, quản lý bản sao, giao dịch mượn-trả, thông báo, wishlist, rating, tìm kiếm và gợi ý.
3. Tích hợp các module thành các luồng nghiệp vụ đầy đủ, ví dụ: tạo ấn phẩm, upload PDF, AI xử lý metadata, người dùng tìm kiếm, gửi yêu cầu mượn, thủ thư xác nhận và hệ thống gửi thông báo.
4. Lặp lại quy trình này để hoàn thiện frontend, backend, AI service và các tích hợp ngoài như S3-compatible storage, Kafka/WebSocket và PayOS.

4.3 Yêu cầu

4.3.1 Yêu cầu chức năng

Các yêu cầu chức năng của hệ thống được đặc tả chi tiết trong bảng dưới đây, phân loại theo từng nhóm chức năng và người dùng tương tác.

Bảng 4.1: Bảng đặc tả yêu cầu hệ thống

ID	Tên yêu cầu	Loại	Mô tả chi tiết
R1	Quản lý thực thể dữ liệu	Dữ liệu	Hệ thống phải lưu trữ và quản lý các nhóm dữ liệu cốt lõi: ấn phẩm , bản sao , người dùng , giao dịch mượn-trả , đặt trước , phí phạt , đánh giá , wishlist , thông báo và dữ liệu phục vụ AI.
R2	Phân biệt Ấn phẩm và Bản sao	Dữ liệu	Hệ thống phải phân biệt giữa ấn phẩm và bản sao vật lý . Mỗi bản sao có barcode riêng, trạng thái riêng, tình trạng sách và vị trí kệ để thủ thư có thể quản lý lưu thông chính xác.
R3	Thuộc tính Ấn phẩm	Dữ liệu	Mỗi ấn phẩm phải có metadata cơ bản như ISBN, tiêu đề, tác giả, nhà xuất bản, danh mục, tag, ngôn ngữ, năm xuất bản, mô tả, ảnh bìa và file PDF nếu có. Hệ thống cũng lưu summary, nhóm đọc giả mục tiêu và tag do AI sinh ra sau khi xử lý tài liệu.
R4	Quản lý vai trò người dùng	Chức năng	Hệ thống phải hỗ trợ xác thực và phân quyền theo vai trò, tối thiểu gồm Bạn đọc và Thủ thư . Bạn đọc được đăng ký, đăng nhập, quản lý hồ sơ cá nhân; thủ thư được truy cập các màn hình quản lý nghiệp vụ.
R5	Nghiệp vụ mượn-trả và phí phạt	Nghiệp vụ	Hệ thống phải hỗ trợ bạn đọc gửi yêu cầu mượn, thủ thư cho mượn trực tiếp hoặc xác nhận nhận sách, trả sách bằng barcode, ghi nhận sách hư/mất, tự tính phí quá hạn hoặc phí sự cố và xử lý thanh toán bằng tiền mặt hoặc PayOS.
R6	Tìm kiếm hybrid	Chức năng	Hệ thống phải cung cấp tìm kiếm catalog theo keyword, tác giả, danh mục, tag, ngôn ngữ, năm xuất bản, chi nhánh và tình trạng bản sao; đồng thời hỗ trợ tìm kiếm ngữ nghĩa trên nội dung PDF thông qua AI service và pgvector.
R7	Chức năng Đặt trước	Chức năng	Bạn đọc phải có khả năng đặt trước sách khi cần. Hệ thống lưu hàng chờ đặt trước, cho phép bạn đọc hủy đặt trước, thủ thư tra cứu yêu cầu và xác nhận nhận sách khi bản sao sẵn sàng.
R8	Gợi ý, wishlist và đánh giá	Chức năng	Hệ thống phải hỗ trợ gợi ý sách cá nhân hóa dựa trên hành vi người dùng, có fallback khi dữ liệu còn ít; đồng thời cho phép bạn đọc thêm sách vào wishlist, đánh giá, bình luận và tương tác với review.

Bảng 4.1 – tiếp theo

ID	Tên yêu cầu	Loại	Mô tả chi tiết
R9	Hệ thống thông báo tự động	Chức năng	Hệ thống phải gửi thông báo cho người dùng về các sự kiện quan trọng như yêu cầu mượn, xác nhận nhận sách, sách sắp đến hạn, sách quá hạn, phí phạt, đặt trước và phản hồi review. Thông báo được lưu trong hệ thống và đẩy realtime qua WebSocket.

4.3.2 Yêu cầu phi chức năng

Hệ thống cần đảm bảo các yêu cầu phi chức năng trọng yếu sau:

- **Tính khả dụng (Usability):** Giao diện người dùng phải nhất quán, trực quan và dễ học cho cả bạn đọc và thủ thư. Frontend cần hỗ trợ các luồng chính như tìm kiếm, xem chi tiết sách, đặt trước, theo dõi sách đang mượn, phí phạt, thông báo, quản lý ấn phẩm và xử lý mượn-trả trên các kích thước màn hình phổ biến.
- **Hiệu năng (Performance):** Các chức năng tương tác phải có thời gian phản hồi nhanh trong điều kiện tải thông thường. Các truy vấn catalog cần được phân trang; semantic search và recommendation cần giới hạn số lượng kết quả, dùng connection pool và timeout để tránh ảnh hưởng đến backend chính.
- **Độ tin cậy (Reliability):** Hệ thống phải hoạt động ổn định ngay cả khi một dịch vụ phụ trợ gặp lỗi. Khi AI service hoặc Gemini lỗi, hệ thống cần ghi nhận trạng thái xử lý, có fallback cho metadata/gợi ý và không làm nghẽn các nghiệp vụ chính như tra cứu, mượn-trả hoặc thanh toán.
- **Bảo mật (Security):** Hệ thống cần có cơ chế xác thực và phân quyền rõ ràng bằng JWT và kiểm tra vai trò ở backend. Các API quản trị chỉ dành cho thủ thư; callback từ AI service và webhook thanh toán cần được kiểm tra chữ ký hoặc cơ chế xác thực tương ứng; upload file phải đi qua đường dẫn có kiểm soát như presigned URL.
- **Khả năng bảo trì và mở rộng (Maintainability & Scalability):** Mã nguồn phải được tổ chức rõ ràng theo frontend, backend và AI service. Backend theo modular monolith giúp tách ranh giới nghiệp vụ; AI service tách riêng giúp mở rộng pipeline PDF, semantic search và recommendation mà không làm phức tạp phần nghiệp vụ thư viện.

4.3.3 Yêu cầu dữ liệu

Hệ thống yêu cầu và quản lý các nhóm dữ liệu chính sau:

- **Dữ liệu lõi (Core Data):** Bao gồm thông tin ấn phẩm, bản sao, barcode, tác giả, nhà xuất bản, danh mục, tag, file bìa, file PDF và thông tin tài khoản người dùng.
- **Dữ liệu nghiệp vụ (Transactional Data):** Gồm các bản ghi yêu cầu mượn, xác nhận nhận sách, trả sách, đặt trước, phí phạt, thanh toán tiền mặt, thanh toán PayOS, trạng thái bản sao và thông báo.
- **Dữ liệu hành vi và AI (Behavioral/AI Data):** Gồm lịch sử tìm kiếm, lượt xem chi tiết sách, wishlist, rating, lịch sử mượn, vector embedding của chunk PDF, summary AI, audience, tag AI và trạng thái ETL. Đây là nguồn dữ liệu phục vụ semantic search và recommendation.

4.4 Kết chương

Chương này đã trình bày tổng quan hệ thống quản lý thư viện trực tuyến tích hợp AI. Hệ thống được định hướng như một sản phẩm hoàn chỉnh, trong đó nghiệp vụ thư viện vẫn là phần lõi, còn AI được dùng để làm giàu dữ liệu, hỗ trợ tìm kiếm ngữ nghĩa và gợi ý tài liệu.

Phần trọng tâm của chương đã đặc tả các yêu cầu cốt lõi của hệ thống, từ quản lý ấn phẩm, bản sao, người dùng, mượn-trả, đặt trước, phí phạt và thông báo cho đến các chức năng thông minh như semantic search, recommendation, wishlist và rating. Đồng thời, các **yêu cầu phi chức năng** như tính khả dụng, hiệu năng, độ tin cậy, bảo mật và khả năng bảo trì cũng được xác định theo đúng kiến trúc frontend-backend-AI service hiện tại.

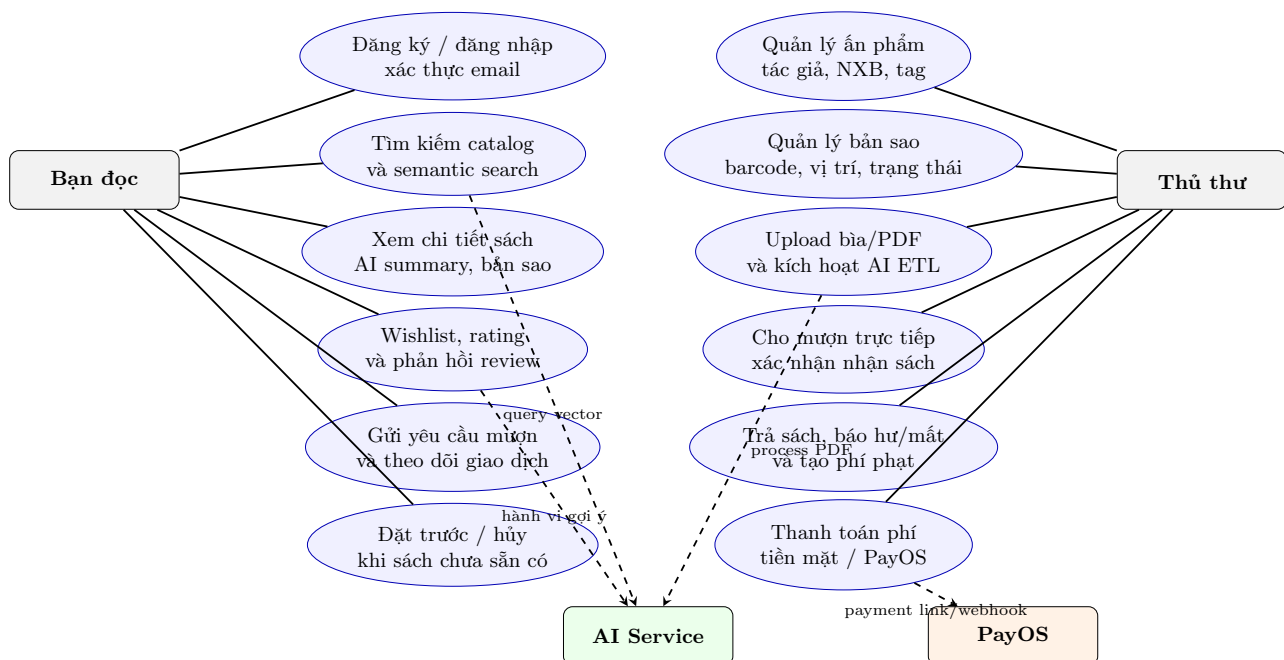
Những yêu cầu này tạo thành một nền tảng vững chắc, là cơ sở để tiến hành quá trình phân tích và thiết kế chi tiết hệ thống sẽ được trình bày ở Chương 5.

Chương 5: Phân tích và Thiết kế

5.1 Phân tích

5.1.1 Mô hình hóa tương tác (Biểu đồ Usecase)

Phần phân tích use case được xây dựng lại theo phạm vi hệ thống đã hiện thực. Hệ thống hiện có hai nhóm tác nhân chính là **Bạn đọc** và **Thủ thư**; ngoài ra còn có các hệ thống phụ trợ như **AI Service** và **PayOS** tham gia vào những luồng chuyên biệt. Các use case trong sơ đồ dưới đây tập trung vào những nghiệp vụ đã có trong frontend, backend và AI service, thay vì mô tả các chức năng dự kiến nhưng chưa triển khai.



Hình 5.1: Biểu đồ Usecase tổng quát theo phạm vi hệ thống đã hiện thực

Các use case được chia thành ba nhóm. Nhóm **khám phá tài liệu** gồm tìm kiếm catalog, semantic search, xem chi tiết, wishlist, rating và recommendation. Nhóm **lưu thông thư viện** gồm gửi yêu cầu mượn, xác nhận nhận sách, mượn trực tiếp tại quầy, đặt trước, trả sách, báo hư/mất và xử lý phí phạt. Nhóm **quản trị dữ liệu** gồm quản lý ấn phẩm, bản sao, metadata, file bìa, file PDF và kích hoạt pipeline AI ETL.

Bảng 5.1: Đối chiếu nhóm use case với hiện thực hệ thống

Nhóm	Use case chính	Hiện thực tương ứng
Khám phá tài liệu	Tìm kiếm, semantic search, xem chi tiết, wishlist, rating, gợi ý	React public pages gọi /api/v1/publications/search, /api/v1/ai/semantic-search, /api/v1/wishlist, /api/v1/publications/{id}/ratings và /api/v1/recommendations.
Lưu thông thư viện	Mượn, xác nhận nhận sách, đặt trước, trả sách, báo sự cố, phí phạt	Circulation module cung cấp /api/v1/transactions/borrow, /borrow-direct, /{id}/confirm-pickup, /return, /{id}/report-issue, /api/v1/reservations và /api/v1/fines.
Quản trị dữ liệu	Quản lý ấn phẩm, bản sao, upload bìa/PDF, dashboard	Catalog và librarian pages dùng /api/v1/publications, /items, presigned URL cho PDF, AI callback và dashboard circulation.

5.1.1.1 Đặc tả chi tiết các Use Case chính

Bảng 5.3: Bảng đặc tả Use Case UC-01: Tìm kiếm sách

Thuộc tính	Mô tả chi tiết
Tên Use Case	Tìm kiếm sách
Mã số	UC-01
Tác nhân	Bạn đọc, Thủ thư
Mô tả ngắn gọn	Cho phép người dùng tìm kiếm ấn phẩm bằng từ khóa, tác giả, danh mục, tag, ngôn ngữ, năm xuất bản, tình trạng bản sao hoặc truy vấn ngôn ngữ tự nhiên thông qua semantic search.
Tiền điều kiện	Người dùng đã truy cập vào giao diện tìm kiếm của hệ thống.
Hậu điều kiện	Thành công: Hệ thống hiển thị danh sách ấn phẩm phù hợp, kèm thông tin bìa, tác giả, đánh giá và tình trạng bản sao. Thất bại: Hệ thống hiển thị thông báo "Không tìm thấy kết quả phù hợp".
Luồng chính	- Người dùng nhập từ khóa hoặc câu hỏi tự nhiên trên giao diện tìm kiếm. - Người dùng có thể chọn bộ lọc như tác giả, danh mục, tag, ngôn ngữ, năm xuất bản, chi nhánh hoặc tình trạng còn bản sao. - Nếu là tìm kiếm catalog, frontend gọi <code>/api/v1/publications/search</code>; backend lọc dữ liệu bằng PostgreSQL và trả kết quả phân trang. - Nếu là semantic search, frontend gọi <code>/api/v1/ai/semantic-search</code>; backend chuyển truy vấn sang AI service để tìm các vector chunk PDF gần nhất, sau đó lấy lại metadata catalog tương ứng. - Hệ thống hiển thị danh sách ấn phẩm kèm bìa, tác giả, đánh giá, trạng thái AI và tình trạng bản sao. Use case kết thúc.
Luồng thay thế	4a. Không có kết quả phù hợp: Hệ thống hiển thị thông báo "Không tìm thấy kết quả phù hợp". Use case kết thúc.

Bảng 5.4: Bảng đặc tả Use Case UC-02: Đăng ký tài khoản

Thuộc tính	Mô tả chi tiết
Tên Use Case	Đăng ký tài khoản
Mã số	UC-02
Tác nhân	Bạn đọc (người dùng mới)
Mô tả ngắn gọn	Cho phép người dùng mới tạo một tài khoản thành viên để sử dụng các dịch vụ của thư viện.
Tiền điều kiện	Người dùng đang ở trang đăng ký của hệ thống.
Hậu điều kiện	Thành công: Một tài khoản mới được tạo ở trạng thái chờ xác thực email. Sau khi người dùng xác thực, tài khoản được kích hoạt để đăng nhập và sử dụng hệ thống. Thất bại: Tài khoản không được tạo, hệ thống hiển thị thông báo lỗi.
Luồng chính	- Người dùng nhập thông tin cá nhân vào biểu mẫu đăng ký (Họ tên, email, mật khẩu,...). - Người dùng nhấn nút "Đăng ký". - Hệ thống kiểm tra tính hợp lệ của thông tin (định dạng email, mật khẩu đủ mạnh). - Hệ thống kiểm tra xem email đã tồn tại trong cơ sở dữ liệu chưa. - Hệ thống tạo tài khoản mới, mã hóa mật khẩu và lưu thông tin vào cơ sở dữ liệu. - Hệ thống gửi email xác thực; khi người dùng mở liên kết xác thực, backend kích hoạt tài khoản và chuyển về trang kết quả xác thực trên frontend. Use case kết thúc.
Luồng thay thế	3a. Thông tin không hợp lệ: - Hệ thống hiển thị thông báo lỗi tương ứng với từng trường thông tin. Người dùng được yêu cầu nhập lại. 4a. Email đã tồn tại: - Hệ thống hiển thị thông báo "Email này đã được sử dụng". Người dùng được yêu cầu sử dụng email khác.

Bảng 5.5: Bảng đặc tả Use Case UC-08: Mượn sách

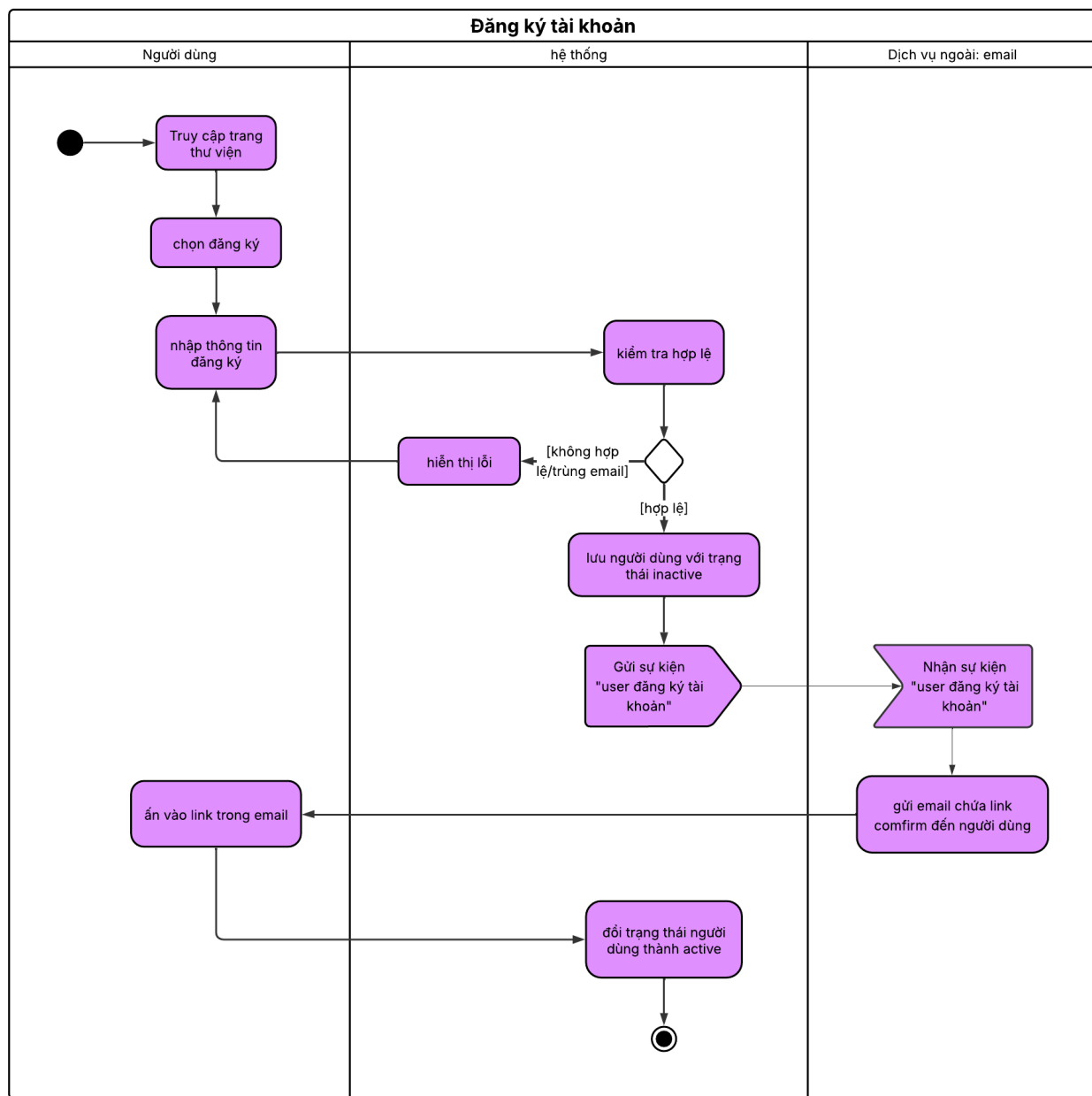
Thuộc tính	Mô tả chi tiết
Tên Use Case	Mượn sách
Mã số	UC-08
Tác nhân	Bạn đọc, Thủ thư
Mô tả ngắn gọn	Bạn đọc có thể gửi yêu cầu mượn một bản sao cụ thể từ giao diện chi tiết sách, hoặc thủ thư có thể tạo giao dịch mượn trực tiếp tại quầy bằng mã sinh viên và barcode bản sao.
Tiền điều kiện	1. Bạn đọc hoặc thủ thư đã đăng nhập vào hệ thống. 2. Bạn đọc có tài khoản đang ở trạng thái hoạt động. 3. Bản sao được chọn đang ở trạng thái AVAILABLE; riêng luồng thủ thư có thể xử lý bản sao RESERVED nếu bản sao đó đang được giữ đúng cho bạn đọc trong hàng đặt trước.
Hậu điều kiện	Thành công: Một yêu cầu mượn ở trạng thái chờ nhận sách hoặc một giao dịch mượn trực tiếp được tạo. Bản sao được chuyển sang RESERVED khi bạn đọc gửi yêu cầu, hoặc BORROWED khi thủ thư giao sách. Thất bại: Trạng thái hệ thống không thay đổi.
Luồng chính	1. Bạn đọc chọn "Mượn sách" trên trang chi tiết hoặc thủ thư mở màn hình mượn sách tại quầy. 2. Hệ thống kiểm tra trạng thái tài khoản, tình trạng phí phạt và số lượng sách đang mượn. 3. Hệ thống khóa bản sao để tránh hai người cùng mượn một barcode, sau đó kiểm tra trạng thái bản sao và các ràng buộc nghiệp vụ. 4. Nếu là yêu cầu từ bạn đọc, hệ thống tạo giao dịch WAITING_FOR_PICKUP, đặt hạn nhận trong 24 giờ và chuyển bản sao sang RESERVED. 5. Nếu là mượn trực tiếp tại quầy, thủ thư nhập/quét barcode, hệ thống tạo giao dịch BORROWING, ghi ngày đến hạn 14 ngày và chuyển bản sao sang BORROWED. 6. Hệ thống gửi thông báo/email; riêng luồng mượn trực tiếp ghi nhận tương tác BORROWED cho recommendation. Use case kết thúc.
Luồng thay thế	4a. Bạn đọc không tồn tại hoặc tài khoản bị khóa: 1. Hệ thống hiển thị thông báo lỗi. Use case kết thúc. 4b. Bản sao sách không tồn tại hoặc không ở trạng thái hợp lệ: 1. Hệ thống hiển thị thông báo lỗi. Use case kết thúc. 4c. Bạn đọc đã mượn đủ số lượng sách tối đa, đã có giao dịch cho cùng ấn phẩm hoặc còn phí phạt chưa xử lý: 1. Hệ thống hiển thị thông báo lỗi. Use case kết thúc.

Bảng 5.6: Bảng đặc tả Use Case UC-09: Trả sách

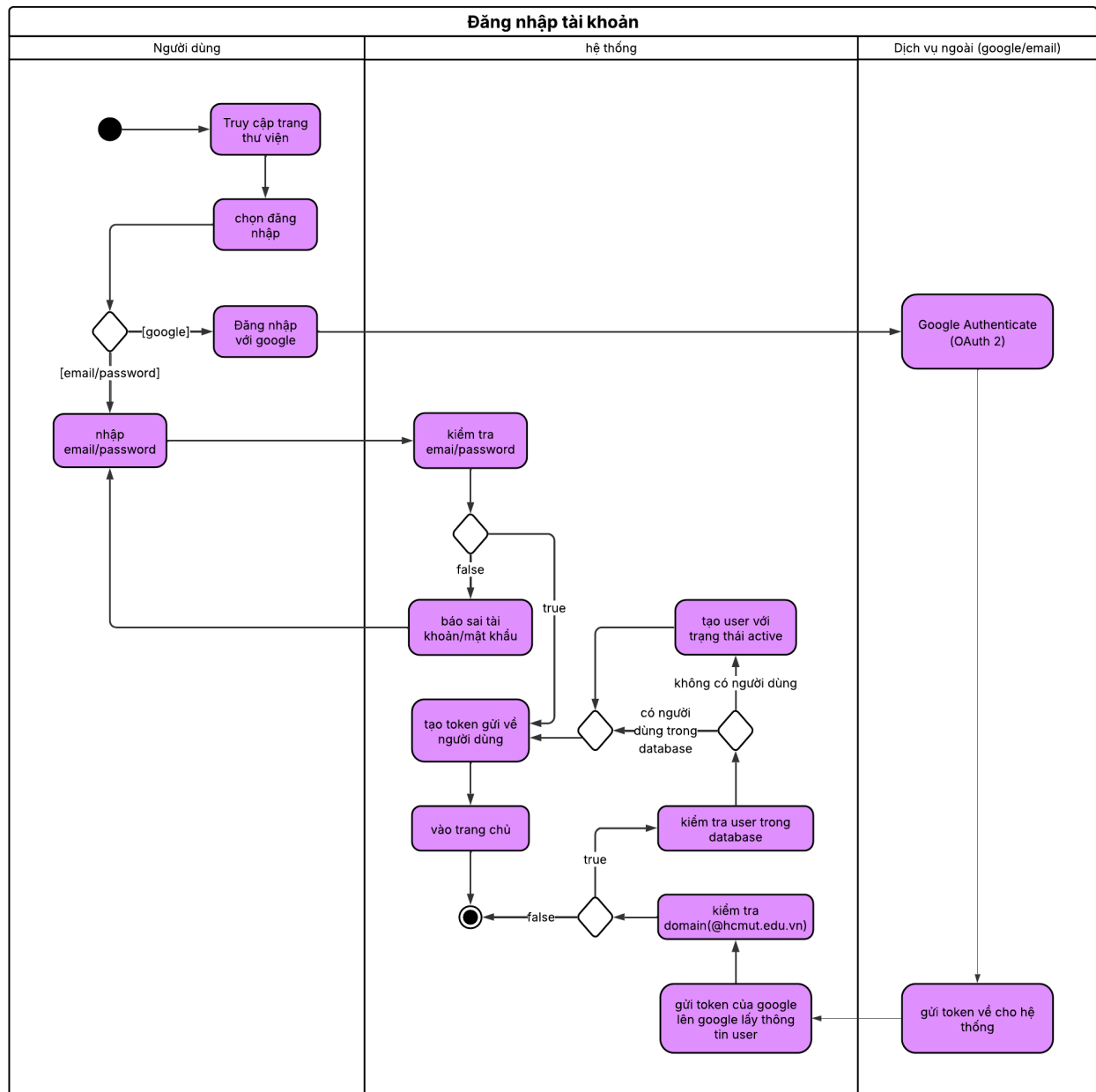
Thuộc tính	Mô tả chi tiết
Tên Use Case	Trả sách
Mã số	UC-09
Tác nhân	Thủ thư
Mô tả ngắn gọn	Thủ thư ghi nhận việc bạn đọc trả lại một bản sao đã mượn. Hệ thống đóng giao dịch, cập nhật trạng thái bản sao và tạo phí phạt nếu trả trễ hoặc sách hư/mất.
Tiền điều kiện	Bản sao có barcode hợp lệ và đang nằm trong một giao dịch BORROWING hoặc OVERDUE.
Hậu điều kiện	Thành công: Giao dịch mượn được đóng với trạng thái RETURNED. Bản sao được trả về AVAILABLE, hoặc được gán cho yêu cầu đặt trước tiếp theo nếu đang có hàng chờ. Nếu quá hạn, hệ thống tạo phí phạt. Thất bại: Trạng thái hệ thống không thay đổi.
Luồng chính	- Thủ thư khởi tạo chức năng "Trả sách". - Hệ thống yêu cầu nhập/quét mã vạch của bản sao sách. - Thủ thư nhập/quét mã vạch sách. - Hệ thống tìm giao dịch đang mượn tương ứng và kiểm tra ngày trả so với ngày đến hạn. - Hệ thống đóng giao dịch, cập nhật trạng thái bản sao và tạo phí quá hạn theo số ngày trễ nếu cần. - Hệ thống kiểm tra hàng đặt trước cho cùng ấn phẩm để gán bản sao vừa trả cho người tiếp theo. - Hệ thống gửi thông báo/email cho bạn đọc, đồng thời mở cơ hội đánh giá sách sau khi trả. Use case kết thúc.
Luồng thay thế	4a. Không tìm thấy giao dịch đang mượn: - Hệ thống báo lỗi và không cập nhật trạng thái. 5a. Sách bị trả muộn: - Hệ thống tạo phí quá hạn; bạn đọc có thể thanh toán sau bằng tiền mặt hoặc PayOS. 5b. Sách hư hoặc mất: - Thủ thư dùng luồng báo sự cố để ghi nhận vi phạm và tạo phí tương ứng.

5.1.2 Mô hình hoá quy trình nghiệp vụ (Activity diagram)

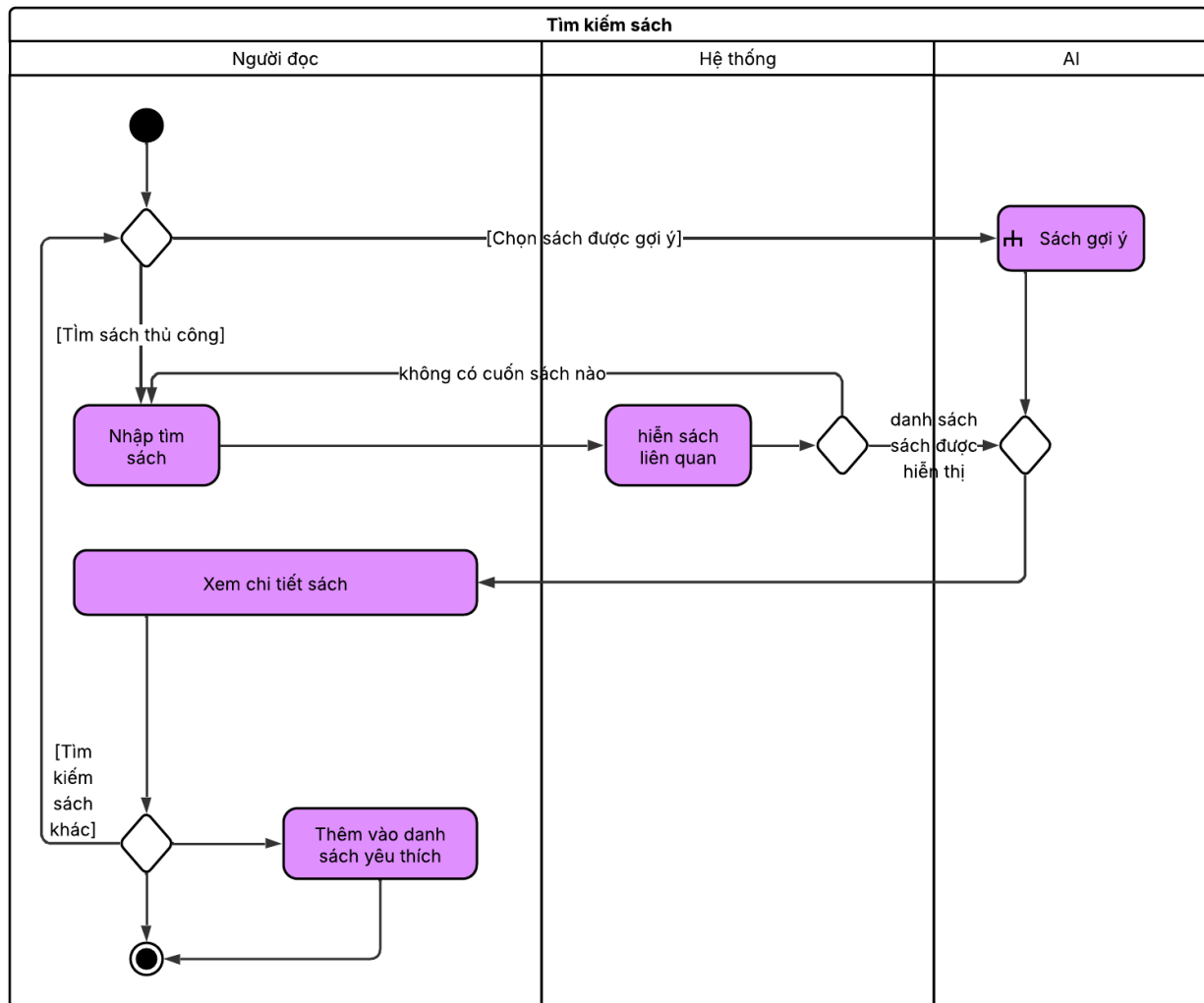
Các activity diagram trong phần này mô tả những luồng nghiệp vụ đã được hiện thực thành màn hình và API. Riêng chức năng **gia hạn sách** không được đưa vào phạm vi hiện tại vì backend chưa có use case/controller hoàn chỉnh cho thao tác này; hệ thống chỉ lưu `renewal_count` trong mô hình dữ liệu để có thể mở rộng về sau.



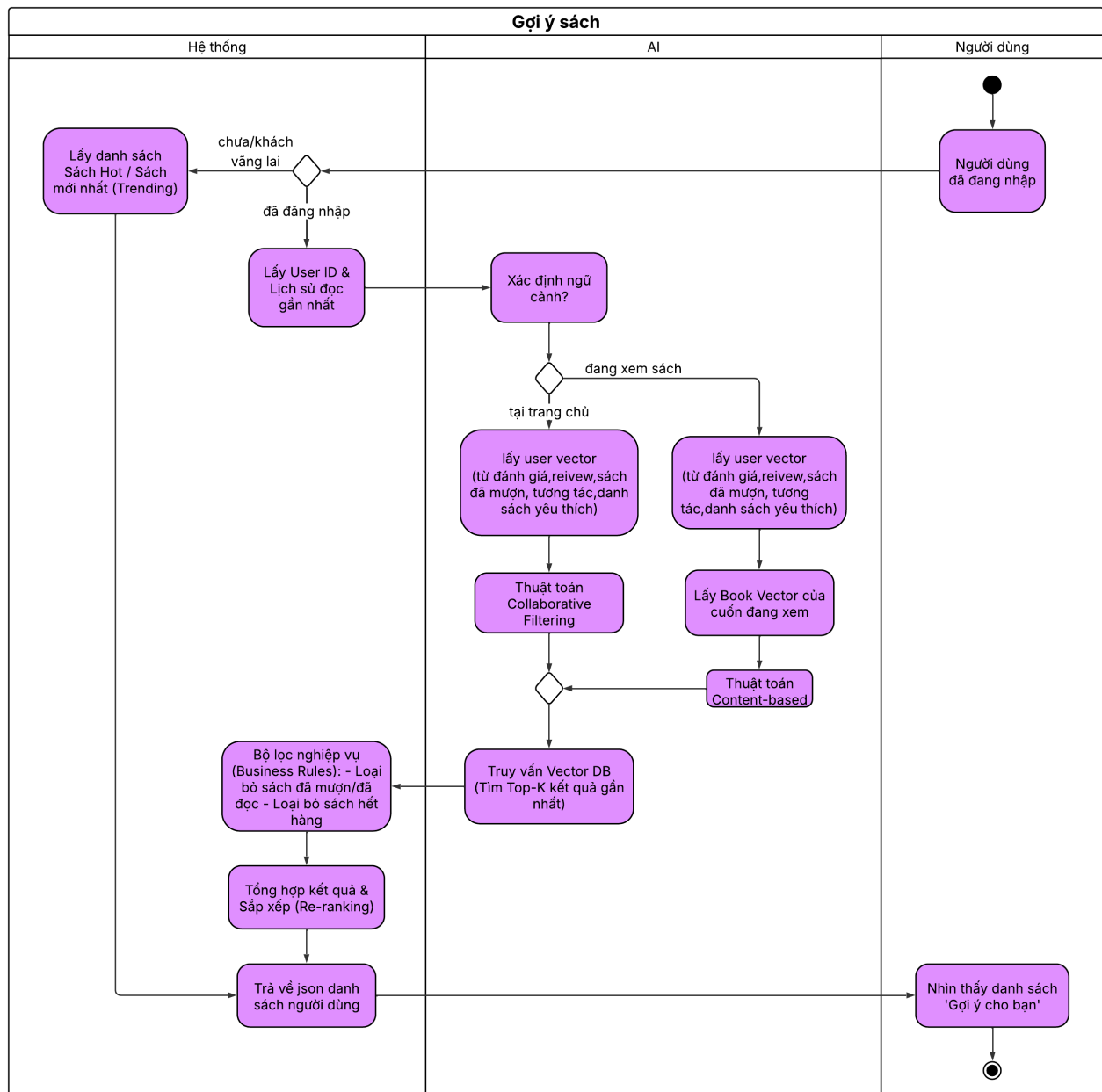
Hình 5.2: Đăng ký tài khoản — Activity Diagram



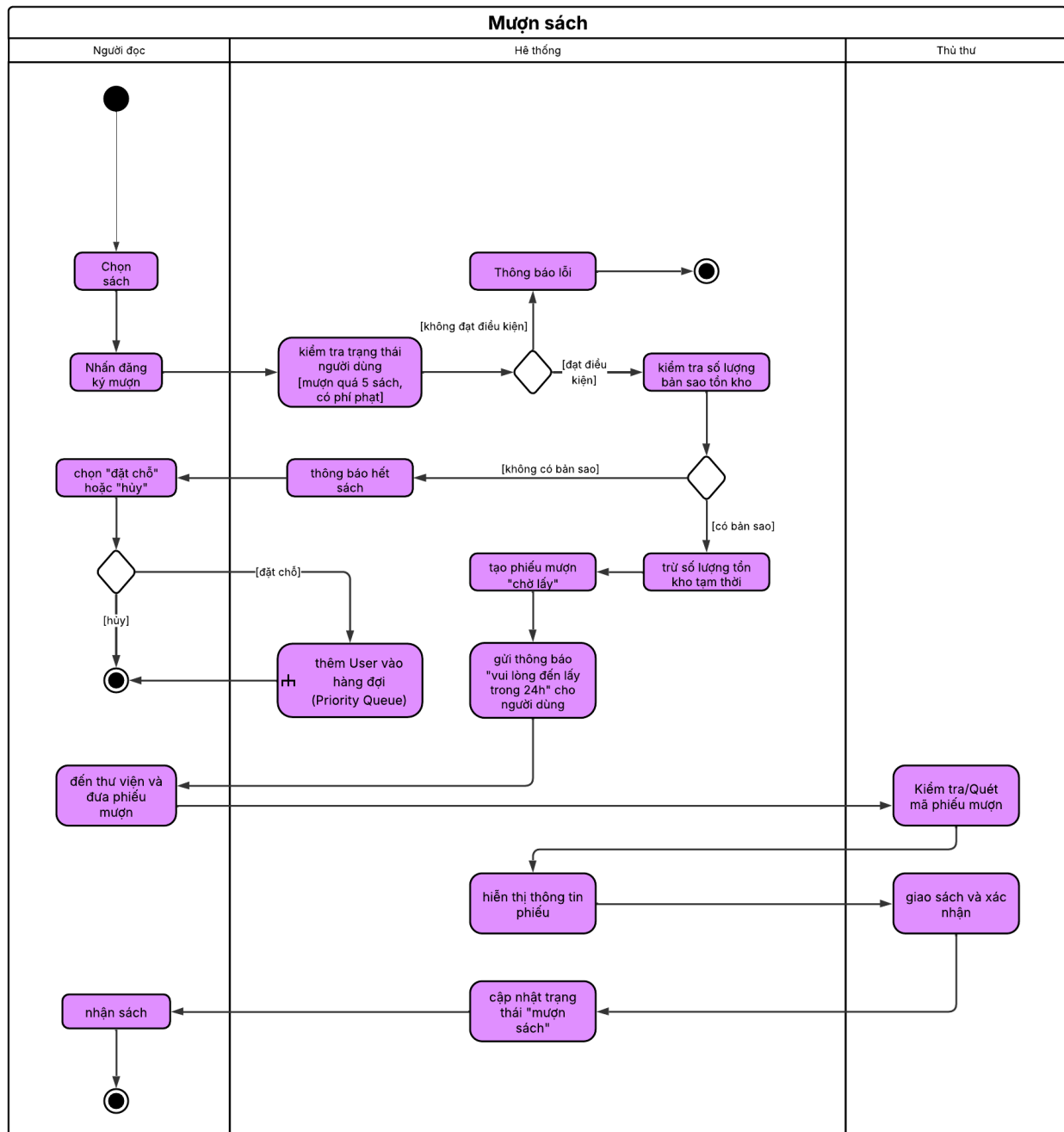
Hình 5.3: Đăng nhập tài khoản — Activity Diagram



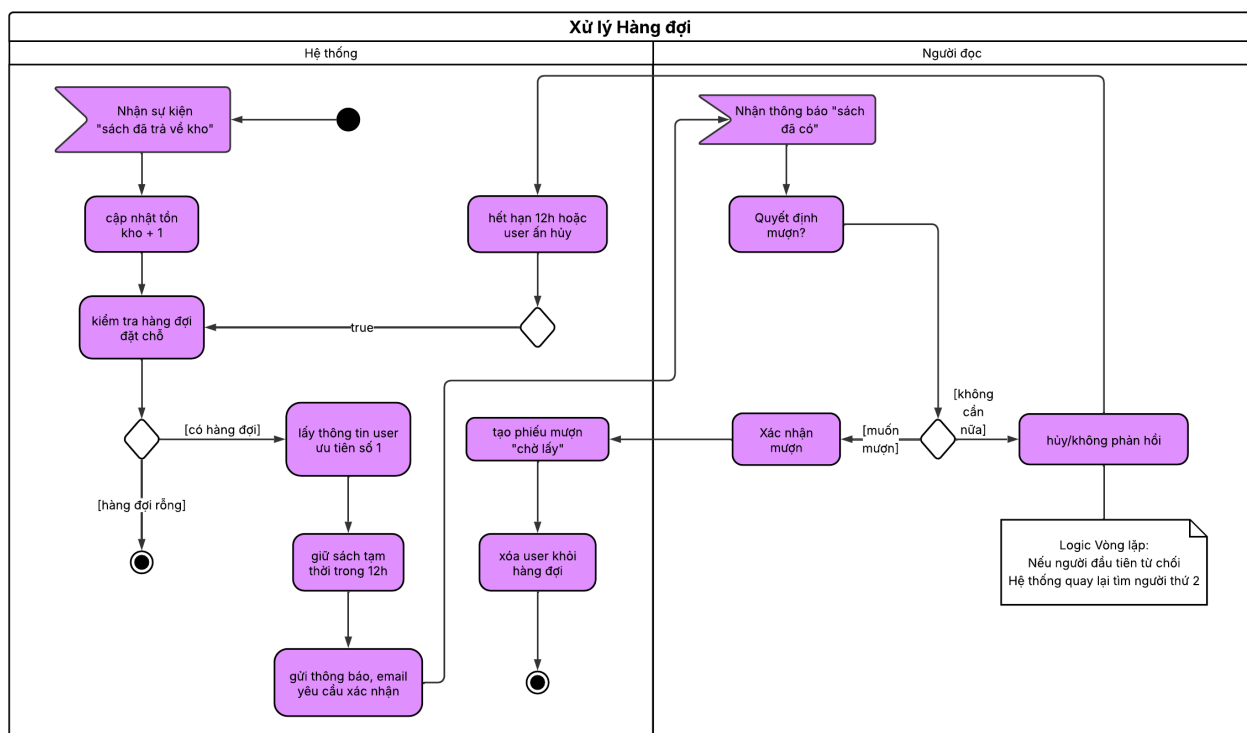
Hình 5.4: Tìm kiếm sách — Activity Diagram



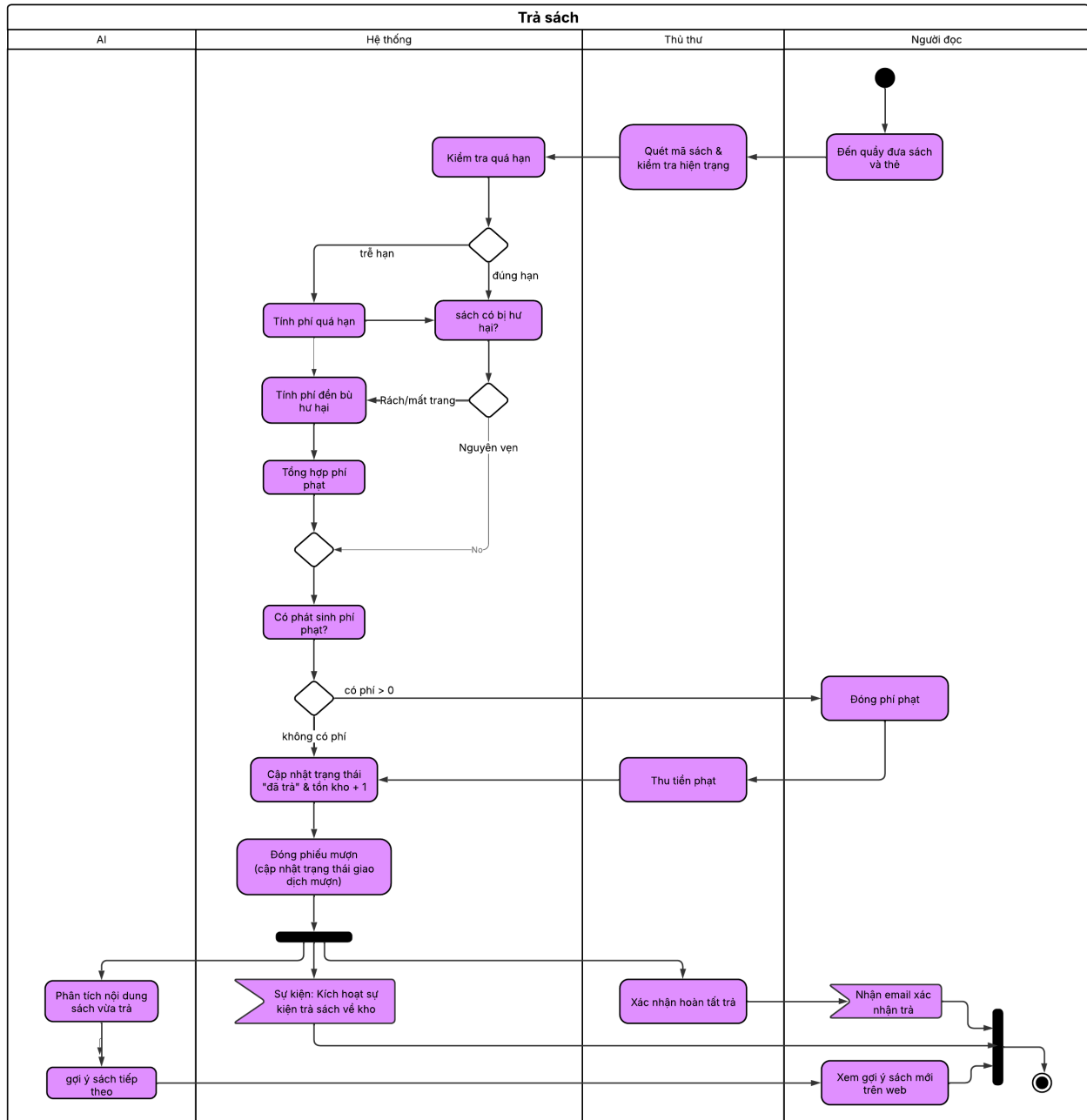
Hình 5.5: Gợi ý sách — Activity Diagram



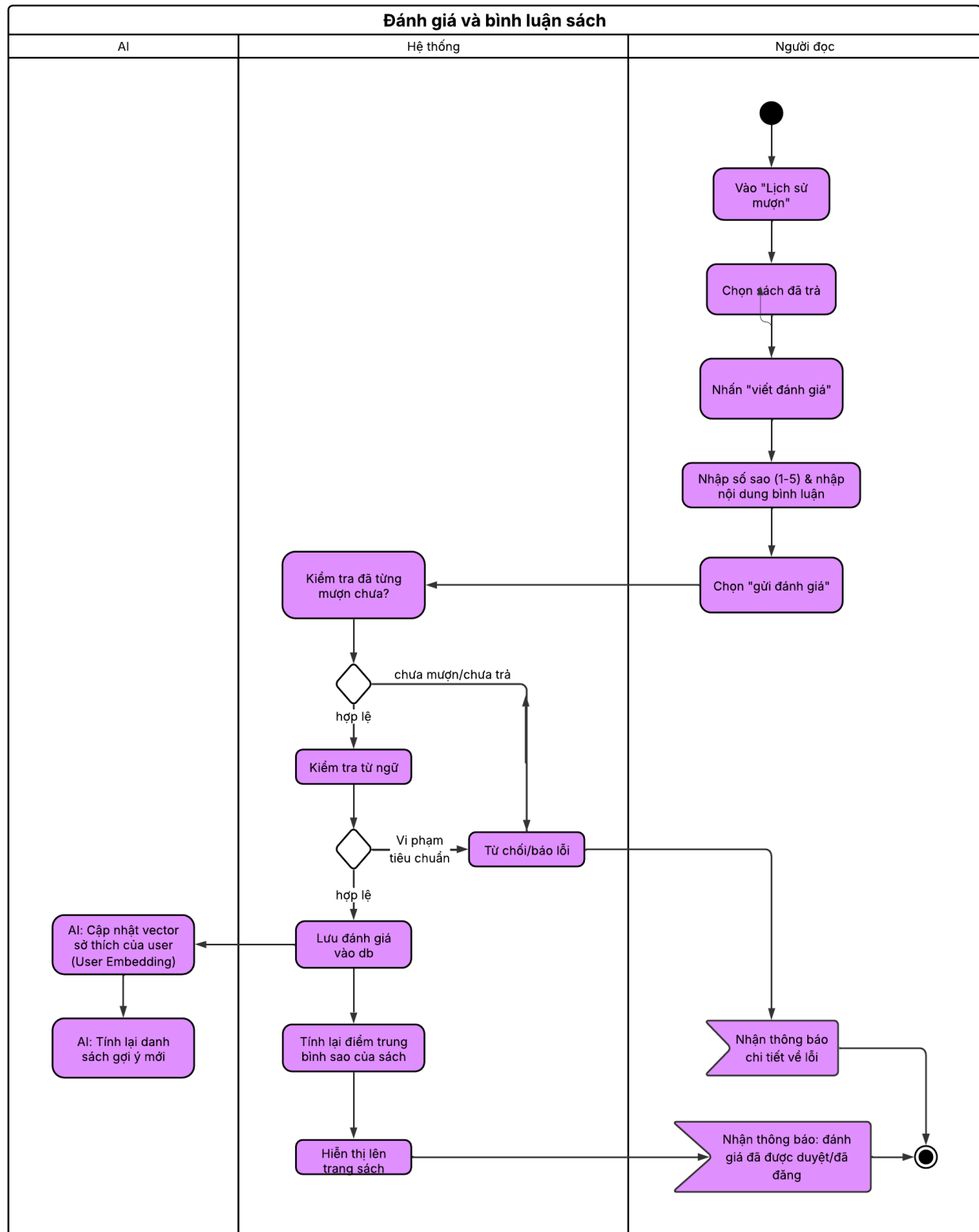
Hình 5.6: Mượn sách — Activity Diagram



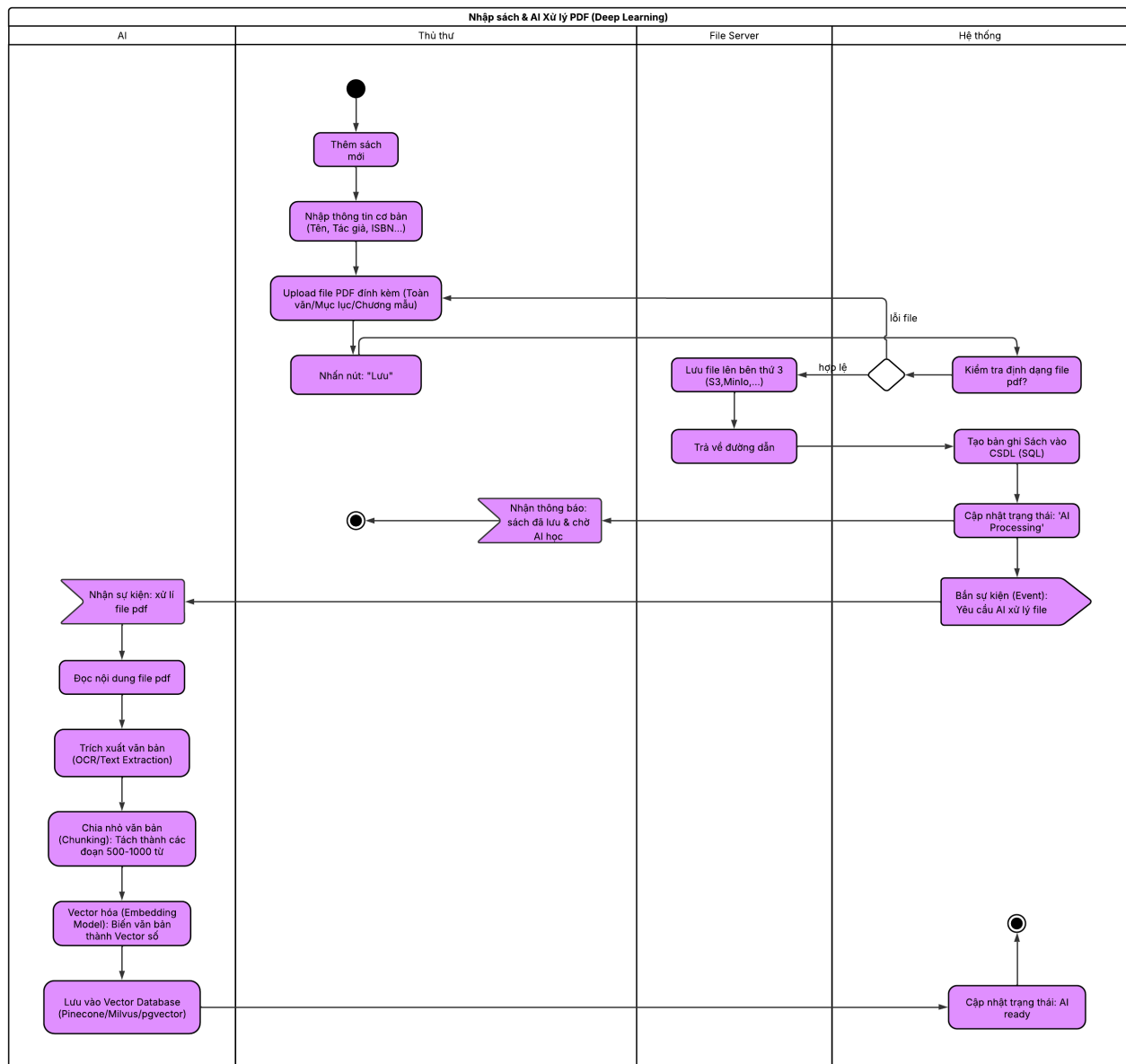
Hình 5.7: Xử lý hàng đợi đặt trước — Activity Diagram



Hình 5.8: Trả sách — Activity Diagram



Hình 5.9: Đánh giá và bình luận sách — Activity Diagram



Hình 5.10: Tạo sách với AI — Activity Diagram

5.2 Giải pháp công nghệ

Việc lựa chọn công nghệ không chỉ dựa trên mức độ phổ biến của framework, mà dựa trên mức độ phù hợp với bài toán thư viện đã hiện thực. Nhóm sử dụng năm tiêu chí chính: **hiệu năng và độ trễ, khả năng bảo trì, bảo mật và phân quyền, trải nghiệm phát triển, và chi phí triển khai trong phạm vi đồ án**. Từ các tiêu chí này, hệ thống được chia thành ba phần rõ ràng: frontend phục vụ trải nghiệm người dùng, backend giữ nghiệp vụ và dữ liệu giao dịch, còn AI service xử lý các workload tính toán nặng.

5.2.1 Front-end

Frontend của hệ thống là một **single-page application** phục vụ cả trang public, khu vực bạn đọc và dashboard thủ thư. Đặc điểm chính của phần này là nhiều form nghiệp vụ, nhiều màn hình trạng thái, gọi REST API liên tục, nhận thông báo realtime và upload file PDF trực tiếp lên S3-compatible storage bằng presigned URL. Vì vậy, công nghệ frontend cần nhẹ khi triển khai, dễ chia component và đủ linh hoạt để bám sát thiết kế Figma.

Nhóm đã so sánh các lựa chọn phổ biến theo đúng bối cảnh này. Bảng 5.8 được đặt ngay trong phần Front-end để tránh tách rời phân tích khỏi kết luận lựa chọn.

Nền tảng	Ưu điểm	Nhược điểm	Kết luận
React + Vite	Phù hợp với SPA nhiều tương tác; build gọn; HMR nhanh; triển khai như static assets; dễ tích hợp REST API, WebSocket/STOMP và presigned upload. Hệ sinh thái React đáp ứng tốt dashboard, form nghiệp vụ và component tái sử dụng.	Không có SSR mặc định; SEO cho public pages không mạnh bằng framework SSR; nhóm phải tự chuẩn hóa routing, state, API layer và cấu trúc component.	Được chọn. Hệ thống chủ yếu là ứng dụng nghiệp vụ sau đăng nhập, không cần SSR phức tạp. React/Vite giúp triển khai đơn giản qua NGINX/Docker, giảm nhu cầu Node runtime và vẫn đủ linh hoạt cho public pages, reader pages và librarian dashboard.
Next.js	Mạnh về SSR/SSG, SEO, route-level data fetching và tối ưu website public lớn. Phù hợp khi nội dung cần index mạnh hoặc cần render phía server.	Cần Node.js runtime, tăng thêm lớp vận hành. Nếu dùng cho hệ thống này, một phần trách nhiệm trình bày và xác thực dễ bị chồng lên Spring Boot API.	Không chọn vì bài toán chính là dashboard và nghiệp vụ thư viện, không phải website nội dung cần SEO mạnh. Lợi ích SSR không đủ bù cho chi phí vận hành thêm.
Nuxt (Vue)	Có SSR/SSG tốt trong hệ sinh thái Vue, cú pháp dễ tiếp cận và tổ chức project rõ ràng.	Không cùng hệ sinh thái với phần frontend đã xây dựng; chuyển sang Vue/Nuxt làm tăng chi phí học lại component, router, test và thư viện UI.	Không chọn vì nhóm đã hiện thực ổn định bằng React và cần tận dụng các thư viện React cho icon, form, realtime notification và dashboard.
Angular	Framework đầy đủ, opinionated, mạnh về form, dependency injection, routing và cấu trúc enterprise.	Boilerplate nhiều, đường cong học tập cao, nặng hơn nhu cầu của nhóm. Việc tùy biến UI nhanh theo Figma kém linh hoạt hơn React trong phạm vi đồ án.	Không chọn vì hệ thống cần tốc độ phát triển và khả năng tinh chỉnh giao diện cao hơn là một framework enterprise lớn ngay từ đầu.

Bảng 5.8: So sánh công nghệ Front-end theo tiêu chí của hệ thống

Lựa chọn: *React + Vite kết hợp với TypeScript, Tailwind CSS, React Router DOM với HashRouter, Axios,*

lucide-react, sonner, SockJS/STOMP và một bộ component tự xây dựng theo thiết kế Figma.

Lý do chọn theo tiêu chí chuyên môn

- **Tốc độ phát triển vượt trội (Developer Experience):** Vite tận dụng cơ chế native ES modules trong môi trường phát triển và dùng esbuild cho nhiều bước tiền xử lý, nhờ đó thời gian khởi động dev server và Hot Module Replacement nhanh hơn đáng kể so với cấu hình Webpack truyền thống. Với một frontend có nhiều trang nghiệp vụ, điều này giúp nhóm lập nhanh khi chỉnh giao diện, form và luồng tương tác.
- **Triển khai đơn giản qua NGINX:** React + Vite tạo ra các file tĩnh (HTML/CSS/JS) sau khi build, có thể phục vụ trực tiếp bởi NGINX mà không cần Node.js server runtime. Điều này giúp giảm tài nguyên máy chủ, đơn giản hóa cấu hình Docker và tận dụng khả năng cache tĩnh của NGINX để tăng hiệu năng.
- **Phù hợp với đặc thù hệ thống:** Phần lớn giao diện của hệ thống là các trang dashboard và luồng nghiệp vụ dành cho người dùng đã đăng nhập, ví dụ quản lý mượn-trả, đặt trước, phí phạt, thông báo và upload tài liệu. Những trang này ưu tiên tốc độ tương tác và tính ổn định hơn SEO. Bảo mật API được đảm bảo ở Backend Spring Boot bằng JWT và kiểm tra vai trò, nên không cần thêm một lớp server-side rendering trung gian.
- **Hệ sinh thái React và tính nhất quán giao diện:** TypeScript giúp giảm lỗi khi tích hợp với API backend, trong khi Tailwind CSS, lucide-react, sonner, react-select và các component tự xây dựng giúp nhóm kiểm soát tốt giao diện mà không phụ thuộc nặng vào một UI framework lớn. Cách làm này phù hợp với hệ thống có nhiều màn hình nghiệp vụ cần tính chỉnh riêng cho bạn đọc và thủ thư.
- **Quản lý trạng thái và Routing:** React Router DOM với HashRouter được dùng để tổ chức các nhóm trang public, reader và librarian, phù hợp với kiểu deploy static qua NGINX. Các trạng thái dùng chung như xác thực, ngôn ngữ, theme, thông báo realtime và upload file được quản lý bằng React Context; các lời gọi API được gom qua Axios service để thống nhất xử lý token, lỗi và phản hồi từ backend.

5.2.2 Back-end

Backend là phần lõi của hệ thống vì chịu trách nhiệm xác thực, phân quyền, quản lý dữ liệu thư viện và đảm bảo tính nhất quán cho các giao dịch mượn-trả. Khác với frontend, backend không chỉ cần tạo API nhanh mà còn phải xử lý transaction, khóa bản sao khi mượn/trả, phí phạt, đặt trước, webhook thanh toán và callback từ AI service. Vì vậy tiêu chí ưu tiên là độ tin cậy, transaction ACID, bảo mật và khả năng tổ chức module rõ ràng.

Nền tảng	Ưu điểm	Nhược điểm	Quyết định
Spring Boot	Mạnh về nghiệp vụ giao dịch, Spring Security, transaction ACID, JPA/JDBC, validation, scheduler, Kafka/WebSocket và tích hợp dịch vụ ngoài. Phù hợp với mô hình dữ liệu quan hệ chặt chẽ của thư viện.	Verbose hơn Node/Python; cần thiết kế module rõ ràng để tránh monolith bị rối khi số use case tăng.	Được chọn. Lỗi hệ thống có nhiều nghiệp vụ cần tính nhất quán: mượn-trả, đặt trước, phí phạt, bản sao, thanh toán và phân quyền. Spring Boot giúp kiểm soát transaction, bảo mật và ranh giới module tốt hơn trong bối cảnh hiện thực hiện tại.
NestJS / Express	Phát triển API nhanh, TypeScript đồng bộ với frontend, hệ sinh thái lớn và dễ tuyển dụng.	Express quá tự do nên cần tự chuẩn hóa nhiều thứ; NestJS tốt hơn nhưng transaction, JPA-style domain modeling và security enterprise không tự nhiên bằng Spring Boot trong bài toán này.	Không được chọn vì lợi thế dùng chung ngôn ngữ không đủ bù cho yêu cầu transaction và phân quyền chặt chẽ của nghiệp vụ thư viện.
Django / DRF	CRUD nhanh, admin mạnh, Python thuận lợi nếu gộp chung backend với AI.	Nếu gộp nghiệp vụ và AI vào một codebase Python, ranh giới giữa transaction nghiệp vụ và workload AI dễ bị lẫn. Khả năng module hóa theo DDD/Clean Architecture trong dự án hiện tại không thuận lợi bằng Spring Boot.	Không được chọn cho core backend; Python được giữ đúng vai trò ở AI service, nơi hệ sinh thái ML phát huy giá trị nhất.
MongoDB (NoSQL)	Schema linh hoạt, dễ lưu tài liệu JSON và mở rộng ngang trong một số bài toán dữ liệu phi cấu trúc.	Không phù hợp làm datastore chính cho quan hệ nhiều chiều giữa ấn phẩm, bản sao, giao dịch mượn, phí phạt, đặt trước và người dùng. Join, constraint và transaction liên bảng là yêu cầu cốt lõi của hệ thống.	Không được chọn vì PostgreSQL đáp ứng tốt hơn cả dữ liệu quan hệ lẫn dữ liệu AI nhờ JSONB và pgvector.

Bảng 5.10: So sánh công nghệ Back-end theo tiêu chí của hệ thống

Lựa chọn: *Spring Boot 3.3, Java 21, Spring Security với JWT/OAuth2, Spring Data JPA kết hợp JDBC Template cho truy vấn đặc thù, PostgreSQL, Flyway, Kafka, WebSocket/STOMP, S3-compatible storage, PayOS và OpenAPI/Swagger. Mô-đun AI/NLP được tách thành một service riêng sử dụng Python/FastAPI.*

Lý do chọn theo tiêu chí chuyên môn

- **Độ tin cậy và Hiệu năng:** Spring Boot là một framework đã được kiểm chứng về độ ổn định và hiệu năng trong các hệ thống enterprise. Kết hợp với **Java Virtual Machine (JVM)**, nó cung cấp khả năng xử lý đồng thời mạnh mẽ, phù hợp với các tác vụ CRUD và giao dịch phức tạp. Các truy vấn catalog được phân trang và tối ưu bằng PostgreSQL; các tác vụ phụ như email, thông báo và một số xử lý nền được tách khỏi luồng chính bằng Kafka, WebSocket và executor bất đồng bộ.

- **Mô hình Dữ liệu và Khả năng mở rộng:** Nghiệp vụ thư viện có bản chất là quan hệ chặt chẽ. **Spring Data JPA** và **Hibernate** cung cấp một tầng ORM (Object-Relational Mapping) mạnh mẽ, cho phép mô hình hóa các thực thể và mối quan hệ phức tạp một cách tự nhiên. Việc sử dụng **Specification Pattern** giúp xây dựng các truy vấn động một cách an toàn và dễ bảo trì, phục vụ cho các yêu cầu lọc và tìm kiếm phức tạp.
- **Bảo mật Toàn diện:** **Spring Security** cho phép triển khai xác thực và phân quyền theo vai trò một cách rõ ràng. Hệ thống sử dụng **JWT** với access token và refresh token cho kiến trúc stateless API; backend chịu trách nhiệm kiểm tra quyền truy cập ở các endpoint quản trị, endpoint mượn-trả, thanh toán và callback từ AI service. Các luồng nhảy cảm như upload file và webhook thanh toán được đặt sau các cơ chế xác thực hoặc chữ ký tương ứng.
- **Ranh giới module rõ ràng:** Backend được tổ chức thành các module Maven như auth, user, catalog, circulation, recommendation và shared. Cách chia này giữ được ưu điểm triển khai đơn giản của modular monolith, nhưng vẫn tạo ranh giới nghiệp vụ đủ rõ để kiểm thử, bảo trì và tách service trong tương lai nếu cần.
- **Dịch vụ AI độc lập:** Việc tách mô-đun AI/NLP thành một service riêng bằng Python/FastAPI là một quyết định kiến trúc có chủ đích. Backend Java giữ phần transaction, phân quyền và nghiệp vụ; AI service giữ phần embedding, semantic search, Gemini reduce và recommendation. Cách chia này tránh nhồi ML workload vào core backend, nhưng vẫn đơn giản hơn việc tách toàn bộ hệ thống thành microservices.

Kết luận về Sự phối hợp giữa hai nền tảng

Kiến trúc tổng thể được thiết kế theo mô hình **frontend tĩnh + backend API + AI service**. **React/Vite SPA** chịu trách nhiệm trải nghiệm người dùng và gọi REST API; **Spring Boot API** giữ vai trò lõi nghiệp vụ, xác thực, phân quyền và điều phối dữ liệu; **FastAPI AI Service** xử lý các tác vụ nặng như embedding, semantic search và recommendation. Sự phân tách này đủ rõ để bảo trì, nhưng vẫn giữ chi phí triển khai thấp hơn so với một kiến trúc microservices đầy đủ.

5.3 Ứng dụng AI vào hệ thống

Trong hệ thống LMS, AI không được đặt vào như một chức năng phụ để “làm đẹp” sản phẩm. AI được thiết kế như một **lớp năng lực hỗ trợ khám phá tri thức**: đọc nội dung PDF, hiểu ý định tìm kiếm của bạn đọc và đề xuất tài liệu phù hợp với hành vi sử dụng thật.

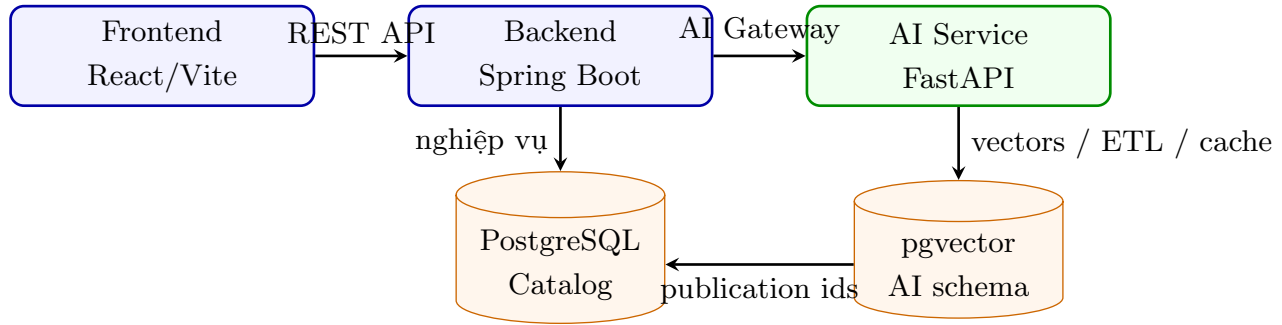
Điểm quan trọng nhất trong thiết kế là **AI không thay thế nghiệp vụ thư viện**. Các nghiệp vụ như mượn-trả, đặt trước, quản lý bản sao, phí phạt, thanh toán và phân quyền vẫn do backend Spring Boot kiểm soát. AI Service chỉ xử lý các bài toán phù hợp với ML/NLP, sau đó trả về **metadata bổ trợ**, **danh sách publication_id** hoặc **chiến lược gợi ý**; backend luôn là nơi lấy dữ liệu cuối cùng trước khi trả về frontend.

- **Module 1 – Xử lý PDF file:** biến file PDF do thủ thư upload thành dữ liệu có thể tìm kiếm và gợi ý.
- **Module 2 – Hỗ trợ tìm kiếm ngữ nghĩa:** giúp bạn đọc tìm theo ý định, không chỉ theo từ khóa chính xác.
- **Module 3 – Gợi ý cá nhân hoá:** kết hợp nội dung sách, hành vi người dùng và fallback nghiệp vụ để đề xuất sách có thể mượn.

5.3.0.0.1 Ranh giới tích hợp AI.

Nhóm chọn mô hình **AI Satellite Service**: backend Java giữ nghiệp vụ và giao dịch; AI Service Python/FastAPI xử lý workload nặng như đọc PDF, sinh embedding, truy vấn pgvector và recommendation. Cách tách này giúp

hệ thống vừa giữ lõi nghiệp vụ ổn định, vừa tận dụng hệ sinh thái AI của Python.



Hình 5.11: Ranh giới trách nhiệm giữa frontend, backend và AI service

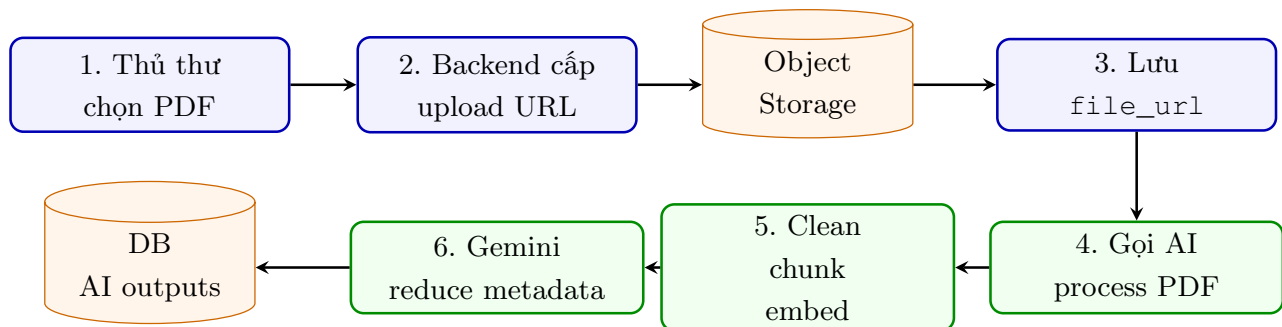
Thành phần	Trách nhiệm chính	Điểm nhấn thiết kế
Frontend	Upload tài liệu, gửi truy vấn, hiển thị trạng thái AI và kết quả đã chuẩn hóa.	Không gọi trực tiếp model AI ; mọi request đi qua backend.
Backend	Xác thực, phân quyền, lưu catalog, quản lý bản sao/giao dịch và lấy metadata cuối cùng.	Là nguồn sự thật nghiệp vụ ; AI không được quyết định trạng thái mượn-trả.
AI Service	Xử lý PDF, sinh embedding, tìm kiếm ngữ nghĩa, tạo metadata AI và recommendation.	Chỉ trả kết quả suy luận ; backend quyết định dữ liệu hiển thị cuối cùng.

Bảng 5.12: Ranh giới trách nhiệm khi tích hợp AI

5.3.1 Module 1: Xử lý PDF file

Mục tiêu của module này là biến một file PDF thô thành dữ liệu có giá trị cho hệ thống: **summary**, **audience**, **tag**, **vector embedding** và **trạng thái xử lý AI**. Đây là module nền tảng: nếu PDF không được xử lý tốt, semantic search và recommendation phía sau sẽ không có đủ dữ liệu để hoạt động.

Luồng xử lý được thiết kế theo hướng **không chặn thao tác của thủ thư**. Thủ thư chỉ cần upload tài liệu; backend lưu URL tài liệu và kích hoạt AI xử lý ở phía sau.



Hình 5.12: Module 1 – Luồng xử lý PDF file sau khi thủ thư upload

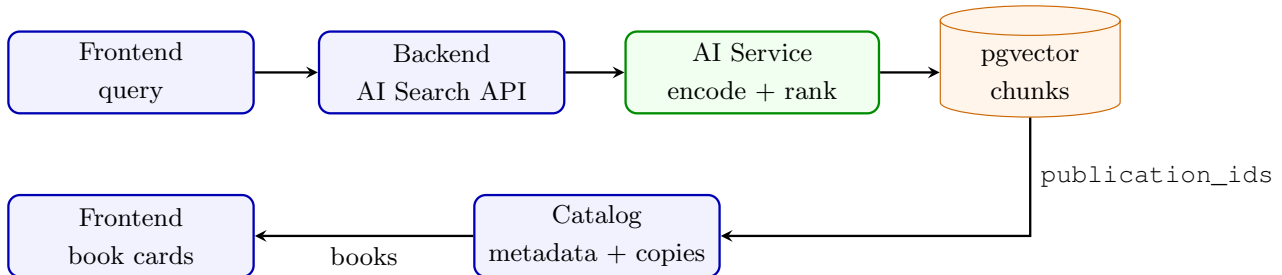
Giai đoạn	Thiết kế	Điểm nhấn báo cáo
Upload và lưu file	Frontend xin presigned upload URL ; file PDF được đẩy lên object storage; backend lưu lại <code>file_url/s3Key</code> cho publication.	File được lưu trước, AI xử lý sau ; thao tác tạo/cập nhật sách không bị khóa bởi pipeline AI.
ETL nội dung	AI Service dùng PyMuPDF để trích xuất text, làm sạch header/footer/số trang, sau đó chia chunk bằng sliding window.	Index nội dung thật của PDF , không chỉ index tiêu đề hay mô tả thủ công.
Embedding và meta-data	Chunk được vector hóa và lưu vào <code>ai_engine.publication_vectors</code> ; Gemini chỉ được gọi ở bước reduce để tạo summary, audience và tag.	Không gọi LLM trong mỗi lần tìm kiếm ; giảm độ trễ, giảm quota và tăng độ ổn định.
Theo dõi trạng thái	Kết quả xử lý được lưu với trạng thái <code>RUNNING, SUCCESS, FAILED</code> ; nếu lỗi, catalog vẫn giữ ấn phẩm.	AI lỗi không làm hỏng nghiệp vụ thư viện .

Bảng 5.14: Thiết kế Module 1 – Xử lý PDF file

5.3.2 Module 2: Hỗ trợ tìm kiếm ngữ nghĩa (Semantic Search)

Mục tiêu của module này là giúp bạn đọc tìm được tài liệu theo **ý định học tập**, không bị phụ thuộc hoàn toàn vào việc nhớ đúng tên sách, tác giả hoặc tag. Ví dụ, truy vấn “sách nhập môn cơ sở dữ liệu dễ hiểu” vẫn có thể tìm đến các tài liệu liên quan, dù cụm từ đó không xuất hiện nguyên văn trong tiêu đề.

Về mặt thiết kế, hệ thống dùng hướng **Bi-Encoder** của Sentence-BERT [3]: vector của chunk PDF được tính trước trong Module 1, còn vector của query được tính tại thời điểm tìm kiếm. Việc so khớp vector được thực hiện bằng **PostgreSQL + pgvector** [4].



Hình 5.13: Module 2 – Luồng thực thi semantic search

Điểm thiết kế	Ý nghĩa
Query đi qua backend	Backend giữ format response, phân quyền và thống nhất cách trả dữ liệu cho frontend.
AI chỉ trả ID	AI Service không tự trả toàn bộ thông tin sách; backend lấy lại metadata, rating và tình trạng bản sao từ catalog.
Kết hợp keyword fallback	Với truy vấn ngắn như tên sách/tác giả, hệ thống vẫn ưu tiên keyword để tránh semantic search làm lệch kết quả rõ ràng.
Tìm trên nội dung thật	Vector được sinh từ chunk PDF, nên hệ thống có thể tìm theo nội dung bên trong sách, không chỉ theo metadata nhập tay.

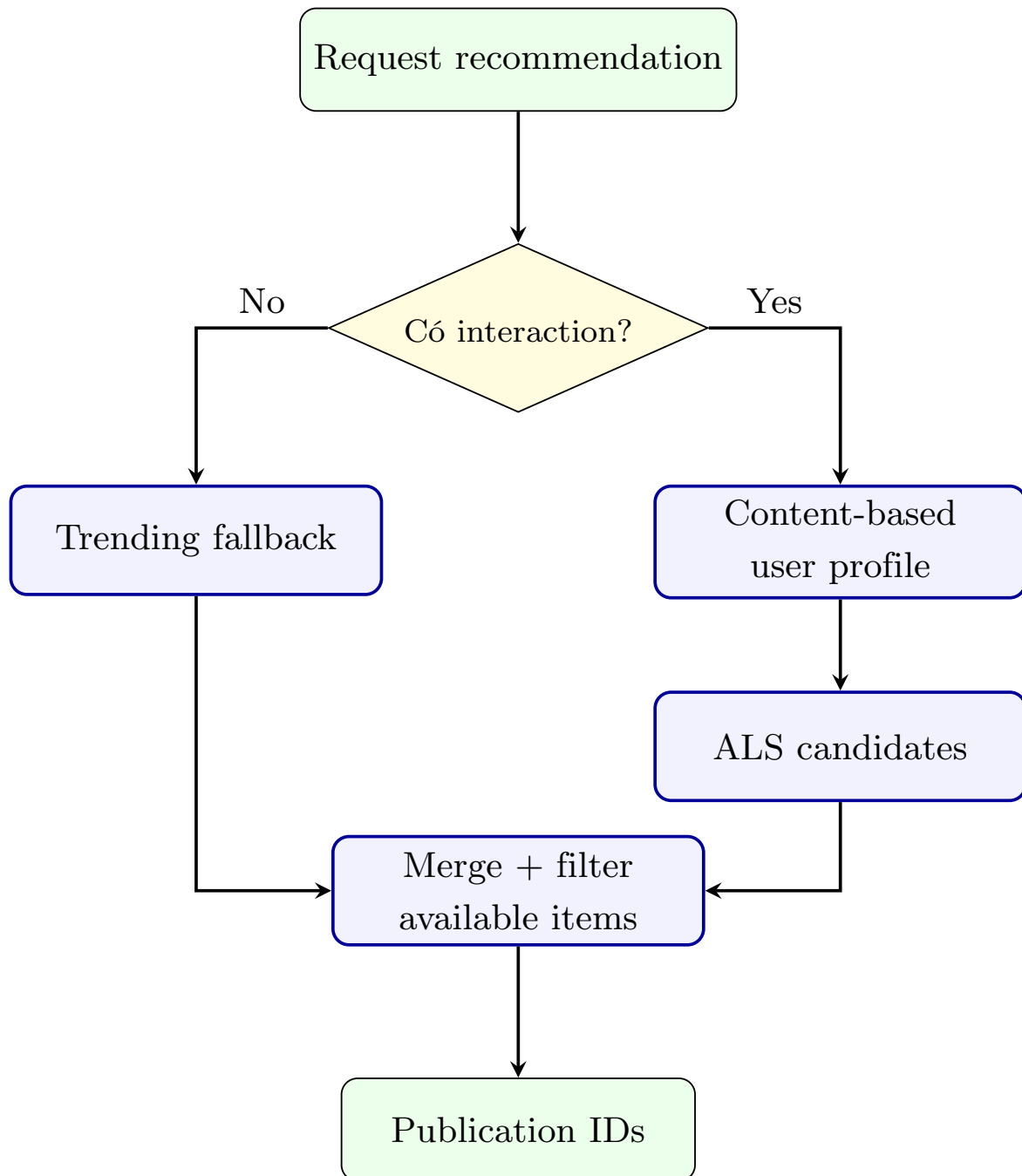
Bảng 5.16: Thiết kế Module 2 – Hỗ trợ tìm kiếm ngữ nghĩa

5.3.3 Module 3: Gợi ý cá nhân hoá (Recommendation)

Mục tiêu của module này là biến dữ liệu tương tác rời rạc thành danh sách sách gợi ý có ý nghĩa. Trong bối cảnh thư viện, dữ liệu thường không dày như các nền tảng thương mại: có người dùng mới, sách mới, ít rating và hành vi không đều. Vì vậy, hệ thống chọn hướng **hybrid recommendation** [5], kết hợp nhiều nguồn tín hiệu thay vì phụ thuộc vào một thuật toán duy nhất.

Nguồn gợi ý	Cách hoạt động	Vai trò
Content-based	Tạo hồ sơ nội dung từ vector của các sách người dùng đã xem, wishlist hoặc mượn.	Phản ứng nhanh với hành vi mới.
ALS implicit feedback	Học quan hệ người dùng–sách từ WATCH, WISHLIST, BORROWED [6].	Khai thác mẫu hành vi tập thể.
Trending fallback	Xếp hạng theo tương tác, lượt mượn, rating và thời gian tạo sách.	Chống cold-start và lỗi AI.

Bảng 5.18: Ba nguồn tín hiệu trong Module 3 – Recommendation



Hình 5.14: Module 3 – Luồng xử lý hybrid recommendation

Điểm nhấn của module này là **gợi ý phải có giá trị nghiệp vụ**, không chỉ đúng về mặt thuật toán. Vì vậy, trước khi trả kết quả, hệ thống luôn lọc các sách không còn bản sao AVAILABLE. Một cuốn sách phù hợp sở thích nhưng không thể mượn ngay sẽ không được ưu tiên trong trải nghiệm người dùng.

Rủi ro AI	Cách kiểm soát trong thiết kế
LLM chậm, lỗi hoặc hết quota	Gemini chỉ chạy trong Module 1, không chạy trong request tìm kiếm/gợi ý; nếu lỗi thì dùng fallback deterministic.
AI Service không phản hồi	Backend vẫn có keyword search, catalog search và cache/trending fallback cho recommendation.
PDF thay đổi hoặc xử lý lại	Pipeline dùng hash file và trạng thái ETL để tránh xử lý trùng; khi cần re-index thì xóa vector/tag cũ trước.
Gợi ý không phù hợp nghiệp vụ	Backend lọc lại theo bản sao khả dụng và lấy metadata cuối cùng từ catalog.

Bảng 5.20: Các điểm kiểm soát rủi ro khi dùng AI

Kết luận thiết kế: hệ thống không biến LLM thành trung tâm. **LLM chỉ hỗ trợ biên mục, semantic search dựa trên vector nội dung thật, recommendation dựa trên hành vi thật**, còn toàn bộ quyết định nghiệp vụ vẫn nằm trong backend. Đây là cách đưa AI vào hệ thống theo hướng **có giá trị, có kiểm soát và có khả năng vận hành**.

5.4 Thiết kế

5.4.1 Kiến trúc hệ thống

5.4.1.1 So sánh và lựa chọn kiến trúc hệ thống

Kiến trúc hệ thống là nền móng quyết định khả năng mở rộng, bảo trì và tiến hóa của toàn bộ sản phẩm phần mềm, việc lựa chọn kiến trúc cần cân nhắc đồng thời ba nhóm yêu cầu then chốt:

- Tính nhất quán giao dịch (Transactional Consistency):** Các nghiệp vụ mượn/trả sách đòi hỏi cập nhật đồng thời nhiều thực thể (trạng thái sách, giao dịch mượn, tồn kho, hàng đợi) trong một transaction ACID để tránh inconsistency.
- Tích hợp AI phức tạp:** Hệ thống cần xử lý các tác vụ machine learning như trích xuất văn bản PDF, tạo vector embeddings, gọi Gemini để reduce metadata, tìm kiếm ngữ nghĩa và huấn luyện mô hình gợi ý. Các workload này có đặc điểm công nghệ và nhu cầu mở rộng khác biệt so với CRUD nghiệp vụ.
- Nguồn lực hạn chế:** Với bối cảnh đồ án sinh viên, cần tối ưu hóa chi phí vận hành (infrastructure cost) và độ phức tạp triển khai, đồng thời vẫn đảm bảo khả năng mở rộng trong tương lai.

A. Các kiểu kiến trúc phổ biến:

Hiện nay, có nhiều kiểu kiến trúc hệ thống phổ biến, mỗi loại có những ưu và nhược điểm riêng:

- Kiến trúc Phân lớp (Layered Architecture):** Một kiểu monolith truyền thống, phân tách logic theo các lớp kỹ thuật (Presentation, Business, Data Access).
- Kiến trúc Monolith dạng Mô-đun (Modular Monolith):** Một khối ứng dụng duy nhất nhưng được cấu trúc nội bộ thành các mô-đun nghiệp vụ (domain) rõ ràng, độc lập tương đối với nhau.
- Kiến trúc Hướng dịch vụ (Service-Based Architecture):** Kiến trúc lai giữa Monolith và Microservices, hệ thống được chia thành một số ít các dịch vụ nghiệp vụ lớn (coarse-grained), thường sử dụng chung cơ sở dữ liệu nhưng có thể triển khai độc lập.
- Kiến trúc Vi dịch vụ (Microservices):** Chia nhỏ hệ thống thành nhiều dịch vụ nhỏ, độc lập, mỗi dịch vụ đảm nhận một chức năng nghiệp vụ cụ thể và có thể được triển khai, mở rộng độc lập.

- **Kiến trúc Hướng sự kiện (Event-Driven Architecture):** Các thành phần trong hệ thống giao tiếp với nhau một cách bất đồng bộ thông qua việc phát và lắng nghe các sự kiện (events).
- **Các kiến trúc khác:** Microkernel, Space-Based Architecture.

B. So sánh các kiến trúc:

Để lựa chọn kiến trúc phù hợp nhất, nhóm đã phân tích các kiểu kiến trúc dựa trên các tiêu chí quan trọng được thể hiện trong biểu đồ so sánh dưới đây.

	layered	modular monolith	microkernel	microservices	service-based	service-oriented	event-driven	space-based
partitioning	technical	domain	domain	domain	domain	technical	technical	technical
cost	\$	\$	\$	\$\$\$\$\$	\$\$	\$\$\$\$	\$\$\$	\$\$\$\$
maintainability	★	★★	★★★	★★★★★	★★★★★	★	★★★	★★★
testability	★★	★★	★★★	★★★★★	★★★★★	★	★★	★
deployability	★	★★	★★★	★★★★★	★★★★★	★	★★★	★★★
simplicity	★★★★★	★★★★★	★★★★★	★	★★★	★	★★	★
scalability	★	★	★	★★★★★	★★★	★★★	★★★★★	★★★★★
elasticity	★	★	★	★★★★★	★★	★★★	★★★★★	★★★★★
responsiveness	★★★	★★★	★★★	★★	★★★	★★	★★★★★	★★★★★
fault-tolerance	★	★	★	★★★★★	★★★★★	★★★	★★★★★	★★★
evolvability	★	★	★★★	★★★★★	★★★★★	★	★★★★★	★★★
abstraction	★	★	★★★	★	★	★★★★★	★★★★★	★
interoperability	★	★	★★★	★★★	★★	★★★★★	★★★	★★

Hình 5.15: So sánh các đặc tính của các kiểu kiến trúc (Nguồn: Mark Richards, Neal Ford, *Fundamentals of Software Architecture*, O'Reilly, 2nd ed., 2020) [1].

C. Phân tích loại trừ: Từ biểu đồ trên kết hợp với các yêu cầu nghiệp vụ đã xác định, nhóm tiến hành loại trừ các kiến trúc không phù hợp:

• Loại bỏ Kiến trúc Vi dịch vụ (Microservices):

- *Ưu điểm từ biểu đồ:* Đạt điểm tuyệt đối về khả năng mở rộng (scalability: 5 sao), triển khai độc lập (deployability: 5 sao), và khả năng chịu lỗi (fault-tolerance: 5 sao).
- *Nhược điểm nghiêm trọng:* Chi phí vận hành ở mức cao nhất (\$\$\$\$\$) và độ phức tạp có điểm thấp nhất (simplicity: 1 sao).
- *Mâu thuẫn với yêu cầu:* Với nguồn lực hạn chế của đồ án sinh viên, việc vận hành hạ tầng phân tán (service discovery, distributed tracing, container orchestration) là không khả thi. Hơn nữa, các nghiệp vụ mượn/trả sách đòi hỏi cập nhật đồng thời nhiều aggregate (giao dịch, tồn kho, hàng đợi) - nếu tách thành microservices sẽ phải xử lý distributed transactions phức tạp (Saga pattern, 2PC:Two-Phase Commit) với rủi ro eventual consistency.

• Loại bỏ Kiến trúc Phân lớp (Layered) truyền thống:

- *Ưu điểm từ biểu đồ:* Đơn giản nhất (simplicity: 5 sao) và chi phí thấp nhất (cost: \$).
- *Nhược điểm nghiêm trọng:* Các chỉ số quan trọng cho sự phát triển dài hạn đều ở mức thấp: khả năng bảo trì (maintainability: 1 sao), khả năng kiểm thử (testability: 2 sao), và tính linh hoạt (evolvability: 1 sao).
- *Mâu thuẫn với yêu cầu:* Kiến trúc phân lớp theo kỹ thuật (technical layers) dễ dẫn đến tight coupling giữa các logic nghiệp vụ. Đối với hệ thống có yêu cầu tích hợp AI phức tạp (xử lý bất đồng bộ PDF, cập nhật vector embeddings, trigger recommendation engine), kiến trúc này dễ dẫn đến "big ball of mud" với business logic rải rác khắp các layer.

- **Cân nhắc về Kiến trúc Hướng dịch vụ (Service-Based):**

- *Ưu điểm từ biểu đồ:* Sự cân bằng tốt với hầu hết chỉ số đạt 3–4 sao (maintainability: 4 sao, testability: 4 sao, deployability: 4 sao) và chi phí vừa phải (cost: \$\$).
- *Hạn chế trong bối cảnh dự án:* Việc tách ngay từ đầu toàn bộ các nghiệp vụ thư viện thành nhiều service riêng biệt (User Service, Catalog Service, Circulation Service, Fine Service) là **over-engineering** cho giai đoạn MVP. Các service này có phụ thuộc lẫn nhau cao: mượn sách cần kiểm tra tài khoản, tình trạng bản sao, đặt trước và phí phạt. Nếu tách quá sớm, hệ thống sẽ phải đánh đổi bằng nhiều inter-service calls, tăng network latency và độ phức tạp debugging.
- *Kết luận:* Service-Based là hướng đi tương lai hợp lý khi hệ thống cần scale, nhưng chưa phù hợp cho giai đoạn đầu.

D. Quyết định: Kiến trúc Modular Monolith kết hợp Dịch vụ vệ tinh AI

Dựa trên phân tích trên, nhóm quyết định áp dụng một cách tiếp cận **thực dụng và có định hướng phát triển**, kết hợp ưu điểm của Modular Monolith với tính linh hoạt của Service-Based Architecture:

1. Lõi hệ thống: Modular Monolith (Spring Boot)

Đây là kiến trúc chủ đạo cho các nghiệp vụ quản lý thư viện (Auth/User, Catalog, Circulation, Fine/Payment, Notification và Engagement).

Lý do lựa chọn dựa trên biểu đồ so sánh:

- **Cân bằng tối ưu:** Modular Monolith cung cấp maintainability (3 sao) và evolvability (3 sao) tốt hơn hẳn Layered Architecture (1 sao) nhưng vẫn giữ được chi phí thấp (cost: \$) và độ đơn giản cao (simplicity: 5 sao).
- **Testability:** Đạt 3 sao - cao hơn đáng kể so với Layered (2 sao) nhờ tách biệt rõ ràng giữa các domain module.
- **Deployability:** Dù chỉ đạt 3 sao (thấp hơn Microservices), nhưng lại phù hợp với nguồn lực dự án: chỉ cần deploy một artifact duy nhất, không cần orchestration phức tạp.

Lợi ích nghiệp vụ cụ thể:

- **Tính nhất quán giao dịch (ACID):** Các use case phức tạp như trả sách (cần atomic update: trạng thái giao dịch + tính phạt + cập nhật tồn kho + trigger hàng đợi) được đảm bảo trong một transaction duy nhất mà không cần distributed transaction protocols.
- **Tính module hóa cao:** Mặc dù là monolith, code được tổ chức theo **domain boundaries** (không phải technical layers). Mỗi module (Auth/User, Catalog, Circulation, Recommendation...) có domain model, use cases, và repositories riêng, giao tiếp thông qua well-defined interfaces. Điều này tạo "exit strategy" - dễ dàng tách module ra thành service riêng trong tương lai nếu cần.
- **DevOps đơn giản:** Chỉ cần một deployment pipeline, một database connection pool, không cần service mesh hay API gateway phức tạp.

2. Dịch vụ vệ tinh: AI Service (Python/FastAPI)

Mô-đun AI/Recommendation System được tách ra thành một dịch vụ độc lập, giao tiếp với Monolith chủ yếu qua REST API. Các tác vụ xử lý PDF được backend kích hoạt bất đồng bộ sau khi lưu URL tài liệu; AI service cũng có khả năng chạy worker nền cho các job xử lý dài nếu cần.

Lý do tách riêng:

- **Công nghệ khác biệt:** Hệ sinh thái Python (HuggingFace/SentenceTransformer, PyMuPDF, im-

plicit ALS, FastAPI) phù hợp hơn cho ML/NLP so với việc nhúng các thư viện này trực tiếp vào Spring Boot.

- **Scaling requirements khác biệt:** AI workload (PyMuPDF, vector similarity search, model inference) tiêu tốn CPU/GPU nhiều hơn CRUD nghiệp vụ. Tách riêng cho phép scale độc lập (horizontal scaling với GPU instances) mà không tốn tài nguyên cho monolith.
- **Bất đồng bộ tự nhiên:** Xử lý PDF, sinh embedding và gọi Gemini không cần chặn luồng thủ thư upload tài liệu. Backend lưu trạng thái ETL, gọi AI service bằng tác vụ nền và có fallback khi AI lỗi.
- **Tính module hóa:** AI Service dùng schema dữ liệu phục vụ AI như `ai_engine.publication_vectors`, `publication_etl_runs` và bảng `ai_recommendations`, nhưng giao tiếp với backend vẫn thông qua API contract rõ ràng. Nhờ đó nhóm có thể thay đổi model embedding hoặc chiến lược recommendation mà không làm xáo trộn nghiệp vụ thư viện.

Migration path: Đây là bước đệm hướng tới Service-Based Architecture. Trong tương lai, nếu cần scale, có thể tách dần các module khác (User Service, Notification Service) ra khỏi monolith theo pattern tương tự.

E. Đánh giá kiến trúc lại theo các chỉ số: Kiến trúc Modular Monolith + AI Satellite này đạt được sự cân bằng tốt nhất cho bối cảnh dự án:

Bảng 5.21: Đánh giá kiến trúc lại so với các chỉ số

Tiêu chí	Điểm	Nhận xét
Cost	\$\$	Giữa Modular Monolith (\$) và Service-Based (\$\$), phù hợp ngân sách
Simplicity	3-4 sao	Đơn giản hơn Microservices nhưng cần quản lý 2 components
Maintainability	3-4 sao	Module hóa domain tốt, dễ sửa lỗi và refactor
Testability	3 sao	Domain logic tách biệt, dễ unit test
Deployability	2-3 sao	Deploy 2 artifacts (Monolith + AI), CI/CD đơn giản
Scalability	3 sao	AI service scale độc lập, monolith scale theo nhu cầu
Evolvability	4 sao	Có thể tách dần modules ra thành services khi cần

Kết luận: Kiến trúc Modular Monolith + AI Satellite Service đạt được ba mục tiêu then chốt: (1) Đảm bảo tính nhất quán giao dịch cho nghiệp vụ core, (2) Tối ưu hóa AI workload với công nghệ và scaling phù hợp, (3) Giữ độ phức tạp và chi phí ở mức chấp nhận được cho đề án, đồng thời tạo nền tảng để mở rộng thành Service-Based Architecture hoặc Microservice Architecture trong tương lai.

5.4.2 Thiết kế cơ sở dữ liệu

Thiết kế cơ sở dữ liệu là quá trình chuyển đổi từ mô hình phân tích yêu cầu và các thực thể đã được đặc tả ở các phần trước thành một cấu trúc lưu trữ logic và vật lý. Mục tiêu là xây dựng một nền tảng dữ liệu nhất quán, toàn vẹn, hiệu quả và có khả năng mở rộng để hỗ trợ toàn bộ nghiệp vụ của hệ thống, từ các giao dịch mượn trả cơ bản đến các tính năng AI phức tạp.

5.4.2.1 Lựa chọn Hệ Quản trị Cơ sở dữ liệu (HQTCSDL)

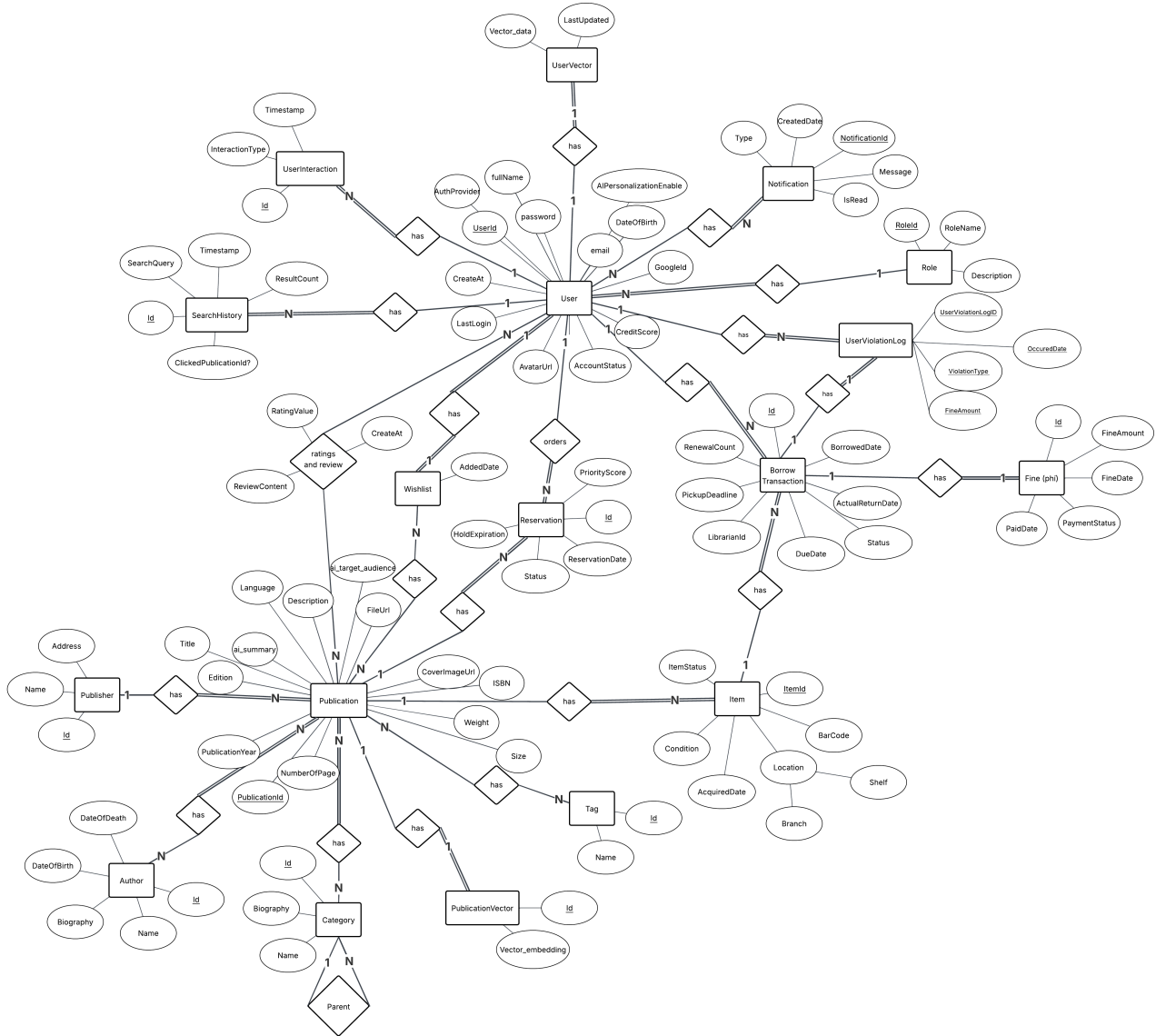
Dựa trên bản chất quan hệ chặt chẽ của dữ liệu (người dùng, sách, lượt mượn), hệ thống lựa chọn **PostgreSQL** làm HQTCSDL chính.

- **Tính tin cậy và toàn vẹn dữ liệu:** PostgreSQL nổi tiếng với sự tuân thủ nghiêm ngặt các tiêu chuẩn ACID (Atomicity, Consistency, Isolation, Durability), đảm bảo các giao dịch nghiệp vụ quan trọng như mượn, trả, và tính phí phạt luôn được thực hiện một cách an toàn.

- **Hỗ trợ các kiểu dữ liệu nâng cao:** PostgreSQL hỗ trợ các kiểu dữ liệu phức tạp như JSONB, rất hữu ích để lưu trữ các metadata linh hoạt. Đặc biệt, thông qua các extension như `pgvector`, nó có khả năng lưu trữ và truy vấn các vector embedding hiệu quả, phục vụ trực tiếp cho tính năng tìm kiếm ngữ nghĩa.
- **Khả năng mở rộng:** Đây là một hệ quản trị CSDL mã nguồn mở mạnh mẽ, đã được chứng minh về hiệu năng trong các hệ thống quy mô lớn.

5.4.2.2 Mô hình Quan hệ Thực thể (ERD)

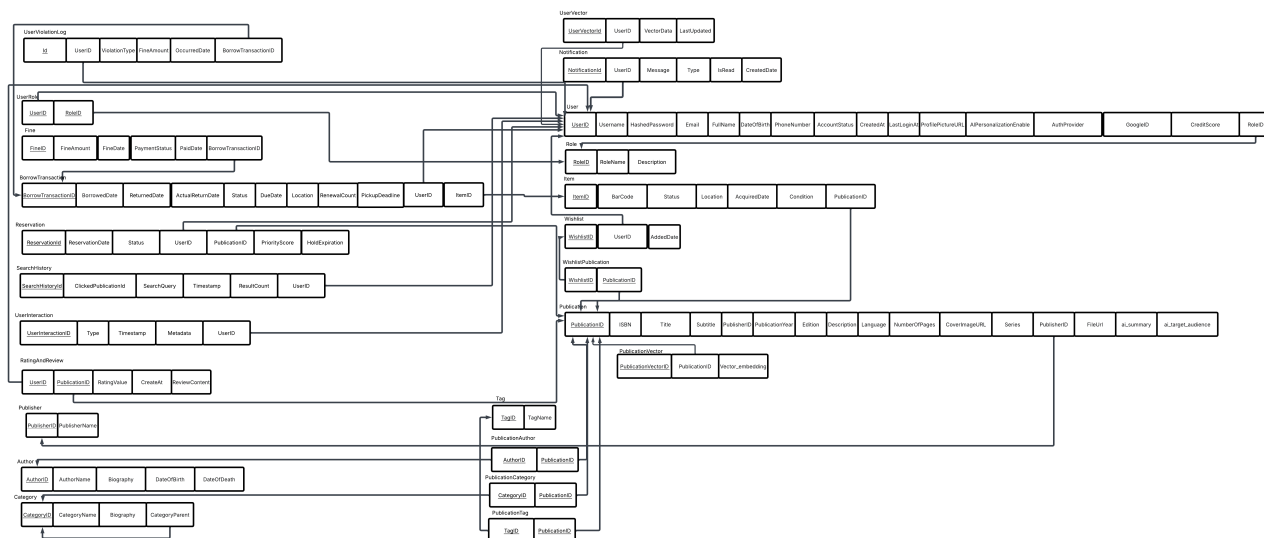
Bên dưới là sơ đồ ERD tổng quan của toàn bộ hệ thống.



Hình 5.16: Sơ đồ Quan hệ Thực thể (ERD) tổng thể của hệ thống

5.4.2.3 Ánh xạ lược đồ cơ sở dữ liệu quan hệ

Sau khi thực hiện quá trình ánh xạ từ mô hình ERD ở trên, ta thu được một lược đồ cơ sở dữ liệu quan hệ như trình bày bên dưới. Trong lược đồ này, các thuộc tính chi tiết đã được lược bớt nhằm tập trung làm rõ mối quan hệ giữa các bảng trong hệ thống.



Hình 5.17: Ảnh xạ cơ sở dữ liệu quan hệ ở dạng tổng quát

5.4.3 Thiết kế chi tiết bên trong

5.4.3.1 Mô hình kiến trúc chi tiết

Sau khi quyết định sử dụng kiến trúc Modular Monolith kết hợp AI Satellite Service, phần này trình bày chi tiết cấu trúc nội bộ của từng thành phần và các nguyên tắc thiết kế được áp dụng.

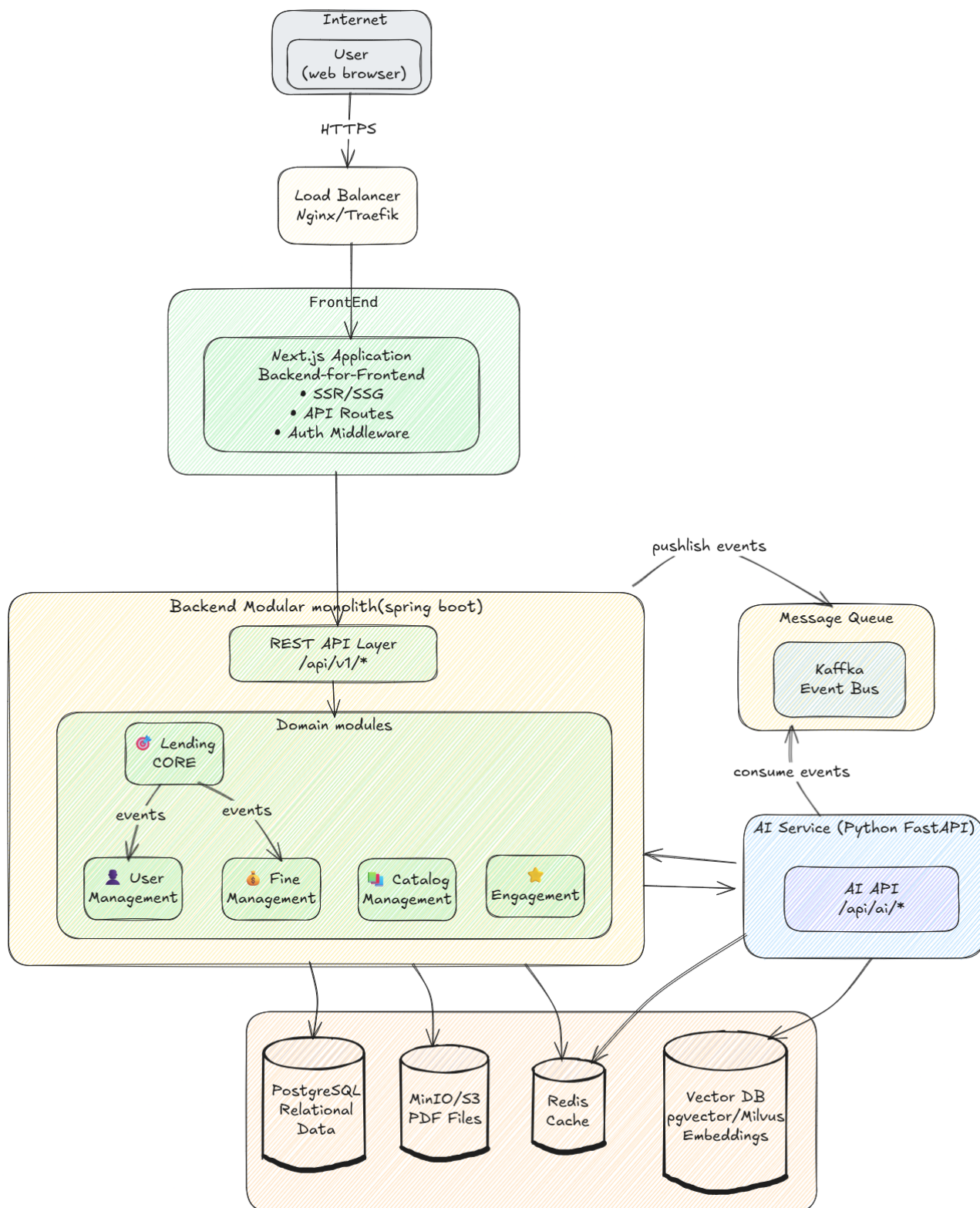
A. Tổng quan kiến trúc hệ thống: Hệ thống bao gồm ba thành phần chính:

- Frontend (React/Vite SPA):** Giao diện người dùng chạy phía trình duyệt, tổ chức route cho public, reader và librarian, gọi REST API trực tiếp đến backend.
- Backend Monolith (Spring Boot):** Lõi nghiệp vụ áp dụng Domain-Driven Design và Clean Architecture.
- AI Service (Python FastAPI):** Dịch vụ độc lập xử lý các tác vụ Machine Learning và Vector Search.

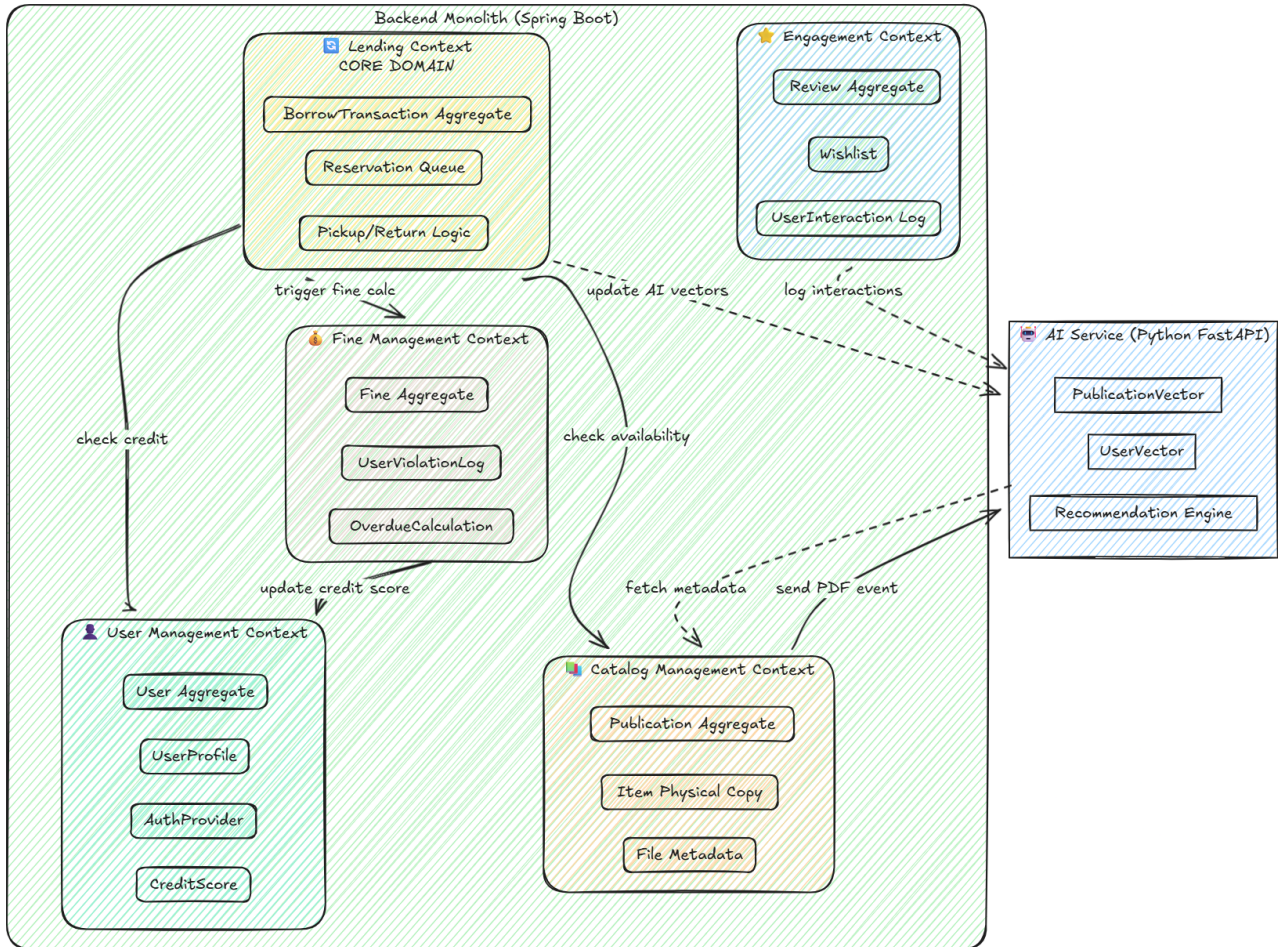
B.1). Kiến trúc Backend Monolith (Spring Boot) Tổ chức theo Bounded Contexts (DDD):

Thay vì tổ chức code theo các lớp kỹ thuật ngang (technical layers), hệ thống được chia thành các module theo ranh giới nghiệp vụ (domain boundaries), mỗi module tương ứng với một Bounded Context trong Domain-Driven Design:

- Auth/User Management Context:** Quản lý đăng ký, đăng nhập, xác thực email, Google OAuth, hồ sơ người dùng, vai trò và phân quyền.
- Catalog Management Context:** Quản lý thông tin xuất bản phẩm (Publication), các bản sao vật lý (Item), metadata sách, và file storage.
- Circulation Context: (Core Domain)** - Quản lý vòng đời mượn/trả sách (BorrowingTransaction), yêu cầu mượn, xác nhận nhận sách, hàng đợi đặt trước (Reservation), trả sách và báo cáo sách hư/mất.
- Fine/Payment Context:** Quản lý phí quá hạn, phí sự cố, thanh toán tiền mặt, thanh toán PayOS và đồng bộ trạng thái webhook.
- Engagement/Recommendation Context:** Quản lý đánh giá sách, phản hồi review, danh sách yêu thích, lịch sử tìm kiếm, log tương tác người dùng, semantic search và recommendation.
- Notification Context:** Lưu thông báo, đếm thông báo chưa đọc và đẩy realtime qua WebSocket/STOMP.



Hình 5.18: Sơ đồ kiến trúc tổng thể hệ thống



Hình 5.19: Các Bounded Contexts trong hệ thống

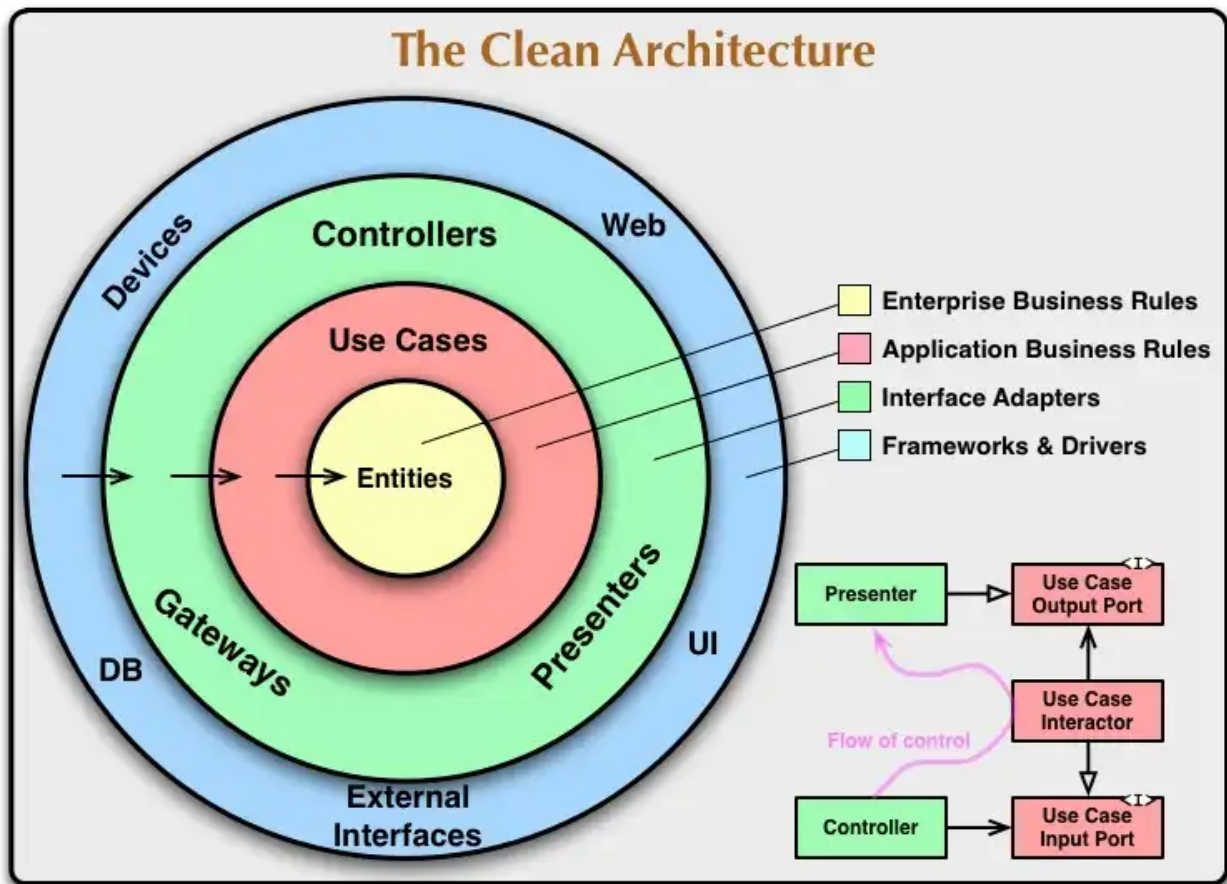
Mỗi Bounded Context được thiết kế như một "mini-service" bên trong monolith, có thể tách ra thành service độc lập trong tương lai mà không cần refactor lớn.

Clean Architecture bên trong mỗi Module:

Bên trong mỗi Bounded Context, code được tổ chức theo 4 layers của Clean Architecture để đảm bảo tách biệt trách nhiệm và independence từ framework/database:

1. Domain Layer (Lõi nghiệp vụ - Innermost):

- **Entities và Aggregates:** Chứa các đối tượng nghiệp vụ cốt lõi với business invariants. Ví dụ:
 - BorrowingTransaction Aggregate: Đảm bảo vòng đời giao dịch từ yêu cầu mượn, xác nhận nhận sách, đang mượn, trả sách đến báo cáo sự cố.
 - Publication Aggregate: Đảm bảo metadata ấn phẩm nhất quán, liên kết đúng tác giả, nhà xuất bản, danh mục, tag, ảnh bìa và tài liệu PDF.
 - Item Aggregate: Đảm bảo mỗi bản sao có barcode riêng, trạng thái riêng và chỉ được mượn khi đang khả dụng.
- **Value Objects:** Các đối tượng bất biến đại diện cho khái niệm nghiệp vụ không có identity riêng, ví dụ Barcode, PublicationId, ItemId hoặc trạng thái bản sao.
- **Domain Services:** Logic nghiệp vụ không thuộc về một Entity/Aggregate cụ thể. Ví dụ:



Hinh 5.20: Clean architecture (R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Pearson Education, 2018 [2])

- ReturnBookUseCase: Tính số ngày quá hạn và tạo phí phạt khi trả sách trễ.
- ReservationAssignmentService: Gán bản sao phù hợp cho yêu cầu đặt trước hoặc nhận sách.
- **Domain Events:** Biểu diễn các sự kiện nghiệp vụ quan trọng để các module khác có thể phản ứng (loose coupling):
 - Sự kiện mượn/trả, phát sinh phí phạt, thay đổi trạng thái đặt trước.
 - Sự kiện gửi thông báo và email qua Kafka.
 - Sự kiện cập nhật trạng thái xử lý AI sau khi PDF được phân tích.
- **Repository Interfaces:** Domain định nghĩa các interface (contract) mà Infrastructure sẽ implement. Domain không quan tâm dùng JPA, JDBC hay MongoDB.
- **Ràng buộc nghiêm ngặt:** Domain Layer KHÔNG ĐƯỢC phụ thuộc vào bất kỳ layer nào khác, không import Spring annotations, không biết HTTP hay Database. Đây là "business logic in pure Java".

2. Application Layer (Use Cases - Orchestrator):

- **Use Case Services:** Mỗi use case là một entry point cho một chức năng nghiệp vụ hoàn chỉnh. Ví dụ:
 - BorrowRequestUseCase: Điều phối luồng gửi yêu cầu mượn từ bạn đọc.
 - BorrowDirectUseCase: Điều phối luồng thủ thư cho mượn trực tiếp tại quầy.

```

1  lending/
2  |
3  |—— domain/
4  |   |—— aggregates/
5  |   |—— entities/
6  |   |—— valueobjects/
7  |   |—— services/
8  |   |—— events/
9  |   |—— repositories/
10 |
11 |—— application/
12 |   |—— usecases/
13 |   |—— services/
14 |   |—— dto/
15 |   |—— commands/
16 |   |—— queries/
17 |
18 |—— infrastructure/
19 |   |—— persistence/
20 |   |   |—— jpa/
21 |   |   |   |—— entities/
22 |   |   |   |—— repositories/
23 |   |   |   |—— mappers/
24 |   |   |—— jooq/
25 |   |   |   |—— repositories/
26 |   |   |   |—— mappers/
27 |   |   |   |—— generated/
28 |   |—— messaging/
29 |   |—— adapters/
30 |   |—— scheduling/
31 |   |—— config/
32 |
33 |—— presentation/
34 |   |—— rest/
35 |   |   |—— controllers/
36 |   |   |—— dto/
37 |   |   |   |—— request/
38 |   |   |   |—— response/
39 |   |   |—— mappers/
40 |   |   |—— exception/

```

Hình 5.21: Cấu trúc Clean Architecture kết hợp Domain-Driven Design trong một Module

- `ConfirmPickupUseCase`: Xác nhận bạn đọc đã nhận sách sau khi có yêu cầu mượn hoặc đặt trước.
- `ReturnBookUseCase`: Điều phối luồng trả sách, cập nhật bản sao và tính phí quá hạn nếu có.
- `ReportIssueUseCase`: Ghi nhận sách hư/mất và tạo phí sự cố.

- **Trách nhiệm chính:**

- Orchestrate workflow: Gọi repositories để load aggregates, gọi domain services, và save kết quả.
- Quản lý transaction boundaries (sử dụng `@Transactional` annotation của Spring).
- Kích hoạt các side effect cần thiết như notification, email hoặc cập nhật trạng thái AI thông qua adapter tương ứng.
- Transform exceptions: Chuyển domain exceptions thành application-level error codes.

- **Application Services:** Các facade cho phép module khác gọi vào (public API giữa các modules).

- **DTOs (Data Transfer Objects):** Chuyển đổi giữa domain objects và external representations (JSON, DTO cho API).

- **Lưu ý:** Application Layer được phép phụ thuộc vào Domain Layer và định nghĩa interfaces cho Infrastructure (Dependency Inversion Principle).

3. Infrastructure Layer (Technical Implementation - Outermost):

- **Repository Implementations:** Hiện thực các repository interfaces từ Domain bằng JPA/Hibernate, JDBC, hoặc bất kỳ persistence technology nào. Ví dụ:

- `BorrowingTransactionJpaRepository`: Truy vấn và cập nhật giao dịch mượn-trả.
- Mapping giữa domain/use case và JPA Entity như `BorrowingTransactionEntity`, `FineEntity`, `PublicationEntity`.

- **External Service Adapters:** Tích hợp với các dịch vụ bên ngoài:

- Email/SMS Service (gửi notification)
- File Storage (S3-compatible storage để lưu ảnh bìa và PDF)
- Payment Gateway (PayOS cho thanh toán phạt trực tuyến)
- OAuth Provider (Google Authentication)

- **Event Infrastructure:** Hiện thực event publishing/subscribing:

- Kafka producer/consumer cho notification, email và các tác vụ cần tách khỏi request chính.
- REST client đến AI Service cho semantic search, recommendation và xử lý PDF.

- **Technical Services:**

- Scheduler (Spring `@Scheduled`) cho background jobs (check overdue daily).
- WebSocket/STOMP cho thông báo realtime.
- Security (JWT token generation/validation, password hashing).

- **Configuration:** Database connection pools, Kafka/WebSocket configuration, S3, PayOS, AI gateway, security settings.

4. Presentation Layer (Interface Adapters):

- **REST Controllers:** Expose các use cases qua HTTP/REST API. Mỗi Bounded Context có controller riêng:

- UserController (/api/v1/users/*)
- PublicationController (/api/v1/publications/*)
- BorrowingTransactionController (/api/v1/transactions/*)
- ReservationController (/api/v1/reservations/*)
- FineController (/api/v1/fines/*)
- RecommendationController (/api/v1/recommendations/*)
- AiSearchController (/api/v1/ai/semantic-search)

- **Trách nhiệm:**

- Nhận HTTP requests, validate syntax (JSON format, required fields, regex).
- Chuyển đổi Request DTOs → Commands/Queries cho Application Layer.
- Authentication & Authorization: Extract user info từ JWT, check permissions.
- Chuyển đổi kết quả từ Application Layer → HTTP Response (JSON).
- Error handling: Chuyển exceptions thành HTTP status codes (400, 401, 403, 404, 500).

- **Nguyên tắc:** Controllers phải "thin" - chỉ làm việc chuyển đổi và delegation, KHÔNG chứa business logic.

Dependency Rule (Nguyên tắc phụ thuộc): Clean Architecture tuân thủ nguyên tắc: **Dependencies chỉ được point inward** (từ ngoài vào trong):

Presentation → Application → Domain ← Infrastructure

- Domain Layer: Không phụ thuộc vào ai (pure business logic).
- Application Layer: Chỉ phụ thuộc Domain, định nghĩa interfaces cho Infrastructure.
- Infrastructure & Presentation: Phụ thuộc cả Domain và Application, implement interfaces.

Điều này đảm bảo business logic (Domain) không bị "ô nhiễm" bởi framework, database hay HTTP concerns.

B. Kiến trúc AI Service (Python FastAPI) Hệ thống AI Service được thiết kế như một *Dịch vụ Vệ tinh Độc lập* (AI Satellite Service), tách biệt khỏi khối Monolith (Spring Boot). Quyết định này nhằm tận dụng sức mạnh của hệ sinh thái Python trong các tác vụ Học máy (Machine Learning) và Xử lý ngôn ngữ tự nhiên (NLP), đồng thời đảm bảo khả năng mở rộng độc lập cho các tác vụ tính toán nặng. **B.1. Chiến lược thiết kế và Lựa chọn công nghệ** Khác với Backend chính tập trung vào tính toàn vẹn dữ liệu giao dịch, AI Service tập trung vào hiệu năng tính toán ma trận và xử lý bất đồng bộ.

- **Web Framework - FastAPI:** Được lựa chọn thay vì Flask/Django vì nhẹ, dễ định nghĩa API contract, hỗ trợ Pydantic để kiểm soát dữ liệu đầu vào và phù hợp với các endpoint inference như semantic search, recommendation và publication processing.
- **Kiến trúc Phi trạng thái (Stateless):** AI Service không lưu trữ session người dùng. Mọi trạng thái cần thiết được truyền qua payload của request hoặc truy xuất từ PostgreSQL/model artifact, giúp dễ dàng mở rộng theo chiều ngang (Horizontal Scaling) bằng cách tăng số lượng container.

B.2. Cấu trúc Phân tầng (Layered Architecture)

Kiến trúc nội bộ của AI Service được tinh gọn thành 3 tầng chính để tối ưu hóa đường đi của dữ liệu:

a. API Layer (Giao tiếp & Định tuyến):

- *Semantic Search Router*: POST /api/v1/semantic-search
 - Xử lý truy vấn tìm kiếm bằng ngôn ngữ tự nhiên.
 - Pipeline: Tiền xử lý text → Encoding → Vector Search.
- *Recommendation Router*: POST /api/v1/recommendations
 - Trả về danh sách gợi ý cá nhân hóa dựa trên ngữ cảnh (Trang chủ, Chi tiết sách).
- *Data Ingestion Router*: POST /api/v1/publications/process
 - Endpoint nhận yêu cầu xử lý tài liệu mới, trích xuất text PDF, sinh metadata AI và lưu embedding.

b. Service Layer (Logic nghiệp vụ ML): Đây là trái tim của hệ thống, chứa các pipeline xử lý dữ liệu phức tạp:

- **Embedding Service:** Sử dụng mô hình `bkai-foundation-models/vietnamese-bi-encoder` hoặc mô hình tương đương để chuyển đổi văn bản tiếng Việt thành vector 768 chiều. Mô hình được load một lần khi service/worker khởi động để giảm độ trễ suy luận.
- **PDF Processing Pipeline:**
 - *Extraction*: Sử dụng **PyMuPDF** để trích xuất văn bản tốc độ cao (nhanh hơn 10x so với `pypdf`).
 - *Chunking*: Áp dụng chiến lược Cửa sổ trượt theo ký tự với kích thước chunk 1500 và overlap 150 để bảo toàn ngữ cảnh giữa các đoạn cắt.
 - *Metadata Reduce*: Chọn các chunk đại diện, gửi một prompt chính đến Gemini để tạo summary, audience và tag; nếu Gemini lỗi, hệ thống dùng fallback deterministic.
- **Recommendation Engine:** Sử dụng thư viện **implicit** để huấn luyện mô hình ALS cho dữ liệu phản hồi ẩn, kết hợp thêm content-based recommendation và trending fallback cho người dùng mới hoặc dữ liệu ít.

c. Data & Storage Layer:

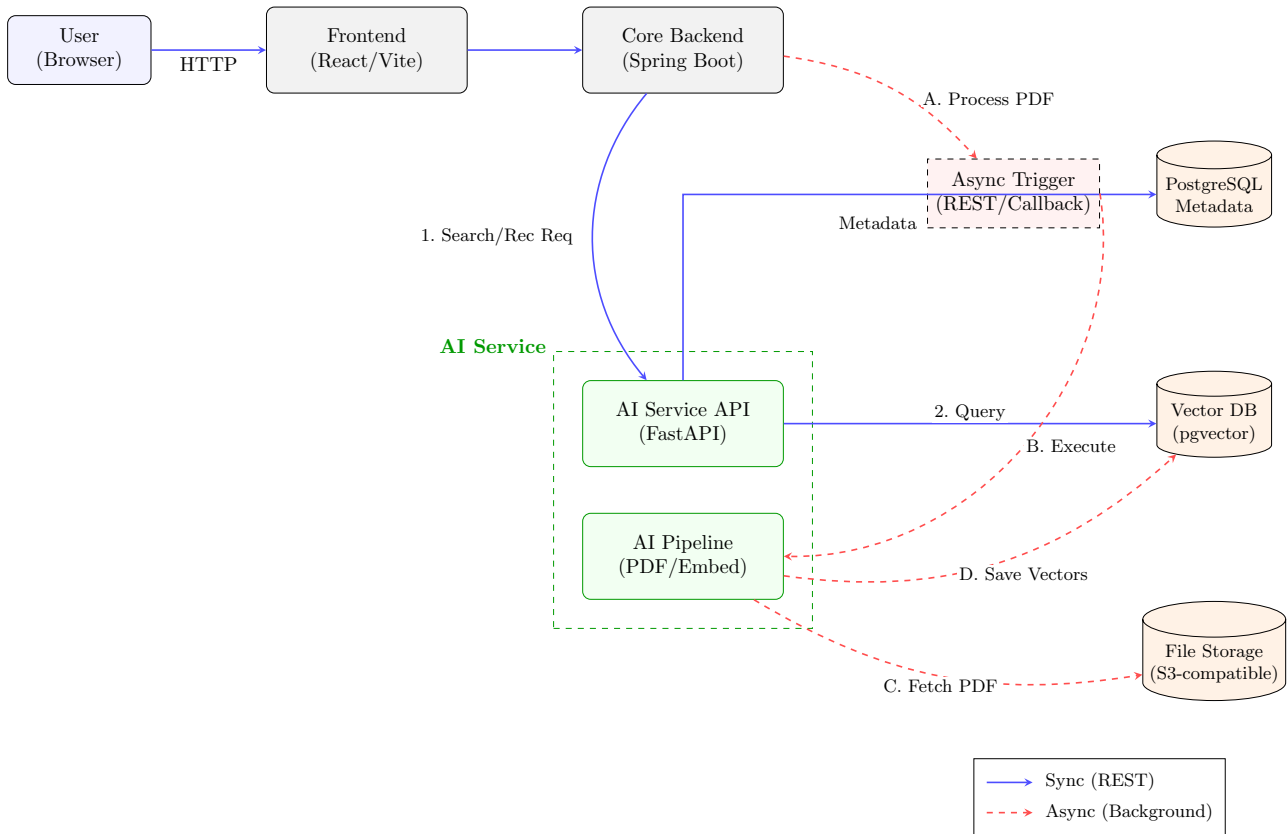
- **Vector Database (PostgreSQL + pgvector):** Lưu trữ embeddings của các chunk nội dung PDF. Sử dụng chỉ mục **HNSW** (Hierarchical Navigable Small World) để tối ưu hóa tốc độ tìm kiếm láng giềng gần nhất (ANN) thay vì quét toàn bộ bảng.
- **Model Store:** Các mô hình nặng (SBERT) được tải sẵn vào RAM khi khởi động (Pre-loaded) để giảm độ trễ (Latency).
- **Recommendation Store:** Mô hình ALS sau khi huấn luyện được lưu dưới dạng artifact `.pkl`; kết quả gợi ý gần nhất có thể được lưu lại trong PostgreSQL để backend có dữ liệu fallback khi AI service tạm thời không phản hồi.

Cấu trúc thư mục của AI service

5.4.3.2 Luồng giao tiếp (Communication Flows) nâng cao

Hệ thống sử dụng hai pattern giao tiếp chính: **Synchronous Communication** (REST API) cho các tương tác cần phản hồi ngay, và **Asynchronous Communication** (Kafka, WebSocket, executor nền và callback) cho các tác vụ phụ hoặc tác vụ AI không nên chặn luồng nghiệp vụ chính.

1. Giao tiếp đồng bộ (Synchronous Communication - REST API):



Hình 5.22: Sơ đồ luồng dữ liệu và Kiến trúc chi tiết AI Service

Được sử dụng khi:

- Frontend cần phản hồi ngay lập tức để hiển thị cho người dùng.
- Thao tác đọc dữ liệu (queries) không có side effects.
- Validation errors cần trả về trực tiếp cho người dùng.
- Cần đảm bảo transaction hoàn thành trước khi tiếp tục workflow.

Ví dụ 1: Semantic Search Flow Luồng tìm kiếm ngữ nghĩa yêu cầu phản hồi nhanh để UX tốt:

1. User nhập query "sách về machine learning" trên giao diện tìm kiếm.
2. React/Vite Frontend gửi POST `/api/v1/ai/semantic-search` đến Backend Monolith với payload JSON chứa query string và limit.
3. Backend (`AiSearchController/AiGatewayService`) xử lý:
 - Validate JWT token và extract user info.
 - Validate query và giới hạn số lượng kết quả.
 - Chuyển tiếp query sang AI Service qua `AiGatewayService`; lịch sử tìm kiếm được ghi nhận ở luồng frontend/backend riêng khi người dùng thực hiện tìm kiếm.
 - Forward request đến AI Service qua internal network.
4. Backend gọi POST `http://ai-service:8000/api/v1/semantic-search` (internal URL, không expose ra internet).
5. AI Service xử lý:

```

1  ai-service/
2  └─ app/
3      └─ api/                                # API Layer (Giao tiếp)
4          ├── __init__.py
5          ├── router.py                      # Định nghĩa các route chính
6          └─ v1/
7              ├── endpoints/
8                  ├── search.py              # Endpoint: /semantic-search
9                  ├── recommend.py           # Endpoint: /recommendations
10                 ├── ingest.py              # Endpoint: /process-document
11                 └─ dependencies.py          # Auth dependencies (API Key/JWT)
12
13     └─ core/                                # Core Layer (Cấu hình & Setting)
14         ├── config.py                      # Load biến môi trường (DB URL, Redis URL)
15         ├── logging.py                     # Cấu hình structured logging
16         └─ exceptions.py                   # Custom exception handlers
17
18     └─ db/                                  # Data Layer (Kết nối CSDL)
19         ├── session.py                     # PostgreSQL/pgvector session management
20         └─ redis.py                         # Redis cache connection
21
22     └─ schemas/                             # Pydantic Models (Data Validation)
23         ├── search_schema.py               # Input/Output cho tìm kiếm
24         ├── book_schema.py                 # Metadata sách
25         └─ user_schema.py                  # User interaction payload
26
27     └─ services/                             # Service Layer (Logic nghiệp vụ ML cốt lõi)
28         ├── __init__.py
29         ├── embedding_service.py           # Wrapper cho mô hình SBERT (bkai-foundation)
30         ├── ocr_service.py                 # Wrapper cho PyMuPDF & Tesseract
31         ├── chunking_service.py            # Logic cắt văn bản (Sliding window)
32         ├── vector_search.py               # Logic truy vấn HNSW với pgvector
33         └─ recommender/
34             ├── collaborative.py           # Thuật toán ALS (LightFM)
35             ├── content_based.py           # Cosine similarity logic
36             └─ hybrid.py                   # Logic Switching Hybrid
37
38     └─ workers/                             # Background Tasks (Celery)
39         ├── celery_app.py                  # Cấu hình Celery instance
40         └─ tasks.py                         # Định nghĩa task: process_pdf_task, retrain_model_task
41
42     └─ models_store/                         # Nơi chứa file model (nếu không load từ HuggingFace)
43         ├── sbert/
44         └─ lightfm_model.pkl                # Model gợi ý đã huấn luyện
45
46     └─ tests/                               # Unit & Integration Tests
47         ├── test_api.py
48         ├── test_embedding.py
49         └─ test_ocr.py
50
51     └─ .env                                 # Biến môi trường (không commit lên git)
52     └─ .gitignore
53     └─ docker-compose.yml                   # Setup dev env (Redis, Postgres)
54     └─ Dockerfile                           # Build image cho AI service
55     └─ main.py                              # Entry point (Khởi chạy Uvicorn)
56     └─ poetry.lock / requirements.txt        # Quản lý thư viện (torch, fastapi, pymupdf...)
57     └─ README.md
58

```

Hình 5.23: Kiến trúc chi tiết AI Service và Luồng dữ liệu

- Embedding Service chuyển query thành vector 768 chiều.
 - Vector Search Service thực hiện ANN search trên bảng vector chunk bằng pgvector.
 - Gom các chunk gần nhất về danh sách publication IDs với similarity scores.
6. Backend nhận response từ AI Service:
- Kiểm tra lỗi/timeout từ AI service và fallback sang tìm kiếm catalog nếu cần.
 - Log request (userId, query, resultCount, responseTime) để phân tích.
7. Backend query PostgreSQL để lấy metadata chi tiết (title, author, cover, availability) của các publication IDs.
8. Backend merge AI results + metadata, trả về Frontend.
9. Frontend render kết quả tìm kiếm.

Đặc điểm kỹ thuật:

- **Request-Response pattern:** Client chờ response trước khi tiếp tục (blocking call).
- **Timeout handling:** Backend cấu hình timeout khi gọi AI service để tránh làm treo request người dùng.
- **Fallback:** Nếu AI service lỗi, hệ thống vẫn có thể trả kết quả từ tìm kiếm keyword/catalog truyền thống.
- **Tách trách nhiệm:** AI service chỉ trả về ranking theo ngữ nghĩa; backend chịu trách nhiệm lấy metadata, tình trạng bản sao và quyền truy cập.

2. Giao tiếp bất đồng bộ - Sự kiện nội bộ và Kafka/WebSocket:

Backend sử dụng các cơ chế bất đồng bộ cho những tác vụ không nên làm chậm request chính, đặc biệt là gửi email, tạo thông báo, đẩy realtime và xử lý tác vụ nền.

Đặc điểm:

- Kafka được dùng để tách các tác vụ notification/email khỏi transaction nghiệp vụ chính.
- WebSocket/STOMP được dùng để đẩy thông báo realtime đến frontend sau khi thông báo được tạo.
- Các tác vụ nền được xử lý bằng executor/scheduler khi cần, ví dụ gọi AI sau khi lưu PDF hoặc kiểm tra trạng thái nghiệp vụ định kỳ.

Ví dụ 3: Quy trình Trả sách và Tính toán Phạt (Return Book with Fine Calculation)

Khi người dùng trả sách, hàng loạt các tác vụ phụ (side effects) cần được thực hiện. Tuy nhiên, để đảm bảo trải nghiệm người dùng, các tác vụ này không được phép làm chặn (block) phản hồi của API chính:

1. Thủ thư thực hiện trả sách thông qua API POST `/api/v1/transactions/return`.
2. ReturnBookUseCase bên trong **Mô-đun Circulation** thực hiện:
 - Tải BorrowingTransaction đang hoạt động từ cơ sở dữ liệu.
 - Kiểm tra bản sao tương ứng với barcode hoặc item id.
 - Cập nhật ngày trả và trạng thái giao dịch thành đã trả.
 - Tính toán trễ hạn dựa trên dueDate; nếu quá hạn, tạo bản ghi phí phạt.
 - Cập nhật trạng thái bản sao về khả dụng hoặc trạng thái phù hợp nếu có sự cố.
 - Gửi thông báo/email cho bạn đọc thông qua hạ tầng notification.

3. Giao dịch cơ sở dữ liệu hoàn tất, API trả kết quả cho thủ thư gồm trạng thái trả sách, ngày trả, số ngày quá hạn và phí phát sinh nếu có.
4. Các tác vụ phụ như gửi thông báo, email hoặc cập nhật dashboard được xử lý sau luồng chính để thao tác trả sách tại quầy không bị chậm.

Lợi ích của Internal Events:

- **Decoupling:** Circulation Module không phải tự xử lý mọi side effect như email, notification realtime hay thống kê.
- **Single Responsibility:** Mỗi module chỉ quan tâm đến domain logic của mình.
- **Extensibility:** Có thể bổ sung loại thông báo mới hoặc kênh gửi mới mà không phải viết lại nghiệp vụ trả sách.
- **Transaction safety:** Trạng thái giao dịch, bản sao và phí phạt được cập nhật trong luồng nghiệp vụ chính trước khi phát sinh side effect.
- **Testability:** Có thể kiểm thử use case trả sách độc lập với kênh gửi thông báo.

3. Giao tiếp bất đồng bộ với AI Service:

Với xử lý PDF, hệ thống không để request upload phải chờ toàn bộ pipeline AI hoàn tất. Backend lưu thông tin tài liệu, ghi trạng thái ETL và kích hoạt AI service bằng REST trong một tác vụ nền. AI service có thể xử lý trực tiếp qua FastAPI hoặc qua worker nền khi cần mở rộng, nhưng hợp đồng chính giữa backend và AI vẫn là API rõ ràng.

Đặc điểm kỹ thuật:

- **Không chặn luồng nghiệp vụ:** Thủ thư có thể hoàn tất thao tác tạo/cập nhật ấn phẩm trước khi AI xử lý xong PDF.
- **Theo dõi trạng thái:** Backend lưu trạng thái QUEUED, SUCCESS hoặc FAILED trong bảng ETL để giao diện có thể hiển thị tiến độ.
- **Fallback:** Nếu AI hoặc Gemini lỗi, ấn phẩm vẫn tồn tại trong catalog và hệ thống vẫn hỗ trợ tìm kiếm keyword.
- **Callback có xác thực:** Backend có endpoint callback được bảo vệ bằng chữ ký HMAC để nhận kết quả từ AI worker khi chạy theo kiểu worker/callback.

Ví dụ 4: Quy trình Thêm sách mới kèm Xử lý PDF (Add Book with PDF Processing)

Quá trình xử lý PDF, sinh embedding và gọi Gemini có thể tốn thời gian, do đó không nên sử dụng cơ chế chặn (blocking) trên request của thủ thư:

1. Thủ thư tạo ấn phẩm mới thông qua API POST `/api/v1/publications` với metadata như tiêu đề, ISBN, tác giả, nhà xuất bản, danh mục, tag, ngôn ngữ và năm xuất bản.
2. `CreatePublicationUseCase` và các use case upload tài liệu trong **Mô-đun Danh mục (Catalog Module)** thực hiện:
 - Kiểm tra hợp lệ metadata và các quan hệ với tác giả, nhà xuất bản, danh mục, tag.
 - Tạo đối tượng `Publication`.
 - Upload ảnh bìa nếu có thông qua endpoint `cover`.
 - Cấp `presigned URL` để frontend upload PDF lên S3-compatible storage.

- Lưu dữ liệu vào PostgreSQL.
 - Khi frontend gửi document URL về backend, hệ thống ghi trạng thái ETL QUEUED và gọi AI service xử lý tài liệu.
3. API trả về mã HTTP 201 Created hoặc kết quả cập nhật document URL ngay sau khi metadata/PDF URL được lưu, không chờ toàn bộ AI pipeline.
4. Thủ thư nhận trạng thái "đang xử lý AI" và có thể tiếp tục thực hiện công việc khác.
5. **Dịch vụ AI** nhận yêu cầu xử lý POST `/api/v1/publications/process`:
- Nhận `publicationId`, `pdfUrl` và cờ `forceReprocess`.
 - Tải PDF từ S3-compatible storage.
 - Trích xuất văn bản bằng PyMuPDF và làm sạch header/footer, số trang, ký tự lỗi.
 - Chia văn bản thành các chunk 1500 ký tự, overlap 150 ký tự.
 - Tạo embedding 768 chiều cho từng chunk và lưu vào `ai_engine.publication_vectors`.
 - Chọn các chunk đại diện, gọi Gemini để tạo summary, audience và tag; nếu lỗi thì dùng fallback deterministic.
 - Cập nhật metadata AI và trạng thái ETL trong PostgreSQL.
6. Sau khi hoàn tất (hoặc thất bại sau tất cả lần thử):
- AI service trả kết quả cho backend hoặc cập nhật qua callback POST `/api/ai/callback` khi chạy theo mô hình worker.
 - Backend cập nhật trạng thái xử lý AI.
 - Nếu cần, hệ thống gửi thông báo cho thủ thư về kết quả xử lý.
7. Kết quả: Nếu xử lý thành công, sách xuất hiện trong kết quả tìm kiếm ngữ nghĩa. Nếu thất bại, sách vẫn có trong danh mục nhưng chỉ hỗ trợ tìm kiếm theo từ khóa truyền thống.

Đặc điểm của mẫu xử lý nền cho AI:

- **Xử lý bất đồng bộ (Async processing):** Backend không bị chặn (non-blocking), trả phản hồi ngay lập tức cho người dùng để tối ưu trải nghiệm.
- **Idempotent processing:** AI pipeline dùng `publicationId`, hash file và cờ `forceReprocess` để tránh xử lý trùng không cần thiết.
- **Khả năng phục hồi:** Khi PDF lỗi, Gemini rate-limit hoặc AI service tạm thời không phản hồi, hệ thống ghi trạng thái thất bại thay vì làm hỏng dữ liệu catalog.
- **Mở rộng từng bước:** Khi tải tăng, AI service có thể bổ sung worker/Celery để xử lý hàng đợi nội bộ mà không cần thay đổi nghiệp vụ frontend/backend.
- **Tách rủi ro AI khỏi nghiệp vụ chính:** Tạo sách, quản lý bản sao, mượn-trả và thanh toán không phụ thuộc tuyệt đối vào kết quả AI.

4. Chuỗi sự kiện lan truyền - Quy trình nghiệp vụ đa mô-đun phức tạp (Event Cascade)

Một số quy trình công việc (workflows) đòi hỏi sự kết hợp giữa cả gọi đồng bộ (sync calls) và sự kiện bất đồng bộ (async events) để đạt được sự cân bằng tối ưu giữa tính nhất quán dữ liệu (consistency) và hiệu năng (performance).

Ví dụ 5: Quy trình Mượn sách hoàn chỉnh (Borrow Book - Complete Workflow)

1. Giai đoạn 1: Kiểm tra hợp lệ Đồng bộ (Synchronous Validation)

- Bạn đọc nhấn "Mượn sách" → Frontend gửi yêu cầu `POST /api/v1/transactions/borrow`; nếu thủ thư cho mượn tại quầy, frontend dùng `POST /api/v1/transactions/borrow-direct`.
- `BorrowingTransactionController` kiểm tra JWT, vai trò và tính hợp lệ của request.
- Use case kiểm tra tài khoản người dùng, phí phạt còn tồn tại, số lượng sách đang mượn và tình trạng bản sao/ấn phẩm.
- Với yêu cầu từ bạn đọc, hệ thống tạo giao dịch chờ xác nhận nhận sách. Với mượn trực tiếp, hệ thống ghi nhận bản sao cụ thể và ngày đến hạn.
- Tạo `BorrowingTransaction`, lưu vào cơ sở dữ liệu, cập nhật trạng thái bản sao khi cần và cam kết giao dịch.
- API trả về mã `201 Created` (tổng thời gian: ~200ms).

2. Giai đoạn 2: Tác vụ phụ sau giao dịch

- Hệ thống tạo thông báo cho bạn đọc và/hoặc thủ thư tùy theo trạng thái giao dịch.
- Kafka/WebSocket giúp đẩy thông báo mà không làm chậm API chính.
- Mô-đun recommendation ghi nhận tương tác mượn để phục vụ mô hình gợi ý.
- Dashboard thủ thư phản ánh số lượng giao dịch mới, sách đang mượn và các yêu cầu cần xử lý.

3. Giai đoạn 3: Tác động đến AI Recommendation

- Dữ liệu tương tác mới được lưu trong PostgreSQL.
- Khi người dùng gọi API gợi ý, backend gọi AI service `POST /api/v1/recommendations`.
- AI service dùng ALS đã train, content-based và trending fallback để trả danh sách mới; mô hình ALS được refresh theo lịch hoặc khi gọi endpoint refresh.

Dòng thời gian xử lý (Timeline):

- **T+0ms:** Người dùng nhấn nút "Mượn sách".
- **T+200ms:** API phản hồi `201 Created` → Người dùng thấy thông báo "Mượn sách thành công".
- **T+300ms:** Thông báo và các tác vụ phụ được xử lý nền.
- **T+vài giây/phút:** Dữ liệu tương tác được AI service sử dụng trong lần suy luận hoặc refresh recommendation tiếp theo.

Lợi ích kiến trúc:

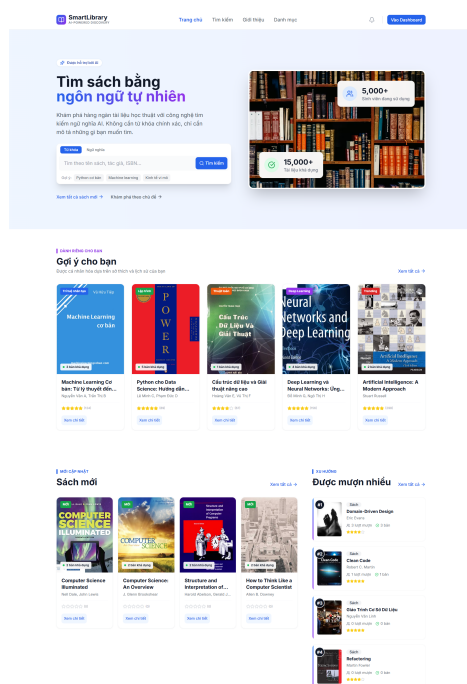
- **Phản hồi nhanh (Fast response):** Người dùng không phải chờ AI xử lý xong mới mượn được sách.
- **Tính nhất quán mạnh (Strong consistency):** Giao dịch cốt lõi (bản ghi mượn sách, kho hàng) đảm bảo nhất quán tuyệt đối nhờ ACID transaction.
- **Tính nhất quán cuối cùng (Eventual consistency):** Dữ liệu gợi ý có thể cập nhật sau nghiệp vụ mượn sách; đây là độ trễ chấp nhận được vì recommendation không quyết định tính đúng sai của giao dịch.
- **Khả năng phục hồi (Resilience):** Nếu Dịch vụ AI gặp sự cố, chức năng mượn sách cốt lõi vẫn hoạt động bình thường, hệ thống chỉ tạm thời dùng gợi ý fallback hoặc kết quả đã lưu trước đó.

5.4.4 Thiết kế giao diện

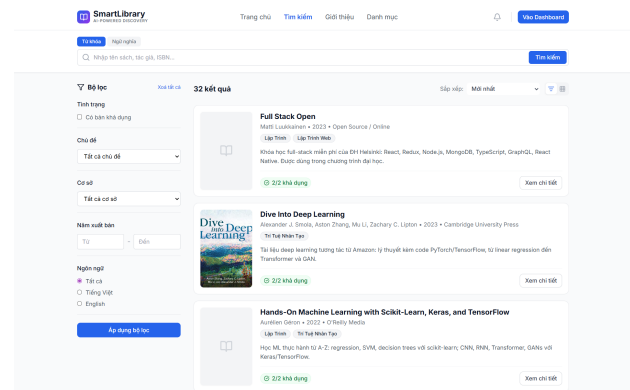
Toàn bộ giao diện hệ thống được thiết kế trên **Figma** trước khi hiện thực, đảm bảo sự thống nhất về phong cách (Design System) và luồng tương tác (User Flow) giữa các màn hình. Bản thiết kế đầy đủ có thể xem tại đường dẫn trong Phụ lục 10.

Hệ thống tuân thủ các nguyên tắc thiết kế cốt lõi: **tính nhất quán** (màu sắc, typography, hành vi component thống nhất toàn ứng dụng), **phản hồi trạng thái** (loading skeleton khi AI xử lý, toast notification sau thao tác), **tối giản** (nội dung trọng tâm là sách và thanh tìm kiếm), và **responsive** (Tailwind CSS Grid đảm bảo hiển thị tốt trên mọi thiết bị). Các component được xây dựng theo thiết kế Figma, kết hợp Tailwind CSS, lucide-react, sonner và một số thư viện nhỏ như react-select, react-day-picker để giữ giao diện linh hoạt nhưng vẫn nhất quán.

5.4.4.1 Giao diện Public (Trang chủ và Tìm kiếm)

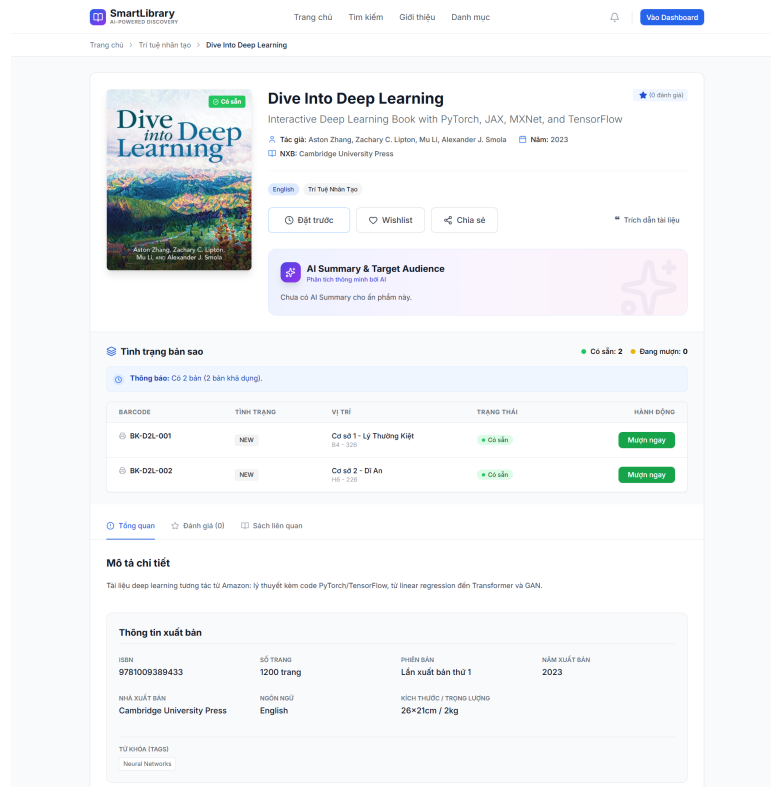


(a) Trang chủ



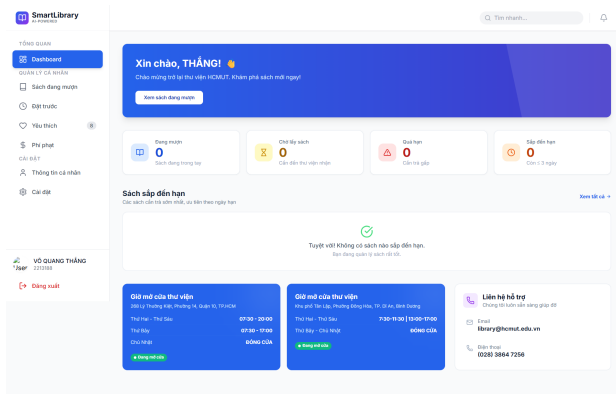
(b) Tìm kiếm sách với bộ lọc

Hình 5.24: Giao diện Public: Trang chủ và Tìm kiếm

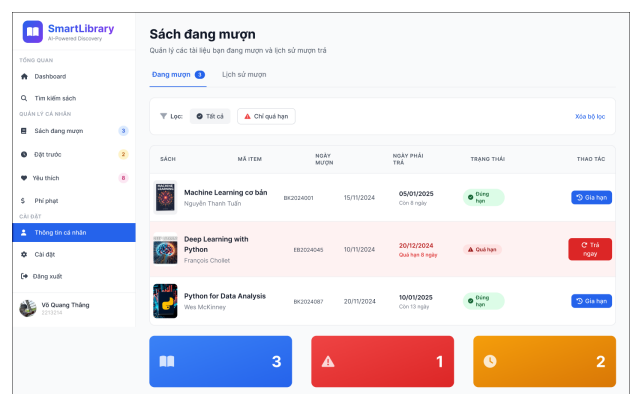


Hình 5.25: Giao diện Chi tiết sách

5.4.4.2 Giao diện Đọc giả (User Pages)



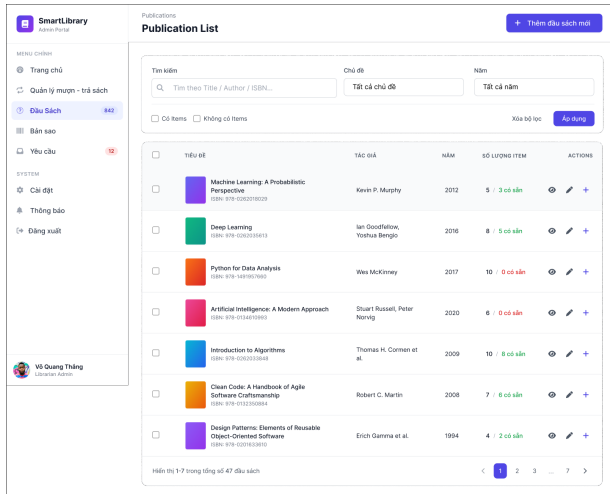
(a) Dashboard tổng quan



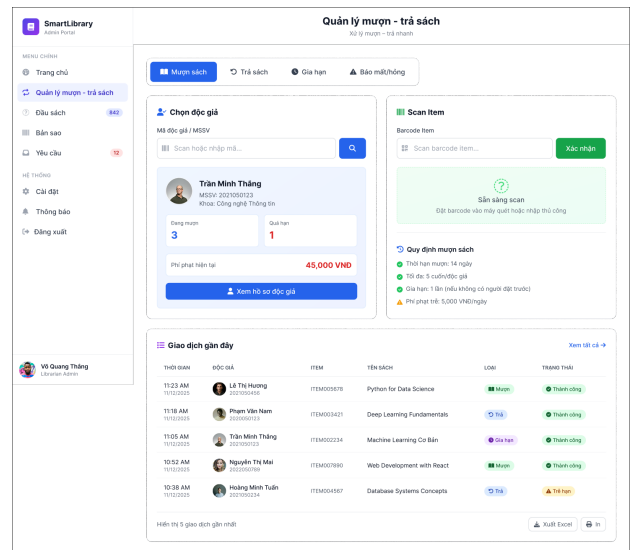
(b) Quản lý sách đang mượn

Hình 5.26: Giao diện Đọc giả: Dashboard và Quản lý mượn sách

5.4.4.3 Giao diện Thủ thư (Librarian Pages)



(a) Quản lý danh mục sách



(b) Ghi nhận mượn sách (Check-out)

Hình 5.27: Giao diện Thủ thư: Quản lý danh mục và Lưu thông

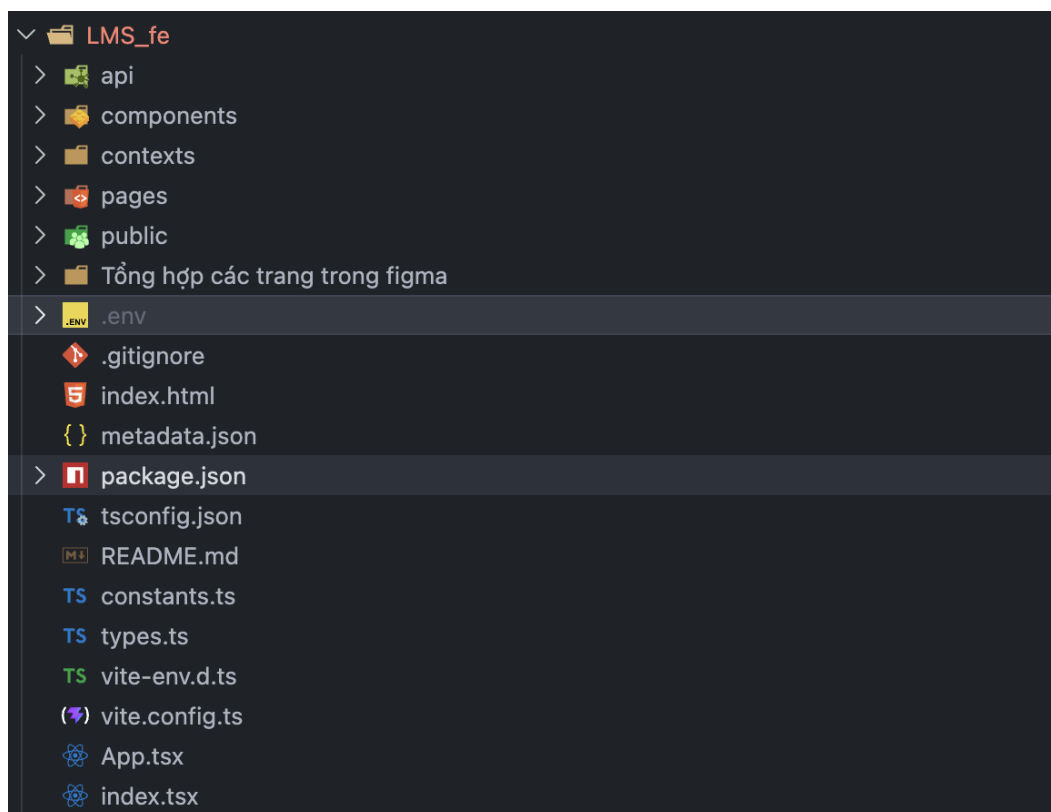
Chương 6: Hiện thực hệ thống

6.1 Hiện thực ứng dụng

Phần hiện thực được tổ chức theo ba lớp triển khai chính của hệ thống: frontend, backend và AI service. Mục này trình bày frontend, backend và AI service, tương ứng với lớp giao diện tiếp nhận thao tác từ độc giả/thủ thư/admin, lớp API xử lý nghiệp vụ và lớp AI phục vụ xử lý tài liệu, tìm kiếm ngữ nghĩa, gợi ý sách.

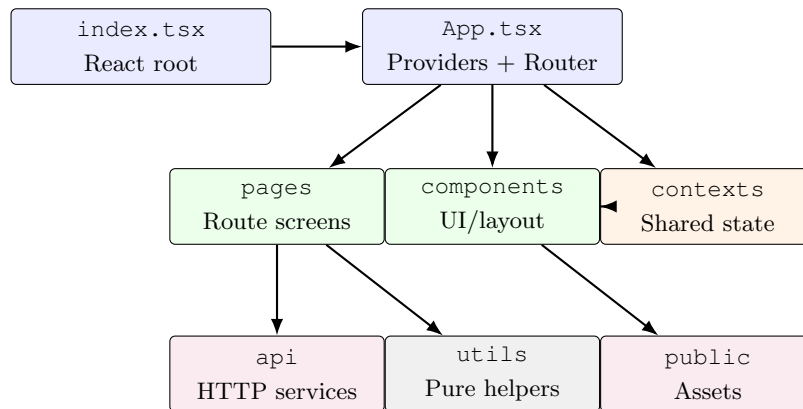
6.1.1 Hiện thực Frontend

Nhóm đã hiện thực frontend bằng React, TypeScript và Vite theo mô hình Single Page Application. Lớp giao diện không chỉ đảm nhiệm hiển thị dữ liệu, mà còn điều phối trạng thái đăng nhập, phân vùng giao diện theo vai trò, gọi API nghiệp vụ, nhận thông báo thời gian thực, hiển thị QR nhận sách và hỗ trợ thanh toán phí phạt qua PayOS. Mã nguồn được chia theo trách nhiệm: `pages` chứa màn hình nghiệp vụ, `components` chứa thành phần tái sử dụng, `contexts` chứa state dùng chung, `api` đóng gói HTTP service và `utils` chứa các hàm xử lý thuần.



Hình 6.1: Cấu trúc thư mục mã nguồn Frontend

6.1.1.1 Kiến trúc mã nguồn và cấu hình build



Hình 6.2: Quan hệ giữa các nhóm module Frontend

Bảng 6.1: Cấu hình triển khai Frontend

Thành phần	Hiện thực
Runtime	React 19, TypeScript, Vite.
Routing	react-router-dom với HashRouter, React.lazy và Suspense.
Network	axios, @stomp/stompjs, sockjs-client.
UI utilities	lucide-react, sonner, react-select, react-day-picker, react-qr-code.
Build chunk	vendor-react, vendor-ui, vendor-network, vendor-utils, vendor-misc.
Deploy	Docker hai giai đoạn: node:20-alpine build artifact, nginx:alpine phục vụ thư mục dist.

File vite.config.ts cấu hình dev server tại cổng 3000, alias @ về thư mục gốc frontend và tách bundle theo nhóm thư viện. File nginx-fe.conf dùng try_files để mọi route phía client được trả về index.html, phù hợp với SPA.

Nhóm đã áp dụng React.lazy và Suspense cho các page ở cấp route. Cách này làm cho bundle ban đầu không phải tải toàn bộ màn hình công khai, độc giả, thủ thư và admin cùng lúc. Khi người dùng truy cập một route, module tương ứng mới được tải về trình duyệt.

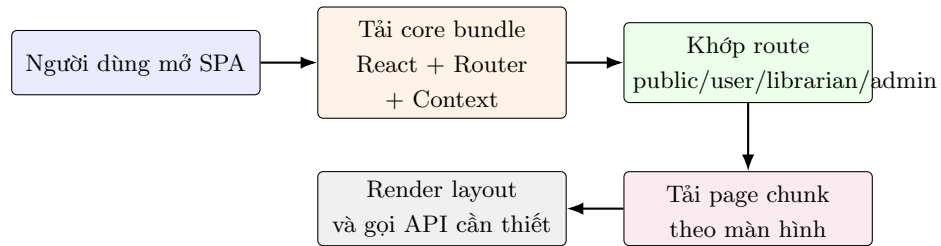
Listing 6.1: Tách màn hình theo route bằng lazy loading

```

const PublicSearchPage = lazy(() => import('./pages/public_pages/SearchPage'));
const DashboardPage = lazy(() => import('./pages/user_pages/DashboardPage'));
const Circulation = lazy(() => import('./pages/librarian_pages/Circulation'));

function App() {
  return (
    <Router>
      <Suspense fallback={<PageFallback />}>
        <AppContent />
      </Suspense>
    </Router>
  );
}
  
```

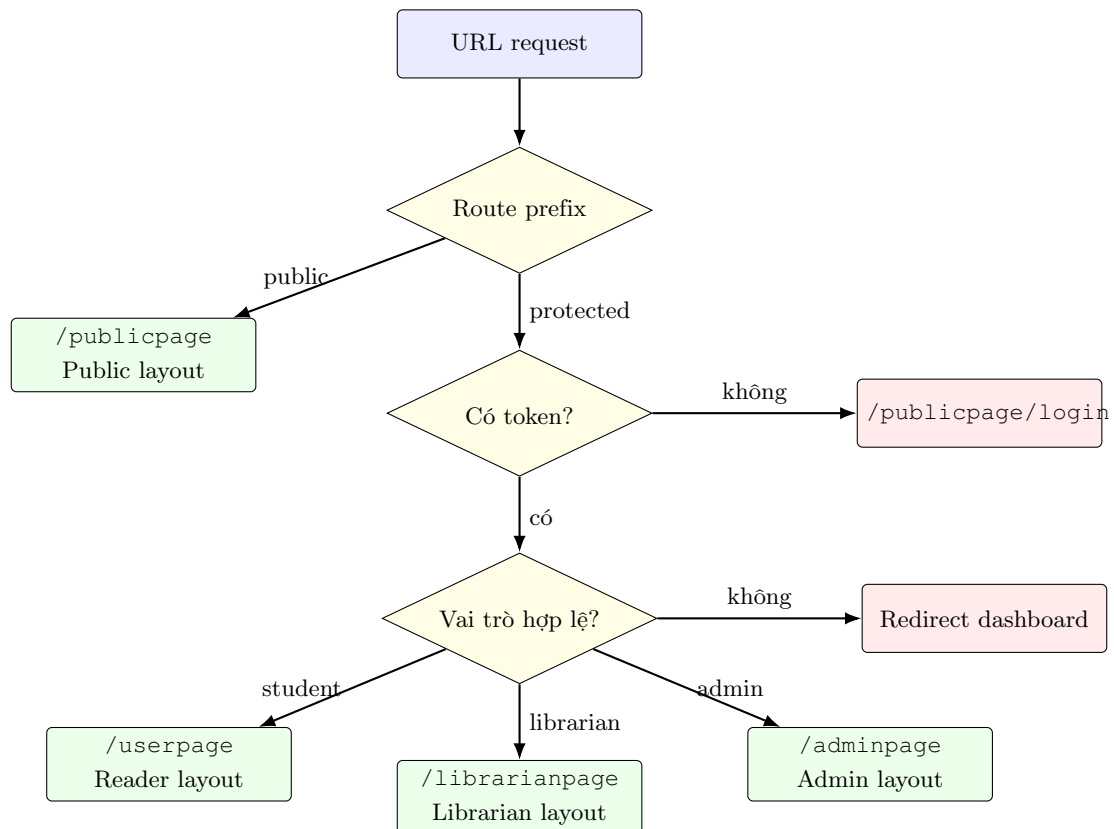
```
);  
}
```



Hình 6.3: Luồng tải màn hình bằng lazy loading

6.1.1.2 Định tuyến và phân quyền giao diện

Định tuyến được chia thành bốn vùng: công khai, độc giả, thủ thư và admin. Component `ProtectedRoute` đọc `userType` từ `AuthContext`; request chưa xác thực được chuyển về trang đăng nhập, còn request sai vai trò được chuyển về dashboard tương ứng.



Hình 6.4: Luồng xử lý route và phân quyền trên Frontend

Bảng 6.3: Phân vùng giao diện theo vai trò

Vùng route	Layout	Vai trò	Màn hình/chức năng chính
/publicpage	Layout	Khách chưa đăng nhập và người dùng xem dữ liệu công khai.	Trang chủ, tìm kiếm, chi tiết sách, danh mục, hướng dẫn, FAQ, liên hệ, đăng nhập, đăng ký, quên mật khẩu, OAuth2 callback.
/userpage	ProtectedLayout	Độc giả đã xác thực với role student.	Dashboard cá nhân, tìm kiếm trong phiên đăng nhập, sách đang mượn, phiếu đặt trước, phí phạt, wishlist, thông báo, hồ sơ, cài đặt.
/librarianpage	LibrarianLayout	Thủ thư đã xác thực với role librarian.	Dashboard vận hành, mượn-trả tại quầy, quản lý ấn phẩm, quản lý bản sao, yêu cầu, lịch sử giao dịch, cài đặt và xem trang công khai trong layout thủ thư.
/adminpage	AdminLayout	Quản trị viên đã xác thực với role admin.	Quản lý tài khoản thủ thư, tài khoản độc giả, khóa/mở khóa/xác minh tài khoản, chỉnh quy định mượn trả, xem nhật ký thao tác và cài đặt tài khoản admin.

Listing 6.2: Điều hướng theo trạng thái đăng nhập và vai trò

```
const ProtectedRoute = ({ children, allowedUserType }) => {
  const { userType } = useAuth();
  const location = useLocation();

  if (!userType) {
    return <Navigate to="/publicpage/login"
      state={{ from: `${location.pathname}${location.search}` }} replace />;
  }

  if (allowedUserType !== 'both' && userType !== allowedUserType) {
    if (userType === 'admin') {
      return <Navigate to="/adminpage/librarians" replace />;
    }
    if (userType === 'librarian') {
      return <Navigate to="/librarianpage/dashboard" replace />;
    }
    return <Navigate to="/userpage/dashboard" replace />;
  }

  return <>{children}</>;
};
```

6.1.1.3 Admin console và kiểm soát vận hành

Vùng /adminpage là khu vực quản trị tập trung của hệ thống. Khác với các màn hình quản lý tĩnh, admin console được nối trực tiếp với backend và cơ sở dữ liệu: mọi thay đổi tài khoản, quy định mượn trả và nhật ký

vận hành đều đi qua API có phân quyền. Nhờ đó admin có thể theo dõi hệ thống đang vận hành như thế nào, ai thực hiện thao tác, thao tác tác động đến đối tượng nào và kết quả ra sao.

Bảng 6.5: Các màn hình chính trong Admin Console

Màn hình	API chính	Vai trò trong vận hành
/adminpage/librarians	/api/v1/admin/librarians, /api/v1/admin/users/{id}/status, /verify.	Tạo tài khoản thủ thư với mã thủ thư, số điện thoại, địa chỉ và ảnh đại diện; khóa/mở khóa và xác minh hồ sơ thủ thư.
/adminpage/users	/api/v1/admin/users.	Tra cứu độc giả, xem trạng thái hồ sơ, khóa hoặc kích hoạt lại tài khoản khi cần kiểm soát truy cập.
/adminpage/policies	/api/v1/circulation-policies/admin.	Cấu hình quy định mượn trả từ DB, gồm hạn nhận sách, số ngày mượn, giới hạn sách/đặt trước và phí trễ hạn theo ngày.
/adminpage/audit-logs	/api/v1/admin/audit-logs.	Xem truy vết vận hành với bộ lọc, tìm kiếm, sắp xếp và nhãn tiếng Việt cho các hành động kỹ thuật/ nghiệp vụ.
/adminpage/settings	/api/v1/users/my-profile, /api/v1/users/change-password, /api/v1/users/avatar.	Cho phép admin tự cập nhật hồ sơ, ảnh đại diện và đổi mật khẩu giống các vai trò khác.

Các quyết định quan trọng trong admin console, ví dụ khóa tài khoản, xác minh hồ sơ hoặc cập nhật quy định mượn trả, đều được bọc bởi dialog xác nhận. Cách làm này giảm rủi ro thao tác nhầm vì hành động của admin thường ảnh hưởng trực tiếp đến quyền truy cập và quy tắc vận hành toàn hệ thống. Bên cạnh đó, sidebar của ba vùng độc giả, thủ thư và admin đều có nút thu gọn/mở rộng, giúp giao diện dashboard có thêm không gian cho các bảng nghiệp vụ nhiều cột.

Với trang nhật ký thao tác, frontend không hiển thị nguyên dạng các mã như POST_AUTH_LOGOUT hay đường dẫn API một cách máy móc. Thay vào đó, giao diện chuyển các hành động phổ biến sang tiếng Việt dễ đọc, đồng thời vẫn giữ phần thông tin kỹ thuật như phân hệ, endpoint, trạng thái phản hồi và mã đối tượng để admin có thể đối soát khi cần. Đây là lớp quan sát quan trọng giúp hệ thống có tính kiểm soát rõ ràng hơn: thay đổi mật khẩu, cập nhật hồ sơ, mượn sách, trả sách, thu phí, rating, reply rating, thêm/sửa ấn phẩm và thao tác tài khoản đều có thể được truy vết tập trung.

6.1.1.4 State management dùng Context

State dùng chung được gom vào các Context ở tầng App. Các page chỉ gọi hook tương ứng, do đó logic xác thực, ngôn ngữ, theme, upload và thông báo không bị lặp trong từng màn hình.

Bảng 6.7: Context được hiện thực trong Frontend

Context	State/API cung cấp	Nguồn đồng bộ
AuthContext	userType, login, socialLogin, logout.	JWT, localStorage.
LanguageContext	language, t(key), chuyển đổi vi/en.	smartlibrary.language.
ThemeContext	themeMode, resolvedTheme, setThemeMode.	prefers-color-scheme, localStorage.
NotificationContext	notifications, unreadCount, đánh dấu đã đọc.	REST API, STOMP/SockJS.
UploadContext	Danh sách upload, tiến độ, trạng thái, hủy và thử lại.	Browser upload event.
AppDialogContext	confirm(...) và alert(...) dạng Promise.	React portal.

Trong các Context trên, UploadContext được dùng cho tác vụ upload tài liệu PDF của thủ thư. Context này lưu danh sách file đang upload, tiến độ, trạng thái và callback hủy/thử lại. Khi còn upload đang chạy, frontend đăng ký beforeunload để cảnh báo người dùng trước khi đóng tab.

Listing 6.3: Quản lý trạng thái upload ở cấp ứng dụng

```
const addUpload = useCallback((item) => {
  setUploads(prev => [...prev, { ...item, progress: 0, status: 'uploading' }]);
}, []);

useEffect(() => {
  const isUploading = uploads.some(u => u.status === 'uploading');
  const handler = (e) => {
    e.preventDefault();
    e.returnValue = '';
  };
  if (isUploading) window.addEventListener('beforeunload', handler);
  return () => window.removeEventListener('beforeunload', handler);
}, [uploads]);
```

6.1.1.5 Cá nhân hóa giao diện: theme và đa ngôn ngữ

Nhóm đã hiện thực hai cơ chế cá nhân hóa ở cấp ứng dụng: light/dark mode và đa ngôn ngữ Việt-Anh. Theme-Context lưu lựa chọn light, dark hoặc system; khi chọn system, giao diện đồng bộ theo prefers-color-scheme. LanguageContext lưu ngôn ngữ hiện tại trong localStorage, cập nhật document.documentElement.lang và cung cấp hàm t(key) cho các component.

Bảng 6.9: Cơ chế cá nhân hóa giao diện

Cơ chế	Hiện thực	Giá trị trong hệ thống
Light/dark/system theme	ThemeContext, localStorage, class dark trên thẻ html.	Giao diện được đồng bộ trên toàn SPA, không cần mỗi page tự lưu trạng thái theme.
Đa ngôn ngữ Việt-Anh	LanguageContext, dictionary key-value, header Accept-Language và metadata translations từ backend.	Một lựa chọn ngôn ngữ được dùng thống nhất ở UI, lỗi API, notification và dữ liệu biên mục có bản dịch/alias.
Định dạng dữ liệu theo locale	Các page đọc language để định dạng ngày, label, trạng thái và thông báo.	Người dùng đọc cùng nghiệp vụ bằng ngôn ngữ đã chọn, không cần tách hai bộ màn hình.

Listing 6.4: Áp dụng theme và ngôn ngữ ở cấp document

```
const applyTheme = (theme) => {
  document.documentElement.classList.toggle('dark', theme === 'dark');
  document.documentElement.dataset.theme = theme;
  document.documentElement.style.colorScheme = theme;
};

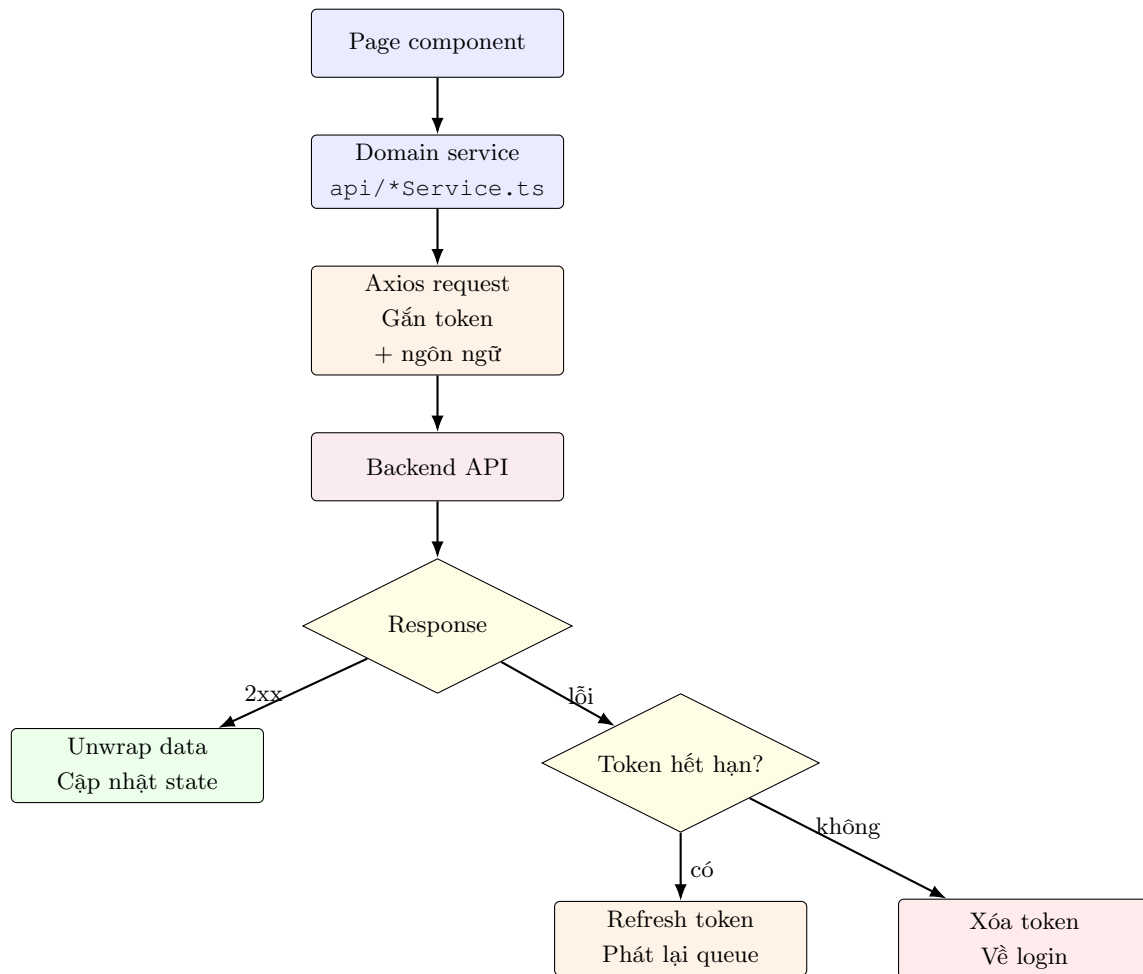
useEffect(() => {
  localStorage.setItem('smartlibrary.language', language);
  document.documentElement.lang = language;
}, [language]);
```

Placeholder: thêm ảnh chuyển đổi light/dark mode và Vietnamese/English tại
picture/frontend/theme-language.png

Hình 6.5: Thiết lập theme và ngôn ngữ trên giao diện

6.1.1.6 Lớp giao tiếp API

Lớp API được tập trung qua api/axiosInstance.ts. Request interceptor gắn Authorization và Accept-Language. Response interceptor unwrap payload thành công, xử lý lỗi nghiệp vụ và tự động refresh token khi backend trả về mã 1403.



Hình 6.6: Pipeline xử lý request và response bằng Axios interceptor

Bảng 6.11: Service layer theo domain nghiệp vụ

Service	Nghiệp vụ được đóng gói
publicationsService	Tìm kiếm, chi tiết sách, bản sao, đánh giá, upload bìa, upload tài liệu, sách tương tự.
transactionsService	Mượn sách, mượn trực tiếp, xác nhận nhận sách, tra cứu giao dịch, trả sách, báo sự cố.
reservationService	Tạo đặt trước, hủy đặt trước, tra cứu phiếu đặt trước, xác nhận nhận sách đặt trước.
fineService	Tra cứu phí phạt, thanh toán tiền mặt, tạo link PayOS, đồng bộ trạng thái thanh toán.
recommendationService semanticSearchService	Gợi ý sách cá nhân hóa và tìm kiếm ngữ nghĩa qua AI service.

Đoạn xử lý có giá trị nhất ở lớp API là cơ chế refresh token có hàng đợi request. Khi nhiều request đồng thời nhận lỗi token hết hạn, frontend chỉ gửi một request refresh. Các request còn lại được đưa vào `failedQueue` và được phát lại sau khi token mới được cấp. Cách này tránh việc nhiều request refresh được gửi song song và

làm sai lệch trạng thái phiên đăng nhập.

Listing 6.5: Refresh token một lần và phát lại các request bị treo

```
let isRefreshing = false;
let failedQueue = [];

if (isTokenExpired && refreshToken && !originalRequest._retry) {
  if (isRefreshing) {
    return new Promise((resolve, reject) => {
      failedQueue.push({ resolve, reject });
    }).then((token) => {
      originalRequest.headers.Authorization = 'Bearer ' + token;
      return axiosInstance(originalRequest);
    });
  }

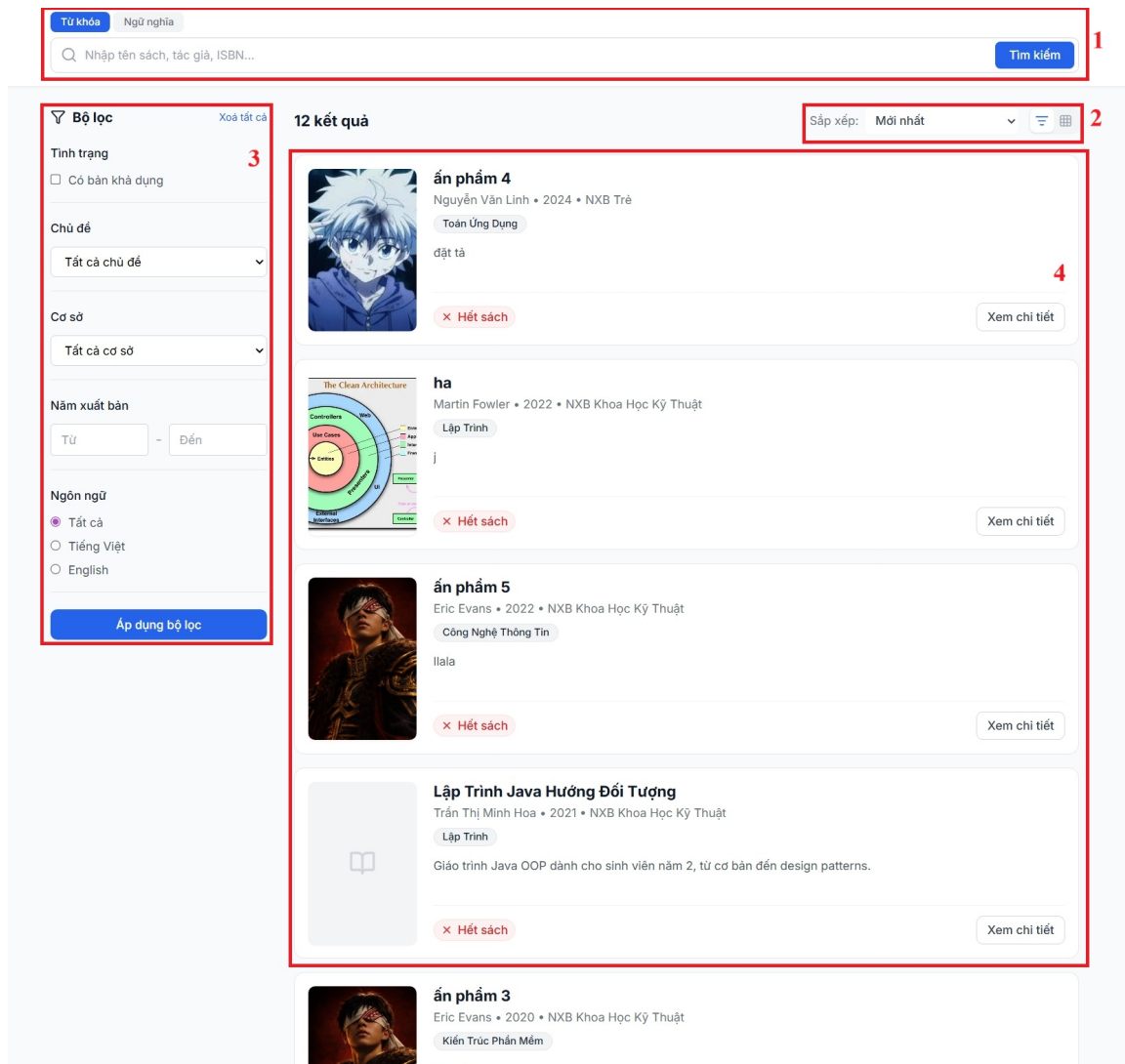
  originalRequest._retry = true;
  isRefreshing = true;
  const res = await axios.post(`${baseUrl}/auth/refresh-accesstoken`, {}, {
    headers: { 're-token': refreshToken }
  });
  localStorage.setItem('accessToken', res.data.data.accessToken);
  processQueue(null, res.data.data.accessToken);
  return axiosInstance(originalRequest);
}
```

6.1.1.7 Hiện thực nhóm trang công khai

Nhóm trang công khai xử lý các tác vụ không yêu cầu phiên đăng nhập. Người dùng có thể tìm sách, xem metadata, xem tình trạng bản sao và đọc thông tin hướng dẫn trước khi đăng nhập. Khi thao tác mượn/đặt trước được kích hoạt từ trang chi tiết sách, frontend chuyển luồng sang yêu cầu xác thực nếu chưa có token.

Bảng 6.13: Chức năng của nhóm trang công khai

Màn hình	Mục đích	Chức năng chính
HomePage	Cổng vào hệ thống và điều hướng tra cứu.	Hiển thị sách mới, sách mượn nhiều, thống kê, danh mục và ô tìm kiếm nhanh.
SearchPage	Tra cứu ấn phẩm ở chế độ công khai.	Tìm kiếm theo từ khóa, tag, danh mục/năm/ngôn ngữ/chi nhánh; kết quả semantic search được dùng như lớp bổ sung sau kết quả lexical.
BookDetailPage	Xem chi tiết một ấn phẩm.	Hiển thị metadata, tag màu sáng có thể bấm để tìm kiếm, danh mục, bản sao, rating, trích dẫn, sách tương tự, wishlist và modal mượn/đặt trước.
Login/Register	Tạo và khôi phục phiên đăng nhập.	Đăng nhập email/mật khẩu, đăng ký, xác thực email, quên mật khẩu, Google OAuth2 và onboarding hồ sơ.
ContactPage	Cung cấp kênh liên hệ và vị trí thư viện.	Form liên hệ, thông tin địa chỉ/điện thoại/email, liên kết gọi nhanh và Google Maps nhúng bằng iframe.



Hình 6.7: Giao diện tìm kiếm sách ở nhóm trang công khai

Trang tìm kiếm được hiện thực theo hướng hybrid nhưng ưu tiên kết quả có bằng chứng trực tiếp từ dữ liệu biên mục. Khi người dùng nhập từ khóa, frontend gọi API tìm kiếm thường để lấy kết quả lexical trước; đồng thời có thể gọi semantic search để lấy thêm các ấn phẩm liên quan theo vector. Sau khi lấy metadata chi tiết cho kết quả semantic, frontend loại trùng và nổi các kết quả này phía sau danh sách lexical. Cách này bảo đảm truy vấn chính xác theo nhan đề, tác giả, tag hoặc danh mục không bị đẩy xuống dưới bởi kết quả vector gần nghĩa. Các filter được đồng bộ vào URL query string để người dùng có thể reload hoặc chia sẻ đường dẫn mà vẫn giữ được trạng thái lọc.

Bảng 6.15: Bộ lọc và sắp xếp trên trang tìm kiếm

Nhóm filter	State frontend	Cách xử lý
Từ khóa	inputValue, activeKeyword.	Gửi keyword vào API tìm kiếm thường; nếu có semantic search thì dùng cùng nội dung làm queryText và chỉ bổ sung kết quả chưa trùng.
Danh mục	categoryIds.	Lưu nhiều categoryIds trong URL; backend lọc theo quan hệ publication_categories, semantic search lọc lại sau khi lấy metadata.
Tình trạng bản sao	available.	Chỉ giữ ấn phẩm còn bản sao khả dụng.
Năm, ngôn ngữ, cơ sở	yearFrom, yearTo, language, branch.	Gửi vào API search; với semantic search, áp dụng filter cục bộ trên danh sách kết quả.
Sắp xếp	sortBy.	Với chế độ liên quan, backend ưu tiên nhan đề/phụ đề, exact tag, partial tag rồi mới đến tiêu chí phụ như năm xuất bản.

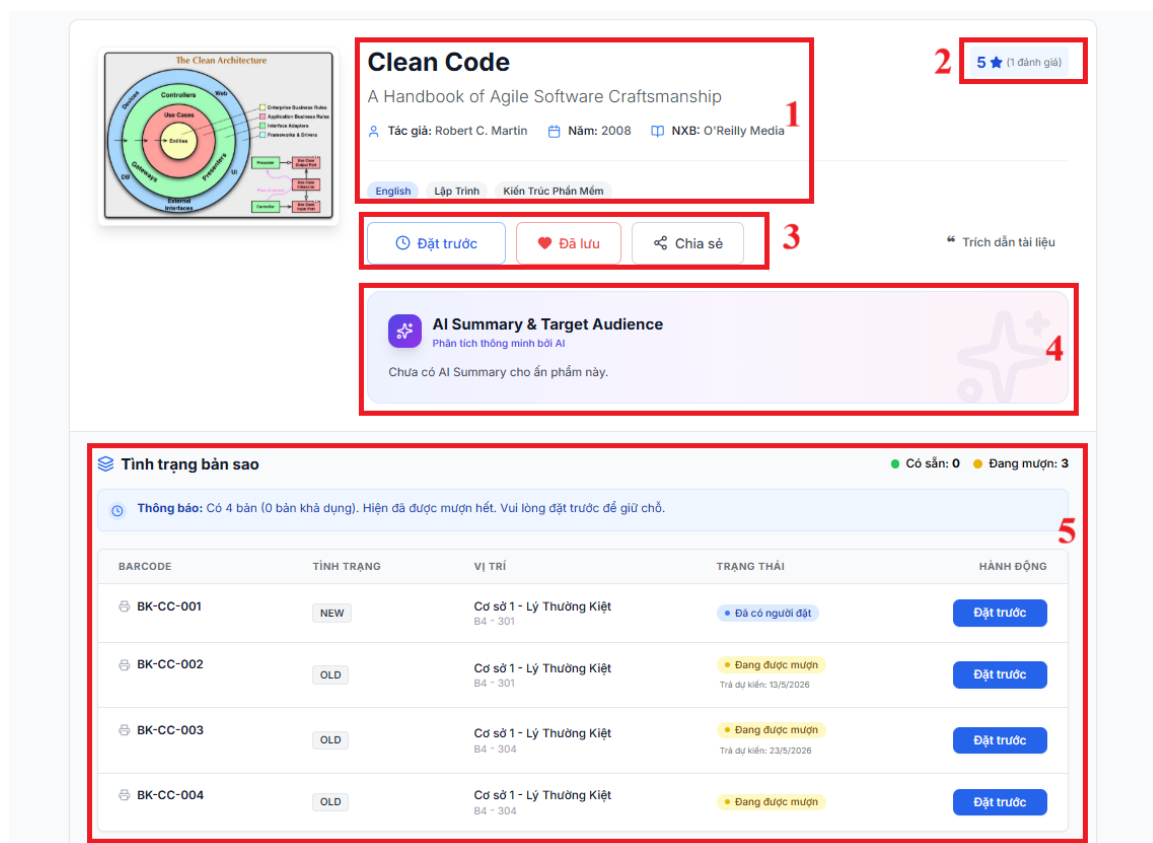
Listing 6.6: Đồng bộ bộ lọc tìm kiếm vào URL và gọi API

```
const syncFilterParams = (filters, keyword = activeKeyword) => {
  const p = new URLSearchParams(searchParams);
  keyword ? p.set('q', keyword) : p.delete('q');
  p.delete('categoryIds');
  filters.categoryIds.forEach(id => p.append('categoryIds', id));
  filters.sortBy === 'relevance' ? p.delete('sort') : p.set('sort', filters.sortBy);
  setSearchParams(p, { replace: true });
};

publicationsService.searchPublications({
  keyword: kw || undefined,
  available: filters.available || undefined,
  language: filters.language || undefined,
  yearFrom: filters.yearFrom ? Number(filters.yearFrom) : undefined,
  yearTo: filters.yearTo ? Number(filters.yearTo) : undefined,
  branch: filters.branch || undefined,
  categoryIds: filters.categoryIds.length ? filters.categoryIds : undefined,
  sortBy: filters.sortBy === 'relevance' ? 'newest' : filters.sortBy,
});
```

Bảng 6.17: State và API của nhóm trang công khai

Màn hình	State chính	API sử dụng
HomePage	searchInput, sách mới, sách mượn nhiều, gợi ý, danh mục, thống kê.	getNewestPublications, getMostBorrowedPublications, getRecommendations.
SearchPage	Từ khóa, danh mục, năm, ngôn ngữ, chỉ nhánh, còn bản sao, sắp xếp, trang, chế độ hiển thị.	searchPublications, semanticSearch, getPublicationById.
BookDetailPage	Metadata, tag, danh mục, bản sao, đánh giá, tóm tắt rating, wishlist, trích dẫn, modal mượn/đặt trước, sách tương tự.	getPublicationById, getPublicationItems, borrow, createReservation, getSimilarPublications.



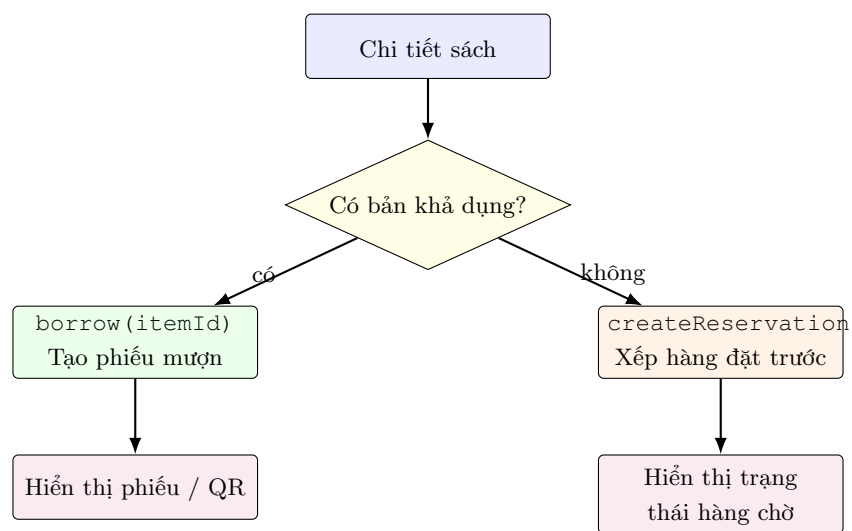
Hình 6.8: Giao diện chi tiết sách

Trên BookDetailPage, tag được đặt ở vùng thông tin đầu trang thay vì ẩn sâu trong tab tổng quan. Mỗi tag hiển thị bằng chip màu sáng được tính ổn định từ tên tag, đồng thời là liên kết tới trang tìm kiếm với tham số $q=<tag>$. Dòng danh mục được trình bày rõ bằng nhãn “Danh mục:” kèm ngôn ngữ tài liệu và các category liên quan. Nút “Trích dẫn tài liệu” được giữ vì phù hợp nghiệp vụ thư viện; thao tác này tạo chuỗi trích dẫn dạng tác giả-năm-nhan đề-nhà xuất bản và sao chép vào clipboard, không mở một luồng nghiệp vụ riêng.

Trên trang liên hệ, Google Maps được nhúng bằng iframe với `loading="lazy"`. Nhóm chọn cách nhúng này vì không cần tích hợp SDK phức tạp, vẫn hiển thị được vị trí thư viện và không làm tăng đáng kể logic frontend.

Placeholder: thêm ảnh trang Contact có Google Maps tại
picture/frontend/contact-map.png

Hình 6.9: Trang liên hệ tích hợp Google Maps



Hình 6.10: Luồng mượn hoặc đặt trước từ trang chi tiết sách

6.1.1.8 Hiện thực đánh giá và phản hồi sách

Chức năng đánh giá được đặt trong BookDetailPage. Frontend hiển thị tóm tắt rating, phân bố số sao, danh sách review có phân trang, filter theo số sao và sắp xếp theo mới nhất/cũ nhất/hữu ích. Độc giả có giao dịch mượn hợp lệ có thể gửi hoặc chỉnh sửa review; thủ thư có thể đọc phản hồi và trả lời review trong cùng màn hình.

Placeholder: thêm ảnh tab đánh giá sách, rating summary và danh sách review tại
picture/frontend/book-rating-review.png

Hình 6.11: Giao diện đánh giá và phản hồi sách

Bảng 6.19: Các thao tác trên hệ thống đánh giá sách

Thao tác	API/service	Xử lý tại Frontend
Tải tóm tắt rating	getPublicationRatingSummary.	Hiển thị điểm trung bình, tổng số đánh giá và thanh phân bố 1–5 sao.
Tải danh sách review	getPublicationRatings.	Gửi page, size, star, sort; render phân trang và filter.
Gửi/chỉnh sửa review	createPublicationRating, up- datePublicationRating.	Kiểm tra nội dung, gửi số sao và comment; sau khi thành công tải lại summary và danh sách.
Đánh dấu hữu ích	toggleRatingHelpful.	Cập nhật helpfulCount và trạng thái đã đánh dấu ngay trên item review.
Thủ thư trả lời	replyToRating.	Hiển thị phản hồi của thủ thư dưới review gốc.

Listing 6.7: Tải review theo bộ lọc sao và thứ tự sắp xếp

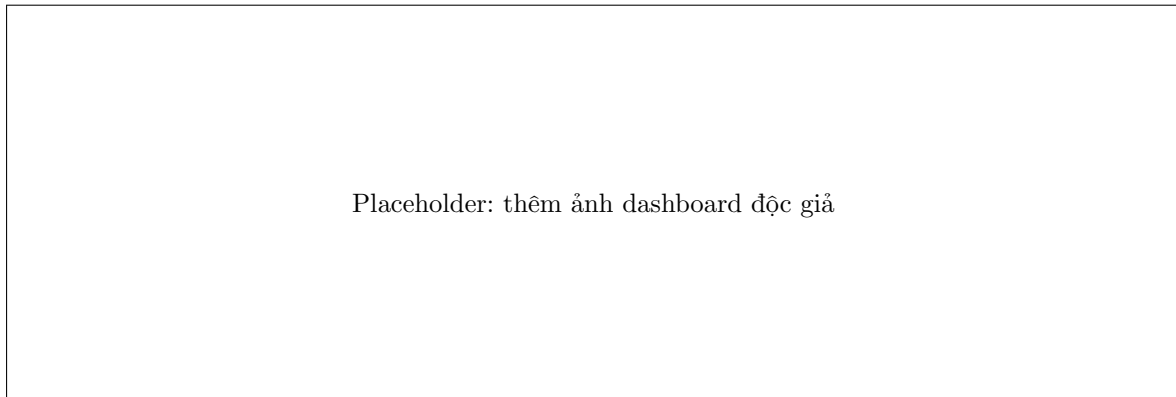
```
const [ratingStarFilter, setRatingStarFilter] = useState('all');
const [ratingSortFilter, setRatingSortFilter] = useState('newest');

useEffect(() => {
  const fetchRatings = async () => {
    const response = await publicationsService.getPublicationRatings(id, ratingsPage, 10,
      {
        star: ratingStarFilter,
        sort: ratingSortFilter,
      });
    if (response.code === 200) setRatingsData(response.data);
  };

  fetchRatings();
}, [id, ratingsPage, ratingStarFilter, ratingSortFilter]);
```

6.1.1.9 Hiện thực nhóm trang độc giả

Nhóm trang độc giả được thiết kế cho các nghiệp vụ cá nhân sau khi đăng nhập. Layout độc giả hiển thị sidebar, badge số lượng thông báo, trạng thái phí phạt và điều hướng đến các trang thường dùng. Các API được gọi theo dữ liệu của người dùng hiện tại, do đó frontend không truyền thủ công mã sinh viên trong những API dạng my-*.

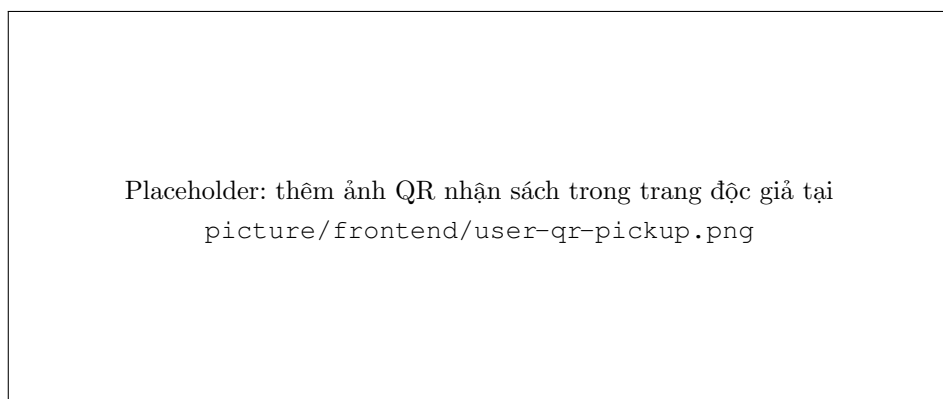


Hình 6.12: Giao diện dashboard độc giả

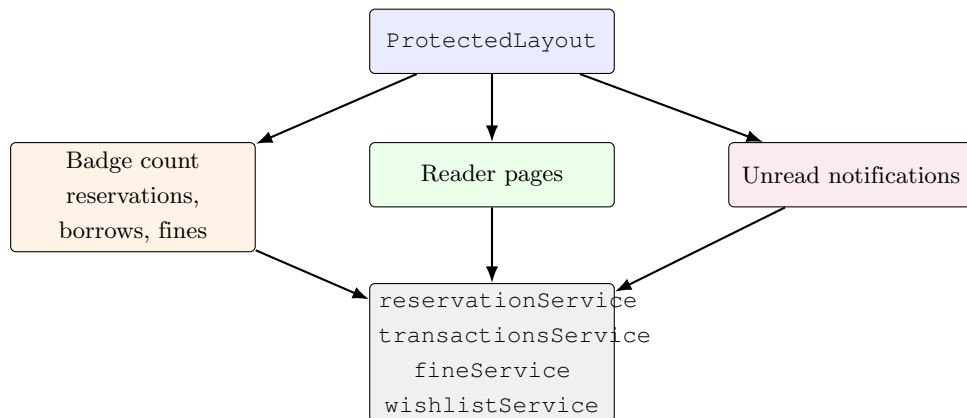
Bảng 6.21: Các màn hình nghiệp vụ của độc giả

Màn hình	Dữ liệu hiển thị	Xử lý chính
DashboardPage	Hồ sơ, giao dịch, đặt trước, sách quá hạn/sắp đến hạn.	Tải song song getMyTransactions và getMyReservations.
MyBooksPage	Sách chờ lấy, đang mượn, đã trả, đã hủy.	Lọc theo tab, tìm kiếm cục bộ, sắp xếp, sinh QR nhận sách.
ReservationsPage	Danh sách đặt trước, trạng thái hàng chờ, hạn nhận.	Hủy đặt trước, hiển thị QR cho sách READY_FOR_PICKUP.
FinesPage	Phí phạt chưa thanh toán và đã thanh toán.	Phân tab theo UNPAID/PAID, tính tổng tiền.
WishlistPage	Danh sách ấn phẩm đã lưu.	Xóa một ấn phẩm hoặc xóa toàn bộ wishlist, cập nhật state cục bộ.
ProfilePage	Thông tin cá nhân và ảnh đại diện.	Cập nhật hồ sơ, upload avatar bằng FormData.

Ở luồng nhận sách, frontend sinh QR từ mã giao dịch hoặc mã đặt trước để thủ thư tra cứu nhanh tại quầy. QR không thay thế kiểm tra quyền ở backend; mã QR chỉ là dữ liệu đầu vào cho API lookup.



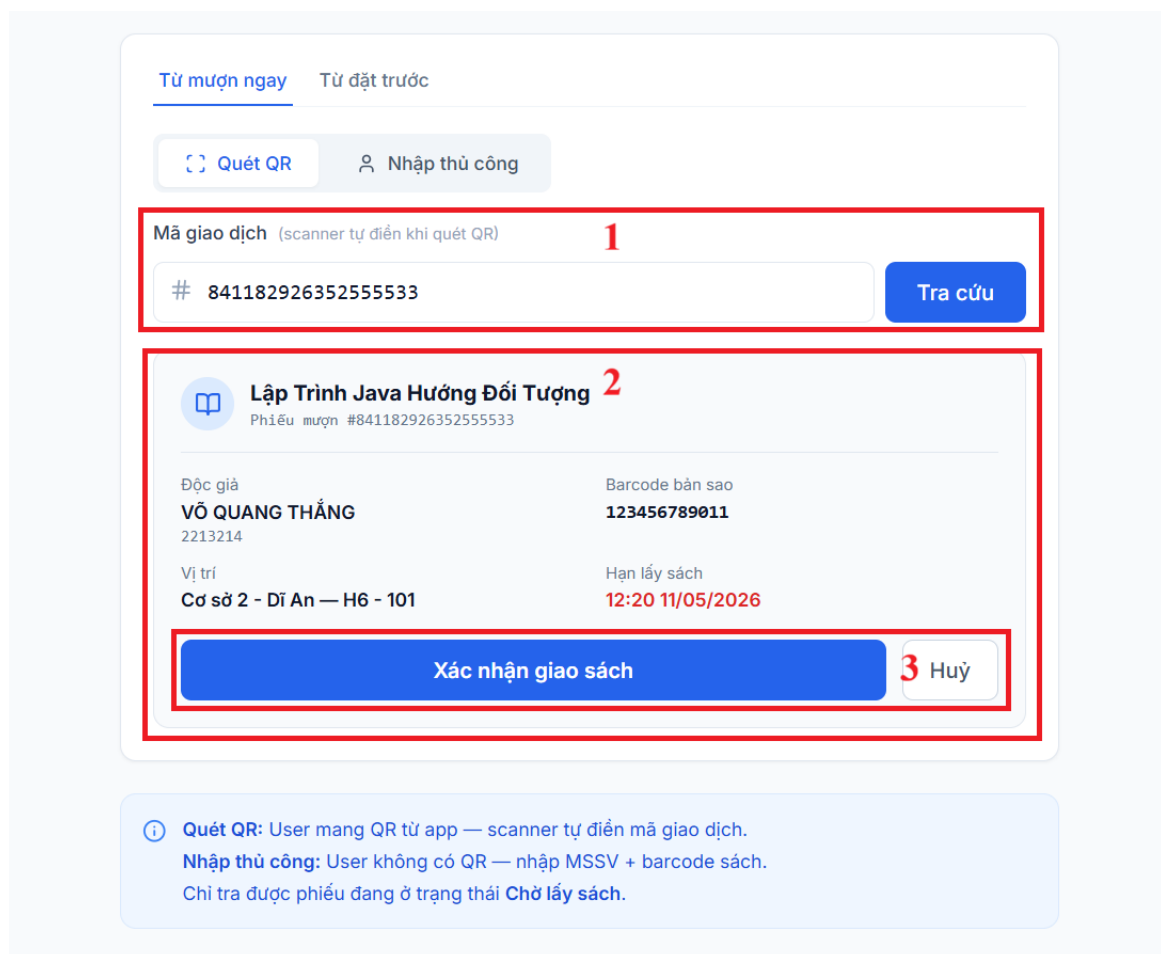
Hình 6.13: QR nhận sách trong giao diện độc giả



Hình 6.14: Dữ liệu điều hướng trong khu vực đọc giả

6.1.1.10 Hiện thực nhóm trang thủ thư

Nhóm trang thủ thư tập trung vào các nghiệp vụ vận hành tại quầy và quản lý dữ liệu thư viện. Khác với trang đọc giả, giao diện thủ thư ưu tiên tra cứu nhanh, thao tác theo barcode/MSSV, cập nhật trạng thái giao dịch và quản trị metadata ấn phẩm.



Từ mượn ngay **Từ đặt trước**

Quét QR **Nhập thủ công**

Mã giao dịch (scanner tự điển khi quét QR) **1**

84118292635255533 **Tra cứu**

Lập Trình Java Hướng Đối Tượng **2**

Phiếu mượn #84118292635255533

Độc giả **Barcode bản sao**

VÔ QUANG THẮNG **123456789011**

2213214

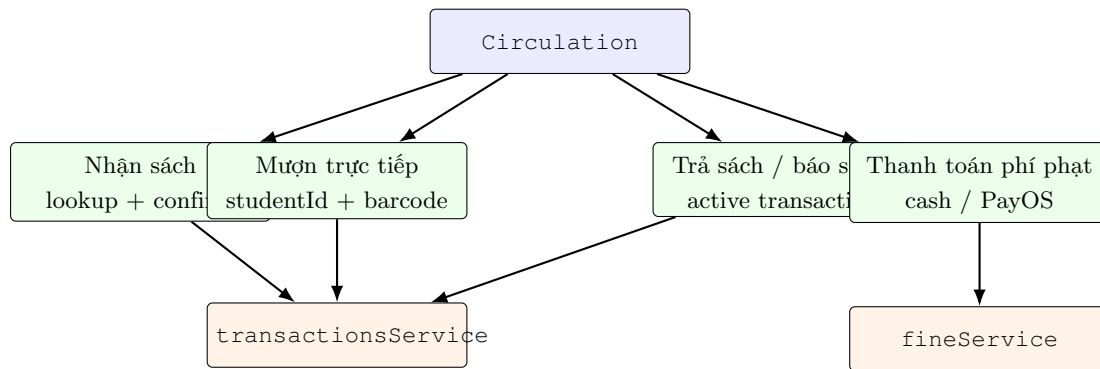
Vị trí **Hạn lấy sách**

Cơ sở 2 - Dĩ An — H6 - 101 **12:20 11/05/2026**

Xác nhận giao sách **3 Hủy**

Quét QR: User mang QR từ app — scanner tự điển mã giao dịch.
Nhập thủ công: User không có QR — nhập MSSV + barcode sách.
Chỉ tra được phiếu đang ở trạng thái **Chờ lấy sách**.

Hình 6.15: Giao diện tra cứu và xác nhận mượn sách của thủ thư

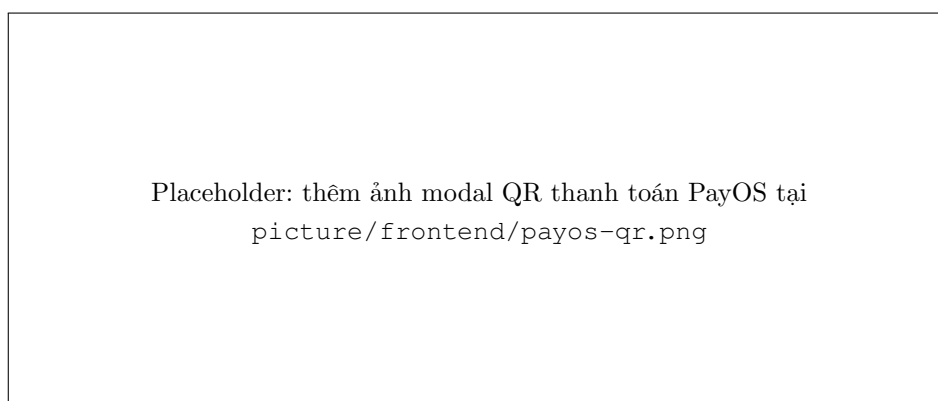


Hình 6.16: Các luồng nghiệp vụ tại quầy thủ thư

Bảng 6.23: Các màn hình nghiệp vụ của thủ thư

Màn hình	Dữ liệu/State chính	API hoặc xử lý chính
Circulation	Tab nhận sách, mượn trực tiếp, trả sách, phí phạt.	lookup, borrowDirect, return-Book, reportIssue, payAllFines-ByCash, syncPayOsFinePayment.
BookList	Từ khóa, danh mục, năm, có bản sao, sắp xếp, trạng thái AI.	getAllPublications; hiển thị AiS-tatusBadge.
BookDetails/CreateMetadata	Mô tả tác giả, tag, danh mục, bìa, tài liệu PDF.	Tạo/cập nhật ấn phẩm, upload cover, upload PDF qua presigned URL.
CopyList	Barcode, tình trạng, trạng thái, vị trí, phân trang.	Gọi /items; lọc, sắp xếp, xuất CSV trang hiện tại.
TransactionList	Lịch sử giao dịch, trạng thái, khoảng ngày, ghi chú.	getAllTransactions, cập nhật và xóa ghi chú giao dịch.

Luồng thu phí phạt được hiện thực với hai phương thức: tiền mặt và chuyển khoản qua PayOS. Khi chọn PayOS, frontend gọi API tạo payment link, hiển thị QR code từ trường qrCode, sau đó tự đồng bộ trạng thái theo chu kỳ 4 giây hoặc cho phép thủ thư bấm kiểm tra thủ công.



Hình 6.17: Modal QR thanh toán phí phạt qua PayOS

Listing 6.8: Tạo QR PayOS và tự đồng bộ trạng thái thanh toán

```
const handleCreatePayOsPayment = async () => {
```

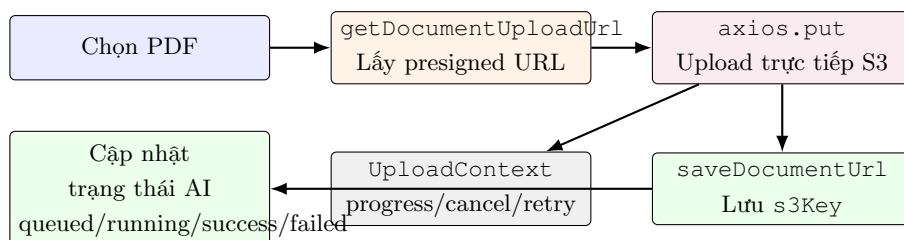
```
const res = await fineService.createPayOsFinePayment(studentData.studentId);
if (res.code === 200) setPayOsPayment(res.data);
};

useEffect(() => {
  if (!payOsPayment) return;
  let stopped = false;

  const timer = window.setInterval(async () => {
    const syncRes = await fineService.syncPayOsFinePayment(payOsPayment.orderCode);
    const refreshed = await refreshStudentFines(payOsPayment.studentId, true);
    if (!stopped && (syncRes.data.paidCount > 0 || refreshed?.fines.length === 0)) {
      setPayOsPayment(null);
      toast.success('payOS payment confirmed. Fines were updated.');
```

Placeholder: thêm ảnh màn hình quản lý ấn phẩm của thủ thư

Hình 6.18: Giao diện quản lý ấn phẩm của thủ thư



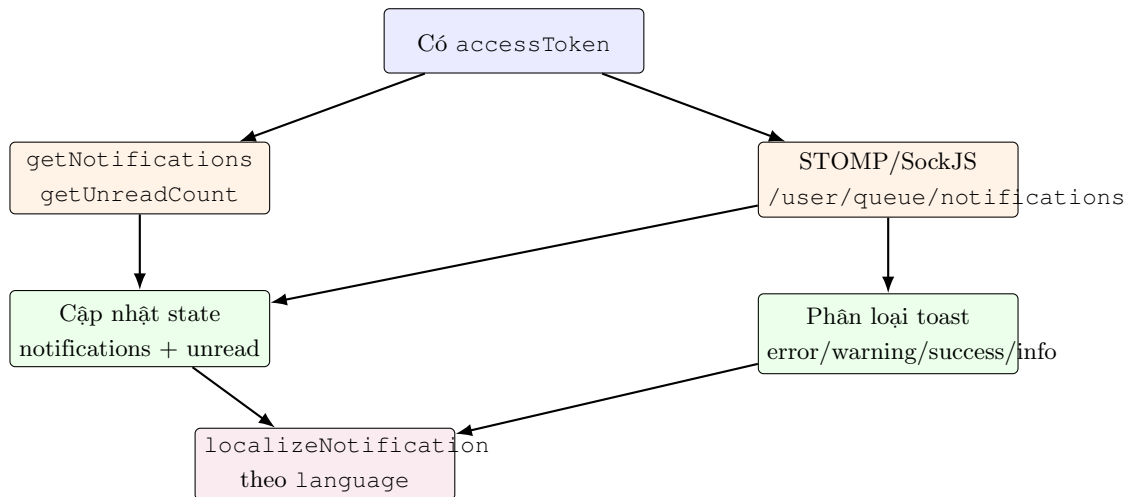
Hình 6.19: Luồng upload tài liệu PDF của thủ thư

Placeholder: thêm ảnh UploadPanel khi upload PDF/cover tại
picture/frontend/upload-panel.png

Hình 6.20: UploadPanel hiển thị tiến độ upload tài liệu

6.1.1.11 Thông báo thời gian thực

Thông báo thời gian thực được hiện thực bằng NotificationContext. Khi có accessToken, frontend tải danh sách thông báo ban đầu qua REST API và mở kết nối STOMP/SockJS đến endpoint WebSocket.



Hình 6.21: Luồng nhận và hiển thị thông báo thời gian thực

Đoạn hiện thực dưới đây cho thấy frontend vừa tải dữ liệu ban đầu bằng REST API, vừa duy trì kênh STOMP/SockJS để nhận message mới. Khi message đến, state danh sách thông báo và số lượng chưa đọc được cập nhật ngay trong Context; toast được phân loại theo loại thông báo.

Listing 6.9: Nhận notification thời gian thực bằng STOMP/SockJS

```

useEffect(() => {
  const accessToken = localStorage.getItem('accessToken');
  if (!accessToken) return;

  fetchNotifications(0, 20);
  fetchUnreadCount();

  const client = new Client({
    websocketFactory: () => new SockJS(wsUrl),
    connectHeaders: { Authorization: `Bearer ${accessToken}` },
    reconnectDelay: 5000,
  });

```

```

    heartbeatIncoming: 4000,
    heartbeatOutgoing: 4000,
  });

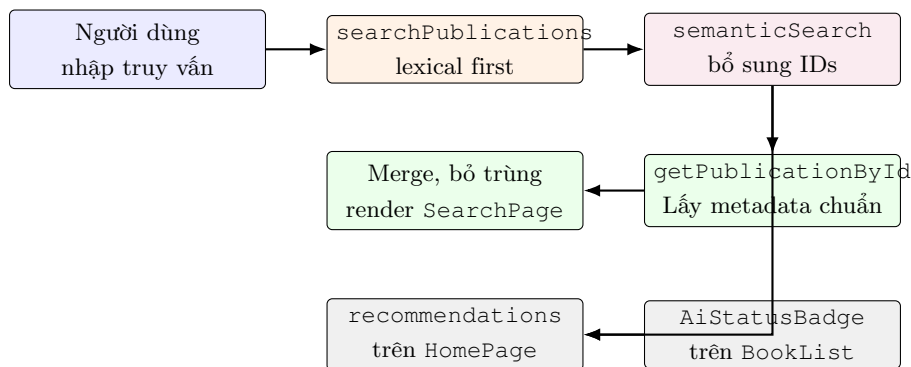
  client.onConnect = () => {
    client.subscribe('/user/queue/notifications', (message) => {
      const notification = normalizeNotification(JSON.parse(message.body));
      setNotifications((prev) => [notification, ...prev]);
      setUnreadCount((prev) => prev + 1);
      showToast(notification);
    });
  };

  client.activate();
  return () => client.deactivate();
}, [userType, language]);

```

6.1.1.12 Tích hợp AI ở tầng giao diện

Frontend không chạy mô hình AI trực tiếp. Các màn hình gửi request đến AI service hoặc đọc trạng thái xử lý AI từ backend, sau đó hiển thị kết quả trong cùng luồng nghiệp vụ. Riêng trang tìm kiếm ưu tiên kết quả lexical từ backend, sau đó mới ghép thêm kết quả semantic chưa trùng để tránh làm yếu các truy vấn chính xác theo tên sách, tag hoặc danh mục.



Hình 6.22: Các điểm tích hợp AI trong Frontend

Bảng 6.25: Tóm tắt điểm tích hợp AI trên giao diện

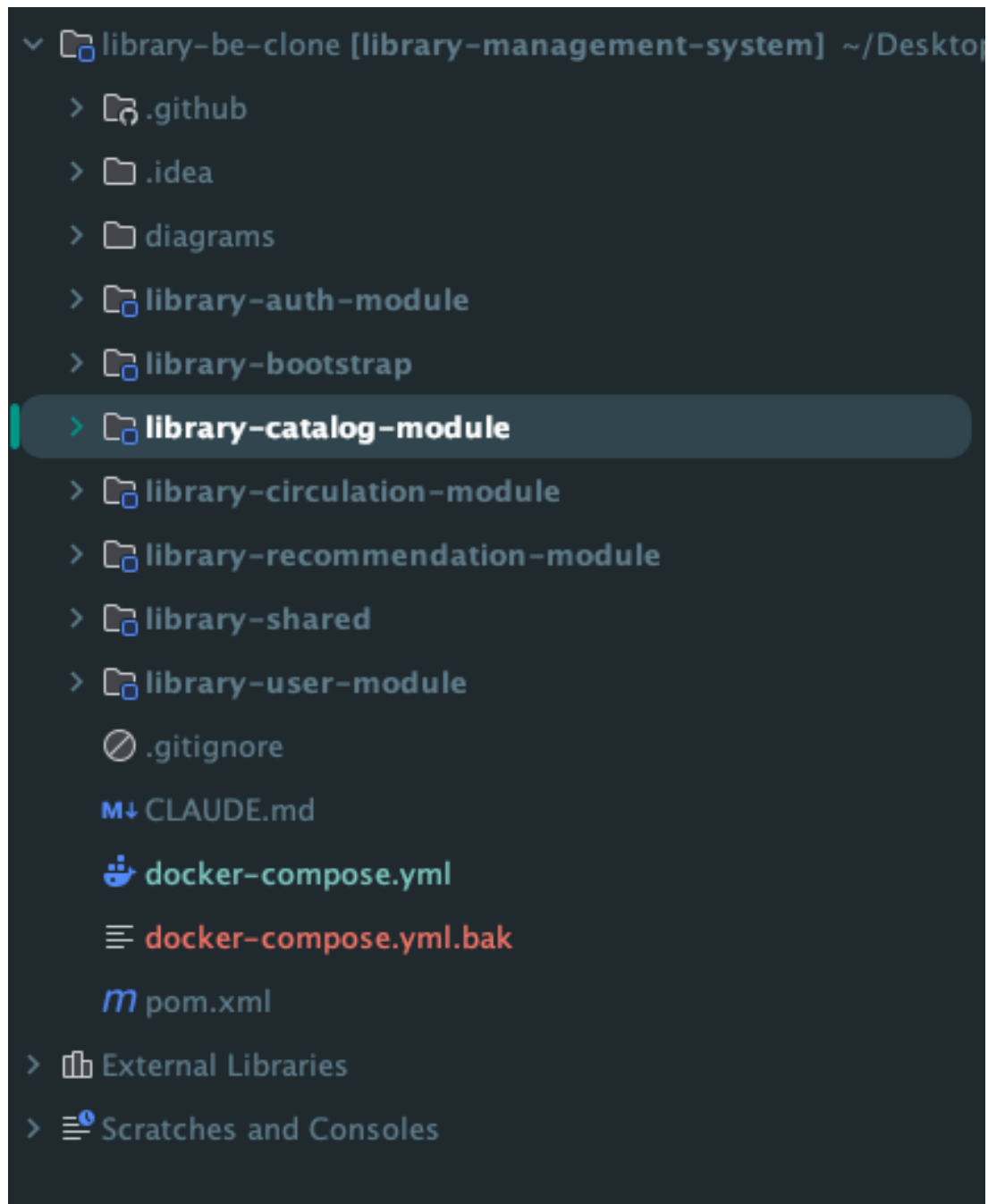
Điểm tích hợp	Nguồn dữ liệu	Cách hiển thị
Tìm kiếm hybrid	GET /publications/search và /ai/semantic-search.	Render kết quả lexical trước; kết quả semantic chỉ bổ sung các ấn phẩm chưa xuất hiện.
Gợi ý sách	/recommendations?limit=....	Hiển thị danh sách sách gợi ý trên HomePage khi có phiên đăng nhập.
Trạng thái xử lý tài liệu	Trường aiProcessingStatus trong dữ liệu ấn phẩm.	Badge trạng thái trên BookList: QUEUED, RUNNING, SUCCESS, FAILED.

6.1.1.13 Tổng kết hiện thực Frontend

Nhóm đã hiện thực frontend với route phân vai trò, state dùng chung qua Context, service layer cho REST API, lazy loading theo trang, upload có tiến độ, QR cho nghiệp vụ nhận sách, QR PayOS cho thanh toán phí phạt, WebSocket cho thông báo và các điểm tích hợp AI ở tìm kiếm, gợi ý và quản lý xử lý tài liệu. Các đoạn hiện thực có giá trị gồm ProtectedRoute, Axios refresh queue, UploadContext, NotificationContext và luồng PayOS polling, vì đây là các điểm kết nối trực tiếp giữa giao diện, trạng thái phiên đăng nhập và nghiệp vụ vận hành thư viện.

6.1.2 Hiện thực Backend

Nhóm đã hiện thực backend bằng Spring Boot 3.3.5 và Java 21 theo mô hình modular monolith. Mã nguồn được chia thành các Maven module theo ranh giới nghiệp vụ thay vì chia theo lớp kỹ thuật toàn cục. Module `library-bootstrap` khởi động ứng dụng Spring Boot và khai báo phụ thuộc đến các module nghiệp vụ; module `library-shared` chứa contract, DTO dùng chung, exception, security annotation, Kafka event và các port dùng giữa các bounded context.

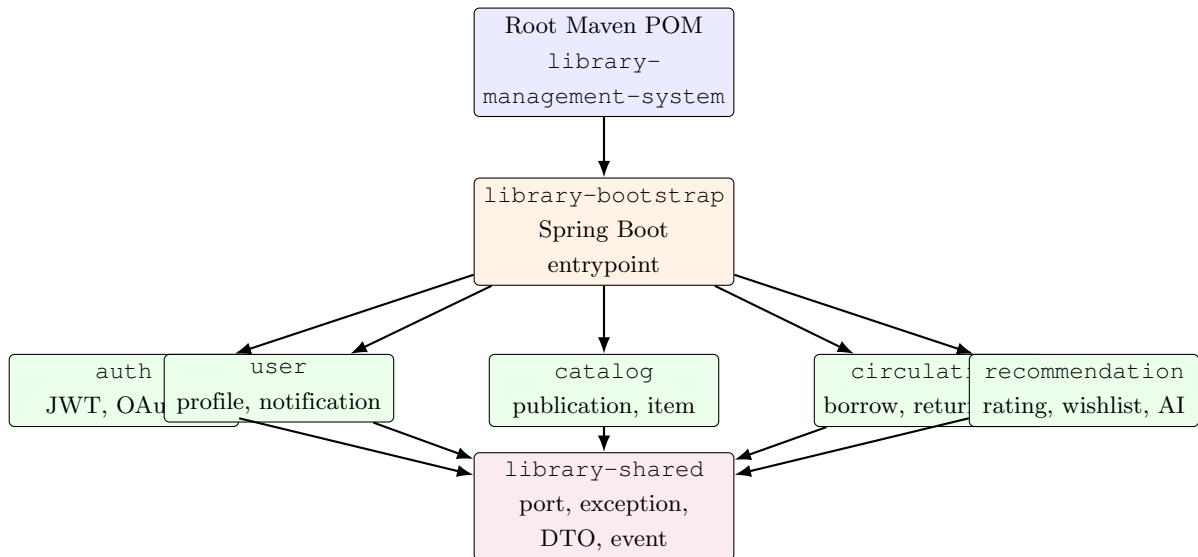


Hình 6.23: Cấu trúc thư mục mã nguồn Backend

Placeholder: thêm ảnh Swagger/OpenAPI tổng quan Backend tại
picture/backend/swagger-overview.png

Hình 6.24: Tài liệu API Backend trên Swagger/OpenAPI

6.1.2.1 Tổ chức Maven module



Hình 6.25: Tổ chức Maven module của Backend

Bảng 6.27: Trách nhiệm của các Backend module

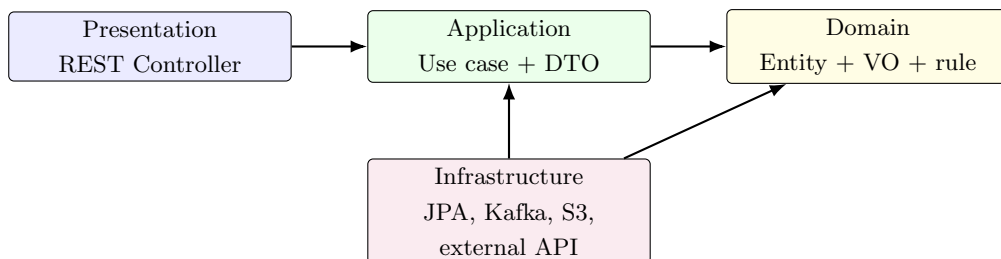
Module	Trách nhiệm hiện thực
library-bootstrap	Khởi động Spring Boot, cấu hình datasource, JPA, Flyway, Kafka, mail, OpenAPI, async và các biến môi trường triển khai.
library-auth-module	Đăng nhập, đăng ký, refresh token, logout, xác thực email, quên mật khẩu, Google OAuth2, Spring Security và WebSocket authentication.
library-user-module	Hồ sơ người dùng, vai trò, thông báo, avatar, Kafka consumer cho notification và WebSocket push.
library-catalog-module	Ấn phẩm, bản sao, tác giả, nhà xuất bản, danh mục, tag, tìm kiếm công khai, upload bìa, presigned URL cho tài liệu PDF và lưu người tạo/người chỉnh sửa cuối.
library-circulation-module	Mượn sách, nhận sách, trả sách, đặt trước, hàng chờ, phí phạt, quy định mượn trả cấu hình từ DB, dashboard thủ thư, scheduler và tích hợp PayOS.
library-recommendation-module	Wishlist, lịch sử tìm kiếm, đánh giá, phản hồi rating của thủ thư, đánh giá hệ thống, sách tương tự, gợi ý sách, semantic search gateway và AI callback.
library-shared	Response wrapper, exception, error code, base entity, shared port, Kafka topic/event, S3 storage port và audit logging aspect.

Bảng 6.29: Lý do lựa chọn mô hình Backend

Phương án	Đặc điểm	Quyết định của nhóm
Layered monolith	Dễ khởi tạo, nhưng các module nghiệp vụ có xu hướng phụ thuộc chéo qua service/repository dùng chung.	Không được chọn làm cấu trúc chính vì khó giữ ranh giới nghiệp vụ khi hệ thống mở rộng.
Microservices đầy đủ	Mỗi service triển khai độc lập, nhưng yêu cầu service discovery, distributed tracing, API gateway và xử lý giao dịch phân tán.	Chưa phù hợp với phạm vi đồ án vì chi phí vận hành cao hơn lợi ích đạt được ở giai đoạn hiện tại.
Modular monolith	Một tiến trình triển khai, nhưng code được chia theo bounded context và giao tiếp qua interface/event.	Được chọn vì giữ đơn giản trong triển khai, đồng thời vẫn tạo đường tách module trong tương lai.

6.1.2.2 Clean Architecture trong từng module

Nhóm đã tổ chức mỗi module nghiệp vụ theo bốn lớp: presentation, application, domain và infrastructure. Controller chỉ nhận request và gọi use case. Use case điều phối nghiệp vụ và transaction. Domain chứa entity, value object và enum nghiệp vụ. Infrastructure hiện thực JPA repository, external client, scheduler, Kafka, S3 hoặc payment adapter.



Hình 6.26: Phân lớp Clean Architecture trong một Backend module

Bảng 6.31: Quy ước hiện thực theo lớp

Lớp	Thành phần mã nguồn	Ví dụ trong hệ thống
Presentation	REST controller, request mapping, security annotation.	PublicationController, BorrowingTransactionController, AuthController.
Application	Interface use case, implementation, mapper, DTO.	BorrowRequestUseCaseImpl, CreateReservationUseCaseImpl, SearchPublicationsUseCaseImpl.
Domain	Entity, enum, value object, domain event.	Publication, Item, BorrowingTransaction, Reservation, Fine.
Infrastructure	JPA entity/repository, adapter, scheduler, external client.	PublicationRepositoryImpl, ItemStatusPortImpl, NotificationEventConsumer, PayOsClient.

Đoạn mã dưới đây thể hiện cách nhóm đưa kiểm tra xác thực và phân quyền ra khỏi từng controller bằng annotation và AOP. Controller chỉ cần khai báo @RequiresRole hoặc @RequiresAuthentication; logic kiểm tra được thực thi tập trung trong SecurityAspect.

Listing 6.10: Hiện thực kiểm tra phân quyền bằng AOP

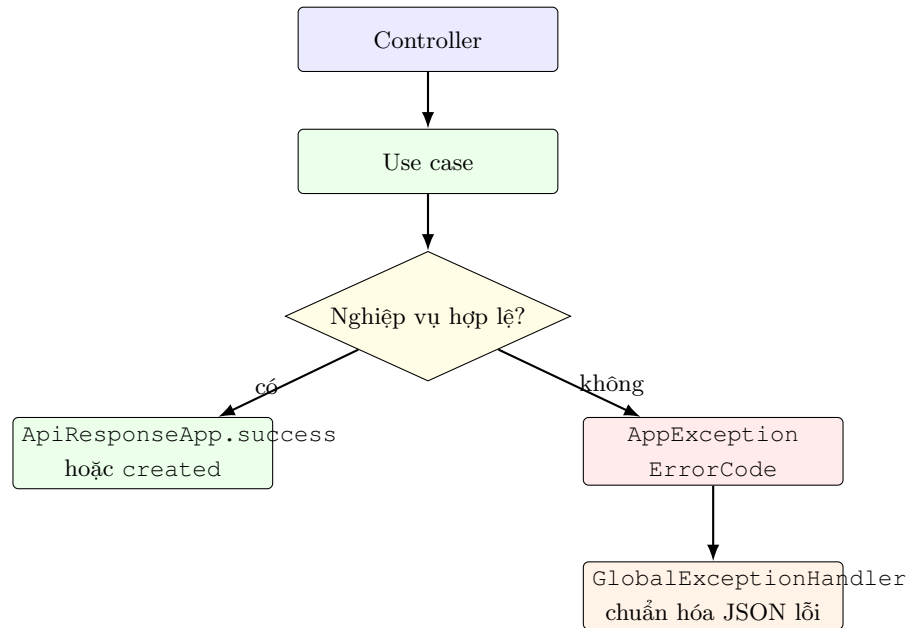
```
@Aspect
@Component
@RequiredArgsConstructor
public class SecurityAspect {
    private final SecurityEvaluator securityEvaluator;

    @Before("@annotation(requiresRole)")
    public void checkRole(RequiresRole requiresRole) {
        String roleNeeded = requiresRole.value();
        if (!securityEvaluator.hasRole(roleNeeded)) {
            throw new AppException(ErrorCode.FORBIDDEN);
        }
    }

    @Before("@annotation(requiresAuthentication)")
    public void checkAuthenticated(RequiresAuthentication annotation) {
        if (!securityEvaluator.isAuthenticated()) {
            throw new AppException(ErrorCode.UNAUTHENTICATED);
        }
    }
}
```

6.1.2.3 Chuẩn hóa response, lỗi và phân trang

Tất cả API trả về cấu trúc ApiResponseApp gồm code, message và data. Các API danh sách dùng PageResponse để thống nhất phân trang giữa frontend và backend. Lỗi nghiệp vụ được ném bằng AppException kèm ErrorCode; GlobalExceptionHandler chuyển exception thành response JSON theo HTTP status tương ứng.



Hình 6.27: Luồng chuẩn hóa response và exception

Bảng 6.33: Nhóm mã lỗi nghiệp vụ

Khoảng mã	Nhóm lỗi
1100–1199	Người dùng: trùng email, tài khoản chưa kích hoạt, sai thông tin đăng nhập.
1400–1499	Xác thực: thiếu token, token hết hạn, refresh token thất bại.
2000–2599	Catalog: ấn phẩm, bản sao, tác giả, nhà xuất bản, danh mục, tag.
2700–2799	Rating: điều kiện đánh giá, cửa sổ chỉnh sửa, trùng đánh giá.
3000–3199	Circulation và notification: giao dịch mượn, đặt trước, phí phạt, thông báo.

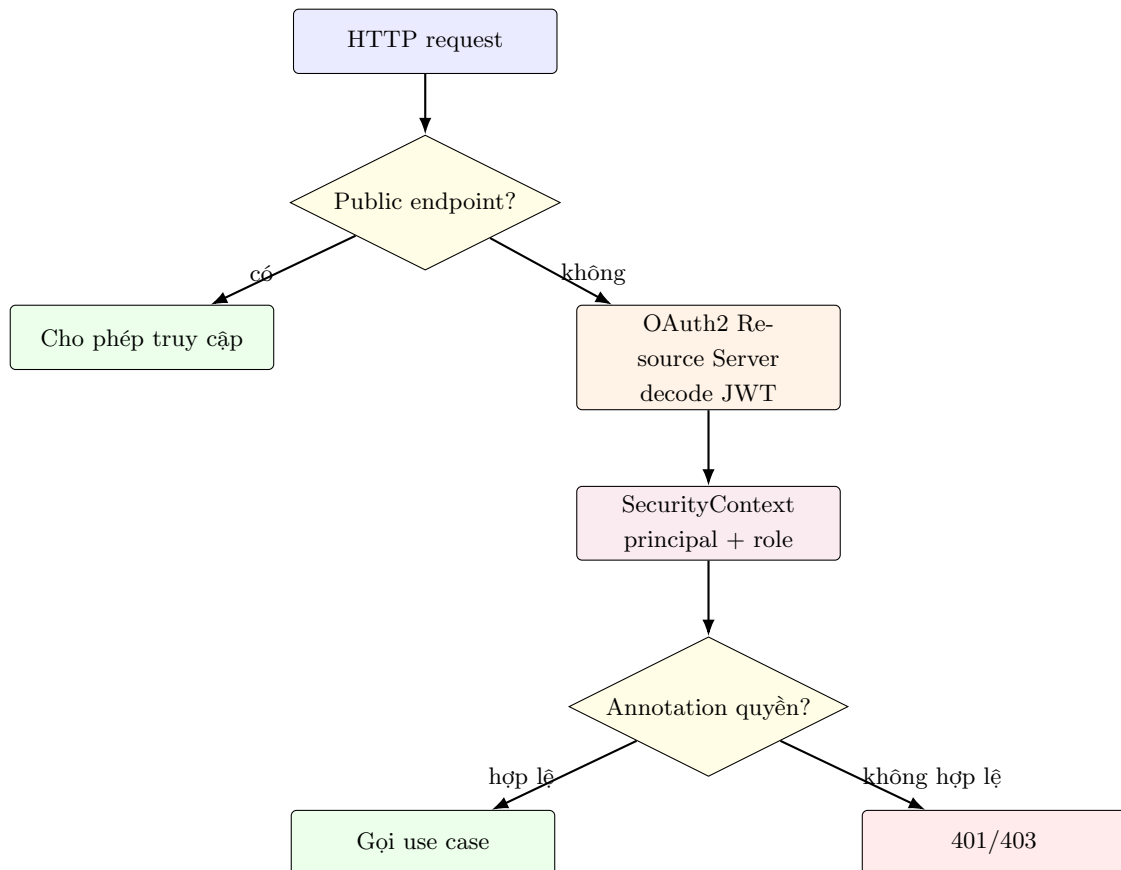
Việc chuẩn hóa lỗi được kết hợp với Accept-Language. Khi frontend gửi header ngôn ngữ, GlobalExceptionHandler chọn thông điệp tiếng Việt hoặc tiếng Anh từ ErrorCode; nhờ đó, frontend không cần tự ánh xạ toàn bộ mã lỗi nghiệp vụ.

6.1.2.4 Xác thực và phân quyền

Backend dùng Spring Security ở chế độ stateless. Các endpoint công khai, Swagger, WebSocket handshake, semantic search và webhook được khai báo trong SecurityConfig; các request còn lại yêu cầu JWT hợp lệ. JWT được giải mã bởi CustomJwtDecoder; CustomAuthenticationConverter chuyển claim trong token thành principal và authority dùng trong SecurityContext.

Phân quyền ở tầng use case/controller được hiện thực bằng annotation @RequiresAuthentication, @RequiresRole và AOP SecurityAspect. Cách này giữ kiểm tra quyền gần với endpoint nghiệp vụ và tái sử dụng cùng một SecurityEvaluator.

Trong bản hiện thực cuối, backend cũng kiểm soát trạng thái tài khoản ngay ở luồng đăng nhập. Khi tài khoản bị khóa hoặc không còn hoạt động, use case xác thực dừng trước bước phát hành token và trả lỗi nghiệp vụ để frontend hiển thị thông báo tài khoản đã bị khóa. Nhờ đó việc admin khóa tài khoản độc giả hoặc thủ thư có hiệu lực ngay ở lớp bảo mật, không chỉ là thay đổi hiển thị trên giao diện.



Hình 6.28: Luồng xác thực và phân quyền Backend

6.1.2.5 Admin, policy và audit vận hành

Để hệ thống có khả năng quản trị tập trung, backend bổ sung nhóm API admin chạy thật thay vì chỉ phục vụ UI tĩnh. Các API này chỉ cho phép role ADMIN truy cập, dùng cùng cơ chế JWT và AOP phân quyền với các module khác. Admin có thể tạo tài khoản thủ thư, xem/quản lý tài khoản độc giả và thủ thư, khóa/mở khóa tài khoản, xác minh hồ sơ, cấu hình quy định mượn trả và xem nhật ký thao tác.

Bảng 6.35: Nhóm API Backend cho Admin và quản trị vận hành

Endpoint	Vai trò hiện thực
GET/POST /api/v1/admin/librarians	Liệt kê và tạo tài khoản thủ thư, bắt buộc thông tin nghiệp vụ như mã thủ thư, số điện thoại, địa chỉ và ảnh đại diện.
GET /api/v1/admin/users	Tra cứu tài khoản đọc giả/thủ thư theo vai trò, trạng thái và từ khóa để phục vụ quản trị truy cập.
PATCH /api/v1/admin/users/{id}	Khóa, kích hoạt lại hoặc vô hiệu hóa tài khoản; trạng thái này được kiểm tra trong luồng đăng nhập.
PATCH /api/v1/admin/users/{id}	Xác minh tài khoản khi hồ sơ đã đầy đủ, giúp phân biệt tài khoản đã được kiểm duyệt với tài khoản mới tạo.
GET/PUT /api/v1/circulation-policies/admin	Đọc và cập nhật quy định mượn trả dùng chung cho đặt mượn online, mượn trực tiếp, xác nhận nhận sách, đặt trước, trả sách và tính phí trễ hạn.
GET /api/v1/admin/audit-logs	Trả nhật ký thao tác có actor, vai trò, action, phân hệ, đối tượng, nội dung tóm tắt và thời điểm để admin lọc/tìm kiếm/sắp xếp.

Quy định mượn trả được lưu trong bảng `circulation_policies` thay vì hard-code trong use case. Các use case mượn, đặt trước, xác nhận nhận sách và trả sách đều đọc qua `CirculationPolicyService`. Điều này bảo đảm khi admin thay đổi hạn nhận sách, số ngày mượn, giới hạn sách đang mượn/đặt trước hoặc mức phí trễ hạn, toàn bộ luồng nghiệp vụ dùng cùng một nguồn cấu hình.

Listing 6.11: Use case đọc quy định mượn trả từ DB

```
CirculationPolicy policy = policyService.getPolicy();

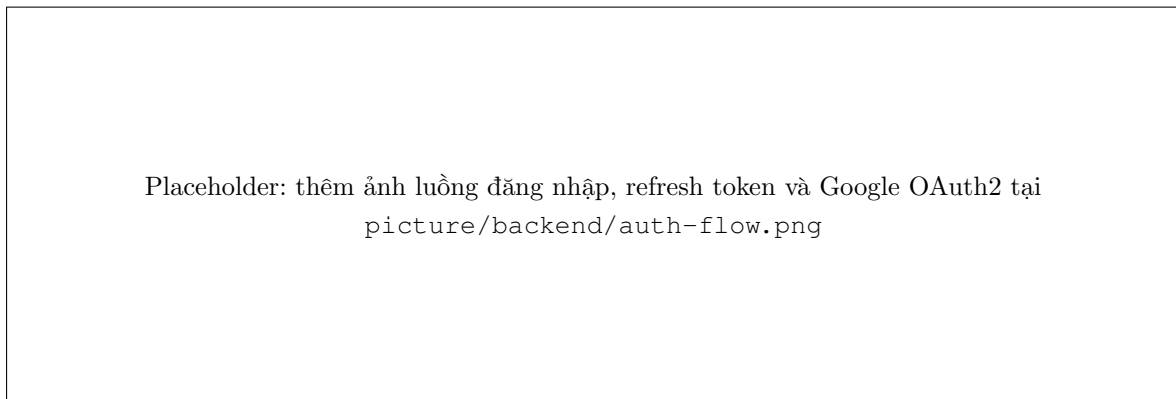
if (activeBorrows >= policy.maxActiveBorrows()) {
    throw new ApplicationException(ErrorCode.USER_BORROW_LIMIT_EXCEEDED);
}

if (Boolean.TRUE.equals(policy.blockBorrowWhenUnpaidFines())) {
    Long unpaidFines = fineQuery.countUnpaidFines(userId);
    if (unpaidFines > 0) {
        throw new ApplicationException(ErrorCode.USER_HAS_UNPAID_FINES);
    }
}
```

Nhật ký vận hành được lưu vào bảng `audit_logs`. Audit logging ghi lại những thao tác quan trọng như đăng nhập/đăng xuất, đổi mật khẩu, cập nhật hồ sơ, upload avatar, mượn/trả sách, thu phí, rating, phản hồi rating, tạo/sửa ấn phẩm, thay đổi quy định và quyết định khóa/xác minh tài khoản. Backend lưu cả thông tin để đọc cho admin và dữ liệu kỹ thuật cần thiết để đối soát: endpoint, HTTP method, mã phản hồi, entity type và entity id.

Bảng 6.37: Các endpoint xác thực chính

Endpoint	Xử lý
POST /api/v1/auth/login	Xác thực email/mật khẩu, phát hành access token và refresh token.
POST .../refresh- accesstoken	Nhận refresh token qua header re-token, cấp cập token mới.
POST /api/v1/auth/logout	Vô hiệu hóa refresh token hiện tại.
GET .../login-with- social	Sinh URL đăng nhập Google OAuth2.
POST .../social- callback/{id}	Xử lý authorization code, tạo hoặc đăng nhập tài khoản social, đồng thời vẫn kiểm tra trạng thái tài khoản trước khi trả token.



Hình 6.29: Luồng xác thực Backend

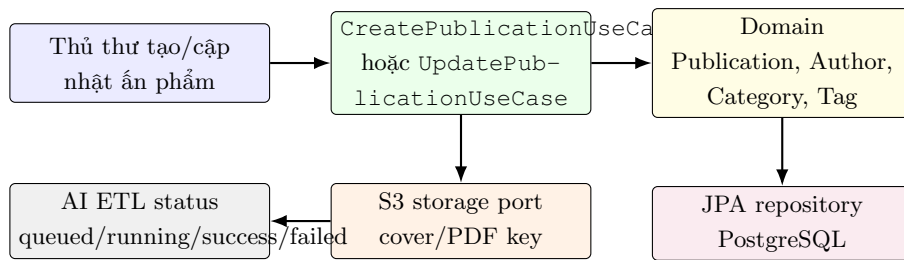
6.1.2.6 Hiện thực Catalog module

Nhóm đã hiện thực Catalog module để quản lý dữ liệu ấn phẩm và bản sao. Controller chính là PublicationController; các controller phụ xử lý tác giả, nhà xuất bản, danh mục, tag và item. Module này cung cấp API công khai cho tìm kiếm/xem chi tiết sách và API thủ thư cho quản lý metadata.

Bảng 6.39: Nhóm API của Catalog module

Nhóm endpoint	Use case được kích hoạt
GET /api/v1/publications/search	Tìm kiếm công khai theo từ khóa, tag, danh mục, ngôn ngữ, năm, chi nhánh, trạng thái còn bản sao; keyword match cả dữ liệu gốc và bảng dịch.
GET /api/v1/publications/{id}	Lấy chi tiết ấn phẩm, metadata dịch theo Accept-Language.
GET .../publications/librarian	Trả danh sách ấn phẩm cho thủ thư kèm số bản, trạng thái xử lý AI và bộ lọc quản trị.
POST/PUT/DELETE .../publications	Tạo, cập nhật và xóa ấn phẩm, kiểm tra ràng buộc bản sao và đặt trước đang hoạt động.
GET .../document-upload-url, PUT .../document-url	Sinh presigned URL cho PDF và lưu s3Key sau khi frontend upload xong.
POST .../{id}/cover	Upload ảnh bìa qua multipart request.

Ở bản final, các thao tác tạo và cập nhật ấn phẩm cũng lưu lại người tạo và người chỉnh sửa cuối cùng. Thông tin này giúp admin hoặc thủ thư trưởng truy vết metadata sách được nhập bởi ai, lần sửa cuối do ai thực hiện, đồng bộ với hướng quản trị vận hành bằng audit log của toàn hệ thống.



Hình 6.30: Luồng hiện thực quản lý ấn phẩm trong Catalog module

Search trong Catalog module được hiện thực bằng SQL động trong SearchPublicationsUseCaseImpl. Khi có keyword, backend match trên nhan đề, phụ đề, mô tả, ISBN, tác giả, tag và category. Đối với tag/category, truy vấn không chỉ dùng bản hiển thị theo Accept-Language; nó match cả tên gốc tiếng Việt trong tags/categories và mọi alias trong tag_translations/category_translations. Nhờ vậy truy vấn web development vẫn tìm được tag Lập Trình Web, và truy vấn tiếng Việt vẫn tìm được tài liệu đang hiển thị ở giao diện tiếng Anh. Khi sắp xếp theo liên quan, thứ tự ưu tiên là nhan đề/phụ đề, exact tag, partial tag, sau đó mới đến tiêu chí phụ như năm xuất bản.

Đối với upload tài liệu PDF, backend không nhận trực tiếp file lớn qua server. GetDocumentUploadUrlUseCase sinh presigned PUT URL thông qua StoragePort. Frontend upload file trực tiếp lên S3, sau đó gọi SaveDocumentUrlUseCase để lưu s3Key. Cách này giảm tải băng thông cho backend và vẫn giữ quyền kiểm soát metadata trong PostgreSQL.

Listing 6.12: Sinh presigned URL để upload tài liệu PDF

```

@Override
public String generatePresignedPutUrl(String s3Key, long ttlSeconds) {
    PutObjectPresignRequest request = PutObjectPresignRequest.builder()
        .signatureDuration(Duration.ofSeconds(ttlSeconds))
        .putObjectRequest(PutObjectRequest.builder()
            .bucket(bucket)
  
```

```
.key(s3Key)
    .build()
    .build();

PresignedPutObjectRequest presigned = s3Presigner.presignPutObject(request);
return presigned.url().toString();
}
```

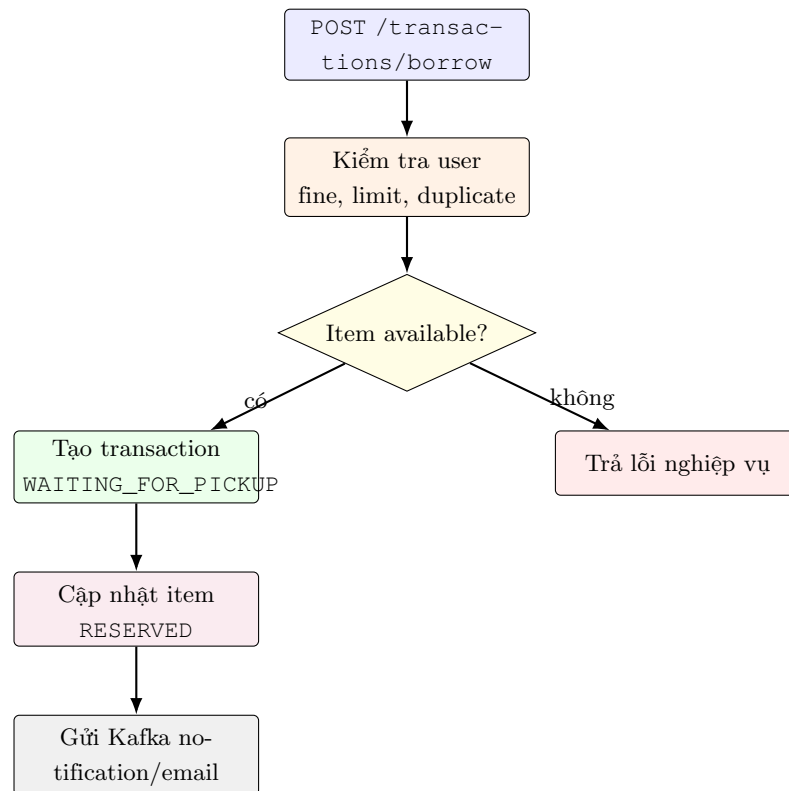
Placeholder: thêm ảnh Swagger nhóm API Catalog tại
picture/backend/catalog-swagger.png

Hình 6.31: Nhóm API Catalog trên Swagger

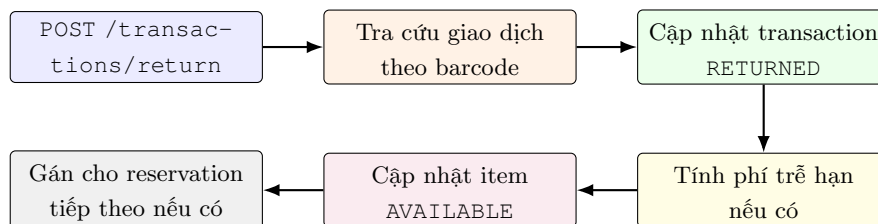
6.1.2.7 Hiện thực Circulation module

Nhóm đã hiện thực Circulation module để xử lý vòng đời mượn-trả, đặt trước, phí phạt và dashboard thủ thư. Các use case trong module này dùng @Transactional để đảm bảo cập nhật giao dịch, bản sao, phí phạt và notification event trong cùng một ranh giới nghiệp vụ.

Các quy định mượn trả trong module này không còn là hằng số cố định trong mã nguồn. BorrowRequestUseCase, DirectBorrowUseCase, ConfirmPickupUseCase, CreateReservationUseCase, ConfirmReservationPickupUseCase và ReturnBookUseCase đều đọc CirculationPolicyService. Nhờ vậy cùng một cấu hình từ admin được áp dụng thống nhất cho mượn online, mượn trực tiếp, đặt trước, hạn nhận sách, giới hạn đang mượn/đang chờ nhận và phí trễ hạn.



Hình 6.32: Luồng xử lý yêu cầu mượn sách



Hình 6.33: Luồng trả sách và xử lý hàng chờ

Bảng 6.41: Các nhóm use case chính trong Circulation module

Nhóm	Use case tiêu biểu
Mượn sách	BorrowRequestUseCase, DirectBorrowUseCase, LookupForPickupUseCase, ConfirmPickupUseCase.
Trả sách	LookupActiveTransactionUseCase, GetStudentActiveTransactionsUseCase, ReturnBookUseCase, ReportIssueUseCase.
Đặt trước	CreateReservationUseCase, CancelReservationUseCase, LookupReservationForPickupUseCase, ConfirmReservationPickupUseCase.
Phí phạt	GetMyFinesUseCase, GetStudentFinesUseCase, PayFineUseCase, PayAllFinesUseCase, FinePaymentService.
Ghi chú giao dịch	TransactionNoteController và bảng transaction_notes lưu nhiều ghi chú theo từng thủ thư trên cùng một giao dịch.
Dashboard	DashboardSummaryUseCase, DashboardChartsUseCase, DashboardRiskyUsersUseCase.

Bên cạnh API CRUD, module này chứa nhiều rule nghiệp vụ có điều kiện. Đoạn mã sau tóm tắt các bước chính trong luồng yêu cầu mượn sách: kiểm tra phí phạt, kiểm tra trạng thái item, tạo transaction, cập nhật item và phát thông báo.

Listing 6.13: Cấu trúc xử lý use case mượn sách

```
@Transactional
public BorrowTransactionResponse execute(Long userId, BorrowRequestCommand command) {
    if (fineRepository.hasUnpaidFine(userId)) {
        throw new AppException(ErrorCode.USER_HAS_UNPAID_FINES);
    }

    ItemSnapshot item = itemStatusPort.getSnapshot(command.itemId());
    if (!"AVAILABLE".equals(item.status())) {
        throw new AppException(ErrorCode.ITEM_NOT_BORROWABLE);
    }

    BorrowingTransaction tx = BorrowingTransaction.waitingForPickup(
        userId,
        command.itemId(),
        LocalDate.now().plusDays(14)
    );
    borrowingTransactionRepository.save(tx);
    itemStatusPort.updateStatus(command.itemId(), "RESERVED");

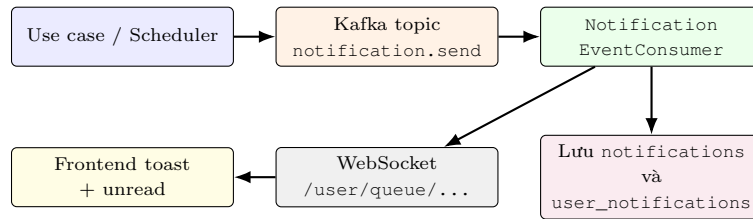
    kafkaTemplate.send(KafkaTopics.NOTIFICATION_SEND,
        NotificationMessage.borrowSuccess(userId, tx.getId()));
    return mapper.toResponse(tx);
}
```

Placeholder: thêm ảnh Swagger nhóm API mượn-trả, đặt trước và phí phạt tại
picture/backend/circulation-swagger.png

Hình 6.34: Nhóm API Circulation trên Swagger

6.1.2.8 Scheduler, notification và WebSocket

Các tác vụ nền trong circulation được hiện thực bằng Spring Scheduler. Scheduler đánh dấu giao dịch quá hạn, gửi nhắc hạn trả, hủy phiếu chờ nhận quá hạn và hủy đặt trước quá hạn. Khi cần gửi thông báo, module phát message lên Kafka topic notification.send; NotificationEventConsumer ở user module lưu thông báo vào database và đẩy dữ liệu đến người dùng qua WebSocket.



Hình 6.35: Luồng phát thông báo bất đồng bộ

Bảng 6.43: Scheduler được hiện thực trong Circulation module

Scheduler	Xử lý	Chu kỳ
OverdueMarkingScheduler	Chuyển giao dịch BORROWING quá hạn sang OVERDUE.	01:00 hằng ngày.
DueDateWarningScheduler	Gửi thông báo/email trước hạn trả 3 ngày và 1 ngày.	08:00 hằng ngày.
ExpiredPickupScheduler	Hủy giao dịch WAITING_FOR_PICKUP quá hạn, trả item về AVAILABLE, thử gán cho hàng chờ.	Mỗi 1 giờ.
ExpiredReservationScheduler	Hủy lượt đặt trước quá hạn và giải phóng lượt tiếp theo trong hàng chờ.	Mỗi 1 giờ.

Đối với thông báo, nhóm đã chọn Kafka thay vì gọi trực tiếp WebSocket trong từng use case. Phương án này tách side effect khỏi transaction chính: use case chỉ phát event, còn user module chịu trách nhiệm lưu notification và đẩy WebSocket.

Listing 6.14: Consumer lưu notification và đẩy WebSocket

```

@KafkaListener(topics = KafkaTopics.NOTIFICATION_SEND,
               groupId = "${spring.kafka.consumer.group-id}")
@Transactional
public void handle(NotificationMessage message) {
    NotificationEntity notification = NotificationEntity.builder()
        .type(NotificationType.valueOf(message.type()))
        .title(message.title())
        .message(message.message())
        .link(message.link())
        .referenceId(message.referenceId())
        .build();
    notificationJpaRepository.save(notification);

    UserNotificationEntity userNotification = UserNotificationEntity.builder()
        .userId(message.userId())
        .notificationId(notification.getId())
        .isRead(false)
        .build();
    userNotificationJpaRepository.save(userNotification);

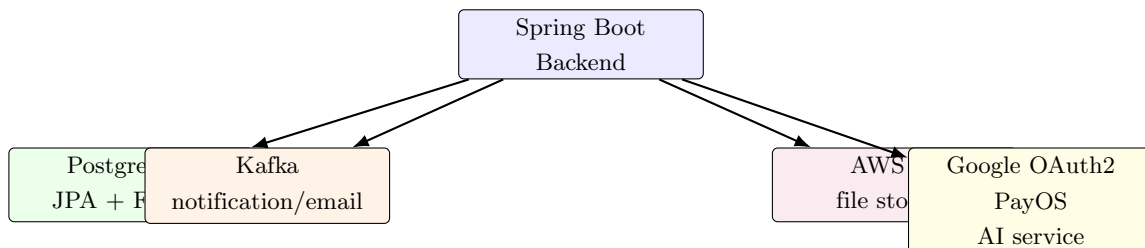
    messagingTemplate.convertAndSendToUser(
        message.userId().toString(),
        "/queue/notifications",
        mapper.toResponse(notification, userNotification)
    );
}
  
```

Placeholder: thêm ảnh bảng notification hoặc WebSocket log tại
picture/backend/notification-db.png

Hình 6.36: Dữ liệu thông báo sau khi Kafka consumer xử lý

6.1.2.9 Tích hợp dịch vụ ngoài

Các dịch vụ ngoài được đặt ở tầng infrastructure và được gọi thông qua adapter hoặc port. S3 được dùng cho ảnh đại diện, bìa sách và tài liệu PDF. PayOS được dùng cho thanh toán phí phạt. Google OAuth2 được dùng cho đăng nhập social. AI service được gọi qua AiGatewayService cho semantic search và cập nhật trạng thái xử lý tài liệu qua callback có chữ ký HMAC.



Hình 6.37: Các tích hợp hạ tầng và dịch vụ ngoài của Backend

Bảng 6.45: Cấu hình hạ tầng Backend

Thành phần	Cấu hình/hiện thực
Database	PostgreSQL, spring.jpa.hibernate.ddl-auto=validate, schema được quản lý bằng Flyway migration.
Kafka	Producer/consumer dùng JSON serializer, group id library-group, topic dùng cho notification và email.
S3	AWS SDK v2, S3Client, S3Presigner, bucket cấu hình qua spring.aws.s3.bucket.
Mail	SMTP Gmail qua Spring Mail, dùng cho xác thực email, quên mật khẩu và nhắc hạn trả.
OpenAPI	springdoc-openapi-starter-webmvc-ui, phục vụ tài liệu API cho kiểm thử và tích hợp frontend.
Deployment	Docker build JAR từ library-bootstrap, ứng dụng chạy cổng 8080.

Đối với callback từ AI service, nhóm không cho phép cập nhật trạng thái xử lý tài liệu nếu request không có chữ ký hợp lệ. Header timestamp được kiểm tra để giảm rủi ro replay; chữ ký được tính bằng HMAC-SHA256 trên chuỗi timestamp.body.

Listing 6.15: Xác minh chữ ký callback từ AI service

```
private void verifySignature(String body, String timestampHeader, String signatureHeader)
{
    if (timestampHeader == null || signatureHeader == null) {
        throw new ResponseStatusException(HttpStatus.UNAUTHORIZED);
    }

    long timestamp = Long.parseLong(timestampHeader);
    long now = clock.instant().getEpochSecond();
    if (Math.abs(now - timestamp) > maxClockSkewSeconds) {
        throw new ResponseStatusException(HttpStatus.UNAUTHORIZED);
    }

    String expected = "sha256=" + hmacSha256(timestampHeader + "." + body,
        callbackSecret);
    if (!MessageDigest.isEqual(
        expected.getBytes(StandardCharsets.UTF_8),
        signatureHeader.getBytes(StandardCharsets.UTF_8))) {
        throw new ResponseStatusException(HttpStatus.UNAUTHORIZED);
    }
}
```

Placeholder: thêm ảnh log hoặc Postman callback AI tại picture/backend/ai-callback.png

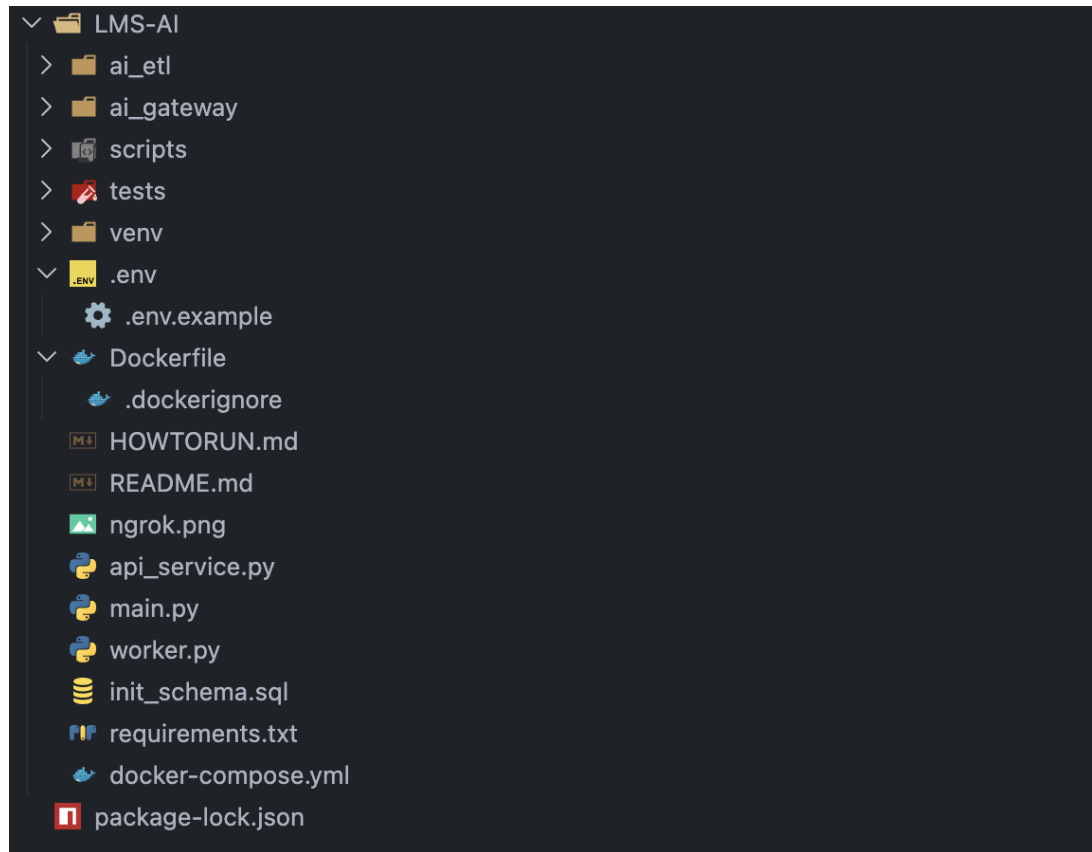
Hình 6.38: Callback cập nhật trạng thái xử lý tài liệu từ AI service

6.1.2.10 Tổng kết hiện thực Backend

Nhóm đã hiện thực backend như một modular monolith có phân tách bounded context rõ ràng. Spring Boot đảm nhiệm REST API, transaction, security, scheduler và tích hợp hạ tầng; từng module nghiệp vụ giữ use case và domain model riêng. Các luồng cốt lõi gồm xác thực, quản lý catalog, mượn-trả, đặt trước, phí phạt, thông báo thời gian thực, upload tài liệu và tích hợp AI đã được nối thông qua service, port, Kafka event và API contract ổn định.

6.1.3 Hiện thực AI Service

Nhóm đã tách AI service thành một tiến trình Python riêng để xử lý các tác vụ có đặc thù tính toán và phụ thuộc mô hình. Service này không thay thế backend nghiệp vụ; backend vẫn là nơi kiểm soát giao dịch, phân quyền và dữ liệu gốc. AI service đảm nhiệm ba nhóm chức năng: ETL tài liệu PDF thành vector và metadata AI, tìm kiếm ngữ nghĩa theo truy vấn tự nhiên, và gợi ý sách theo tương tác người dùng.



Hình 6.39: Cấu trúc thư mục mã nguồn AI service

6.1.3.1 Tổ chức module AI

Bảng 6.47: Các module chính của AI service

Module/file	Trách nhiệm hiện thực
api_service.py	FastAPI gateway cho semantic search, similar publications, recommendation và xử lý publication theo request đồng bộ.
worker.py	Celery worker nhận message xử lý sách, tải PDF, chạy ETL, gửi callback về backend bằng chữ ký HMAC.
ai_etl/pdf_processing.py	Tải hoặc đọc PDF, trích xuất text bằng PyMuPDF, làm sạch header/footer/số trang và chia chunk sliding window.
ai_etl/pipeline.py	Orchestrator của pipeline ETL: hash skip, embedding, chọn chunk đại diện, reduce bằng LLM, fallback local và ghi database.
ai_etl/embedding_client.py	Load SentenceTransformer và sinh vector 768 chiều đã normalize.
ai_etl/llm_client.py	Gọi Gemini, throttle/retry, ép output JSON và chuẩn hóa summary, audience, tag tiếng Việt cùng alias tags_en.
ai_etl/tag_quality.py	Lọc tag rác, chuẩn hóa alias Anh-Việt và map tag tiếng Việt sang thuật ngữ tiếng Anh phục vụ search song ngữ.
ai_gateway/recommender.py	Huấn luyện và lưu mô hình ALS implicit feedback từ bảng tương tác người dùng.
init_schema.sql	Khởi tạo schema ai_engine, bảng vector, ETL run, search log và index pgvector; dữ liệu tag/recommendation dùng bảng canonical ở schema public.

Bảng 6.49: Các quyết định kỹ thuật trong AI service

Quyết định	Phương án đã dùng	Lý do
Tách AI khỏi backend	FastAPI/Celery service riêng.	Backend Java giữ nghiệp vụ ổn định; AI có thể thay model, scale worker và cache model độc lập.
Vector database	PostgreSQL + pgvector thay vì vector DB riêng.	Dữ liệu thư viện đã nằm trong PostgreSQL, pgvector đủ cho phạm vi đồ án và giảm chi phí vận hành.
Embedding tiếng Việt	bkai-foundation-models/vietnamese-bi-encoder.	Mô hình phù hợp truy vấn tiếng Việt, sinh vector 768 chiều và hỗ trợ semantic search.
LLM reduce	Gemini với prompt JSON bắt buộc, có fallback local.	Tạo summary nội dung, audience, 5 tag tiếng Việt và tags_en trong cùng một lần gọi; pipeline vẫn hoàn tất khi LLM lỗi hoặc quá tải.
Recommendation	Hybrid: content-based realtime, ALS, trending fallback.	Phủ được cả người dùng mới, người dùng có ít tương tác và người dùng có lịch sử đủ lớn.

6.1.3.2 Schema dữ liệu AI và chỉ mục vector

Nhóm đã tạo schema `ai_engine` cho dữ liệu đặc thù AI như vector, ETL run và search log. Các dữ liệu biên mục dùng chung với backend như tag, liên kết tag-ấn phẩm và recommendation được chuẩn hóa về schema `public` để tránh tồn tại hai nguồn sự thật. Bảng vector của ấn phẩm lưu từng chunk của tài liệu cùng embedding vector (768). Chỉ mục HNSW trên embedding phục vụ truy vấn cosine distance; chỉ mục GIN trigram trên `chunk_text` phục vụ fallback keyword search.

Trong thiết kế cuối cùng, schema `ai_engine` không định nghĩa bảng tag hoặc bảng cache recommendation riêng. Các bảng `tags`, `publication_tags`, `tag_translations` và `ai_recommendations` thuộc schema `public`, vì đây là dữ liệu được backend, frontend và AI service cùng sử dụng.

Bảng 6.51: Các bảng dữ liệu AI quan trọng

Bảng	Vai trò
<code>ai_engine.publication_vectors</code>	Lưu <code>publication_id</code> , <code>chunk_text</code> , embedding vector (768) và thời điểm tạo vector.
<code>ai_engine.publication_etl_runs</code>	Theo dõi <code>file_hash</code> , trạng thái <code>RUNNING/SUCCESS/FAILED</code> , số chunk, số vector và lỗi nếu có.
<code>public.tags</code> , <code>publication_tags</code>	Lưu tag canonical bằng tiếng Việt và liên kết tối đa 5 tag với mỗi ấn phẩm.
<code>public.tag_translations</code>	Lưu alias tiếng Anh của tag để search hai chiều Anh-Việt mà không cần lưu tag tiếng Anh như một tag riêng.
<code>ai_engine.search_logs</code>	Ghi lại truy vấn semantic search và danh sách ấn phẩm trả về để phân tích sau.
<code>public.ai_recommendations</code>	Bảng chia sẻ kết quả gợi ý cho backend/frontend sử dụng trực tiếp.

Listing 6.16: Schema vector và các index chính

```
CREATE TABLE IF NOT EXISTS ai_engine.publication_vectors (
  id BIGSERIAL PRIMARY KEY,
  publication_id BIGINT REFERENCES public.publications(id) ON DELETE CASCADE,
  chunk_text TEXT NOT NULL,
  embedding vector(768) NOT NULL,
```

```
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX IF NOT EXISTS idx_publication_vectors_hnsw  
ON ai_engine.publication_vectors  
USING hnsw (embedding vector_cosine_ops);  
  
CREATE INDEX IF NOT EXISTS idx_publication_vectors_chunk_text_trgm  
ON ai_engine.publication_vectors  
USING gin (chunk_text gin_trgm_ops);
```

Placeholder: thêm ảnh database bảng ai_engine.publication_vectors tại
picture/ai/database-vectors.png

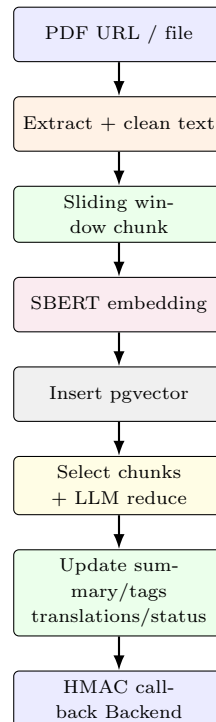
Hình 6.40: Dữ liệu vector của tài liệu trong PostgreSQL/pgvector

Placeholder: thêm ảnh database bảng ai_engine.publication_etl_runs tại
picture/ai/database-etl-runs.png

Hình 6.41: Trạng thái ETL của từng ấn phẩm trong database

6.1.3.3 Pipeline ETL tài liệu PDF

Pipeline ETL được kích hoạt khi thủ thư upload tài liệu PDF cho ấn phẩm. Backend đưa message vào queue hoặc gọi API xử lý; AI service tải file PDF, trích xuất text, chia chunk, sinh embedding, ghi vector, chọn chunk đại diện, gọi LLM để tạo summary nội dung, audience, tag tiếng Việt, alias tag tiếng Anh và callback về backend.



Hình 6.42: Luồng ETL tài liệu PDF trong AI service

Bảng 6.53: Các tham số ETL chính

Tham số	Giá trị/ý nghĩa
AI_ETL_CHUNK_SIZE	Mặc định 1500 ký tự, tạo đoạn đủ dài để giữ ngữ cảnh.
AI_ETL_CHUNK_OVERLAP	Mặc định 150 ký tự, tránh mất ý tại biên chunk.
AI_ETL_VECTOR_INSERT_BATCH_SIZE	Mặc định 256, ghi vector theo lô để giảm số round-trip database.
AI_ETL_REDUCE_MAX_CHUNKS	Mặc định 12, giới hạn số chunk đưa vào LLM.
AI_ETL_REDUCE_TOTAL_CHARS_BUDGET	Mặc định 10000, kiểm soát token/cost của prompt.
AI_ETL_ENABLE_HASH_SKIP	Nếu hash PDF không đổi và đã có output, pipeline bỏ qua reprocess.
AI_ETL_ENABLE_ENGLISH_TRANSLATION	Mặc định tắt; chỉ bật khi muốn gọi thêm LLM để dịch toàn bộ title/description/summary. Alias tag tiếng Anh vẫn được sinh trong reduce chính nên không cần call riêng.

6.1.3.4 Làm sạch PDF và chia chunk

PDF thường chứa header/footer lặp, số trang, lỗi xuống dòng và ký tự null. Nhóm đã xử lý nhiều trước khi chunk để vector biểu diễn nội dung thật của tài liệu thay vì thông tin trình bày. Sau khi làm sạch, text được chia bằng sliding window theo chunk_size và overlap.

Listing 6.17: Làm sạch PDF và chia chunk sliding window

```
def sliding_window_chunk(text, chunk_size, overlap):
    if not text:
```

```
return []

step = chunk_size - overlap
chunks = []
start = 0
index = 1

while start < len(text):
    end = min(start + chunk_size, len(text))
    chunk_text = text[start:end].strip()
    if chunk_text:
        chunks.append(Chunk(chunk_id=f"c{index:05d}", text=chunk_text))
        index += 1
    if end >= len(text):
        break
    start += step

return chunks
```

6.1.3.5 Sinh embedding và ghi vector

Embedding được sinh bằng SentenceTransformer. Các vector được normalize trước khi ghi database, phù hợp với cosine distance trong pgvector. Pipeline không ghi từng vector đơn lẻ mà gom theo batch để giảm thời gian insert.

Listing 6.18: Sinh embedding cho các chunk và ghi batch

```
chunk_texts = [chunk.text for chunk in chunks]
vectors = embedding_client.encode(chunk_texts)

vector_rows = list(zip(chunk_texts, vectors))
for batch_rows in iter_batches(vector_rows, vector_insert_batch_size):
    database.bulk_insert_vectors(conn, publication_id, batch_rows)
```

6.1.3.6 Chọn chunk đại diện và reduce bằng LLM

Nếu đưa toàn bộ PDF vào LLM, chi phí và độ trễ sẽ tăng theo độ dài tài liệu. Nhóm đã chọn chiến lược reduce-first: chỉ chọn một số chunk đại diện dựa trên độ gần với centroid của toàn bộ tài liệu và ràng buộc đa dạng để tránh chọn nhiều chunk trùng ý. Sau đó từng chunk được cắt theo ngân sách ký tự trước khi đưa vào Gemini.

Listing 6.19: Chọn chunk đại diện bằng relevance và diversity

```
embeddings = np.array(vectors, dtype=np.float32)
target = min(max_chunks, max(8, int(math.sqrt(len(chunks)) * 2)), len(chunks))

centroid = embeddings.mean(axis=0)
centroid = centroid / np.linalg.norm(centroid)
similarities = embeddings @ centroid
selected = []
for seed_idx in range(min(2, len(chunks))):
    selected.append(seed_idx)
```

```
centroid_idx = int(np.argmax(similarities))
if centroid_idx not in selected:
    selected.append(centroid_idx)

lambda_weight = 0.65
while len(selected) < target:
    best_idx, best_score = None, float("-inf")
    for idx in candidate_indices:
        relevance = float(similarities[idx])
        diversity = max(float(embeddings[idx] @ embeddings[s]) for s in selected)
        score = lambda_weight * relevance - (1.0 - lambda_weight) * diversity
        if score > best_score:
            best_idx, best_score = idx, score
    selected.append(best_idx)
    candidate_indices.remove(best_idx)
```

LLM được yêu cầu trả về đúng một JSON object gồm master_summary, audience, tags và tags_en. Prompt bắt buộc summary chỉ dựa trên CONTENT_EXCERPTS, không ghép nhan đề/tác giả/category để tạo câu mô tả giả. Tag gốc luôn là tiếng Việt, đúng 5 tag, không dùng tên sách, tên tác giả, tên category rộng hoặc các nhãn chung chung. tags_en là alias tiếng Anh theo cùng thứ tự, dùng thuật ngữ người học hay tìm kiếm như Web Development, Machine Learning. Nếu Gemini lỗi, hết quota hoặc trả JSON không hợp lệ, pipeline chuyển sang fallback local dựa trên nội dung trích xuất, từ khóa chuyên ngành và heuristic khoa; fallback cũng sinh alias tiếng Anh bằng bảng map cấu hình sẵn.

Bảng 6.55: Kết quả AI ghi ngược vào dữ liệu ẩn phẩm

Output	Nguồn sinh	Cách sử dụng trong hệ thống
master_summary	LLM reduce hoặc fallback local.	Hiển thị tóm tắt nội dung và hỗ trợ keyword fallback search.
audience	LLM chọn một mã khoa hoặc heuristic keyword.	Gợi ý nhóm đọc giả phù hợp với ấn phẩm.
tags	LLM sinh đúng 5 tag tiếng Việt hoặc fallback từ nội dung/chuyên ngành.	Ghi vào public.tags và publication_tags; frontend hiển thị chip tag và cho bấm tìm kiếm.
tags_en	LLM trả alias tiếng Anh cùng thứ tự hoặc map từ tag_quality.py.	Ghi vào public.tag_translations để truy vấn Anh-Việt match cùng một tag canonical.
file_hash, chunks_count, vectors_count	Pipeline ETL.	Theo dõi trạng thái xử lý và quyết định skip khi file không đổi.

Placeholder: thêm ảnh database summary, audience và tag AI tại
picture/ai/database-tags-summary.png

Hình 6.43: Summary, audience và tag AI sau khi ETL hoàn tất

6.1.3.7 Worker bất đồng bộ và callback an toàn

Tác vụ xử lý PDF có thể kéo dài nên nhóm đã hiện thực Celery worker cho luồng bất đồng bộ. Worker nhận message gồm book_id, pdf_url và webhook_url, tải PDF về thư mục tạm, chạy pipeline và gửi callback SUCCESS hoặc FAILED. Callback được ký bằng HMAC-SHA256 trên chuỗi timestamp.body; backend kiểm tra chữ ký trước khi cập nhật trạng thái AI.

Listing 6.20: Tạo chữ ký HMAC cho callback về Backend

```
def build_webhook_headers(payload_body, secret):
    timestamp = str(int(time.time()))
    signed_payload = f"{timestamp}.{payload_body}".encode("utf-8")
    signature = hmac.new(
        secret.encode("utf-8"),
        signed_payload,
        hashlib.sha256
    ).hexdigest()
    return {
        "Content-Type": "application/json",
        "X-AI-Callback-Timestamp": timestamp,
        "X-AI-Callback-Signature": f"sha256={signature}",
    }
```

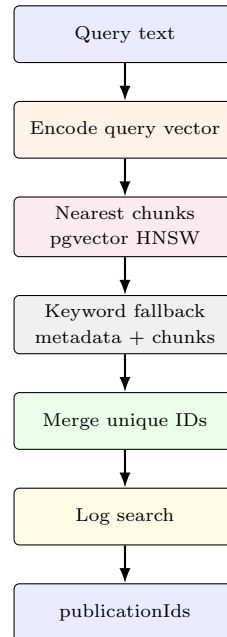
6.1.3.8 Tìm kiếm ngữ nghĩa

Semantic search nhận truy vấn tự nhiên từ frontend/backend. AI service encode truy vấn thành vector, tìm các chunk gần nhất bằng toán tử $\leq$ của pgvector, gom kết quả theo publication_id, lấy khoảng cách nhỏ nhất của mỗi ấn phẩm và trả về danh sách ID. Đây là lớp mở rộng độ phủ khi keyword search không bắt được cách diễn đạt gần nghĩa. Ở trải nghiệm tìm kiếm cuối cùng, frontend/backend vẫn ưu tiên lexical search theo keyword/tag/category trước, semantic search chỉ bổ sung kết quả chưa trùng. Để tăng độ ổn định, service kết hợp ba nguồn kết quả: nearest vector chunks, keyword fallback trên metadata và keyword search trên chunk_text.

Listing 6.21: Truy vấn semantic search bằng pgvector

```
WITH nearest_chunks AS (
    SELECT publication_id, embedding <=> %s::vector AS distance
    FROM ai_engine.publication_vectors
```

```
ORDER BY embedding <=> %s::vector
LIMIT %s
)
SELECT publication_id, MIN(distance) AS min_distance
FROM nearest_chunks
GROUP BY publication_id
HAVING MIN(distance) <= %s
ORDER BY min_distance ASC
LIMIT %s;
```



Hình 6.44: Luồng xử lý semantic search

Placeholder: thêm ảnh Postman/Swagger gọi semantic search tại
picture/ai/semantic-search-api.png

Hình 6.45: Kết quả API semantic search

6.1.3.9 Tìm sách tương tự

Với chức năng sách tương tự, service lấy tối đa 80 vector đầu tiên của ấn phẩm, tính vector hồ sơ bằng trung bình vector, normalize và tìm các chunk gần nhất của ấn phẩm khác. Kết quả chỉ giữ các sách còn bản sao AVAILABLE; nếu không đủ số lượng, hệ thống bổ sung bằng trending books.

Listing 6.22: Tính profile vector cho sách tương tự

```
vectors = np.array([np.asarray(row[0], dtype=np.float32) for row in rows])
profile = vectors.mean(axis=0)
norm = np.linalg.norm(profile)
if norm == 0:
    return []
profile = profile / norm
```

6.1.3.10 Gợi ý sách hybrid

Recommendation service được thiết kế theo nhiều lớp để xử lý các tình huống dữ liệu khác nhau. Nếu người dùng chưa có tương tác, hệ thống dùng trending fallback. Nếu đã có tương tác, service tạo profile vector realtime từ các sách đã xem, wishlist hoặc mượn; đồng thời gọi mô hình ALS đã huấn luyện từ implicit feedback. Kết quả content-based được ưu tiên, sau đó hợp nhất với ALS và bổ sung trending nếu chưa đủ số lượng.

Bảng 6.57: Chiến lược gợi ý sách

Strategy	Điều kiện dùng	Nguồn xếp hạng
TRENDING _FALLBACK	Người dùng chưa có tương tác hoặc không có vector/model phù hợp.	Tổng hợp lượt xem, wishlist, mượn, rating và ngày tạo.
REALTIME _CONTENT_BASED	Người dùng có tương tác và có vector sách tương ứng.	Trung bình có trọng số của vector sách đã tương tác.
ALS_HYBRID	Mô hình ALS có user/item profile đủ dữ liệu.	Collaborative filtering từ ma trận implicit feedback.
REALTIME_CBF _ALS_HYBRID	Cả content-based và ALS đều trả kết quả.	Hợp nhất CBF trước, ALS sau, loại trùng và lọc sách còn bản sao.

Listing 6.23: Tạo profile người dùng từ tương tác có trọng số

```
SELECT pv.embedding,
       CASE ui.type
         WHEN 'WATCH' THEN 1.0
         WHEN 'WISHLIST' THEN 5.0
         WHEN 'BORROWED' THEN 10.0
         ELSE 1.0
       END AS weight
FROM public.user_interactions ui
JOIN ai_engine.publication_vectors pv
  ON pv.publication_id = ui.publication_id
WHERE ui.user_id = %s
LIMIT 300;

profile = np.average(vectors, axis=0, weights=weights)
profile = profile / np.linalg.norm(profile)
```

Listing 6.24: Huấn luyện ALS từ implicit feedback

```
for user_id, publication_id, score in rows:
    row_idx.append(user_id_to_idx[int(user_id)])
    col_idx.append(item_id_to_idx[int(publication_id)])
    data.append(1.0 + als_alpha * float(score))
```

```
user_items = csr_matrix((data, (row_idx, col_idx)), shape=(n_users, n_items))
model = implicit.als.AlternatingLeastSquares(
    factors=als_factors,
    regularization=als_regularization,
    iterations=als_iterations,
    random_state=42,
)
model.fit(user_items.T.tocsr())
```

Placeholder: thêm ảnh database public.ai_recommendations tại
picture/ai/database-recommendation-cache.png

Hình 6.46: Kết quả gợi ý sách trong bảng public.ai_recommendations

6.1.3.11 API contract và kiểm thử độ trễ

AI service cung cấp các API contract ổn định để backend gọi. Các response chỉ trả về danh sách mã ấn phẩm và strategy; backend giữ trách nhiệm lấy metadata đầy đủ và kiểm soát quyền truy cập. Nhóm cũng hiện thực test hợp đồng để kiểm tra /health, semantic search và recommendation với ngưỡng độ trễ mặc định 200ms trong môi trường live test.

Bảng 6.59: API chính của AI service

Endpoint	Input	Output
GET /health	Không có.	{"status":"ok"}.
POST /api/v1/publications/process	publication_id, pdf_url, force_reprocess.	Trạng thái xử lý, số chunk/vector, tags, tags_en và audience.
POST /api/v1/semantic-search	query_text, limit.	publication_ids.
GET /api/v1/publications/{id}/similar	publication_id, limit.	publication_ids.
POST /api/v1/recommendations	user_id, limit.	publication_ids, strategy.

6.1.3.12 Tổng kết hiện thực AI Service

Nhóm đã hiện thực AI service như một lớp xử lý độc lập nhưng liên kết chặt với backend qua database, API và callback. Các điểm có giá trị chính gồm: ETL PDF có làm sạch text và hash skip, lưu embedding bằng pgvector/HNSW, chọn chunk đại diện trước khi gọi LLM, fallback local khi LLM lỗi, semantic search kết hợp vector và keyword, similar books theo profile vector, recommendation hybrid giữa content-based, ALS và trending fallback. Các bảng publication_vectors, publication_etl_runs, search_logs và ai_recommendations giúp kết quả AI có thể được kiểm chứng trực tiếp từ database thay vì chỉ quan sát trên giao diện.

Chương 7: Kiểm thử phần mềm

7.1 Kiểm thử Frontend

Nhóm đã kiểm thử frontend ở mức unit, context, component và service contract bằng Jest, Testing Library và môi trường jsdom. Mục tiêu của phần kiểm thử này là bắt các lỗi dễ tái xuất hiện trong giao diện: ánh xạ lỗi nghiệp vụ từ backend sang thông báo người dùng, trạng thái đăng nhập, đa ngôn ngữ Việt-Anh, light/dark/system theme, upload panel, nội dung notification, Google Maps ở trang liên hệ, service layer gọi đúng endpoint, admin audit/policy contract và các helper xử lý trạng thái đặt trước.

Khác với kiểm thử thủ công chỉ quan sát giao diện, các test này kiểm tra trực tiếp logic dùng chung trong utils và contexts. Đây là những vị trí có phạm vi ảnh hưởng rộng: nếu LanguageContext sai, nhiều trang bị sai ngôn ngữ; nếu ThemeContext sai, toàn bộ SPA bị lệch theme; nếu getFriendlyErrorMessage sai, cùng một lỗi backend có thể bị hiển thị thành thông báo không đúng nghiệp vụ. Ngoài ra, các test service contract giúp nhóm kiểm soát URL, query string và payload gửi về backend/AI service trước khi chạy tích hợp thật.

7.1.1 Môi trường và cấu hình kiểm thử

Frontend dùng jest.config.cjs để chạy các file *.test.ts và *.test.tsx. Môi trường jsdom được chọn vì các context cần thao tác với localStorage, document.html, matchMedia và DOM giống trình duyệt. File src/test/setup.ts nạp @testing-library/jest-dom để assertion có thể viết gần với hành vi giao diện hơn, ví dụ toHaveTextContent và toHaveClass.

Bảng 7.1: Cấu hình kiểm thử Frontend

Thành phần	Cấu hình sử dụng
Test runner	Jest, chạy bằng script npm test --runInBand.
Biên dịch test	ts-jest với JSX mode react-jsx.
Môi trường DOM	jest-environment-jsdom, hỗ trợ kiểm tra DOM, localStorage và document.
Assertion UI	@testing-library/jest-dom.
Mock style	identity-obj-proxy cho file CSS/SCSS.
Alias import	@/(.*) trở về thư mục gốc frontend.

Listing 7.1: Cấu hình Jest cho Frontend

```
module.exports = {
  preset: 'ts-jest',
  testEnvironment: 'jest-environment-jsdom',
  setupFilesAfterEnv: ['<rootDir>/src/test/setup.ts'],
  moduleNameMapper: {
    '\\.(css|less|scss|sass)$': 'identity-obj-proxy',
    '^@/(.*)$': '<rootDir>/$1',
  },
  transform: {
```

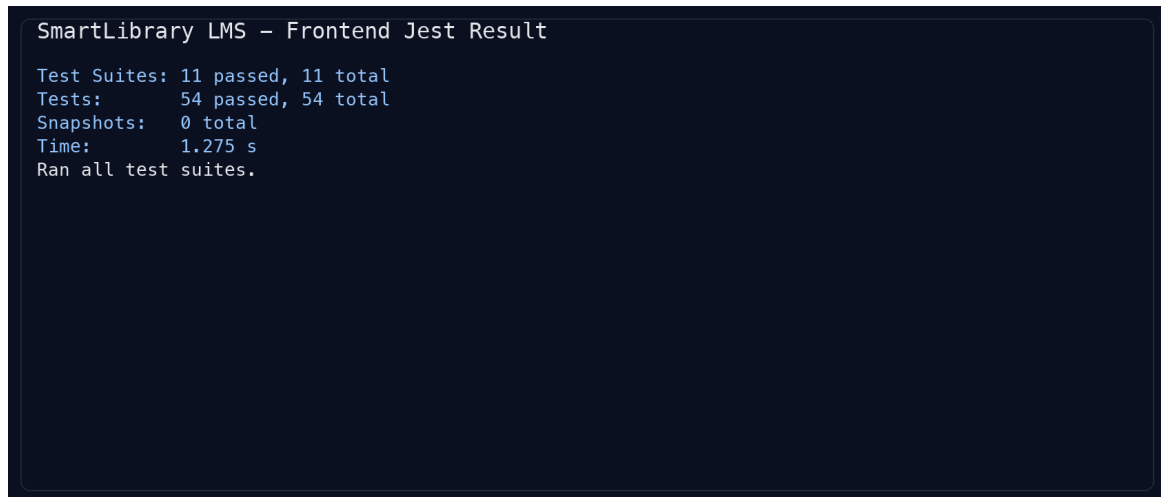
```
'^.+\\. (ts|tsx)$': ['ts-jest', {
  useESM: false,
  tsconfig: { jsx: 'react-jsx', esModuleInterop: true },
}],
},
testMatch: ['<rootDir>/src/**/*.test.{ts,tsx}'],
};
```

7.1.2 Phạm vi test đã hiện thực

Nhóm ưu tiên các phần logic có khả năng ảnh hưởng nhiều màn hình thay vì chỉ kiểm từng nút bấm riêng lẻ. Bộ test cuối cùng bao phủ 11 suite, gồm logic dùng chung, context toàn cục, component nhỏ, trang liên hệ và service layer.

Bảng 7.3: Các file test Frontend đã hiện thực

File test	Nội dung kiểm thử	Số test
src/utils/ errorMessages.test.ts	Ảnh xạ mã lỗi backend sang thông báo phù hợp cho duplicate email, duplicate review và text lỗi chung.	4
src/utils/ notification Localization.test.ts	Localize notification theo ngôn ngữ, giữ dữ liệu động như tên sách, số tiền và vị trí hàng chờ.	3
src/utils/ reservationUtils.test.ts	Nhân/màu trạng thái đặt trước, format ngày, quyền hủy reservation và deadline nhận sách.	18
src/contextes/ LanguageContext .test.tsx	Ngôn ngữ mặc định, fallback text, chuyển sang English, lưu localStorage và cập nhật document.lang.	2
src/contextes/ ThemeContext .test.tsx	Theme mặc định system, dark/light mode, prefers-color-scheme, class dark trên thẻ html.	3
src/contextes/ AuthContext.test.tsx	Load role đã lưu, login bằng mật khẩu/social, parse role từ JWT và logout xóa token.	5
src/contextes/ UploadContext .test.tsx	Thêm/cập nhật/xóa upload, progress/status và cảnh báo beforeunload khi upload đang chạy.	5
src/components/ UiComponents.test.tsx	Star rating, password visibility toggle và disabled button.	3
src/components/ Switchers.test.tsx	Language switcher và theme mode toggle thao tác đúng context toàn cục.	2
src/pages/ ContactPage.test.tsx	Thông tin liên hệ, iframe Google Maps lazy loading và submit form liên hệ.	2
src/api/ frontendServices.test.ts	Hợp đồng endpoint/payload cho search, semantic search, PayOS, wishlist, S3 upload, rating, admin audit, circulation policy, system review và ghi chú giao dịch.	10
Tổng cộng	11 test suites	58



Hình 7.1: Kết quả chạy unit/context test Frontend

7.1.3 Nhóm test ánh xạ lỗi nghiệp vụ

Một lỗi thực tế nhóm đã phát hiện là frontend từng ánh xạ lỗi trùng email sang thông báo trùng đánh giá. Nguyên nhân là cả hai lỗi đều có dạng conflict và có text gần giống already exists. Nhóm đã điều chỉnh helper getFriendlyErrorMessage theo nguyên tắc: ưu tiên code nghiệp vụ do backend trả về trước, sau đó mới fallback theo text.

Bảng 7.5: Test case cho ánh xạ lỗi nghiệp vụ

Mã	Dữ liệu vào	Kết quả mong đợi	Mức ưu tiên
FE-ERR-01	Backend trả code=1103, message tiếng Anh cho email đã tồn tại.	Hiển thị This email is already registered.	Cao
FE-ERR-02	Backend trả code=1103, message tiếng Việt cho email đã tồn tại.	Hiển thị Email đã được sử dụng.	Cao
FE-ERR-03	Lỗi chỉ có text Email already exists.	Vẫn hiểu là lỗi đăng ký email, không nhầm sang review.	Cao
FE-ERR-04	Backend trả code=2700 cho rating đã tồn tại.	Hiển thị lỗi giao dịch mượn đã được đánh giá.	Cao

Listing 7.2: Test regression cho duplicate email và duplicate review

```
it('maps duplicate email backend code to the English registration message', () => {
  const error = {
    status: 409,
    message: 'Email already exists',
    data: { code: 1103, message: 'Email already exists' },
  };

  expect(getFriendlyErrorMessage(error, 'en'))
    .toBe('This email is already registered.');
```

```
});

it('keeps duplicate review mapped to the review-specific message', () => {
  const error = {
```

```
status: 409,
message: 'User already has a rating for this publication',
data: { code: 2700 },
};

expect(getFriendlyErrorMessage(error, 'en'))
  .toBe('This loan transaction has already been reviewed.');
```

Giá trị của nhóm test này là bảo vệ trải nghiệm đăng ký và đánh giá sách. Khi backend thay đổi message nhưng vẫn giữ `ErrorCode`, frontend vẫn hiển thị đúng thông báo. Điều này cũng giúp báo cáo lỗi rõ hơn: lỗi đăng ký email, lỗi tài khoản tồn tại và lỗi đánh giá trùng không còn bị gom chung thành một thông báo mơ hồ.

7.1.4 Nhóm test đa ngôn ngữ và notification

Frontend có hai nguồn nội dung cần localize: text tĩnh trong giao diện và notification phát sinh từ backend. Với text tĩnh, nhóm kiểm thử `LanguageContext`; với notification, nhóm kiểm thử helper localize notification. Cách tách này giúp bảo đảm thông báo thời gian thực vẫn hiển thị đúng khi người dùng đổi ngôn ngữ sau khi notification đã được lưu trong database.

Bảng 7.7: Test case cho đa ngôn ngữ

Mã	Kịch bản	Kết quả mong đợi
FE-LANG-01	Mở provider khi chưa có cấu hình trong <code>localStorage</code> .	Ngôn ngữ mặc định là <code>vi</code> , tiêu đề cài đặt hiển thị Cài đặt.
FE-LANG-02	Gọi <code>t()</code> với key không tồn tại nhưng có fallback.	Hiển thị fallback text thay vì key thô.
FE-LANG-03	Chuyển từ tiếng Việt sang English.	UI đổi sang English, lưu <code>smartlibrary.language=en</code> , cập nhật <code>document.lang</code> .
FE-NOTI-01	Notification đặt trước sách ở tiếng Việt.	Giữ nguyên title/message tiếng Việt.
FE-NOTI-02	Notification đặt trước sách ở English.	Dịch title/message, giữ đúng vị trí hàng chờ.
FE-NOTI-03	Notification thanh toán phí phạt ở English.	Dịch câu thông báo nhưng giữ tên sách và số tiền.

Listing 7.3: Test `LanguageContext` lưu ngôn ngữ và cập nhật document

```
it('switches to English and persists language selection', () => {
  render(
    <LanguageProvider>
      <LanguageProbe />
    </LanguageProvider>
  );

  fireEvent.click(screen.getByText('English'));

  expect(screen.getByTestId('language')).toHaveTextContent('en');
  expect(screen.getByTestId('settings-title')).toHaveTextContent('Settings');
  expect(localStorage.getItem('smartlibrary.language')).toBe('en');
  expect(document.documentElement.lang).toBe('en');
```

```
});
```

Listing 7.4: Test notification giữ dữ liệu động khi dịch

```
it('translates paid fine notification with amount and title', () => {
  const result = localizeNotification(notification({
    type: 'FINE_PAID',
    title: 'Fine paid',
    message: "Fine for book 'Dive Into Deep Learning' (80,000d) was paid.",
  }), 'en');

  expect(result.title).toBe('Fine payment recorded');
  expect(result.message).toBe(
    'The fine for "Dive Into Deep Learning" (80,000d) has been marked as paid.'
  );
});
```

7.1.5 Nhóm test theme light/dark/system

Theme là state toàn cục nên nhóm kiểm thử ở cấp ThemeContext. Test không chỉ kiểm giá trị state, mà còn kiểm side effect trên thẻ `document.documentElement`. Đây là vị trí Tailwind đọc class `dark`; nếu class này không được thêm hoặc xóa đúng, nhiều trang sẽ hiển thị sai màu dù state React vẫn đúng.

Bảng 7.9: Test case cho ThemeContext

Mã	Kịch bản	Kết quả mong đợi
FE-THEME-01	Chưa chọn theme, hệ điều hành đang light mode.	<code>themeMode=system</code> , <code>resolvedTheme=light</code> , thẻ <code>html</code> không có class <code>dark</code> .
FE-THEME-02	Người dùng chọn dark mode.	<code>themeMode=dark</code> , <code>resolvedTheme=dark</code> , thêm class <code>dark</code> , lưu <code>smartlibrary.theme=dark</code> .
FE-THEME-03	Chọn system mode khi <code>prefers-color-scheme</code> là dark.	<code>resolvedTheme=dark</code> , thẻ <code>html</code> có class <code>dark</code> .

Listing 7.5: Mock `prefers-color-scheme` trong ThemeContext test

```
const installMatchMedia = (matches) => {
  Object.defineProperty(window, 'matchMedia', {
    writable: true,
    value: jest.fn().mockImplementation((query) => ({
      matches,
      media: query,
      addEventListener: jest.fn(),
      removeEventListener: jest.fn(),
    })),
  });
};

it('applies dark mode and persists the selected mode', () => {
```

```
render(<ThemeProvider><ThemeProbe /></ThemeProvider>);  
fireEvent.click(screen.getByText('Dark'));  
  
expect(document.documentElement).toHaveClass('dark');  
expect(localStorage.getItem('smartlibrary.theme')).toBe('dark');  
});
```

7.1.6 Nhóm test helper đặt trước

Trong trang đặt trước, frontend cần hiển thị đúng trạng thái hàng chờ và quyết định phiếu nào còn được hủy. Nhóm đã kiểm thử các helper trong `reservationUtils`: cấu hình nhãn/màu, format ngày, điều kiện hủy và deadline nhận sách. Đây là nhóm test có số lượng lớn nhất vì trạng thái đặt trước có nhiều nhánh nhỏ nhưng ảnh hưởng trực tiếp đến thao tác của độc giả.

Bảng 7.11: Các nhánh kiểm thử reservation utility

Hàm/nhóm dữ liệu	Nhánh đã kiểm thử
STATUS_CONFIG	PENDING, READY_FOR_PICKUP, CANCELLED, COMPLETED, EXPIRED; kiểm nhãn và màu tương ứng.
fmtDate	Input null, chuỗi ngày không hợp lệ, ISO string hợp lệ và năm hiển thị trong kết quả.
isActive Cancellable	Cho phép hủy PENDING, READY_FOR_PICKUP; không cho hủy COMPLETED, CANCELLED, EXPIRED.
calcHold DeadlineLabel	Không có hạn nhận, đã hết hạn, còn dưới 24 giờ và còn trên 24 giờ.

Listing 7.6: Test điều kiện hủy reservation

```
it('PENDING can be cancelled', () => {  
  expect(isActiveCancellable('PENDING')).toBe(true);  
});  
  
it('COMPLETED cannot be cancelled', () => {  
  expect(isActiveCancellable('COMPLETED')).toBe(false);  
});  
  
it('returns an empty label when hold deadline is missing', () => {  
  expect(calcHoldDeadlineLabel(null)).toBe('');  
});
```

7.1.7 Nhóm test xác thực và phiên đăng nhập

Bộ kiểm thử bao phủ `AuthContext` vì đây là nơi quyết định toàn bộ trạng thái phiên đăng nhập của SPA. Test không gọi backend thật mà mock `authService`; nhờ vậy có thể kiểm tra rõ các nhánh: load role từ `localStorage`, login demo, login bằng mật khẩu, social login và logout. Với login thật, test tạo JWT giả có payload scope để xác minh frontend parse đúng vai trò `student/librarian`.

Bảng 7.13: Test case cho AuthContext

Mã	Kịch bản	Kết quả mong đợi
FE-AUTH-01	localStorage đã có userType=librarian.	Provider khởi tạo với role thủ thư.
FE-AUTH-02	Login demo bằng chuỗi student.	Cập nhật state và lưu userType=student.
FE-AUTH-03	Login bằng mật khẩu nhận JWT có scope LIBRARIAN.	Lưu access/refresh token và xác định role librarian.
FE-AUTH-04	Social login trả isNewUser=true.	Gọi đúng callback service, lưu token và role.
FE-AUTH-05	Logout khi có refresh token.	Gọi API logout, xóa token và đưa userType về null.

Listing 7.7: Test AuthContext parse role từ JWT và lưu token

```
it('parses a librarian token after password login and stores tokens', async () => {
  authService.login.mockResolvedValue({
    data: {
      accessToken: makeToken({ scope: 'ROLE_LIBRARIAN' }),
      refreshToken: 'refresh-token',
    },
  });

  render(<AuthProvider><AuthProbe /></AuthProvider>);
  fireEvent.click(screen.getByText('Password login'));

  await waitFor(() =>
    expect(screen.getByTestId('user-type')).toHaveTextContent('librarian')
  );
  expect(localStorage.getItem('refreshToken')).toBe('refresh-token');
});
```

7.1.8 Nhóm test upload và component dùng chung

Với upload tài liệu PDF, frontend cần giữ progress ở cấp ứng dụng và cảnh báo người dùng khi đóng tab trong lúc upload còn chạy. Nhóm đã kiểm thử UploadContext cho các thao tác thêm, cập nhật progress, đổi trạng thái, xóa upload và đăng ký beforeunload. Bên cạnh đó, nhóm bổ sung test cho các component dùng chung như StarRating, Input dạng password, Button, LanguageSwitcher và ThemeModeToggle.

Bảng 7.15: Test case cho upload và component dùng chung

Mã	Kịch bản	Kết quả mong đợi
FE-UP-01	Thêm upload mới.	Item có status=uploading, progress=0.
FE-UP-02	Cập nhật progress và chuyển sang done.	State upload phản ánh đúng tiến độ và trạng thái cuối.
FE-UP-03	Upload đang chạy.	Đăng ký listener beforeunload.
FE-UI-01	StarRating nhận rating=3.6.	Làm tròn và tô 4 sao, hiển thị review count.
FE-UI-02	Input password bấm nút hiển thị.	Type đổi từ password sang text.
FE-UI-03	Switch language/theme từ component.	Context và localStorage được cập nhật đúng.

7.1.9 Nhóm test trang liên hệ

Trang liên hệ là trang public nhưng có nhiều điểm tích hợp nhỏ: thông tin gọi nhanh, email, form liên hệ và Google Maps nhúng bằng `iframe`. Nhóm đã kiểm thử trang này để bảo đảm các link liên hệ không bị sai, bản đồ được lazy load và submit form hiển thị trạng thái thành công.

Listing 7.8: Test ContactPage có Google Maps và link liên hệ

```
it('renders contact channels and Google Maps iframe', () => {
  render(<LanguageProvider><ContactPage /></LanguageProvider>);

  expect(screen.getByRole('link', { name: '+84 88 676 5392' }))
    .toHaveAttribute('href', 'tel:+84886765392');
  expect(screen.getByTitle('Ban do Truong Dai hoc Bach khoa TP.HCM'))
    .toHaveAttribute('loading', 'lazy');
});
```

7.1.10 Nhóm test service contract

Các service trong thư mục `api` là lớp hợp đồng giữa frontend và backend. Nếu service tạo sai query string hoặc gọi sai endpoint, UI có thể vẫn render được nhưng request gửi đi sai. Nhóm đã mock `axiosInstance` để kiểm tra URL/payload của các service có giá trị cao: tìm kiếm public, semantic search, PayOS, wishlist, upload PDF qua presigned URL, rating, admin audit, policy cấu hình từ DB, system review công khai và thread ghi chú giao dịch.

Bảng 7.17: Test case cho service layer

Mã	Service	Điểm kiểm tra
FE-SVC-01	publicationsService. searchPublications	Tạo query string gồm keyword, categoryIds, language, year, branch, available, sort và pagination.
FE-SVC-02	semanticSearchService. search và publicationsService. semanticSearch	Gửi queryText và limit đến /ai/semantic-search; wrapper trong publicationsService là hàm được trang Search dùng khi ghép lexical-semantic.
FE-SVC-03	fineService	Tạo payment PayOS và sync đúng order-Code.
FE-SVC-04	wishlistService	Add/remove/clear/status gọi đúng endpoint wishlist.
FE-SVC-05	uploadDocument ToS3	Dùng axios thuần với Content-Type: application/pdf, không đi qua interceptor.
FE-SVC-06	Rating service	Gọi đúng endpoint list/create/helpful rating và filter sao/sắp xếp.
FE-SVC-07	adminService	Gọi đúng endpoint nhật ký audit, khóa tài khoản và xác minh hồ sơ người dùng/thủ thư.
FE-SVC-08	circulationPolicyService	Trang đọc giả/thủ thư đọc quy định mượn trả từ /circulation-policies; admin cập nhật qua endpoint admin riêng.
FE-SVC-09	systemReviewsService	Trang công khai lấy đánh giá nổi bật, thống kê tổng quan và lưu đánh giá cá nhân đúng endpoint.
FE-SVC-10	transactionsService	Ghi chú giao dịch dùng endpoint thread để nhiều thủ thư cùng trao đổi và vãn tách được người ghi chú.

Listing 7.9: Test service contract cho semantic search và PayOS

```

await semanticSearchService.search({ queryText: 'machine learning', limit: 8 });
expect(axiosInstance.post).toHaveBeenCalledWith('/ai/semantic-search', {
  queryText: 'machine learning',
  limit: 8,
});

await publicationsService.semanticSearch('web development', 50);
expect(axiosInstance.post).toHaveBeenCalledWith('/ai/semantic-search', {
  queryText: 'web development',
  limit: 50,
});

await fineService.createPayOsFinePayment('22520001');
expect(axiosInstance.post).toHaveBeenCalledWith(
  '/fines/payments/payos',
  null,
  { params: { studentId: '22520001' } }
);

```

);

7.1.11 Kết quả thực thi

Nhóm đã chạy lại bộ test frontend và đồng thời sinh ảnh kết quả bằng script `test:report`. Script này chạy Jest, lưu log text và tạo ảnh PNG để báo cáo có thể nhúng trực tiếp mà không cần chụp màn hình thủ công.

Listing 7.10: Lệnh chạy test Frontend và tạo ảnh kết quả

```
cd LMS_fe
npm run test:report
```

Bảng 7.19: Kết quả kiểm thử Frontend

Chỉ số	Kết quả
Test suites	11 passed / 11 total.
Test cases	58 passed / 58 total.
Snapshot	0 total.
Thời gian chạy	Khoảng 1.424s trên môi trường local.
Kết luận	Bộ test frontend pass toàn bộ ở tầng unit/context.

7.1.12 Đánh giá và giới hạn

Phần kiểm thử frontend đã bao phủ các logic dùng chung và các hợp đồng giao tiếp có giá trị cao: localize lỗi backend, notification đa ngôn ngữ, theme toàn cục, language toàn cục, xác thực, upload, contact map, helper đặt trước và service layer cho search/AI/PayOS/wishlist/rating. Các test này giúp nhóm tự tin hơn khi sửa UI hoặc đổi message từ backend, vì những lỗi sai ngôn ngữ, sai role, sai endpoint hoặc sai payload sẽ được phát hiện trước khi demo.

Tuy nhiên, nhóm vẫn phân biệt rõ giữa test frontend trong môi trường jsdom và kiểm thử E2E trên trình duyệt thật. Bộ test này đủ cho tầng unit/context/component/service contract, nhưng các luồng end-to-end như đăng nhập thật, tìm kiếm sách, đặt trước, nhận sách, thanh toán PayOS và nhận WebSocket notification vẫn nên được kiểm thử bằng Playwright hoặc kịch bản demo tích hợp sau khi backend/AI service cùng chạy.

7.2 Kiểm thử Backend

Backend được kiểm thử bằng JUnit 5, Mockito, MockMvc, Spring Boot Test và Testcontainers. Nhóm tách kiểm thử backend thành hai tầng: tầng unit/controller chạy nhanh với mock để kiểm tra nhánh nghiệp vụ và hợp đồng API; tầng integration chạy với PostgreSQL thật trong Docker để xác minh migration, dữ liệu seed và các truy vấn native SQL quan trọng.

7.2.1 Môi trường và cấu hình kiểm thử Backend

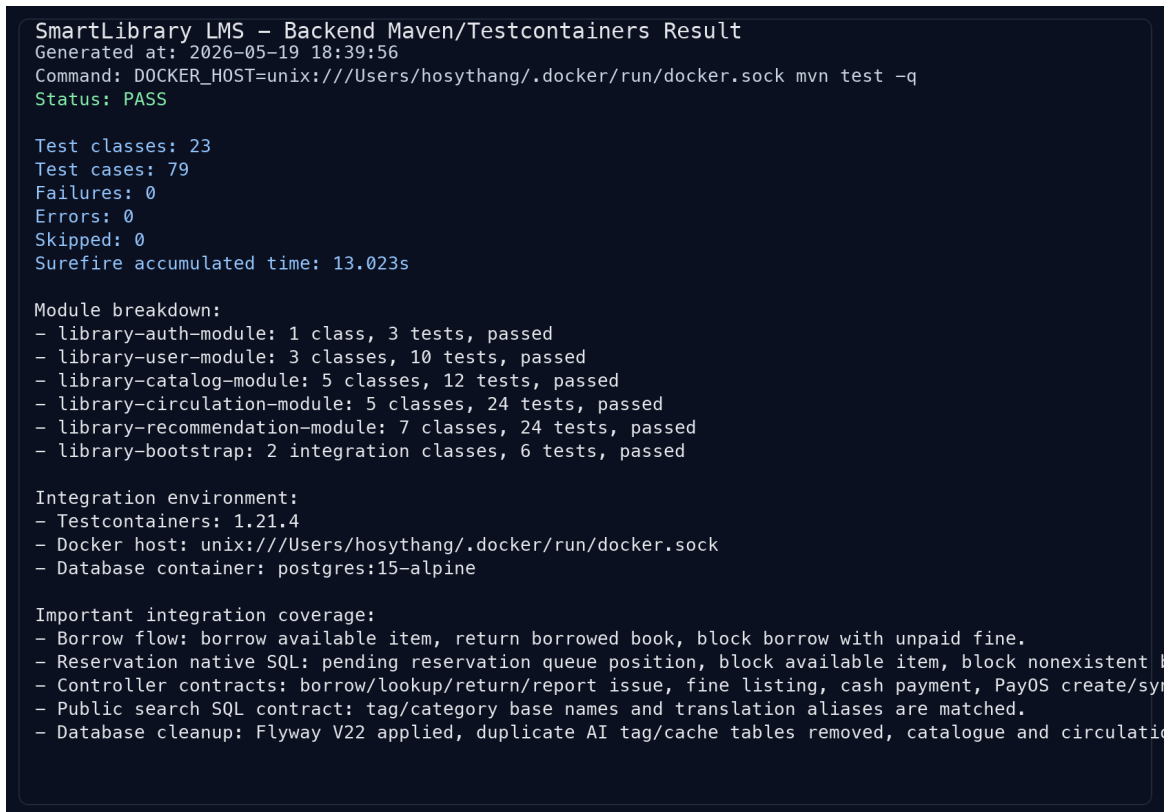
Các module backend dùng Maven Surefire để chạy test. Với test tích hợp trong `library-bootstrap`, nhóm dùng Testcontainers để khởi động `postgres:15-alpine`, sau đó để Flyway chạy toàn bộ migration từ V1 đến V22. Cách này giúp kiểm thử trên schema thật của hệ thống thay vì dùng database giả hoặc H2 không tương thích hoàn toàn với PostgreSQL.

Bảng 7.21: Cấu hình kiểm thử Backend

Thành phần	Cấu hình sử dụng
Test runner	JUnit 5, Maven Surefire, chạy bằng <code>mvn test</code> .
Mock unit test	Mockito và AssertJ cho use case/service.
Controller test	MockMvc standalone, kiểm request binding, validation và response JSON.
Integration test	Spring Boot Test với Testcontainers 1.21.4.
Database test	Docker Engine 29.4.3, container <code>postgres:15-alpine</code> .
Migration	Flyway chạy 29 migration trên database test, gồm migration dọn schema/index, meta-data AI, admin/policy/audit và transaction notes cuối cùng.
Test profile	<code>application-test.yml</code> , tắt Kafka listener và scheduler để test không phụ thuộc dịch vụ ngoài.

Listing 7.11: Lệnh chạy toàn bộ test Backend

```
cd LMS_BE
DOCKER_HOST=unix:///Users/hosythang/.docker/run/docker.sock mvn test -q
```



```
SmartLibrary LMS – Backend Maven/Testcontainers Result
Generated at: 2026-05-19 18:39:56
Command: DOCKER_HOST=unix:///Users/hosythang/.docker/run/docker.sock mvn test -q
Status: PASS

Test classes: 23
Test cases: 79
Failures: 0
Errors: 0
Skipped: 0
Surefire accumulated time: 13.023s

Module breakdown:
- library-auth-module: 1 class, 3 tests, passed
- library-user-module: 3 classes, 10 tests, passed
- library-catalog-module: 5 classes, 12 tests, passed
- library-circulation-module: 5 classes, 24 tests, passed
- library-recommendation-module: 7 classes, 24 tests, passed
- library-bootstrap: 2 integration classes, 6 tests, passed

Integration environment:
- Testcontainers: 1.21.4
- Docker host: unix:///Users/hosythang/.docker/run/docker.sock
- Database container: postgres:15-alpine

Important integration coverage:
- Borrow flow: borrow available item, return borrowed book, block borrow with unpaid fine.
- Reservation native SQL: pending reservation queue position, block available item, block nonexistent
- Controller contracts: borrow/lookup/return/report issue, fine listing, cash payment, PayOS create/syn
- Public search SQL contract: tag/category base names and translation aliases are matched.
- Database cleanup: Flyway V22 applied, duplicate AI tag/cache tables removed, catalogue and circulatio
```

Hình 7.2: Kết quả chạy bộ test Backend

7.2.2 Phạm vi test Backend đã hiện thực

Bộ test backend có 23 test class với 79 test case. Phạm vi được chọn theo các luồng cốt lõi nhóm đã xây dựng: đăng ký/xác thực, notification, catalog tài liệu, mượn-trả sách, đặt trước, rating/wishlist, AI gateway và callback từ AI service. Với admin console và quy định mượn trả cấu hình từ DB, các test circulation sử dụng mock `CirculationPolicyService`, bảo đảm unit test phản ánh đúng cách runtime đọc policy thay vì giả

định hằng số cứng.

Bảng 7.23: Tổng hợp test Backend theo module

Module	Số class	Số test	Nội dung chính
library-auth-module	1	3	Controller đăng ký, payload hợp lệ và validation email/student id.
library-user-module	3	10	Signup use case, notification page, unread count, mark read và mark all read.
library-catalog-module	5	12	Tạo publication/item, document upload URL, save document URL, public search SQL contract và publication controller.
library-circulation-module	5	24	Borrow request, return book, overdue fine, unpaid fine, borrow/reservation limit lấy từ CirculationPolicyService, reservation use case, transaction controller và fine/PayOS controller.
library-recommendation-module	7	24	Wishlist, rating, AI gateway fallback, semantic/recommendation contract và signed AI callback.
library-bootstrap	2	6	Integration với PostgreSQL thật: borrow/return flow và reservation native SQL.
Tổng cộng	23	79	Toàn bộ pass.

7.2.3 Nhóm test Catalog

Catalog là phần backend ảnh hưởng trực tiếp đến tìm kiếm sách, quản lý ấn phẩm, upload tài liệu PDF và dữ liệu hiển thị ở frontend. Bộ test có các hợp đồng có rủi ro cao: ISBN phải được normalize trước khi lưu, bảng liên kết author/category/tag phải được insert đúng, item mới phải có trạng thái AVAILABLE, barcode không được trùng, upload URL phải tạo đúng S3 key và public search phải sinh đúng SQL filter.

Bảng 7.25: Test case Backend cho Catalog

Mã	Kịch bản	Kết quả mong đợi
BE-CAT-01	Tạo publication có ISBN, TOC, author, category và tag.	Lưu publication, normalize ISBN và insert đúng các bảng liên kết.
BE-CAT-02	Tạo item cho publication tồn tại và barcode duy nhất.	Item được lưu với trạng thái AVAILABLE.
BE-CAT-03	Tạo item với publication không tồn tại.	Ném PUBLICATION_NOT_FOUND, không lưu item.
BE-CAT-04	Tạo item với barcode đã tồn tại.	Từ chối trước khi gọi save.
BE-CAT-05	Xin presigned URL upload document với filename có khoảng trắng.	S3 key được sanitize và URL có TTL 15 phút.
BE-CAT-06	Lưu S3 key sau khi upload PDF.	Lưu public URL, đánh dấu AI ETL QUEUED và gọi AI processing.
BE-CAT-07	Public search có keyword, category, language, year, branch, available và sort rating.	SQL chứa đúng filter, normalize ngôn ngữ, giới hạn page size tối đa 50, đồng thời match tag/category gốc và alias trong tag_translations/category_translations.
BE-CAT-08	Mock result set của public search.	Map đúng các cột SQL sang PublicSearchResult.

Trong quá trình viết test, nhóm phát hiện một lỗi contract ở endpoint lưu URL tài liệu: controller trả URL vào trường message thay vì data. Lỗi này đã được sửa để frontend có thể đọc URL đã lưu một cách nhất quán. Riêng public search, test SQL contract cũng khóa hành vi tìm kiếm Anh-Việt hai chiều: keyword được so khớp

trên tên tag/category tiếng Việt trong bảng gốc và alias tiếng Anh trong bảng translation, sau đó thứ tự liên quan vẫn ưu tiên nhan đề/phụ đề trước exact tag và partial tag.

7.2.4 Nhóm test Circulation và Reservation

Các use case mượn-trả và đặt trước có nhiều điều kiện nghiệp vụ nên được kiểm ở cả unit test, controller contract test và integration test. Unit test kiểm các nhánh ràng buộc bằng mock: item không sẵn sàng, độc giả còn nợ phí, vượt giới hạn mượn, đã đặt trước sách, hoặc sách đang có bản AVAILABLE. Controller test kiểm endpoint mà frontend/thủ thư gọi trực tiếp như tạo phiếu mượn, tra cứu giao dịch, trả sách, báo cáo sự cố, danh sách phí phạt, thanh toán tiền mặt và PayOS. Integration test chạy trên PostgreSQL thật để kiểm các câu SQL có FOR UPDATE, queue position và trạng thái item/transaction sau khi mượn hoặc trả.

Bảng 7.27: Test case Backend cho mượn-trả và đặt trước

Mã	Kịch bản	Kết quả mong đợi
BE-BOR-01	Mượn item AVAILABLE.	Tạo transaction WAITING_FOR_PICKUP, item chuyển RESERVED, hạn nhận và hạn trả lấy từ policy hiện hành.
BE-BOR-02	Mượn item không ở trạng thái AVAILABLE.	Ném ITEM_NOT_BORROWABLE.
BE-BOR-03	User còn fine UNPAID.	Ném USER_HAS_UNPAID_FINES khi policy bật chặn mượn với phí chưa trả.
BE-BOR-04	User đã đạt giới hạn mượn cấu hình bởi admin.	Ném USER_BORROW_LIMIT_EXCEEDED.
BE-RET-01	Trả sách đúng hạn.	Transaction RETURNED, item về AVAILABLE.
BE-RET-02	Trả sách quá hạn.	Tạo fine quá hạn theo mức phí/ngày trong policy và cập nhật trạng thái trả.
BE-TRX-01	Controller tạo phiếu mượn với itemId hợp lệ.	HTTP 200, lấy đúng user hiện tại và gọi use case tạo transaction.
BE-TRX-02	Controller trả sách và báo cáo sự cố.	Bind đúng librarian hiện tại, transaction id, ghi chú, mức phạt và danh sách phí tạo ra.
BE-FINE-01	Lấy fine của sinh viên, đánh dấu đã thanh toán và thanh toán tiền mặt.	Gọi đúng use case, trả response schema thống nhất cho frontend.
BE-FINE-02	Tạo/sync PayOS payment và nhận webhook sai chữ ký.	Tạo đúng payment URL, sync đúng orderCode; webhook không hợp lệ bị từ chối HTTP 401.
BE-RES-01	Đặt trước khi không có item available.	Tạo reservation PENDING, kiểm giới hạn đặt trước từ policy và tính đúng queue position bằng native SQL.
BE-RES-02	Đặt trước khi sách đang có item available.	Ném RESERVATION_BOOK_AVAILABLE.
BE-RES-03	Đặt trước ở branch không có item.	Ném RESERVATION_BRANCH_NO_ITEMS.

7.2.5 Nhóm test Auth, User, Recommendation và AI

Các module còn lại được kiểm theo hướng hợp đồng nghiệp vụ và API. Auth controller bảo đảm chỉ payload đăng ký hợp lệ được chuyển xuống use case. User module kiểm signup và notification state. Recommendation module kiểm wishlist, rating, duplicate rating, AI gateway khi thành công hoặc lỗi, semantic search contract và HMAC callback từ AI service.

Bảng 7.29: Test case Backend cho Auth/User/Recommendation/AI

Mã	Kịch bản	Kết quả mong đợi
BE-AUTH-01	POST register với payload hợp lệ.	HTTP 200 và gọi SignUpUseCase.
BE-AUTH-02	Register email không phải @hcmut.edu.vn.	HTTP 400 trước khi gọi use case.
BE-AUTH-03	Register student id sai định dạng.	HTTP 400 trước khi gọi use case.
BE-NOTI-01	Lấy notification của user hiện tại.	Trả page response đúng dữ liệu.
BE-NOTI-02	Mark read/mark all read.	Gọi use case với đúng user id hiện tại.
BE-RATE-01	Tạo rating và list/summary rating.	Controller bind đúng publication id, user id, filter và response schema.
BE-RATE-02	Star rating ngoài khoảng 1-5.	HTTP 400 trước khi gọi use case.
BE-AI-01	AI gateway trả response snake case.	Map đúng publication ids, strategy và metadata.
BE-AI-02	AI gateway lỗi 500.	Backend fallback, không làm sập request chính.
BE-AI-03	AI callback có chữ ký HMAC hợp lệ/không hợp lệ.	Chấp nhận callback hợp lệ, từ chối callback thiếu hoặc sai chữ ký.

7.2.6 Đối soát test với các tính năng admin final

Các tính năng admin cuối cùng tạo thêm hai nhóm kiểm thử cần phân biệt rõ trong báo cáo. Nhóm thứ nhất đã có trong bộ test tự động: các use case circulation không còn phụ thuộc hằng số cứng mà đọc policy từ `CirculationPolicyService`. Do đó các test mượn, đặt trước và trả sách mock policy để giữ đúng hành vi runtime và tránh lỗi giả do thiếu dependency trong unit test.

Nhóm thứ hai là các API quản trị như khóa/mở khóa tài khoản, xác minh hồ sơ, tạo thủ thư, lọc audit log và cập nhật policy. Các API này nằm trong test case kiểm thử hệ thống và được khóa hợp đồng ở frontend service-contract, tức frontend phải gọi đúng endpoint/payload. Trong phạm vi 79 test backend tự động, nhóm chưa tăng số test class cho `AdminManagementController`; vì vậy báo cáo không tính riêng admin controller vào số 79 test. Nếu mở rộng CI sau đề án, phần nên bổ sung tiếp theo là MockMvc test cho `GET/POST /api/v1/admin/librarians`, `PATCH /api/v1/admin/users/{id}/status`, `PATCH /api/v1/admin/users/{id}/verify` và `GET /api/v1/admin/audit-logs`.

7.2.7 Kết quả thực thi Backend

Bộ test backend cuối cùng chạy thành công toàn bộ. Kết quả chi tiết được lưu tại thư mục `figures/test-results`, gồm file tóm tắt `be-test-result.txt` và log Maven đầy đủ `be-full-test-result.txt`.

Bảng 7.31: Kết quả kiểm thử Backend

Chỉ số	Kết quả
Test classes	23.
Test cases	79 passed / 79 total.
Failures	0.
Errors	0.
Skipped	0.
Thời gian Surefire	Khoảng 13.023s.
Kết luận	Bộ test backend pass toàn bộ ở tầng unit/controller và integration PostgreSQL.

7.2.8 Đánh giá và giới hạn

Bộ test backend đã bao phủ các thành quả chính của hệ thống: quản lý ấn phẩm, upload tài liệu, tìm kiếm public, đăng ký người dùng, notification, mượn-trả sách, đặt trước, phí phạt/thanh toán, rating, wishlist và

tích hợp AI. Đặc biệt, nhóm đã kiểm thử integration với PostgreSQL thật bằng Testcontainers để chứng minh các native SQL và migration hoạt động trên môi trường gần với triển khai thực tế.

Giới hạn còn lại là bộ test chưa thay thế hoàn toàn kiểm thử E2E nhiều dịch vụ cùng lúc với frontend, backend, AI service, Kafka và PayOS sandbox đều đang chạy. Tuy nhiên, ở phạm vi backend độc lập, bộ test đủ để chứng minh các use case cốt lõi, controller contract và truy vấn database quan trọng hoạt động đúng.

7.3 Kiểm thử AI Service

AI service là phần chịu trách nhiệm xử lý tài liệu PDF, tạo vector embedding, sinh metadata học thuật, tìm kiếm ngữ nghĩa và gợi ý sách. Vì vậy nhóm không chỉ kiểm thử API trả đúng schema, mà còn kiểm thử các ràng buộc chất lượng ở bên trong pipeline: làm sạch PDF, chia chunk ổn định, reduce không dùng metadata catalog để viết summary, tag tiếng Việt đúng 5 nhãn, alias tiếng Anh khớp 1-1, SQL ghi vào đúng schema `public/ai_engine`, hybrid search ưu tiên lexical trước semantic với truy vấn ngắn, recommender có fallback và worker callback có chữ ký HMAC.

Riêng phần schema AI, test contract xác nhận pipeline ghi vector vào `ai_engine.publication_vectors`, ghi tag vào `public.tags/public.publication_tags`, ghi alias vào `public.tag_translations` và không phụ thuộc các bảng ngoài schema final `ai_engine.tags`, `ai_engine.publication_tags`, `ai_engine.recommender`.

7.3.1 Môi trường kiểm thử AI

Bộ test AI được viết bằng `pytest`. Các test chính chạy offline, không gọi Gemini thật, không cần PostgreSQL, RabbitMQ hoặc backend đang chạy. Những test tích hợp live cho API vẫn được giữ trong thư mục `tests/integration` và chỉ chạy khi bật biến môi trường `RUN_LIVE_AI_CONTRACT_TESTS=1`. Cách tách này giúp nhóm có một bộ regression nhanh để chạy thường xuyên, đồng thời vẫn có đường kiểm thử end-to-end khi cần đo latency với service thật.

Bảng 7.33: Cấu hình kiểm thử AI Service

Thành phần	Cấu hình sử dụng
Test runner	<code>pytest 8.3.5</code> , chạy bằng <code>PYTHONPATH=. ./venv/bin/python -m pytest -q</code> .
Mock LLM	Stub <code>GeminiClient._call</code> để trả JSON cố định, không gọi API ngoài.
Mock database	Fake connection/cursor để bắt SQL và tham số, không cần PostgreSQL thật.
Mock gateway	Monkeypatch các hàm search/recommendation để kiểm thử tự merge, limit và fallback.
Mock worker	Mock Celery signal/app và HTTP request để kiểm payload, download và HMAC callback.
Live integration	3 test contract/latency được skip mặc định, chỉ bật khi có AI API thật qua <code>AI_API_BASE_URL</code> .

Listing 7.12: Lệnh chạy test AI Service

```
cd LMS-AI
PYTHONPATH=. ./venv/bin/python -m pytest -q
```



Hình 7.3: Kết quả chạy bộ test AI Service

7.3.2 Phạm vi test AI đã hiện thực

Bộ test AI có 9 file test với 45 test case được collect. Trong đó 42 test offline pass và 3 test live integration được skip mặc định. Các test được chia theo module thay vì chỉ theo endpoint, vì lỗi trong AI thường xuất hiện ở giữa pipeline: PDF nhiều làm chunk sai, LLM trả JSON lệch, tag bị generic, SQL ghi sai schema hoặc hybrid search merge sai thứ tự.

Bảng 7.35: Các file test AI Service đã hiện thực

File test	Nội dung kiểm thử	Số test
tests/test_ai_metadata_quality.py	Fallback tag, summary chỉ dựa trên PDF content, lọc tag generic/title/category và alias tiếng Anh.	5
tests/test_pdf_processing.py	Làm sạch header/footer/số trang, bỏ ký tự null, ghép từ bị xuống dòng, chunk id ổn định và validate chunk_overlap.	4
tests/test_llm_client.py	Parse JSON từ Gemini, lỗi response rỗng, reduce metadata, lọc tag xấu, giữ alias tags_en đúng cặp và reject summary kiểu catalog.	6
tests/test_pipeline_core.py	Chọn representative chunks, áp budget reduce, fallback khi LLM lỗi, optional English metadata translation và hash file/URL.	7
tests/test_database_sql_contract.py	SQL contract cho vector, publication tags, public.tags, public.tag_translations và điều kiện đủ AI output.	5
tests/test_api_service_contract.py	pgvector literal, tokenize tiếng Việt không dấu, merge hybrid search, lexical-first cho query ngắn và fallback recommendation.	7
tests/test_recommender_engine.py	ALS bundle settings, trọng số interaction, cold user và danh sách publication id từ model recommendation.	4
tests/test_worker_contract.py	Validate payload RabbitMQ, download PDF tạm, webhook JSON compact và chữ ký HMAC timestamp.body.	4
tests/integration/test_api_contract.py	Healthcheck, hybrid search và recommendation contract/latency với AI API thật; skip mặc định.	3 skip
Tổng cộng	42 pass, 3 skip	45

7.3.3 Nhóm test PDF processing và chunking

PDF là đầu vào gốc của pipeline nên nhóm kiểm thử kỹ bước làm sạch trước khi embedding. Test xác minh hệ thống loại bỏ header/footer lặp lại, số trang, ký tự null và khoảng trắng nhiều; đồng thời vẫn giữ phần nội dung thật. Nhóm cũng thêm kiểm tra tham số chunk_size/chunk_overlap để tránh lỗi vòng lặp nếu overlap lớn hơn hoặc bằng chunk size.

Listing 7.13: Test làm sạch PDF và chia chunk ổn định

```
def test_clean_pdf_pages_removes_repeated_page_furniture_and_noise():
    pages = [
        "Smart Library PDF\n1\nMachine learn-\ning models infer patterns.\nConfidential footer",
        "Smart Library PDF\n2\nRegression and classification workflows.\nConfidential footer",
        "Smart Library PDF\n3\nFeature engineering improves model quality.\nConfidential footer",
    ]
```

```
cleaned = _clean_pdf_pages(pages)

assert "Smart Library PDF" not in cleaned
assert "Confidential footer" not in cleaned
assert "Machine learning models" in cleaned
```

Nhóm test này phát hiện một lỗi thật: với trang rất ngắn, cùng một dòng có thể được tính hai lần khi lấy `lines[:3] + lines[-3:]`, khiến một dòng nội dung đầu trang bị nhận nhầm là header lặp. Nhóm đã sửa bằng cách chỉ đếm một signature một lần trên mỗi trang.

7.3.4 Nhóm test LLM metadata quality

Đây là nhóm test quan trọng nhất vì nó khóa chất lượng đầu ra của Gemini. Test không chỉ kiểm parse JSON, mà còn kiểm các rule nghiệp vụ: summary không được mô tả metadata/thư viện/file PDF, tag phải là tag tiếng Việt có giá trị tìm kiếm, tag generic bị loại, tag title/category bị loại và `tags_en` phải khớp thứ tự với tag tiếng Việt sau khi lọc.

Bảng 7.37: Test case chất lượng metadata AI

Mã	Kịch bản	Kết quả mong đợi
AI-META-01	LLM trả JSON trong markdown fence hoặc có text thừa.	Trích được JSON hợp lệ, lỗi rõ khi output không phải JSON.
AI-META-02	LLM trả tag generic như Công Nghệ hoặc tag trùng title.	Loại tag xấu và backfill bằng tag chuyên môn từ nội dung PDF.
AI-META-03	LLM trả <code>tags_en</code> theo danh sách ban đầu nhưng một số tag Việt bị loại.	Alias tiếng Anh vẫn đi cùng đúng tag Việt sau lọc, không bị lệch index.
AI-META-04	Summary giống mô tả catalog: nhắc metadata, thư viện, file PDF.	Ném lỗi reduce để pipeline dùng fallback an toàn.
AI-META-05	LLM lỗi hoặc không gọi được.	Fallback summary/tag dùng <code>CONTENT_EXCERPTS</code> , đủ 5 tag và có alias tiếng Anh.

Listing 7.14: Test giữ đúng cặp tag tiếng Việt và alias tiếng Anh

```
result = client._reduce_once(
    "cheap",
    ["Relational database design covers SQL, normalization, indexes and transactions."],
    "Title: Database System Concepts\nCategories: Computer Science",
)

assert result["tags"][0:2] == ["SQL", "Chuan Hoa Du Lieu"]
assert result["tags_en"][0:2] == ["SQL", "Database Normalization"]
```

Nhóm test này cũng phát hiện một lỗi thật trong `GeminiClient._reduce_once`: khi tag xấu bị loại, danh sách `tags_en` vẫn được align theo index cũ, làm alias tiếng Anh bị lệch sang tag khác. Nhóm đã sửa bằng cách giữ cặp tag-alias trong lúc lọc, sau đó mới backfill và gọi `align_english_tags`.

7.3.5 Nhóm test ETL pipeline và database contract

Pipeline AI có nhiều bước liên tiếp: hash file, extract text, chunk, embedding, ghi vector, chọn representative chunk, reduce metadata, cập nhật summary, tag và alias. Các test pipeline kiểm phần logic thuần như chọn chunk đại diện theo centroid/diversity, cắt budget reduce, fallback khi LLM lỗi và tùy chọn dịch metadata

tiếng Anh. Các test database dùng fake cursor để xác minh SQL ghi đúng schema final: vector nằm trong `ai_engine.publication_vectors`, tag canonical nằm trong `public.tags`, liên kết nằm trong `public.publication_tags`, alias tiếng Anh nằm trong `public.tag_translations`.

Bảng 7.39: Test case ETL pipeline và SQL contract

Mã	Kịch bản	Kết quả mong đợi
AI-ETL-01	Chọn representative chunks từ nhiều vector.	Giữ context mở đầu, chọn chunk gần centroid và đa dạng theo MMR.
AI-ETL-02	Reduce input vượt giới hạn ký tự.	Cắt từng chunk theo <code>AI_ETL_REDUCE_MAX_CHARS_PER_CHUNK</code> và dừng trước tổng budget.
AI-ETL-03	LLM reduce lỗi.	Pipeline sinh fallback metadata, không ghép title/author/category thành summary.
AI-ETL-04	Bật dịch metadata tiếng Anh.	Ghi <code>publication_translations</code> và <code>tag_translations</code> ; nếu dịch lỗi thì trả False không làm hỏng pipeline chính.
AI-SQL-01	Xóa dữ liệu AI cũ trước khi reprocess.	Xóa vector trong <code>ai_engine</code> và tag link trong <code>public</code> .
AI-SQL-02	Upsert tag và alias.	Dùng <code>public.tags</code> , <code>public.publication_tags</code> , <code>public.tag_translations</code> ; không còn phụ thuộc <code>ai_engine.tags</code> .

7.3.6 Nhóm test API gateway, recommender và worker

Ở tầng API, nhóm kiểm thử các hành vi quyết định trải nghiệm tìm kiếm và gợi ý: query tiếng Việt được normalize bỏ dấu, query ngắn ưu tiên kết quả lexical trước semantic, query dài ưu tiên semantic/chunk keyword hơn, kết quả merge không trùng và vẫn giữ thứ tự rank. Với recommendation, test kiểm cold-start fallback sang trending, warm user merge content-based với ALS, ghi cache vào `public.ai_recommendations` và ALS model trả publication id theo mapping đúng. Với worker, test kiểm payload RabbitMQ, download PDF tạm, callback JSON compact và chữ ký HMAC theo chuỗi `timestamp.body`.

Listing 7.15: Test hybrid semantic search ưu tiên lexical cho query ngắn

```
response = semantic_search(SemanticSearchRequest(
    query_text="web development",
    limit=5,
))

assert response.publication_ids == [1, 2, 8, 3, 9]
```

Listing 7.16: Test chữ ký webhook callback của AI worker

```
headers = _build_webhook_headers({'book_id':12,"status":"SUCCESS"}, "secret")

expected_signature = hmac.new(
    b"secret",
    b'1700000000.{"book_id":12,"status":"SUCCESS"}',
    hashlib.sha256,
).hexdigest()

assert headers["X-AI-Callback-Signature"] == f"sha256={expected_signature}"
```

7.3.7 Kết quả thực thi AI

Bộ test AI cuối cùng chạy thành công toàn bộ phần offline. Ba test integration live được collect nhưng skip mặc định vì chúng yêu cầu AI API thật đang chạy và có dữ liệu/embedding trong database.

Bảng 7.41: Kết quả kiểm thử AI Service

Chỉ số	Kết quả
Test files	9.
Test cases collected	45.
Passed	42.
Skipped	3 live integration tests.
Failures/Errors	0.
Thời gian chạy	Khoảng 2.84s trên môi trường local.
Kết luận	Bộ test AI pass toàn bộ ở tầng unit/contract offline; test live được giữ để chạy khi có môi trường tích hợp.

7.3.8 Kiểm thử tích hợp Gemini thật

Bên cạnh bộ regression offline, nhóm hiện thực kịch bản live để gọi Gemini thật trên một PDF mẫu. Script `scripts/run_live_gemini_report.py` đọc file PDF bằng cùng hàm `extract_pdf_text`, chia chunk, gọi `GeminiClient.reduce_publication`, đo thời gian xử lý và ghi kết quả JSON ra thư mục `figures/test-results`. Artifact JSON được thiết kế để lưu các trường cần nghiệm thu: `master_summary`, `tags`, `tags_en`, `audience`, số chunk, số ký tự extract và thời gian xử lý.

Listing 7.17: Lệnh chạy live Gemini report

```
cd LMS-AI
PYTHONPATH=. ./venv/bin/python scripts/run_live_gemini_report.py \
  --pdf test_pdfs/gemini_live_sample.pdf \
  --output ../latex/figures/test-results/ai-gemini-live-result.json
```



Hình 7.4: Artifact chạy live Gemini trên PDF mẫu

Kịch bản live này kiểm tra trực tiếp khả năng gọi Gemini và sinh metadata từ PDF mẫu. Khi khóa API còn quota, artifact thành công chứa `master_summary`, `tags`, `tags_en`, `audience`, số chunk và thời gian xử lý. Nếu Gemini trả lỗi quota/rate-limit, script vẫn ghi artifact lỗi có redact API key để chứng minh đường kiểm thử được thực thi mà không làm lộ thông tin nhạy cảm.

7.3.9 Đánh giá và giới hạn

Bộ test AI đã bao phủ các tính năng chính của service: xử lý PDF, chunking, metadata reduce, tag quality, fallback khi LLM lỗi, alias tiếng Anh, ghi SQL đúng schema, semantic search hybrid, recommendation

CBF/ALS/trending và worker callback bảo mật. Quan trọng hơn, các test đã bắt được lỗi thật trong pipeline PDF và align tags_en, qua đó giúp chất lượng AI output ổn định hơn trước khi demo.

Giới hạn còn lại là các test offline chưa đo chất lượng ngôn ngữ thực tế của Gemini và chưa đo latency thật trên tập dữ liệu lớn. Vì vậy nhóm vẫn giữ 3 test integration live để chạy khi PostgreSQL, model embedding và AI API đang hoạt động; các test này phù hợp cho giai đoạn nghiệm thu tích hợp hoặc benchmark riêng, còn bộ 42 test offline phù hợp để chạy thường xuyên sau mỗi lần sửa code.

Chương 8: Triển khai hệ thống

8.1 Container hóa với Docker

8.1.1 Giới thiệu Docker

Docker là một nền tảng mã nguồn mở được thiết kế để phát triển, đóng gói và chạy các ứng dụng [7]. Docker giải quyết bài toán “chạy được trên máy tôi nhưng không chạy được trên máy bạn” bằng cách đóng gói ứng dụng cùng toàn bộ phụ thuộc—thư viện runtime, biến môi trường, file cấu hình—vào một đơn vị tiêu chuẩn gọi là *container*. Đối với dự án này, Docker là thành phần thiết yếu vì nó đảm bảo tính nhất quán tuyệt đối giữa môi trường phát triển cục bộ, kiểm thử và sản xuất, đồng thời đơn giản hóa đáng kể quy trình triển khai trên máy chủ.



Hình 8.1: Docker

8.1.2 Cấu hình môi trường phát triển cục bộ

Trong giai đoạn phát triển, Docker Compose được sử dụng để quản lý kiến trúc đa container của hệ thống. File `docker-compose.yml` định nghĩa hai dịch vụ hạ tầng cốt lõi:

- **PostgreSQL 15:** Cơ sở dữ liệu quan hệ chính của hệ thống, được cấu hình với health check để đảm bảo sẵn sàng trước khi ứng dụng kết nối. Dữ liệu được lưu trữ bền vững qua Docker volume `postgres_data`.
- **Kafka 4.0.0 (KRaft mode):** Message broker xử lý giao tiếp bất đồng bộ cho các luồng email và thông báo. Dự án sử dụng chế độ KRaft (không cần ZooKeeper), giảm đáng kể độ phức tạp vận hành. Dữ liệu Kafka được lưu trong volume `kafka_data`.

Ứng dụng Spring Boot và giao diện React được chạy trực tiếp trên máy của nhà phát triển (không trong container) để tận dụng tính năng hot-reload, giúp chu kỳ phát triển nhanh hơn.

```
# Khởi động hạ tầng
docker compose up -d

# Chạy Backend
cd LMS_be && mvn spring-boot:run -pl library-bootstrap

# Chạy Frontend
cd LMS_fe && npm run dev
```

8.2 Dockerfile đa giai đoạn

Để triển khai lên môi trường sản xuất, mỗi thành phần được đóng gói thành Docker image riêng sử dụng kỹ thuật *multi-stage build*—tách biệt giai đoạn biên dịch và giai đoạn chạy nhằm tối thiểu hóa kích thước image cuối cùng [8].

8.2.1 Backend Dockerfile

Dockerfile của Spring Boot bao gồm hai giai đoạn:

- **Giai đoạn Build:** Sử dụng image `maven:3.9-eclipse-temurin-21-alpine`. Các file `pom.xml` được sao chép trước để Docker cache lớp phụ thuộc Maven—chỉ tải lại khi `pom.xml` thay đổi, không phải mỗi lần build. Sau đó toàn bộ mã nguồn được sao chép và build với lệnh `mvn package -DskipTests -pl library-bootstrap -am`.
- **Giai đoạn Runtime:** Sử dụng image `eclipse-temurin:21-jre-alpine`—chỉ chứa Java Runtime Environment, không có Maven hay mã nguồn. File JAR được sao chép từ giai đoạn build sang. Kích thước image cuối cùng giảm đáng kể so với image chứa đầy đủ JDK và Maven.

8.2.2 Frontend Dockerfile

Dockerfile của React cũng sử dụng hai giai đoạn:

- **Giai đoạn Build:** Sử dụng `node:20-alpine`, cài đặt phụ thuộc qua `npm ci` và chạy `npm run build` (Vite) để tạo ra thư mục `/dist` chứa các file tĩnh đã được tối ưu. URL API được truyền vào lúc build qua build argument `VITE_API_BASE_URL=/api/v1`.
- **Giai đoạn Runtime:** Sử dụng `nginx:alpine` phục vụ các file tĩnh từ `/usr/share/nginx/html`. Cấu hình Nginx của frontend được thiết lập để chuyển hướng tất cả các route về `index.html`, hỗ trợ React Router hoạt động đúng khi người dùng truy cập trực tiếp vào URL.

8.3 Nginx — Reverse Proxy

8.3.1 Vai trò của Nginx

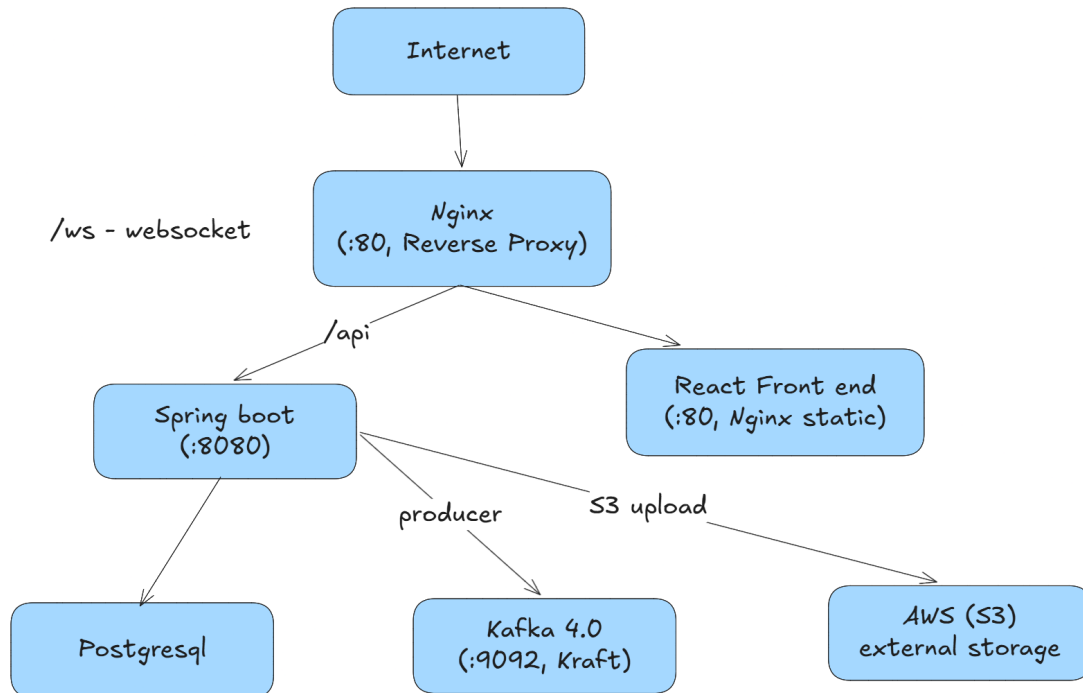
Nginx đóng vai trò là reverse proxy trung tâm, tiếp nhận toàn bộ traffic từ bên ngoài trên cổng 80 và định tuyến đến đúng dịch vụ nội bộ [9]. Kiến trúc này mang lại hai lợi ích chính: thứ nhất, các cổng nội bộ của từng container (8080 cho Backend, 80 cho Frontend) không bị lộ ra internet; thứ hai, client chỉ cần giao tiếp với một địa chỉ duy nhất.

8.3.2 Cấu hình định tuyến

File cấu hình `nginx/nginx.conf` định nghĩa ba quy tắc định tuyến:

- `/ws` → Backend (WebSocket): Định tuyến các kết nối WebSocket STOMP/SockJS đến Spring Boot. Cần thiết lập `proxy_http_version 1.1` và header `Upgrade` để nâng cấp kết nối HTTP thành WebSocket. Thời gian chờ `proxy_read_timeout` được đặt là 3600 giây để duy trì kết nối real-time lâu dài.
- `/api/` → Backend (REST API): Tất cả các yêu cầu REST API được chuyển tiếp đến Spring Boot tại `http://spring-boot:8080`. Header `X-Real-IP` và `X-Forwarded-For` được thêm vào để Spring Boot ghi nhận địa chỉ IP thực của client.

- / → Frontend (React SPA): Toàn bộ các yêu cầu còn lại được chuyển đến container React. Frontend Nginx nội bộ xử lý việc phục vụ file tĩnh và hỗ trợ React Router.



Hình 8.2: Kiến trúc triển khai hệ thống

8.4 Môi trường sản xuất

8.4.1 Docker Compose sản xuất

File `docker-compose.prod.yml` mở rộng cấu hình phát triển bằng cách thêm hai container mới—spring-boot và frontend—vào cùng mạng Docker nội bộ với PostgreSQL, Kafka và Nginx. Tất cả năm dịch vụ giao tiếp với nhau qua tên service (DNS nội bộ của Docker), không cần địa chỉ IP cố định.

Một điểm khác biệt quan trọng so với cấu hình phát triển là giá trị `KAFKA_CFG_ADVERTISED_LISTENERS` được thay đổi từ `PLAINTEXT://localhost:9092` thành `PLAINTEXT://kafka:9092`. Trong môi trường phát triển, Spring Boot chạy trên máy host nên dùng `localhost`; trong môi trường container, Spring Boot nằm trong một container khác và phải kết nối đến Kafka qua tên service Docker.

```
# Triển khai toàn bộ hệ thống
docker compose -f docker-compose.prod.yml up -d --build
```

8.4.2 Quản lý biến môi trường và bảo mật

Tất cả thông tin nhạy cảm của hệ thống được quản lý thông qua file `.env` và không bao giờ được đưa vào mã nguồn (được liệt kê trong `.gitignore`). Bảng 8.1 liệt kê các nhóm biến môi trường chính.

Khi triển khai lên máy chủ, file `.env` được sao chép trực tiếp lên server và được tham chiếu qua chỉ thị `env_file` trong Docker Compose. Cách tiếp cận này đảm bảo không có thông tin đăng nhập nào được nhúng vào Docker image hoặc lộ trong lịch sử git.

Bảng 8.1: Các nhóm biến môi trường của hệ thống

Nhóm	Biến	Mục đích
Cơ sở dữ liệu	DATABASE_URL DATABASE_USERNAME DATABASE_PASSWORD	Kết nối PostgreSQL
JWT	EXP_TOKEN EXP_REFRESH_TOKEN	Thời hạn Access/Refresh token
OAuth2 Google	GOOGLE_CLIENT_ID GOOGLE_CLIENT_SECRET OAUTH2_REDIRECT_URI	Đăng nhập Google
Email	MAIL_SENDER_USERNAME MAIL_SENDER_PASSWORD	Gửi email qua SMTP
AWS S3	AWS_ACCESS_KEY AWS_SECRET_KEY	Lưu trữ file (ảnh bìa, tài liệu)

8.4.3 Flyway Migration tự động

Hệ thống sử dụng Flyway để quản lý phiên bản schema cơ sở dữ liệu. Khi container Spring Boot khởi động, Flyway tự động thực thi các file migration chưa được áp dụng theo thứ tự phiên bản trước khi ứng dụng nhận request. Điều này đảm bảo schema cơ sở dữ liệu luôn đồng bộ với phiên bản ứng dụng mà không cần can thiệp thủ công.

```
db/migration/
+-- V1__create_schema.sql      # Toàn bộ schema hệ thống
\-- V2__seed_data.sql         # Dữ liệu mẫu ban đầu
```

8.4.4 Sơ đồ kiến trúc triển khai

Hình 8.3 mô tả chi tiết luồng traffic từ người dùng đến từng thành phần của hệ thống trong môi trường sản xuất.

Toàn bộ năm container chạy trong cùng một Docker network nội bộ, giao tiếp qua DNS tên service. Chỉ có Nginx container được ánh xạ cổng ra ngoài (cổng 80), đảm bảo các dịch vụ nội bộ như PostgreSQL (5432) và Kafka (9092) không bị truy cập trực tiếp từ internet.

8.5 Quy trình triển khai

Quy trình triển khai hệ thống lên một máy chủ Linux mới bao gồm các bước sau:

1. **Chuẩn bị máy chủ:** Cài đặt Docker Engine và Docker Compose Plugin trên máy chủ Ubuntu.

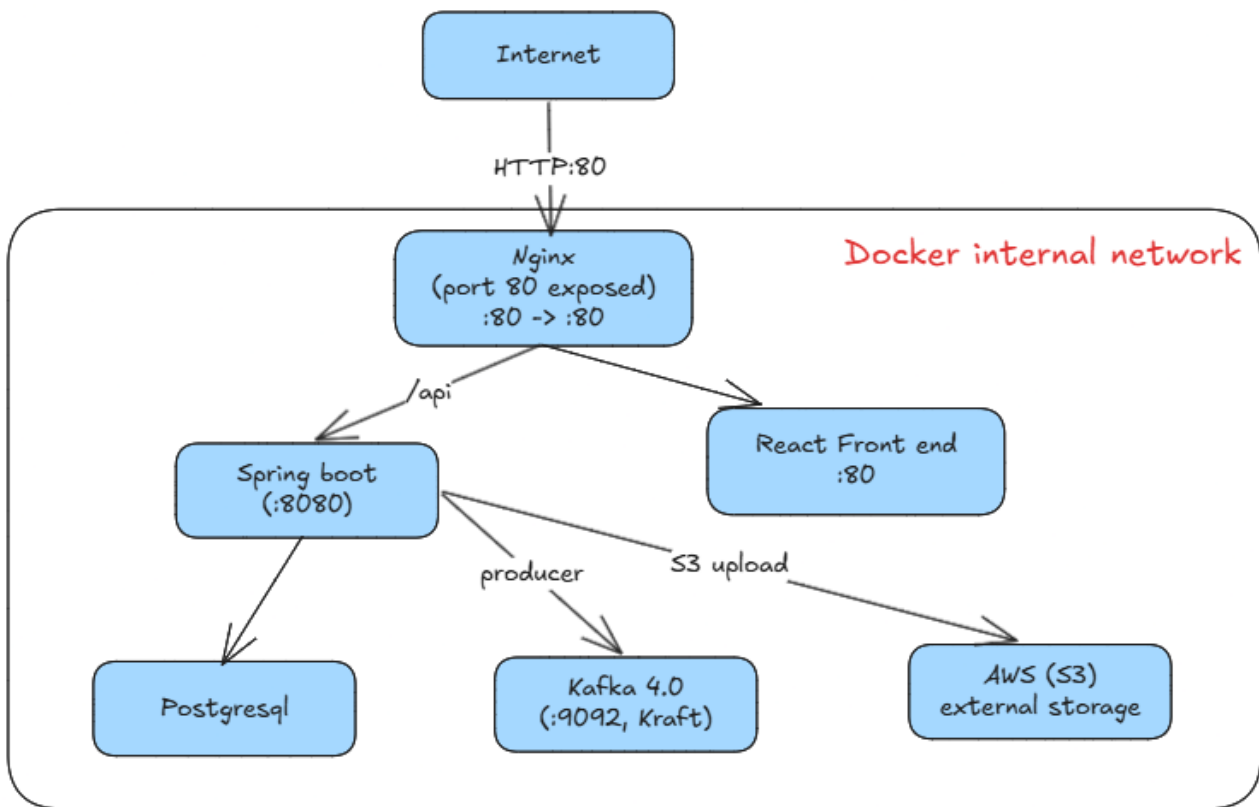
```
curl -fsSL https://get.docker.com | sh
```

2. **Sao chép mã nguồn:** Clone repository về máy chủ.

```
git clone <repository-url>
cd doan
```

3. **Cấu hình môi trường:** Tạo file .env từ template và điền các giá trị thực tế (database credentials, API keys, v.v.).

```
cp LMS_be/.env.example LMS_be/.env
```



Hình 8.3: Luồng traffic qua các container trong môi trường sản xuất

Chỉnh sửa LMS_be/.env với giá trị thực tế

4. Khởi động hệ thống: Build image và khởi động tất cả container.

```
docker compose -f docker-compose.prod.yml up -d --build
```

5. Kiểm tra trạng thái: Xác nhận tất cả container đang chạy.

```
docker compose -f docker-compose.prod.yml ps
docker compose -f docker-compose.prod.yml logs spring-boot
```

Sau khi khởi động, Flyway tự động chạy các migration, sau đó hệ thống sẵn sàng phục vụ tại địa chỉ <http://<server-ip>>.

8.6 Hạ tầng mạng: VPS, Domain và HTTPS

8.6.1 Máy chủ ảo (VPS)

Để một ứng dụng web có thể truy cập từ internet, hệ thống cần được triển khai trên một máy chủ có địa chỉ IP công cộng hoạt động liên tục 24/7. Giải pháp phổ biến nhất là thuê *Virtual Private Server* (VPS)—một máy chủ ảo được cấp phát tài nguyên CPU, RAM và ổ đĩa riêng biệt trên hạ tầng vật lý của nhà cung cấp dịch vụ đám mây [10].

Đối với hệ thống thư viện này, cấu hình tối thiểu được đề xuất là **2GB RAM** do Kafka yêu cầu khoảng 512MB

bộ nhớ để hoạt động ổn định cùng với Spring Boot và PostgreSQL. Bảng 8.2 so sánh một số nhà cung cấp VPS phổ biến.

Bảng 8.2: So sánh các nhà cung cấp VPS phổ biến

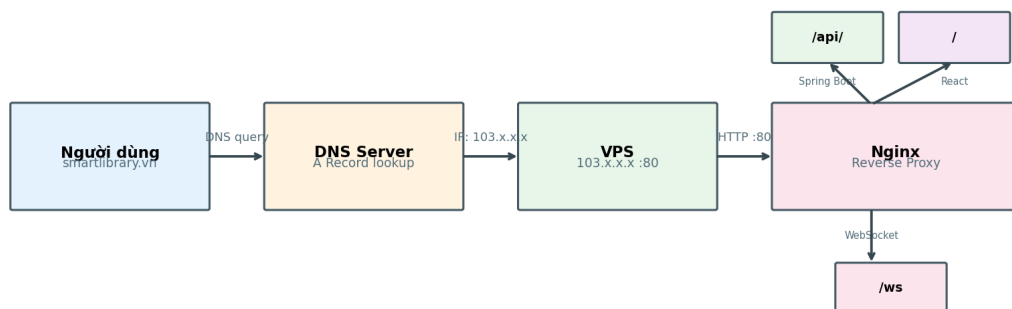
Nhà cung cấp	RAM	Giá	Ghi chú
Hetzner Cloud (CX22)	4 GB	€4/tháng	Giá rẻ, datacenter EU
DigitalOcean (Basic)	1 GB	\$6/tháng	Phổ biến, dễ sử dụng
Vultr (Regular)	1 GB	\$5/tháng	Nhiều khu vực
Oracle Cloud Free Tier	1 GB	Miễn phí	Vĩnh viễn, ARM architecture

8.6.2 Tên miền (Domain)

Sau khi có VPS với địa chỉ IP công cộng (ví dụ: 103.45.67.89), hệ thống đã có thể truy cập trực tiếp qua IP. Tuy nhiên, để có địa chỉ thân thiện và dễ nhớ, cần mua thêm một *tên miền*—ví dụ smartlibrary.vn hoặc lms-hcmut.com. Tên miền được đăng ký thông qua các nhà cung cấp như GoDaddy, Namecheap hoặc các nhà đăng ký tên miền Việt Nam.

Sau khi mua tên miền, cần cấu hình bản ghi DNS loại **A record** trỏ tên miền về địa chỉ IP của VPS. Quá trình phân giải DNS diễn ra tự động, thông thường có hiệu lực trong vòng vài phút đến 24 giờ. Hình 8.4 minh họa luồng kết nối từ người dùng đến hệ thống.

Luồng kết nối từ tên miền đến hệ thống



Hình 8.4: Luồng kết nối từ tên miền đến hệ thống

8.6.3 HTTPS và SSL/TLS Certificate

Các trình duyệt hiện đại cảnh báo người dùng khi truy cập trang web qua HTTP không mã hóa. Để bảo mật dữ liệu truyền tải—đặc biệt là JWT token, thông tin đăng nhập và dữ liệu cá nhân—hệ thống cần được cấu hình HTTPS thông qua SSL/TLS certificate [11].

Let's Encrypt là tổ chức phi lợi nhuận cung cấp SSL certificate hoàn toàn miễn phí và tự động. Công cụ **Certbot** tích hợp trực tiếp với Nginx, tự động xác minh quyền sở hữu tên miền, cấp certificate và cập nhật cấu hình Nginx để lắng nghe trên cổng 443 (HTTPS):

```
# Cài đặt Certbot
apt install certbot python3-certbot-nginx

# Cấp certificate và tự động cấu hình Nginx
certbot --nginx -d smartlibrary.vn -d www.smartlibrary.vn
```

Certificate Let's Encrypt có hiệu lực 90 ngày và được Certbot tự động gia hạn qua cron job, không cần can thiệp thủ công.

8.6.4 Tổng chi phí vận hành

Bảng 8.3 tổng hợp chi phí ước tính để vận hành hệ thống trong môi trường sản xuất với tên miền và HTTPS đầy đủ.

Bảng 8.3: Chi phí vận hành hệ thống ước tính

Thành phần	Chi phí	Chu kỳ	Ghi chú
VPS (Hetzner CX22)	90.000đ	Tháng	4GB RAM, 2 vCPU
Tên miền .com	300.000đ	Năm	Qua Namecheap, GoDaddy
SSL Certificate	Miễn phí	90 ngày	Let's Encrypt, tự gia hạn
AWS S3 (lưu trữ)	23.000đ	Tháng	1GB storage + transfer
Tổng	≈140.000đ	Tháng	Chưa tính domain năm đầu

Chi phí vận hành thấp này cho thấy hệ thống hoàn toàn khả thi để triển khai thực tế trong môi trường trường đại học với ngân sách hạn chế.

Chương 9: Tổng kết

Dự án “Hệ thống Quản lý Thư viện Thông minh tích hợp AI” (Mã dự án: HK251-DAGD1-512) được xây dựng với mục tiêu hiện đại hóa hoạt động quản lý thư viện truyền thống, đồng thời nâng cao trải nghiệm tra cứu và gợi ý sách cho người dùng thông qua các kỹ thuật Trí tuệ nhân tạo. Sau toàn bộ quá trình nghiên cứu, thiết kế, hiện thực và kiểm thử, nhóm đã hoàn thành một hệ thống phần mềm toàn diện từ lớp kiến trúc đến lớp triển khai.

9.1 Kết quả đạt được

9.1.1 Kiến trúc phần mềm

Nhóm đã thiết kế và hiện thực hóa thành công kiến trúc **Modular Monolith** kết hợp **Clean Architecture** cho lõi Backend (Spring Boot). Mã nguồn được phân chia theo ranh giới nghiệp vụ (Bounded Contexts) theo nguyên tắc Domain-Driven Design gồm các module: Quản lý sách, Đọc giả, Lưu thông, Đặt chỗ, Phạt vi phạm và Tương tác người dùng. Bên trong mỗi module, các tầng *Domain*, *Application*, *Infrastructure* và *Presentation* được tổ chức nghiêm ngặt theo Dependency Rule, giúp logic nghiệp vụ độc lập hoàn toàn với Framework và Cơ sở dữ liệu [12].

Giao diện người dùng được xây dựng bằng **React**, **TypeScript** và **Vite** theo mô hình Single Page Application. Frontend phân vùng rõ ràng cho độc giả, thủ thư và admin, gọi trực tiếp các API nghiệp vụ của backend, nhận thông báo thời gian thực và hiển thị các trạng thái vận hành quan trọng. Dịch vụ AI được triển khai độc lập bằng **Python FastAPI** và worker nền, giao tiếp với Backend qua REST, message queue và callback có chữ ký HMAC, giúp tách workload embedding/LLM khỏi lõi nghiệp vụ Spring Boot.

9.1.2 Tích hợp Trí tuệ nhân tạo

- **Tìm kiếm ngữ nghĩa:** Tích hợp thành công mô hình `bkai-foundation-models/vietnamese-bi-encoder` (dựa trên SBERT) để mã hóa văn bản tiếng Việt thành vector 768 chiều. Kết hợp với chỉ mục HNSW trên PostgreSQL (pgvector), hệ thống đạt tốc độ tìm kiếm láng giềng gần nhất (ANN) vượt trội so với phương pháp quét toàn bảng [13].
- **Gợi ý sách cá nhân hóa:** Xây dựng pipeline gợi ý lai ghép, kết hợp content-based realtime, ALS implicit feedback và trending fallback. Cách tiếp cận này phục vụ được cả người dùng mới, người dùng có ít tương tác và người dùng đã có lịch sử mượn/đánh giá đủ lớn [14].
- **Xử lý tài liệu bất đồng bộ:** Xây dựng pipeline xử lý PDF, làm sạch văn bản, chia chunk, sinh embedding, chọn chunk đại diện và gọi Gemini ở bước reduce metadata. Pipeline ghi vector vào PostgreSQL/pgvector, ghi tag/alias vào schema canonical và có fallback khi LLM lỗi hoặc hết quota.

9.1.3 Kiểm thử

Nhóm áp dụng chiến lược kiểm thử phân tầng nhằm đảm bảo chất lượng xuyên suốt toàn bộ hệ thống:

- **Kiểm thử đơn vị (Unit Testing):** Viết bộ test case cho các Use Case và Domain Service cốt lõi bằng JUnit 5 và Mockito (Backend), Jest (Frontend). Mọi phụ thuộc ngoài đều được thay thế bằng đối tượng giả lập (mock) để đảm bảo tính độc lập và tốc độ thực thi.

- **Kiểm thử tích hợp (Integration Testing):** Sử dụng Testcontainers để khởi tạo môi trường PostgreSQL trong container tạm thời, kiểm tra các luồng nghiệp vụ và migration Flyway trên database thật mà không cần môi trường bên ngoài cố định.
- **Kiểm thử hợp đồng API và service:** Kiểm tra request/response, payload, cơ chế xác thực JWT, service contract frontend và SQL contract của AI service.
- **Kiểm thử AI:** Kiểm thử PDF processing, chunking, chất lượng metadata, fallback khi Gemini lỗi, hybrid semantic search, recommender và kịch bản live Gemini report khi có quota API.

9.1.4 Triển khai hệ thống

Toàn bộ các dịch vụ (Backend, AI Service, PostgreSQL, Redis, RabbitMQ, NGINX) được đóng gói thành Docker images và điều phối thông qua Docker Compose. Áp dụng kỹ thuật **Multi-stage Build** để tối ưu kích thước image, tách biệt môi trường build và runtime. Hệ thống được triển khai thành công lên máy chủ VPS, sử dụng NGINX làm Reverse Proxy và cấu hình SSL/TLS qua Let's Encrypt, đảm bảo kết nối HTTPS an toàn.

9.2 Nhận xét và đánh giá

Qua quá trình thực hiện, nhóm nhận thấy việc lựa chọn kiến trúc Modular Monolith kết hợp Clean Architecture là phù hợp với quy mô đồ án: đủ nghiêm ngặt để duy trì chất lượng mã nguồn, nhưng không đánh đổi sự phức tạp vận hành của Microservices thuần túy. Mô hình SBERT tiếng Việt chứng tỏ hiệu quả rõ rệt so với tìm kiếm từ khóa truyền thống, đặc biệt với các truy vấn ngữ nghĩa và từ đồng nghĩa.

Hạn chế lớn nhất của hệ thống hiện tại là chất lượng gợi ý phụ thuộc nhiều vào lượng dữ liệu tương tác, dẫn đến vấn đề Cold Start đối với người dùng và đầu sách mới. Ngoài ra, chi phí tính toán của việc mã hóa (encoding) toàn bộ kho sách khi cập nhật lớn đòi hỏi cơ chế cập nhật gia tăng (incremental update) trong tương lai.

Nhìn chung, dự án đã đạt được mục tiêu đề ra: xây dựng một hệ thống quản lý thư viện hoàn chỉnh, có khả năng vận hành thực tế, tích hợp thành công các kỹ thuật AI hiện đại phù hợp với đặc thù tiếng Việt, và được kiểm thử nghiêm túc trước khi triển khai.

Chương 10: Tài nguyên và Mã nguồn dự án

Để phục vụ cho quá trình thiết kế, kiểm thử và đánh giá, nhóm thực hiện xin đính kèm các đường dẫn đến tài nguyên của dự án:

Loại tài nguyên	Đường dẫn truy cập
Mã nguồn (Source Code)	frontend: https://github.com/imsythang/LMS_fe backend: https://github.com/quangthangk4/LMS_BE
Thiết kế giao diện (Figma)	https://www.figma.com/design/aSHkyebIWfWN7LMgTeY0be/Library-Management-System?node-id=0-1&t=mf5nzMKPeV3M6Vyp-1

Bảng 10.1: Danh sách các tài nguyên trực tuyến của hệ thống

Tài liệu tham khảo

- [1] M. Richards and N. Ford, *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, Inc., 2020.
- [2] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Pearson Education, 2018.
- [3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 3982–3992.
- [4] A. Kane, *pgvector: Open-source vector similarity search for PostgreSQL*, 2024, accessed: 2026-05-01. [Online]. Available: <https://github.com/pgvector/pgvector>
- [5] R. Burke, "Hybrid recommender systems: Survey and experiments," *User Modeling and User-Adapted Interaction*, vol. 12, no. 4, pp. 331–370, 2002.
- [6] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in *8th IEEE International Conference on Data Mining (ICDM 2008)*. IEEE, 2008, pp. 263–272.
- [7] Docker Inc., "Docker: Accelerated container application development," <https://www.docker.com/>, 2024, accessed: 2026-05-01.
- [8] —, "Use multi-stage builds," <https://docs.docker.com/build/building/multi-stage/>, 2024, accessed: 2026-05-01.
- [9] F5, Inc., "NGINX documentation," <https://nginx.org/en/docs/>, 2024, accessed: 2026-05-01.
- [10] DigitalOcean, LLC, "What is a VPS?" <https://www.digitalocean.com/resources/articles/what-is-a-vps>, 2024, accessed: 2026-05-01.
- [11] Internet Security Research Group, "Let's encrypt: Free, automated, and open certificate authority," <https://letsencrypt.org/>, 2024, accessed: 2026-05-01.
- [12] S. Newman, *Building microservices: designing fine-grained systems*. " O'Reilly Media, Inc.", 2015.
- [13] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992.
- [14] C. C. Aggarwal, *Recommender systems*. Springer, 2016.
- [15] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional Transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, 2019, pp. 4171–4186.
- [16] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.
- [17] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*

- (EMNLP). Association for Computational Linguistics (ACL), 2014, pp. 1532–1543. [Online]. Available: <https://nlp.stanford.edu/pubs/glove.pdf>
- [18] L. N. Thanh, “Clean architecture - gỡ khổng lồ sinh sau đẻ muộn,” <https://tech.cybozu.vn/clean-architecture-ga-khong-lo-sinh-sau-de-muon-phan-1-51c96/>, 2025, truy cập: 17-06-2025.
- [19] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 42, no. 4, pp. 824–836, 2018.
- [20] E. Evans, *Domain-driven design: tackling complexity in the heart of software*. Addison-Wesley Professional, 2004.
- [21] D. Q. Nguyen and N. A. Tuan, “Phobert: Pre-trained language models for vietnamese,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1037–1042.
- [22] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. John Wiley & Sons, 2011.
- [23] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
- [24] G. Meszaros, *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley, 2007.
- [25] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Addison-Wesley, 2018.
- [26] Postman Inc., “What is API testing?” <https://www.postman.com/api-platform/api-testing/>, 2024, accessed: 2026-05-01.
- [27] I. Molyneaux, *The Art of Application Performance Testing*, 2nd ed. O’Reilly Media, 2014.
- [28] JUnit Team, “JUnit 5 user guide,” <https://junit.org/junit5/docs/current/user-guide/>, 2024, accessed: 2026-05-01.
- [29] Mockito Contributors, “Mockito framework site,” <https://site.mockito.org/>, 2024, accessed: 2026-05-01.
- [30] Meta Platforms, Inc., “Jest: Delightful JavaScript testing,” <https://jestjs.io/>, 2024, accessed: 2026-05-01.
- [31] Testcontainers Contributors, “Testcontainers for Java,” <https://java.testcontainers.org/>, 2024, accessed: 2026-05-01.
- [32] Postman Inc., “Postman API platform,” <https://www.postman.com/>, 2024, accessed: 2026-05-01.
- [33] Grafana Labs, “k6: Open source load testing tool,” <https://k6.io/>, 2024, accessed: 2026-05-01.